

Microservicios con Spring Boot y Nube de primavera

Cree microservicios resistentes y escalables utilizando
Spring Cloud, Istio y Kubernetes

Cuarta edición

Magnus Larsson

◁packt▷



Microservicios con Spring Boot y Spring Cloud

Cuarta edición

Cree microservicios resistentes y escalables utilizando Spring Cloud, Istio y Kubernetes

Magnus Larsson



Microservicios con Spring Boot y Spring Cloud

Cuarta edición

Derechos de autor © 2025 Packt Publishing

Todos los derechos reservados. Ninguna parte de este libro podrá reproducirse, almacenarse en un sistema de recuperación de datos ni transmitirse en ninguna forma ni por ningún medio sin la autorización previa por escrito del editor, excepto en el caso de citas breves incluidas en artículos críticos o reseñas.

En la preparación de este libro se ha hecho todo lo posible para garantizar la precisión de la información presentada. Sin embargo, la información contenida en este libro se vende sin garantía, ni expresa ni implícita. Ni el autor, ni Packt Publishing, ni sus distribuidores serán responsables de ningún daño causado o presuntamente causado, directa o indirectamente, por este libro.

Packt Publishing se ha esforzado por proporcionar información sobre las marcas comerciales de todas las empresas y productos mencionados en este libro mediante el uso adecuado de mayúsculas. Sin embargo, Packt Publishing no puede garantizar la exactitud de esta información.

Director de cartera: Ashwin Nair

Responsable de relaciones: Aaron Lazar

Ingeniero de contenido : Kinnari Chohan

Gerente de proyecto : Ruvika Rao

Editor técnico: Sweety Pagaria

Editor de copias: Safis Editing

Indexador: Hemangini Bari

Corrector: Kinnari Chohan

Diseñador de producción: Ganesh Bhadwalkar

Líder de crecimiento: Anamika Singh

Primera publicación: septiembre de 2019

Segunda edición: septiembre de 2021

Tercera edición: agosto de 2023

Cuarta edición: agosto de 2025

Referencia de producción: 1290725

Publicado por Packt Publishing Ltd.

Casa Grosvenor

11 Plaza de San Pablo

Birmingham B3

1RB, Reino Unido.

ISBN 978-1-80580-127-6

www.packtpub.com

Colaboradores

Acerca del autor

Magnus Larsson, veterano del sector de TI desde 1986, ha sido consultor de importantes empresas suecas como Volvo, Ericsson y AstraZeneca. A pesar de las dificultades pasadas con los sistemas distribuidos, las herramientas de código abierto actuales (como Spring Cloud, Kubernetes e Istio) ofrecen soluciones eficaces. Durante los últimos 10 años, Magnus ha ayudado a los clientes a utilizar estas herramientas y ha compartido sus conocimientos mediante presentaciones y entradas de blog.

Me gustaría agradecer a las siguientes personas:

A Aaron Lazar, Kinnari Chohan y Ruvika Rao, de Packt Publishing, por su apoyo. A mi esposa, María, gracias por todo su apoyo y comprensión durante la escritura de este libro.

Acerca del revisor

K. Siva Prasad Reddy trabaja como promotor de desarrollo en JetBrains. Cuenta con más de 18 años de experiencia en el desarrollo de aplicaciones empresariales distribuidas y escalables con Java, Go y Node.js.

Es autor publicado y conferenciante. Cuenta con una amplia experiencia en el desarrollo de sistemas de software utilizando arquitecturas modernas y tecnologías nativas de la nube.

Siva es un sólido practicante de prácticas ágiles como TDD, CI/CD y DevOps.

Comparte sus conocimientos en su blog en <https://sivalabs.in> y a través de su canal de YouTube <https://youtube.com/sivalabs>.

Tabla de contenido

Prefacio	xxv
Parte 1: Introducción a los microservicios Desarrollo con Spring Boot	1
Capítulo 1: Introducción a los microservicios	3
Requisitos técnicos	3
Cómo aprovechar al máximo este libro: conozca sus beneficios gratuitos	3
Lector de próxima generación • 4	
Asistente interactivo de IA (beta) • 4	
Versión PDF o ePub sin DRM • 5	
Mi camino hacia los microservicios	5
Beneficios de los componentes de software autónomos • 7	
Desafíos con los componentes de software autónomos • 9	
Introduzca los microservicios	10
Un ejemplo de paisaje de microservicios • 12	
Definición de un microservicio	13
Desafíos con los microservicios	15
Patrones de diseño para microservicios	16
Descubrimiento de servicios • 17	
Problema • 17	
Solución • 18	
Requisitos de la solución • 18	

Servidor perimetral • 19
Problema • 19
Solución • 19
Requisitos de la solución • 20
Microservicios reactivos • 20
Problema • 20
Solución • 20
Requisitos de la solución • 20
Configuración central • 21
Problema • 21
Solución • 22
Requisitos de la solución • 22
Análisis de registros centralizado • 22
Problema • 22
Solución • 23
Requisitos de la solución • 23
Rastreo distribuido • 24
Problema • 24
Solución • 25
Requisitos de la solución • 26
Disyuntor • 26
Problema • 26
Solución • 26
Requisitos de la solución • 26
Bucle de control • 27
Problema • 27
Solución • 28
Requisitos de la solución • 28
Monitoreo centralizado y alarmas • 28
Problema • 28

Solución • 28	
Requisitos de la solución • 29	
Habilitadores de software	30
Otras consideraciones importantes	31
Resumen	34
Capítulo 2: Introducción a Spring Boot	35
Requisitos técnicos	36
Bota de primavera	36
Convención sobre configuración y archivos JAR pesados	37
Ejemplos de código para configurar una aplicación Spring Boot	38
La anotación mágica @SpringBootApplication • 38	
Escaneo de componentes • 39	
Configuración basada en Java • 40	
¿Qué novedades hay en Spring Boot 3.0 a 3.5? • 41	
Migración de una aplicación Spring Boot 2	44
WebFlux de primavera	45
Ejemplos de código de configuración de un servicio REST	46
Dependencias de inicio • 46	
Expedientes de propiedad • 47	
Ejemplo RestController • 48	
springdoc-openapi • 48	
Datos de primavera • 50	
Entidad • 51	
Repositorios • 52	
Corriente de nubes de primavera	54
Ejemplos de código para enviar y recibir mensajes	54
Estibador	56
Resumen	59
Preguntas	59

Capítulo 3: Creación de un conjunto de microservicios cooperativos	61
Requisitos técnicos	61
Introducción al panorama de microservicios	62
Información manejada por los microservicios • 62	
El producto servicio • 62	
El servicio de reseñas • 63	
El servicio de recomendaciones • 63	
El servicio compuesto de productos • 63	
Información relacionada con la infraestructura • 63	
Reemplazo temporal del descubrimiento de servicios • 64	
Generando microservicios esqueléticos	64
Uso de Spring Initializr para generar código esqueleto • 64	
Configuración de compilaciones de múltiples proyectos en Gradle • 69	
Agregar API RESTful	71
Agregar una API y un proyecto de utilidad • 71	
El proyecto API • 72	
El proyecto utilitario • 74	
Implementando nuestra API • 74	
Agregar un microservicio compuesto	77
Clases de API • 78	
Propiedades • 79	
El componente de integración • 80	
Implementación de API compuesta • 83	
Añadiendo gestión de errores	84
El manejador de excepciones del controlador REST global • 85	
Manejo de errores en implementaciones de API • 86	
Manejo de errores en el cliente API • 86	
Probar las API manualmente	87
Agregar pruebas de microservicios automatizadas de forma aislada	90
Adición de pruebas semiautomatizadas de un entorno de microservicios	94
Probando el script de prueba • 95	

Tabla de contenido	ix
Resumen	96
Preguntas	97
Capítulo 4: Implementación de nuestros microservicios mediante Docker	99
Requisitos técnicos	100
Introducción a Docker	100
Ejecutando nuestros primeros comandos Docker	101
Ejecución de Java en Docker • 103	
Limitación de CPU disponibles • 103	
Limitar la memoria disponible • 104	
Usando Docker con un microservicio	106
Cambios en el código fuente • 107	
Construyendo una imagen de Docker • 110	
Puesta en marcha del servicio • 111	
Ejecución del contenedor en modo separado	114
Gestión de un paisaje de microservicios mediante Docker Compose	115
Cambios en el código fuente • 115	
Puesta en marcha del panorama de microservicios • 117	
Automatizar pruebas de microservicios cooperativos	120
Solución de problemas durante una ejecución de prueba	123
Resumen	124
Preguntas	125
Capítulo 5: Agregar una descripción de API mediante OpenAPI	127
Requisitos técnicos	128
Introducción al uso de springdoc-openapi	128
Añadiendo springdoc-openapi al código fuente	130
Agregar dependencias a los archivos de compilación de Gradle • 131	
Agregar configuración de OpenAPI y documentación general de API a ProductCompositeService • 131	
Agregar documentación específica de la API a la interfaz ProductCompositeService • 134	

Construcción e inicio del panorama de microservicios	138
Probando la documentación de OpenAPI	140
Resumen	146
Preguntas	146
Capítulo 6: Añadiendo persistencia	149
Requisitos técnicos	150
Objetivos del capítulo	150
Agregar una capa de persistencia a los microservicios centrales	152
Añadiendo dependencias • 153	
Almacenamiento de datos con clases de entidad • 154	
Definición de repositorios en Spring Data • 157	
Escritura de pruebas automatizadas que se centran en la persistencia	158
Uso de contenedores de prueba • 159	
Escritura de pruebas de persistencia • 161	
Uso de la capa de persistencia en la capa de servicio	167
Registro de la URL de conexión a la base de datos • 167	
Agregar nuevas API • 168	
Llamada a la capa de persistencia desde la capa de servicio • 169	
Declaración de un asignador de beans Java • 171	
Actualización de las pruebas de servicio • 172	
Ampliación de la API de servicio compuesto	174
Agregar nuevas operaciones a la API de servicio compuesto • 174	
Agregar métodos a la capa de integración • 177	
Implementación de las nuevas operaciones de API compuestas • 178	
Actualización de las pruebas de servicio compuestas • 179	
Agregar bases de datos al entorno de Docker Compose	180
La configuración de Docker Compose • 180	
Configuración de la conexión a la base de datos • 182	
Las herramientas CLI de MongoDB y MySQL • 184	
Pruebas manuales de las nuevas API y la capa de persistencia	185

Actualización de las pruebas automatizadas del entorno de microservicios 188

Resumen 191

Preguntas 191

Capítulo 7: Desarrollo de microservicios reactivos

193

Requisitos técnicos 194

Elegir entre API síncronas sin bloqueo y servicios asíncronos controlados por eventos 194

Desarrollo de API REST síncronas sin bloqueo 195

 Una introducción al Proyecto Reactor • 196

 Persistencia sin bloqueo usando Spring Data para MongoDB • 198

 Cambios en el código de prueba • 199

 API REST sin bloqueo en los servicios principales • 200

 Cambios en las API • 200

 Cambios en las implementaciones del servicio • 200

 Cambios en el código de prueba • 202

 Cómo lidiar con el código de bloqueo • 202

 API REST sin bloqueo en los servicios compuestos • 205

 Cambios en la API • 205

 Cambios en la implementación del servicio • 205

 Cambios en la capa de integración • 206

 Cambios en el código de prueba • 208

Desarrollo de servicios asíncronos basados en eventos 209

 Cómo afrontar los desafíos de la mensajería • 210

 Grupos de consumidores • 211

 Reintentos y colas de mensajes muertos • 213

 Orden y particiones garantizadas • 214

 Definición de temas y eventos • 215

 Cambios en los archivos de compilación de Gradle • 217

 Consumir eventos en los servicios principales • 218

 Declaración de procesadores de mensajes • 218

 Cambios en las implementaciones del servicio • 220

Agregar configuración para consumir eventos • 221	
Cambios en el código de prueba • 222	
Publicación de eventos en el servicio compuesto • 224	
Publicación de eventos en la capa de integración • 224	
Agregar configuración para publicar eventos • 225	
Cambios en el código de prueba • 226	
Ejecución de pruebas manuales del entorno de microservicios reactivos	229
Guardado de eventos • 230	
Agregar una API de salud • 230	
Uso de RabbitMQ sin usar particiones • 234	
Uso de RabbitMQ con particiones • 238	
Uso de Kafka con dos particiones por tema • 241	
Ejecución de pruebas automatizadas del panorama de microservicios reactivos	244
Resumen	245
Preguntas	245

Parte 2: Aprovechar Spring Cloud para
Administrar microservicios

247

Capítulo 8: Introducción a Spring Cloud

249

Requisitos técnicos	250
La evolución de Spring Cloud	250
Uso de Netflix Eureka para el descubrimiento de servicios	252
Uso de Spring Cloud Gateway como servidor perimetral	253
Uso de Spring Cloud Config para la configuración centralizada	254
Usando Resilience4j para mejorar la resiliencia	256
Ejemplo de uso del disyuntor en Resilience4j • 257	
Uso del trazado micrométrico y Zipkin para el trazado distribuido	259
Resumen	262
Preguntas	262

Capítulo 9: Agregar descubrimiento de servicios mediante Netflix Eureka 263

Requisitos técnicos	263	
Introducción al descubrimiento de servicios	264	
El problema con el descubrimiento de servicios basado en DNS •	264	
Desafíos del descubrimiento de servicios •	266	
Descubrimiento de servicios con Netflix Eureka en Spring Cloud •	267	
Configuración de un servidor Netflix Eureka	269	
Conexión de microservicios a un servidor Eureka de Netflix		270
Configuración para uso en desarrollo	272	
Parámetros de configuración de Eureka •	273	
Configuración del servidor Eureka •	274	
Configuración de clientes en el servidor Eureka •	275	
Probando el servicio de descubrimiento		275
Ampliación •	276	
Reducción de escala •	279	
Pruebas disruptivas con el servidor Eureka •	280	
Detener el servidor Eureka •	280	
Puesta en marcha de una instancia adicional del servicio del producto •	281	
Reiniciando el servidor Eureka	282	
Resumen	283	
Preguntas	283	

Capítulo 10: Uso de Spring Cloud Gateway para Ocultar microservicios detrás de un servidor perimetral 285

Requisitos técnicos	286	
Agregar un servidor de borde a nuestro panorama de sistemas		286
Configuración de Spring Cloud Gateway	287	
Agregar una comprobación de salud compuesta •	288	
Configuración de Spring Cloud Gateway •	290	
Reglas de enrutamiento •	291	

xiv	Tabla de contenido
Probando el servidor de borde	296
Examinando lo que se expone fuera del motor Docker • 297	
Probando las reglas de enrutamiento • 298	
Llamada a la API compuesta del producto a través del servidor perimetral • 298	
Llamar a la interfaz de usuario Swagger a través del servidor perimetral • 299	
Llamar a Eureka a través del servidor perimetral • 300	
Enrutamiento basado en el encabezado del host • 301	
Resumen	303
Preguntas	303
 Capítulo 11: Protección del acceso a las API	 305
Requisitos técnicos	305
Introducción a OAuth 2.0 y OpenID Connect	306
Presentación de OAuth 2.0 • 306	
Presentación de OpenID Connect • 310	
Asegurar el panorama del sistema	311
Protegiendo la comunicación externa con HTTPS	313
Reemplazo de un certificado autofirmado en tiempo de ejecución • 315	
Asegurar el acceso al servidor de descubrimiento	317
Cambios en el servidor Eureka • 317	
Cambios en los clientes de Eureka • 319	
Agregar un servidor de autorización local	319
Protección de API mediante OAuth 2.0 y OpenID Connect	321
Cambios tanto en el servidor de borde como en el servicio compuesto por producto • 322	
Cambios únicamente en el servicio compuesto por producto • 324	
Cambios para permitir que Swagger UI adquiera tokens de acceso • 325	
Cambios en el script de prueba • 327	
Pruebas con el servidor de autorización local	328
Construcción y ejecución de pruebas automatizadas • 328	
Prueba del servidor de descubrimiento protegido • 328	
Adquisición de tokens de acceso • 330	

Tabla de contenido	xv
Adquisición de tokens de acceso mediante el flujo de concesión de credenciales del cliente • 330	
Adquisición de tokens de acceso mediante el flujo de concesión de código de autorización • 331	
Llamadas a API protegidas mediante tokens de acceso • 334	
Prueba de la interfaz de usuario de Swagger con OAuth 2.0 • 337	
Pruebas con un proveedor externo de OpenID Connect 339	
Configuración de una cuenta en Auth0 • 339	
Aplicación de los cambios necesarios para utilizar Auth0 como proveedor de OpenID • 344	
Cambiar la configuración en los servidores de recursos OAuth • 345	
Cambiar el script de prueba para que adquiera tokens de acceso de Auth0 • 346	
Ejecución del script de prueba con Auth0 como proveedor de OpenID Connect • 347	
Adquisición de tokens de acceso mediante el flujo de concesión de credenciales del cliente • 349	
Adquisición de tokens de acceso mediante el flujo de concesión de código de autorización • 349	
Llamada a API protegidas mediante tokens de acceso Auth0 • 353	
Obtener información adicional sobre el usuario • 353	
Resumen 354	
Preguntas 355	
Capítulo 12: Configuración centralizada	357
Requisitos técnicos 357	
Introducción a Spring Cloud Config Server 358	
Selección del tipo de almacenamiento del repositorio de configuración • 359	
Decidir sobre la conexión inicial del cliente • 359	
Asegurar la configuración • 360	
Asegurar la configuración en tránsito • 360	
Asegurar la configuración en reposo • 360	
Presentación de la API del servidor de configuración • 361	
Configuración de un servidor de configuración 361	
Configuración de una regla de enrutamiento en el servidor perimetral • 363	
Configuración del servidor de configuración para su uso con Docker • 363	
Configuración de clientes de un servidor de configuración 364	
Configuración de la información de conexión • 366	
Estructuración del repositorio de configuración • 367	

xvi	Tabla de contenido
Probando Spring Cloud Config Server	367
Creación y ejecución de pruebas automatizadas • 368	
Obtener la configuración mediante la API del servidor de configuración • 368	
Cifrado y descifrado de información confidencial • 370	
Resumen	371
Preguntas	372
Capítulo 13: Mejorar la resiliencia mediante Resilience4j	373
Requisitos técnicos	374
Presentando los mecanismos de resiliencia de Resilience4j	374
Presentación del disyuntor • 375	
Presentamos el limitador de tiempo • 378	
Introducción al mecanismo de reintento • 378	
Añadiendo los mecanismos de resiliencia al código fuente	379
Adición de retrasos programables y errores aleatorios • 380	
Cambios en las definiciones de API • 381	
Cambios en el microservicio compuesto por producto • 381	
Cambios en el microservicio del producto • 382	
Adición de un disyuntor y un limitador de tiempo • 384	
Agregar dependencias al archivo de compilación • 384	
Agregar anotaciones en el código fuente • 385	
Adición de lógica de respaldo rápido ante fallos • 386	
Añadiendo configuración • 387	
Agregar un mecanismo de reintento • 388	
Añadiendo la anotación de reintento • 388	
Añadiendo configuración • 388	
Añadiendo pruebas automatizadas • 389	
Probando el disyuntor y el mecanismo de reintento	393
Construcción y ejecución de pruebas automatizadas • 394	
Verificación de que el circuito esté cerrado en condiciones normales de funcionamiento • 394	
Forzar la apertura del disyuntor cuando las cosas salen mal • 395	
Cerrando de nuevo el disyuntor • 396	

Tabla de contenido	xvii
Probando reintentos causados por errores aleatorios • 397	
Resumen	399
Preguntas	400
Capítulo 14: Comprensión del rastreo distribuido	401
Requisitos técnicos	401
Introducción al rastreo distribuido con Micrometer Tracing y Zipkin	402
Añadiendo seguimiento distribuido al código fuente	404
Agregar dependencias a los archivos de compilación • 404	
Agregar configuración para trazado micrométrico y Zipkin • 405	
Agregar Zipkin a los archivos de Docker Compose • 406	
Añadiendo soluciones alternativas para la falta de soporte de clientes reactivos • 407	
Agregar intervalos personalizados y etiquetas personalizadas a intervalos existentes • 409	
Agregar un lapso personalizado • 410	
Agregar etiquetas personalizadas a intervalos existentes • 413	
Probando el seguimiento distribuido	416
Puesta en marcha del panorama del sistema • 416	
Envío de una solicitud de API exitosa • 417	
Envío de una solicitud de API fallida • 421	
Envío de una solicitud de API que activa el procesamiento asíncrono • 422	
Resumen	427
Preguntas	427
 Parte 3: Desarrollo de peso ligero	
Microservicios que utilizan Kubernetes	429
Capítulo 15: Introducción a Kubernetes	431
Requisitos técnicos	431
Introducción a los conceptos de Kubernetes	432
Introducción a los objetos de la API de Kubernetes	433
Presentación de los componentes de tiempo de ejecución de Kubernetes	436

xviii	Tabla de contenido
Creación de un clúster de Kubernetes con minikube	439
Trabajar con perfiles de minikube • 440	
Trabajar con la CLI de Kubernetes, kubectl • 441	
Trabajar con contextos de kubectl • 442	
Creación de un clúster de Kubernetes • 443	
Probando una implementación de muestra	445
Administración de un clúster local de Kubernetes	453
Hibernación y reanudación de un clúster de Kubernetes • 454	
Terminar un clúster de Kubernetes • 455	
Resumen	455
Preguntas	456
Capítulo 16: Implementación de nuestros microservicios en Kubernetes	457
Requisitos técnicos	457
Reemplazo de Netflix Eureka con servicios de Kubernetes	458
Introducción a cómo se utilizará Kubernetes	461
Uso del soporte de Spring Boot para un apagado elegante y sondas de actividad y preparación	461
Presentando Helm	463
Ejecución de comandos de Helm • 465	
Mirando un gráfico de Helm • 465	
Plantillas y valores de Helm • 466	
El cuadro de la biblioteca común • 468	
La plantilla ConfigMap • 469	
Ejemplo de uso de la plantilla ConfigMap • 470	
La plantilla Secretos • 471	
Ejemplo de uso de la plantilla Secretos • 472	
La plantilla de servicio • 474	
Ejemplo de uso de la plantilla de Servicio • 476	
La plantilla de implementación • 478	
Ejemplo de uso de la plantilla de implementación • 482	

Tabla de contenido	XIX
Los gráficos de componentes • 484	
Los gráficos del medio ambiente • 485	
Implementación en Kubernetes para desarrollo y pruebas	487
Construyendo imágenes de Docker • 487	
Resolución de dependencias de gráficos de Helm • 488	
Implementación en Kubernetes • 489	
Cambios en el script de prueba para su uso con Kubernetes • 492	
Prueba de la implementación • 492	
Prueba del soporte de Spring Boot para el apagado elegante y sondas de actividad y preparación • 493	
Implementación en Kubernetes para ensayo y producción	500
Cambios en el código fuente • 501	
Implementación en Kubernetes • 503	
Limpieza • 504	
Resumen	505
Preguntas	506
 Capítulo 17: Implementación de funciones de Kubernetes para Simplificar el panorama del sistema	 509
Requisitos técnicos	510
Reemplazo del servidor de configuración de Spring Cloud	510
Cambios necesarios para reemplazar el servidor de configuración de Spring Cloud • 512	
Reemplazo de Spring Cloud Gateway	517
Cambios necesarios para reemplazar Spring Cloud Gateway • 518	
Automatización del aprovisionamiento de certificados	523
Pruebas con Kubernetes ConfigMaps, Secrets, Ingress y cert-manager	525
Certificados rotativos • 529	
Implementación en Kubernetes para puesta en escena y producción • 531	
Verificar que los microservicios funcionan sin Kubernetes	532
Cambios en los archivos de Docker Compose • 533	
Pruebas con Docker Compose • 535	

Resumen	537
Preguntas	538
 Capítulo 18: Uso de una malla de servicios para Mejorar la observabilidad y la gestión	 539
Requisitos técnicos	540
Introducción a las mallas de servicios mediante Istio	540
Presentando Istio • 541	
Inyección de proxies de Istio en microservicios • 543	
Presentación de los objetos de la API de Istio • 546	
Simplificando el panorama de microservicios	547
Reemplazo del controlador de ingreso de Kubernetes con una puerta de enlace de ingreso de Istio • 547	
Reemplazo del servidor Zipkin con el componente Jaeger de Istio	548
Implementación de Istio en un clúster de Kubernetes	549
Configuración del acceso a los servicios de Istio • 554	
Creación de la malla de servicios	558
Cambios en el código fuente • 558	
Contenido en la plantilla _istio_base.yaml • 558	
Contenido en la plantilla _istio_dr_mutual_tls.yaml • 561	
Ejecución de comandos para crear la malla de servicio • 562	
Registro de propagación de identificadores de seguimiento y de intervalo • 564	
Observando la malla de servicio	566
Asegurar una malla de servicio	571
Protección de puntos finales externos con HTTPS y certificados • 571	
Autenticación de solicitudes externas mediante tokens de acceso OAuth 2.0/OIDC • 574	
Protección de la comunicación interna mediante autenticación mutua con mTLS • 577	
Cómo garantizar que una malla de servicios sea resiliente	579
Prueba de resiliencia mediante la inyección de fallas • 580	
Poniendo a prueba la resiliencia mediante la inyección de retrasos • 581	
Realizar actualizaciones sin tiempo de inactividad	584
Cambios en el código fuente • 586	
Servicios virtuales y normas de destino • 586	

Despliegues y servicios • 588

Uniendo elementos en el diagrama de Helm del entorno de producción • 589

Implementación de las versiones v1 y v2 de los microservicios con enrutamiento a la versión v1 • 590

Verificar que todo el tráfico se dirige inicialmente a la versión v1 de los microservicios • 593

Ejecución de pruebas canarias • 594

Ejecución de una implementación azul-verde • 596

Una breve introducción al comando kubectl patch • 597

Realizando la implementación azul-verde • 598

Ejecución de pruebas con Docker Compose 602

Resumen 603

Preguntas 603

Capítulo 19: Registro centralizado con la pila EFK

605

Requisitos técnicos 606

Presentación de Fluentd 606

Descripción general de Fluentd • 606

Configuración de Fluentd • 608

Implementación de la pila EFK en Kubernetes 617

Construyendo e implementando nuestros microservicios • 617

Implementación de Elasticsearch y Kibana • 619

Un recorrido por los archivos de manifiesto • 620

Ejecución de los comandos de implementación • 622

Implementación de Fluentd • 623

Un recorrido por los archivos de manifiesto • 623

Ejecución de los comandos de implementación • 626

Probando la pila EFK 627

Inicializando Kibana • 628

Análisis de los registros de registro • 629

Descubriendo los registros de microservicios • 634

Realización de análisis de causa raíz • 640

Resumen 646

Preguntas 647

Capítulo 20: Monitoreo de microservicios	651
Requisitos técnicos	651
Introducción a la monitorización de aplicaciones utilizando Prometheus y Grafana	652
Cambios en el código fuente para recopilar métricas de la aplicación	654
Construyendo e implementando los microservicios	656
Monitoreo de microservicios usando paneles de Grafana	657
Instalación de un servidor de correo local para pruebas •	658
Configuración de Grafana •	659
Iniciando la prueba de carga •	660
Uso de los paneles de control integrados de Kiali •	660
Importación de paneles de Grafana existentes •	663
Desarrollando sus propios paneles de control de Grafana •	665
Examinando las métricas de Prometheus •	665
Creación del panel de control •	666
Exportación e importación de paneles de Grafana •	674
Configuración de alarmas en Grafana	676
Configuración de un punto de contacto basado en correo •	676
Configuración de políticas de notificación predeterminadas •	678
Configuración de una alarma en el disyuntor •	679
Probando la alarma del disyuntor •	684
Resumen	688
Preguntas	689
Capítulo 21: Instrucciones de instalación para macOS	693
Requisitos técnicos	693
Instalación de las herramientas necesarias	694
Instalación de SDKMan, Java y la CLI de Spring Boot •	695
Instalación de Homebrew •	696
Uso de Homebrew para instalar herramientas •	697
Instalar herramientas sin Homebrew •	697
Instalación de herramientas en una Mac con procesador Intel •	697

Tabla de contenido	xxiii
Instalación de herramientas en una Mac basada en silicio de Apple • 698	
Acciones posteriores a la instalación • 698	
Verificación de las instalaciones • 700	
Accediendo al código fuente	701
Usando un IDE • 702	
La estructura del código • 702	
Resumen	703
Capítulo 22: Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu 705	
Requisitos técnicos	705
Instalación de las herramientas necesarias	706
Instalación de herramientas en Windows • 707	
Instalación de WSL 2 con un servidor Ubuntu predeterminado • 707	
Instalación de un nuevo servidor Ubuntu 24.04 en WSL 2 • 708	
Instalación de Windows Terminal • 708	
Instalación de Docker Desktop para Windows • 709	
Instalación de Visual Studio Code y su extensión para WSL remoto • 711	
Instalación de herramientas en el servidor Linux en WSL 2 • 711	
Instalación de herramientas mediante apt install • 711	
Instalación de la CLI de Java y Spring Boot mediante SDKMan • 712	
Instalación de las herramientas restantes mediante curl e install • 713	
Verificación de las instalaciones • 714	
Accediendo al código fuente	715
La estructura del código • 716	
Resumen	717
Capítulo 23: Microservicios de Java compilados de forma nativa	719
Requisitos técnicos	720
Cuándo compilar de forma nativa el código fuente de Java	720
Presentación del proyecto GraalVM	721
Presentación del motor AOT de Spring	722

xxiv	Tabla de contenido
Manejo de problemas con la compilación nativa	725
Cambios en el código fuente	727
Actualizaciones de los archivos de compilación de Gradle • 728	
Proporcionar metadatos de accesibilidad y sugerencias personalizadas • 729	
Habilitación de beans Spring en tiempo de compilación en archivos application.yml • 730	
Propiedades de tiempo de ejecución actualizadas • 732	
Configuración del agente de seguimiento de imágenes nativas de GraalVM • 732	
Actualizaciones del script de verificación test-em-all.bash • 733	
Prueba y compilación de imágenes nativas	734
Ejecución del agente de rastreo • 734	
Ejecución de pruebas nativas • 735	
Creación de una imagen nativa para el sistema operativo actual • 736	
Creación de una imagen nativa como una imagen de Docker • 737	
Pruebas con Docker Compose	739
Prueba de microservicios basados en Java VM con el modo AOT deshabilitado • 740	
Prueba de microservicios basados en Java VM con el modo AOT habilitado • 742	
Prueba de microservicios compilados de forma nativa • 743	
Pruebas con Kubernetes	745
Resumen	749
Preguntas	750
Capítulo 24: Desbloquea los beneficios exclusivos de tu libro	751
Cómo desbloquear estos beneficios en tres sencillos pasos	751
Paso 1 • 751	
Paso 2 • 752	
Paso 3 • 752	
¿Necesitas ayuda? • 753	
Otros libros que te pueden gustar	757
Índice	761

Prefacio

Este libro trata sobre cómo crear microservicios listos para producción utilizando Spring Boot y Spring Cloud.

Hace doce años, cuando comencé a explorar los microservicios, estaba buscando un libro como este.

Este libro se desarrolló después de que aprendí y dominé el software de código abierto utilizado para desarrollar, probar, implementar y administrar paisajes de microservicios cooperativos.

Este libro abarca principalmente Spring Boot, Spring Cloud, Docker, Kubernetes, Istio, la pila EFK, Prometheus y Grafana. Cada una de estas herramientas de código abierto funciona de maravilla por sí sola, pero puede resultar difícil comprender cómo usarlas juntas de forma ventajosa. En algunos aspectos se complementan, pero en otros se solapan, y no es evidente cuál elegir para una situación concreta.

Este es un libro práctico que describe paso a paso cómo utilizar estas herramientas de código abierto juntas.

Este es el libro que estaba buscando hace diez años cuando comencé a aprender sobre microservicios, pero con versiones actualizadas de las herramientas de código abierto que cubre.

Para quién es este libro

Este libro está dirigido a desarrolladores y arquitectos de Java y Spring que desean aprender a crear entornos de microservicios desde cero e implementarlos localmente o en la nube, utilizando Kubernetes como orquestador de contenedores e Istio como malla de servicios. No se requieren conocimientos de arquitectura de microservicios para comenzar a leerlo.

Qué cubre este libro

Parte 1, Introducción al desarrollo de microservicios con Spring Boot. En esta primera parte del libro, aprenderá a utilizar algunas de las características más importantes de Spring Boot para desarrollar microservicios.

El capítulo 1, Introducción a los microservicios, le ayudará a comprender la premisa básica del libro (los microservicios) junto con los conceptos esenciales y los patrones de diseño que los acompañan.

El capítulo 2, Introducción a Spring Boot, le presentará Spring Boot y otros proyectos de código abierto que se utilizarán en la primera parte del libro: Spring WebFlux para desarrollar API RESTful, springdoc-openapi para producir documentación basada en OpenAPI para las API, Spring Data para almacenar datos en bases de datos SQL y NoSQL, Spring Cloud Stream para microservicios basados en mensajes y Docker para ejecutar los microservicios como contenedores.

El Capítulo 3, "Creación de un Conjunto de Microservicios Cooperativos", le enseñará a crear un conjunto de microservicios cooperativos desde cero. Utilizará Spring Initializr para crear proyectos esqueleto basados en Spring Framework y Spring Boot. La idea es crear tres servicios principales (que gestionarán sus propios recursos) y un servicio compuesto que utilice los tres servicios principales para agregar un resultado compuesto. Hacia el final del capítulo, aprenderá a agregar API RESTful muy básicas basadas en Spring WebFlux. En los siguientes capítulos, se ampliará la funcionalidad. añadido a estos microservicios.

El Capítulo 4, "Implementación de nuestros microservicios con Docker", te enseñará a implementarlos. Aprenderás a agregar archivos Dockerfiles y archivos docker-compose para iniciar todo el entorno de microservicios con un solo comando. Después, aprenderás a usar múltiples perfiles de Spring para gestionar configuraciones con y sin Docker.

El capítulo 5, "Añadir una descripción de API con OpenAPI", te ayudará a documentar rápidamente las API expuestas por un microservicio con OpenAPI. Utilizarás la herramienta springdoc-openapi para anotar los servicios y crear documentación de API basada en OpenAPI sobre la marcha. El punto clave será cómo probar las API en un navegador web con Swagger UI.

El capítulo 6, "Añadir persistencia", le mostrará cómo añadir persistencia a los datos de los microservicios. Utilizará Spring Data para configurar y acceder a los datos en una base de datos documental MongoDB para dos de los microservicios principales, y para el resto, en una base de datos relacional MySQL. Los contenedores de prueba se utilizarán para iniciar bases de datos cuando se ejecuten pruebas de integración.

El Capítulo 7, Desarrollo de Microservicios Reactivos, le enseñará por qué y cuándo es importante un enfoque reactivo y cómo desarrollar servicios reactivos de extremo a extremo. Aprenderá a desarrollar y probar tanto API RESTful síncronas no bloqueantes como servicios asíncronos basados en eventos. También aprenderá a usar el controlador reactivo no bloqueante para MongoDB y a usar código de bloqueo convencional para MySQL.

Parte 2, Cómo aprovechar Spring Cloud para administrar microservicios: en esta primera parte del libro, comprenderá cómo se puede usar Spring Cloud para gestionar los desafíos que se enfrentan al desarrollar microservicios (es decir, construir un sistema distribuido).

El capítulo 8, Introducción a Spring Cloud, le presentará Spring Cloud y los componentes de Spring Cloud que se utilizarán en este libro.

El capítulo 9, "Añadir descubrimiento de servicios con Netflix Eureka", le mostrará cómo usar Netflix Eureka en Spring Cloud para añadir capacidades de descubrimiento de servicios. Esto se logrará añadiendo un servidor de descubrimiento de servicios basado en Netflix Eureka al entorno del sistema. A continuación, configurará los microservicios para que usen Spring Cloud LoadBalancer y encuentren otros microservicios. Comprenderá cómo se registran automáticamente los microservicios y cómo el tráfico a través de Spring Cloud LoadBalancer se balancea automáticamente a las nuevas instancias cuando están disponibles.

El capítulo 10, " Uso de Spring Cloud Gateway para ocultar microservicios tras un servidor perimetral", le guiará en el proceso de ocultar los microservicios tras un servidor perimetral mediante Spring Cloud Gateway y exponer únicamente ciertas API a usuarios externos. También aprenderá a ocultar la complejidad interna de los microservicios a usuarios externos. Esto se logrará añadiendo un servidor perimetral basado en Spring Cloud Gateway al entorno del sistema y configurándolo para exponer únicamente las API públicas.

El capítulo 11, "Asegurando el acceso a las API", explicará cómo proteger las API expuestas mediante OAuth 2.1 y OpenID Connect. Aprenderá a añadir un servidor de autorización OAuth basado en Spring Authorization Server al entorno del sistema y a configurar el servidor perimetral y el servicio compuesto para que requieran tokens de acceso válidos emitidos por dicho servidor. Aprenderá a exponer el servidor de autorización a través del servidor perimetral y a proteger su comunicación con usuarios externos mediante HTTPS. Finalmente, aprenderá a reemplazar el servidor de autorización OAuth interno con un proveedor externo de OpenID Connect de Auth0.

El capítulo 12, "Configuración centralizada", explicará cómo recopilar los archivos de configuración de todos los microservicios en un repositorio central y usar el servidor de configuración para distribuir la configuración a los microservicios en tiempo de ejecución. También aprenderá a agregar un servidor Spring Cloud Config al entorno del sistema y a configurar todos los microservicios para que lo usen y obtengan su configuración.

El capítulo 13, "Mejora de la resiliencia con Resilience4j", explicará cómo usar las capacidades de Resilience4j para prevenir, por ejemplo, el antipatrón de "cadena de fallos". Aprenderá a añadir un mecanismo de reintento y un disyuntor al servicio compuesto, a configurar el disyuntor para que falle rápidamente cuando el circuito esté abierto y a utilizar un método de respaldo para crear una respuesta óptima.

El capítulo 14, "Comprensión del rastreo distribuido", le mostrará cómo usar Zipkin para recopilar y visualizar información de rastreo. También usará el rastreo Micrometer para agregar identificadores de rastreo a las solicitudes, de modo que se puedan visualizar las cadenas de solicitudes entre microservicios que cooperan.

Parte 3, Desarrollo de microservicios livianos con Kubernetes, esta parte lo ayudará a comprender la importancia de Kubernetes como plataforma de tiempo de ejecución para cargas de trabajo en contenedores.

El capítulo 15, Introducción a Kubernetes, explicará los conceptos fundamentales de Kubernetes y cómo realizar una implementación de ejemplo. También aprenderá a configurar Kubernetes localmente para desarrollo y pruebas con Minikube.

El capítulo 16, "Implementación de nuestros microservicios en Kubernetes", mostrará cómo implementar microservicios en Kubernetes. También aprenderá a usar Helm para empaquetar y configurar microservicios para su implementación en Kubernetes. Helm se usará para implementar los microservicios en diferentes entornos de ejecución, como entornos de prueba y producción. Finalmente, aprenderá a reemplazar Netflix Eureka con la compatibilidad integrada en Kubernetes para el descubrimiento de servicios, basada en objetos de servicio de Kubernetes y el componente de ejecución kube-proxy.

El capítulo 17, "Implementación de las funciones de Kubernetes para simplificar el entorno del sistema", explicará cómo usar las funciones de Kubernetes como alternativa a los servicios de Spring Cloud presentados en los capítulos anteriores. Aprenderá por qué y cómo reemplazar Spring Cloud Config Server con Kubernetes Secrets y ConfigMaps. También aprenderá por qué y cómo reemplazar Spring Cloud Gateway con objetos de Kubernetes Ingress, y cómo agregar cert-manager para aprovisionar y rotar automáticamente certificados para endpoints HTTPS externos.

El capítulo 18, "Uso de una malla de servicios para mejorar la observabilidad y la gestión", presentará el concepto de malla de servicios y explicará cómo usar Istio para implementar una malla de servicios en tiempo de ejecución con Kubernetes. Aprenderá a usar una malla de servicios para mejorar aún más la resiliencia, la seguridad, la gestión del tráfico y la observabilidad del entorno de microservicios.

El capítulo 19, Registro centralizado con la pila EFK, explicará cómo usar Elasticsearch, Fluentd y Kibana (la pila EFK) para recopilar, almacenar y visualizar flujos de registros de microservicios. Aprenderá a implementar la pila EFK en Minikube y a usarla para analizar los registros recopilados y encontrar la salida de los registros de todos los microservicios involucrados en el procesamiento de una solicitud que abarca varios microservicios. También aprenderá a realizar análisis de causa raíz utilizando la pila EFK.

El capítulo 20, "Monitoreo de microservicios", le mostrará cómo monitorear los microservicios implementados en Kubernetes con Prometheus y Grafana. Aprenderá a usar los paneles de control de Grafana para monitorear diferentes tipos de métricas y a crear sus propios paneles. Finalmente, aprenderá a crear alertas en Grafana que se usarán para enviar correos electrónicos con alertas cuando se superen los umbrales configurados para las métricas seleccionadas.

El capítulo 21, Instrucciones de instalación para macOS, le mostrará cómo instalar las herramientas utilizadas en este libro en una Mac. Se cubren tanto las Mac basadas en Intel como en Apple Silicon (ARM64).

El capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu, le mostrará cómo instalar las herramientas utilizadas en este libro en una PC con Windows usando el Subsistema de Windows para Linux (WSL) v2.

El capítulo 23, "Microservicios Java nativos compilados", le mostrará cómo crear microservicios basados en Spring compilados a código nativo. Aprenderá a usar la compatibilidad con imágenes nativas en Spring Framework y Spring Boot, así como el compilador de imágenes nativas GraalVM. En comparación con la Máquina Virtual de Java (JVM), esto generará microservicios que se inician casi al instante.

Al final de cada capítulo, encontrarás preguntas sencillas que te ayudarán a repasar parte del contenido del capítulo. El archivo "Evaluaciones" se encuentra en el repositorio de GitHub y contiene las respuestas a estas preguntas.

Para aprovechar al máximo este libro

Se recomienda tener conocimientos básicos de Java y Spring. Para ejecutar todo el contenido del libro, necesita una Mac con procesador Intel o Apple Silicon, o una PC con al menos 16 GB de memoria, aunque se recomienda tener al menos 24 GB, ya que el entorno de microservicios se vuelve más complejo y requiere más recursos hacia el final del libro. Para obtener una lista completa de los requisitos de software e instrucciones detalladas para configurar su entorno y poder seguir este libro, consulte el Capítulo 21 (para macOS) y el Capítulo 22 (para Windows).

Descargue los archivos de código de ejemplo

El paquete de código del libro está alojado en GitHub en [https://github.com/PacktPublishing/Microservicios con Spring Boot y Spring Cloud](https://github.com/PacktPublishing/Microservicios-con-Spring-Boot-y-Spring-Cloud) (cuarta edición). También tenemos otros paquetes de códigos de nuestro rico catálogo de libros y videos disponibles en <https://github.com/PacktPublishing>. ¡Échales un vistazo!

Descargar las imágenes en color

También proporcionamos un archivo PDF que tiene imágenes en color de las capturas de pantalla/diagramas utilizados en este libro. Puedes descargarlo aquí: <https://packt.link/gbp/9781805801276>.

Convenciones utilizadas

A lo largo de este libro se utilizan varias convenciones textuales.

CodeInText: Indica palabras clave en el texto, nombres de tablas de bases de datos, nombres de carpetas, nombres de archivos, extensiones de archivos, rutas de acceso, URL ficticias, entradas del usuario y cuentas de Twitter. Por ejemplo: "Ejecutar el comando docker ps :"

Un bloque de código se establece de la siguiente manera:

```
@SpringBootApplication
clase pública ProductoServicioAplicación {
    público estático void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

Cuando deseamos llamar su atención sobre una parte particular de un bloque de código, las líneas relevantes o

Los elementos están en negrita:

```
@SpringBootTest
clase ProductServiceApplicationTests {
    @Prueba
    void contextLoads() {
    }
}
```

Cualquier entrada o salida de la línea de comandos se escribe de la siguiente manera:

```
./gradlew compilación
```

Negrita: Indica un término nuevo, una palabra importante o palabras que se ven en pantalla. Por ejemplo, las palabras en menús o cuadros de diálogo aparecen en el texto de esta manera. Por ejemplo: "Usaremos Spring Initializr para generar un proyecto esqueleto para cada microservicio".



Las advertencias o notas importantes aparecen así.



Los consejos y trucos aparecen así:

Ponte en contacto con nosotros

Los comentarios de nuestros lectores siempre son bienvenidos.

Comentarios generales: si tiene preguntas sobre cualquier aspecto de este libro o tiene algún comentario general, envíenos un correo electrónico a customer-care@packt.com y mencione el título del libro en el asunto de su mensaje.

Erratas: Aunque hemos tomado todas las precauciones para garantizar la precisión de nuestro contenido, pueden producirse errores. Si encuentra algún error en este libro, le agradeceríamos que nos lo comunicara.

Por favor visite <http://www.packt.com/submit-errata>, Haga clic en Enviar erratas y complete el formulario.

Piratería: Si encuentra copias ilegales de nuestras obras en internet, en cualquier formato, le agradeceríamos que nos proporcionara la dirección o el nombre del sitio web. Por favor, contáctenos en copyright@packt.com con el enlace al material.

Si está interesado en convertirse en autor: si hay un tema en el que es experto y está interesado en escribir o contribuir a un libro, visite <http://authors.packt.com/>.

Comparte tus pensamientos

Una vez que hayas leído Microservicios con Spring Boot y Spring Cloud, Cuarta Edición , ¡nos encantaría conocer tu opinión! Haz clic aquí para ir directamente a la página de reseñas de Amazon. para este libro y comparte tus comentarios.

Su reseña es importante para nosotros y para la comunidad tecnológica y nos ayudará a garantizar que ofrecemos contenido de excelente calidad.

Parte 1

Introducción a los microservicios

Desarrollo utilizando

Bota de primavera

En esta primera parte del libro, aprenderá a utilizar algunas de las características más importantes de Spring Boot para desarrollar microservicios.

Esta parte del libro incluye los siguientes capítulos:

- Capítulo 1, Introducción a los microservicios
- Capítulo 2, Introducción a Spring Boot
- Capítulo 3, Creación de un conjunto de microservicios cooperativos
- Capítulo 4, Implementación de nuestros microservicios mediante Docker
- Capítulo 5, Agregar una descripción de API mediante OpenAPI
- Capítulo 6, Añadiendo persistencia
- Capítulo 7, Desarrollo de microservicios reactivos

1

Introducción a los microservicios

Este libro no elogia ciegamente los microservicios. En cambio, trata sobre cómo podemos aprovechar sus beneficios y, al mismo tiempo, afrontar los desafíos de construir servicios escalables, resilientes y manejables. microservicios.

A modo de introducción a este libro, en este capítulo se tratarán los siguientes temas:

- Mi camino hacia los microservicios
- ¿Qué es una arquitectura basada en microservicios?
- Desafíos con los microservicios
- Patrones de diseño para afrontar desafíos
- Habilitadores de software que pueden ayudarnos a afrontar estos desafíos
- Otras consideraciones importantes que no se tratan en este libro

Requisitos técnicos

No se requiere ninguna instalación para este capítulo. Sin embargo, puede que le interese consultar las convenciones del modelo C4: <https://c4model.com> ya que las ilustraciones de este capítulo están inspiradas en el modelo C4.

Este capítulo no contiene ningún código fuente.

Cómo aprovechar al máximo este libro: conozca sus beneficios gratuitos

Desbloquea beneficios gratuitos exclusivos que vienen con tu compra, diseñados cuidadosamente para potenciar tu recorrido de aprendizaje y ayudarte a aprender sin límites.

Aquí tienes una rápida descripción general de lo que obtienes con este libro:

Lector de próxima generación

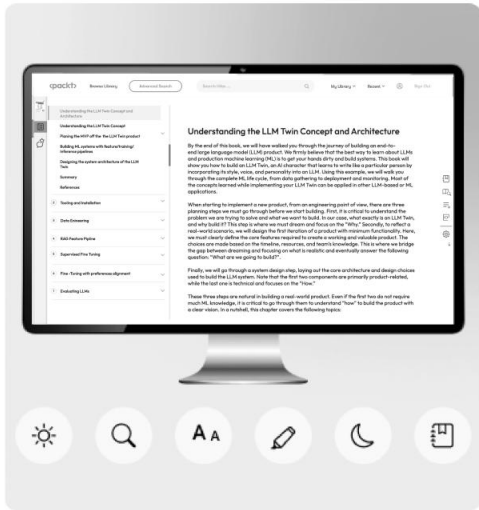


Figura 1.1: Ilustración de las características del lector Packt de próxima generación

Nuestro lector basado en web, diseñado para ayudarle a aprender de manera efectiva, viene con las siguientes características:

 Sincronización de progreso en varios dispositivos: aprenda de cualquier dispositivo con sincronización de progreso perfecta.

 Resaltar y tomar notas: Convierte tu lectura en conocimiento duradero.

 Marcadores: Revisa tus favoritos

Aprendizajes importantes en cualquier momento.

 Modo oscuro: enfoque con mínima fatiga visual cambiando al modo oscuro o sepia.

Asistente interactivo de IA (beta)



Figura 1.2: Ilustración del asistente de IA de Packt

Nuestro asistente interactivo de IA ha sido entrenado

Sobre el contenido de este libro, para maximizar su experiencia de aprendizaje. Incluye las siguientes características:

 Resumirlo: Resume secciones clave o un capítulo completo.

 Explicadores de código de IA: en el lector Packt de próxima generación, haga clic en el botón Explicar sobre cada bloque de código para obtener explicaciones del código impulsadas por IA.

Nota: El asistente de inteligencia artificial es parte del Packt Reader de próxima generación y todavía está en versión beta.

Versión PDF o ePub sin DRM



Aprende sin límites con los siguientes beneficios incluidos en tu compra:

 Aprende desde cualquier lugar con una copia PDF de este libro sin DRM.

 Utilice su lector electrónico favorito para aprender a usar un Versión ePub sin DRM de este libro.

Figura 1.3: PDF y ePub gratuitos

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock. Luego, busca este libro por nombre. Asegúrate de que sea la edición correcta.

Nota: Tenga a mano la factura de compra antes de comenzar.

UNLOCK NOW



Mi camino hacia los microservicios

Cuando aprendí por primera vez sobre el concepto de microservicios en 2014, me di cuenta de que había estado desarrollando microservicios (bueno, más o menos) durante varios años sin saber que estaba tratando con microservicios.

Participé en un proyecto que comenzó en 2009, donde desarrollamos una plataforma basada en un conjunto de funciones independientes. La plataforma se entregó a varios clientes que la implementaron localmente. Para facilitar a los clientes la elección de las funciones que querían usar de la plataforma, cada función se desarrolló como un componente de software autónomo; es decir, contaba con sus propios datos persistentes y solo se comunicaba con otros componentes mediante API bien definidas.

Como no puedo analizar características específicas de la plataforma de este proyecto, he generalizado los nombres de los componentes, que están etiquetados del Componente A al Componente F. La composición de la plataforma como un conjunto de componentes se ilustra de la siguiente manera:

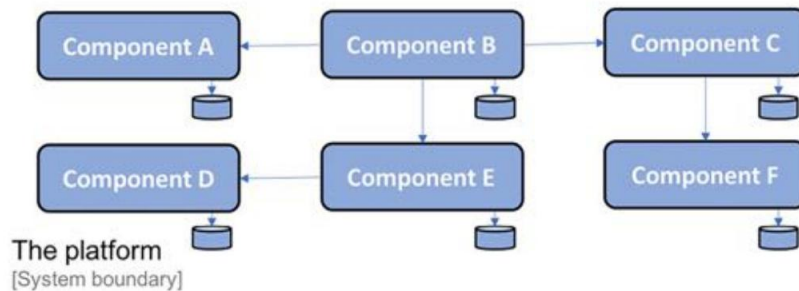


Figura 1.4: La composición de la plataforma

En la ilustración, también podemos ver que cada componente tiene su propio almacenamiento para datos persistentes y no comparte bases de datos con otros componentes.

Cada componente se desarrolla con Java y Spring Framework, se empaqueta como un archivo WAR y se implementa como una aplicación web en un contenedor web Java EE, por ejemplo, Apache Tomcat. Según las necesidades específicas del cliente, la plataforma puede implementarse en uno o varios servidores.

Una implementación de dos nodos podría verse así:

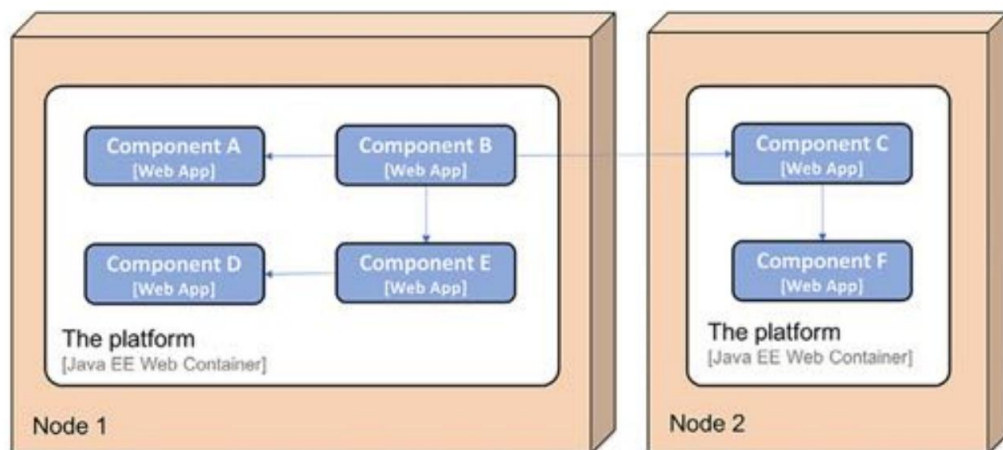


Figura 1.5: Un escenario de implementación de dos nodos

Beneficios de los componentes de software autónomos

En este proyecto aprendí que descomponer la funcionalidad de la plataforma en un conjunto de componentes de software autónomos ofrece una serie de beneficios:

- Un cliente puede implementar partes de la plataforma en su propio entorno de sistemas, integrándola con sus sistemas existentes utilizando sus API bien definidas.
- El siguiente es un ejemplo en el que un cliente decidió implementar el Componente A, el Componente B, el Componente D y el Componente E desde la plataforma e integrarlos con dos sistemas existentes, el Sistema A y el Sistema B, en el panorama de sistemas del cliente:

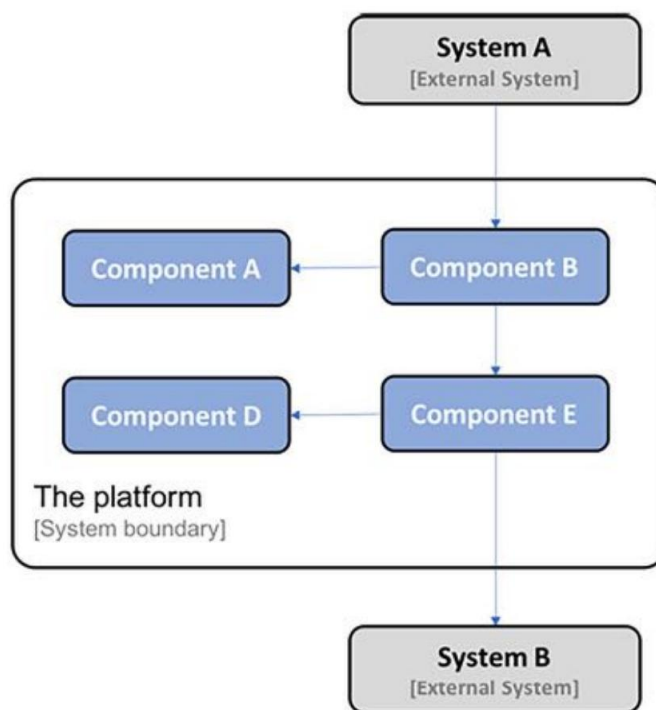


Figura 1.6: Despliegue parcial de la plataforma

Otro cliente podría optar por reemplazar partes de la funcionalidad de la plataforma con implementaciones ya existentes en su infraestructura de sistemas, lo que podría requerir la adopción de la funcionalidad existente en las API de la plataforma. El siguiente es un ejemplo de un cliente que ha reemplazado los componentes C y F de la plataforma con su propia implementación:

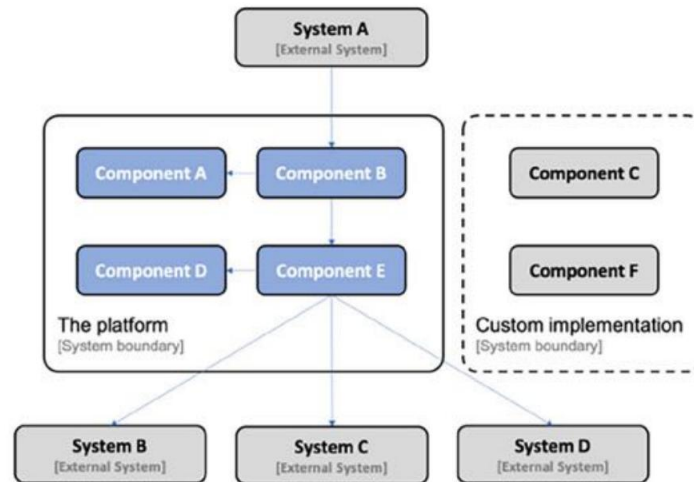


Figura 1.7: Reemplazo de piezas de la plataforma

Cada componente de la plataforma puede entregarse y actualizarse por separado. Gracias al uso de API bien definidas, un componente puede actualizarse a una nueva versión sin depender del ciclo de vida de los demás.

El siguiente es un ejemplo donde el componente A se ha actualizado desde la versión v1.1 a la v1.2. El componente B, que llama al componente A, no necesita actualizarse, ya que utiliza una API bien definida; es decir, sigue siendo el mismo después de la actualización (o al menos es retrocompatible).

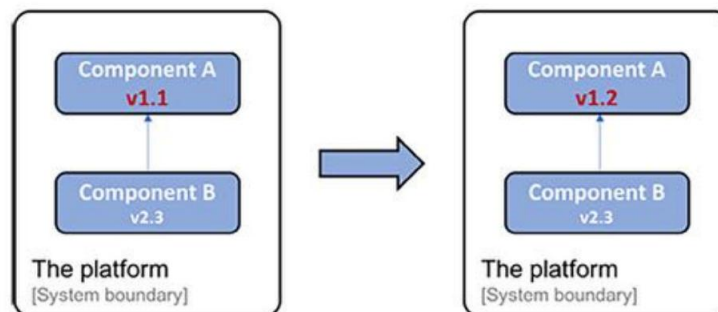


Figura 1.8: Actualización de un componente específico

Gracias al uso de API bien definidas, cada componente de la plataforma también puede escalarse horizontalmente a múltiples servidores independientemente de los demás componentes. El escalado puede realizarse para cumplir con requisitos de alta disponibilidad o para gestionar un mayor volumen de solicitudes. En este proyecto específico, se logró configurando manualmente balanceadores de carga frente a varios servidores, cada uno ejecutando un contenedor web Java EE. Un ejemplo donde el Componente A tiene Se ha escalado a tres instancias y su aspecto es el siguiente:

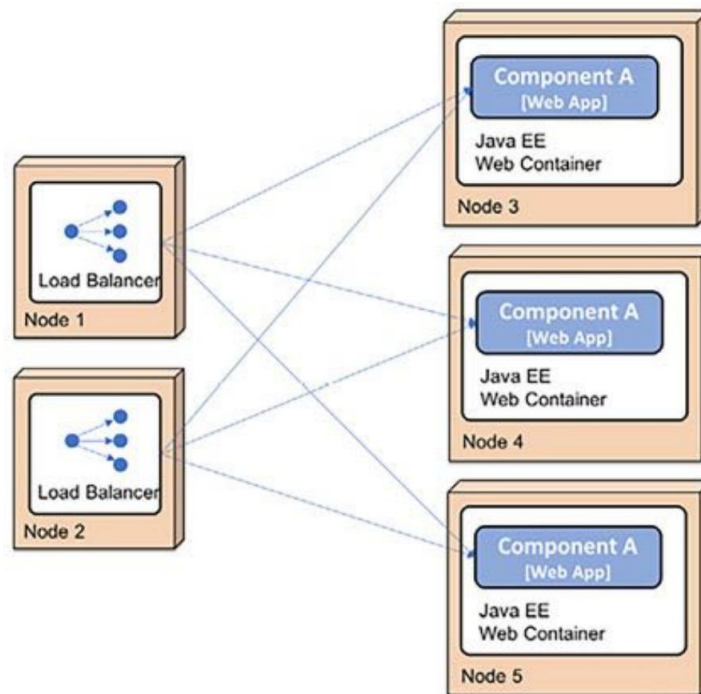


Figura 1.9: Escalamiento horizontal de la plataforma

Desafíos de los componentes de software autónomos

Mi equipo también aprendió que descomponer la plataforma introdujo una serie de nuevos desafíos a los que no estábamos expuestos (al menos no en el mismo grado) cuando desarrollamos aplicaciones más tradicionales y monolíticas:

- Añadir nuevas instancias a un componente requería configurar manualmente los balanceadores de carga y los nuevos nodos. Esta tarea era lenta y propensa a errores.

Inicialmente, la plataforma era propensa a errores causados por los demás sistemas con los que se comunicaba. Si un sistema dejaba de responder a las solicitudes enviadas desde la plataforma de forma oportuna, esta agotaba rápidamente recursos cruciales, como los subprocesos del sistema operativo, especialmente al exponerse a un gran número de solicitudes simultáneas. Esto provocaba que los componentes de la plataforma se bloquearan o incluso fallaran. Dado que la mayor parte de la comunicación en la plataforma se basaba en comunicación síncrona, la falla de un componente podía provocar fallos en cascada; es decir, los clientes de los componentes que fallaban también podían fallar después de un tiempo. Esto se conoce como cadena de fallos.

Mantener la configuración consistente y actualizada en todas las instancias de los componentes se convirtió rápidamente en un problema, lo que generaba mucho trabajo manual y repetitivo. Esto ocasionaba ocasionalmente problemas de calidad.

- Monitorear el estado de la plataforma en términos de problemas de latencia y uso de hardware (por ejemplo, uso de CPU, memoria, discos y red) era más complicado en comparación con monitorear una sola instancia de una aplicación monolítica.
- Recopilar archivos de registro de varios componentes distribuidos y correlacionar eventos de registro relacionados de los componentes también fue difícil, pero factible ya que el número de componentes Los componentes eran fijos y conocidos de antemano.

Con el tiempo, abordamos la mayoría de los desafíos mencionados en la lista anterior con una combinación de herramientas desarrolladas internamente e instrucciones bien documentadas para gestionarlos manualmente. En general, la escala de la operación permitía que los procedimientos manuales para el lanzamiento de nuevas versiones de los componentes y la gestión de problemas de tiempo de ejecución fueran aceptables, aunque no deseables.

Introduzca los microservicios

Aprender sobre arquitecturas basadas en microservicios en 2014 me hizo darme cuenta de que otros proyectos también habían enfrentado desafíos similares (en parte por razones distintas a las que describí antes, por ejemplo, los grandes proveedores de servicios en la nube que cumplían con los requisitos de escala web). Muchos pioneros de los microservicios habían publicado detalles de las lecciones aprendidas. Fue muy interesante...

Aprende de estas lecciones.

Muchos de los pioneros desarrollaron inicialmente aplicaciones monolíticas que les dieron un gran éxito empresarial. Sin embargo, con el tiempo, estas aplicaciones monolíticas se volvieron cada vez más difíciles de mantener y desarrollar. También se volvió difícil escalarlas más allá de las capacidades de las máquinas más grandes disponibles (lo que se conoce como escalado vertical).

Con el tiempo, los pioneros comenzaron a encontrar maneras de dividir las aplicaciones monolíticas en componentes más pequeños que pudieran publicarse y escalarse de forma independiente. El escalado de componentes pequeños se puede realizar mediante el escalado horizontal, es decir, implementando un componente en varios servidores más pequeños y colocando un balanceador de carga delante. Si se realiza en la nube, la capacidad de escalado es potencialmente ilimitada; solo es cuestión de cuántos servidores virtuales se incorporen (dado que el componente puede escalar horizontalmente en un gran número de instancias, pero hablaremos de esto más adelante).

En 2014, también conocí varios proyectos nuevos de código abierto que ofrecían herramientas y marcos de trabajo que simplificaban el desarrollo de microservicios y podían utilizarse para afrontar los retos que conlleva una arquitectura basada en microservicios. Algunos de ellos son los siguientes:

- Pivotal lanzó Spring Cloud, que envuelve partes del OSS de Netflix para brindar capacidades como descubrimiento dinámico de servicios, gestión de configuración, seguimiento distribuido, interrupción de circuitos y más.
- También aprendí sobre Docker y la revolución de los contenedores, lo cual es fundamental para minimizar la brecha entre el desarrollo y la producción. Poder empaquetar un componente no solo como un artefacto de tiempo de ejecución implementable (por ejemplo, un archivo Java .war o .jar), sino como una imagen completa, lista para ejecutarse como contenedor en un servidor con Docker, fue un gran avance para el desarrollo y las pruebas.



Por ahora, piense en un contenedor como un proceso aislado. Aprenderemos más sobre los contenedores en el Capítulo 4, "Implementación de nuestros microservicios con Docker".

Un motor de contenedores, como Docker, no es suficiente para usar contenedores en un entorno de producción. Se necesita algo que garantice que todos los contenedores estén en funcionamiento y que permita escalarlos en varios servidores, proporcionando así alta disponibilidad y mayores recursos computacionales.

Este tipo de productos se conocieron como orquestadores de contenedores. Diversos productos han evolucionado en los últimos años, como Apache Mesos, Docker en modo enjambre, Amazon ECS, HashiCorp Nomad y Kubernetes. Kubernetes fue desarrollado inicialmente por Google. Cuando Google lanzó la versión 1.0 en 2015, también donó Kubernetes a CNCF (<https://www.cncf.io/>). Durante 2018, Kubernetes se convirtió en una especie de estándar de facto, disponible tanto en formato empaquetado para uso local como como servicio de la mayoría de los principales proveedores de nube.



Como se explica en <https://kubernetes.io/blog/2015/04/borg-predecessor-to-kubernetes/>, Kubernetes es en realidad una reescritura basada en código abierto de un orquestador de contenedores interno, llamado Borg, utilizado por Google durante más de una década antes de que se fundara el proyecto Kubernetes.

- En 2018, comencé a aprender sobre el concepto de una malla de servicios y cómo puede complementar un orquestador de contenedores para descargar aún más los microservicios de responsabilidades para hacerlos manejables y resilientes.

Un ejemplo de paisaje de microservicios

Dado que este libro no puede cubrir todos los aspectos de las tecnologías que acabo de mencionar, me centraré en las partes que han demostrado ser útiles en los proyectos de clientes en los que he participado desde 2014. Describiré cómo se pueden utilizar juntas para crear microservicios cooperativos que sean manejables, escalables y resilientes.

Cada capítulo de este libro abordará un tema específico. Para demostrar cómo se integran los elementos, utilizaré un pequeño conjunto de microservicios cooperativos que desarrollaremos a lo largo del libro. El entorno de microservicios se describirá en el Capítulo 3, «Creación de un conjunto de microservicios cooperativos»; por ahora, basta con saber que se presenta así:

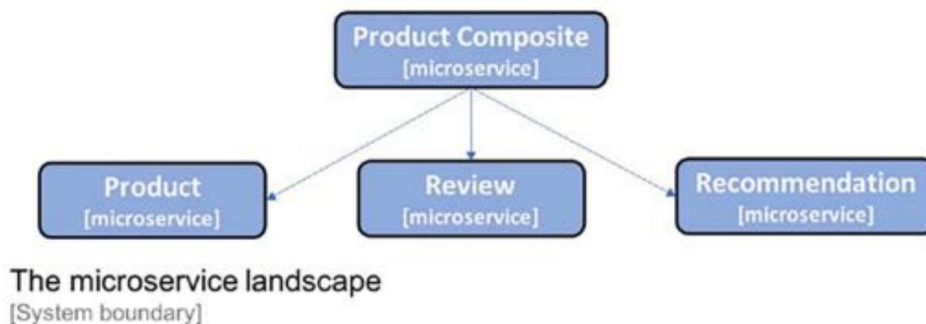


Figura 1.10: El panorama del sistema basado en microservicios utilizado en el libro



Tenga en cuenta que este es un entorno de sistema muy pequeño de microservicios cooperativos. Los servicios de soporte que añadiremos en los próximos capítulos podrían parecer excesivamente complejos para estos pocos microservicios. Sin embargo, tenga en cuenta que las soluciones presentadas en este libro buscan dar soporte a un entorno de sistema mucho más amplio.

Ahora que conocemos los beneficios y desafíos potenciales de los microservicios, veamos... Empecemos a analizar cómo se puede definir un microservicio.

Definición de un microservicio

Una arquitectura de microservicios consiste en dividir aplicaciones monolíticas en componentes más pequeños, lo que logra dos objetivos principales:

- Desarrollo más rápido, lo que permite implementaciones continuas
- Más fácil de escalar, manual o automáticamente.

Un microservicio es esencialmente un componente de software autónomo, actualizable, reemplazable y escalable de forma independiente. Para funcionar como tal, debe cumplir ciertos criterios, como se indica a continuación:

- Debe ajustarse a una arquitectura de nada compartido; es decir, los microservicios no comparten datos. ¡En bases de datos entre sí!
- Debe comunicarse únicamente a través de interfaces bien definidas, ya sea mediante API y servicios síncronos o, preferiblemente, enviando mensajes de forma asíncrona. Las API y los formatos de mensaje utilizados deben ser estables, estar bien documentados y evolucionar siguiendo una estrategia de control de versiones definida.
- Debe implementarse como procesos de ejecución independientes. Cada instancia de un microservicio se ejecuta en un proceso de ejecución independiente; por ejemplo, un contenedor Docker.
- Las instancias de microservicio no tienen estado, por lo que las solicitudes entrantes a un microservicio pueden ser manejado por cualquiera de sus instancias.

Al utilizar un conjunto de microservicios que cooperan, podemos implementar en varios servidores más pequeños en lugar de vernos obligados a implementar en un único servidor grande, como tenemos que hacer cuando implementamos una aplicación monolítica.

Dado que se cumplen los criterios anteriores, es más fácil ampliar un solo microservicio a más instancias (por ejemplo, utilizando más servidores virtuales) en comparación con ampliar una gran aplicación monolítica.

Utilizar las capacidades de escalado automático disponibles en la nube también es una posibilidad, pero no suele ser viable para una aplicación monolítica de gran tamaño. Además, es más fácil actualizar o incluso reemplazar un solo microservicio que actualizar una aplicación monolítica de gran tamaño.

Esto se ilustra en el siguiente diagrama, donde una aplicación monolítica se ha dividido en seis microservicios, cada uno de los cuales se ha implementado en servidores separados. Algunos microservicios también se han escalado independientemente de los demás:

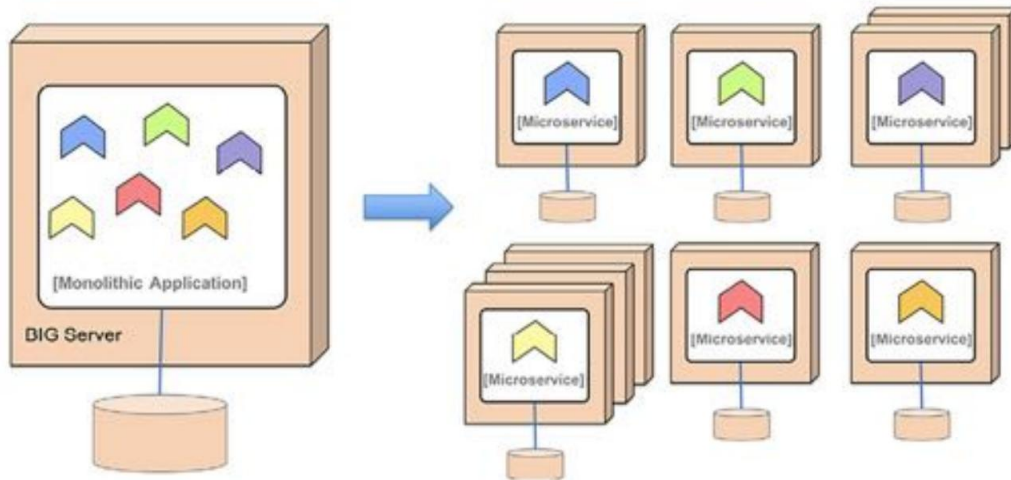


Figura 1.11: División de un monolito en microservicios

Una pregunta muy frecuente que recibo de los clientes es:



¿Qué tamaño debe tener un microservicio?

Intento utilizar las siguientes reglas generales:

- Lo suficientemente pequeño para que un desarrollador pueda realizar un seguimiento
- Lo suficientemente grande como para no poner en riesgo el rendimiento (es decir, la latencia) y/o la consistencia de los datos (las claves externas SQL entre datos almacenados en diferentes microservicios ya no son algo que se pueda dar por sentado)

En resumen, la arquitectura de microservicios es, en esencia, un estilo arquitectónico en el que descomponemos una aplicación monolítica en un grupo de componentes de software autónomos que cooperan entre sí. La motivación es permitir un desarrollo más rápido y facilitar la escalabilidad de la aplicación.

Con una mejor comprensión de cómo definir un microservicio, podemos avanzar y detallar los desafíos que conlleva un panorama de sistemas de microservicios.

Desafíos con los microservicios

En la sección Desafíos con componentes de software autónomos , ya hemos visto algunos de los desafíos que pueden traer los componentes de software autónomos (y todos se aplican también a los microservicios), como se indica a continuación:

- Muchos componentes pequeños que utilizan comunicación sincrónica pueden provocar una cadena de fallos problema, especialmente bajo carga alta
- Mantener la configuración actualizada para muchos componentes pequeños puede ser un desafío
- Es difícil realizar un seguimiento de una solicitud que se está procesando e involucra muchos componentes, por ejemplo, cuando se realiza un análisis de causa raíz, donde cada componente almacena registros de forma local.
- Analizar el uso de los recursos de hardware a nivel de componentes también puede ser un desafío.
- La configuración y gestión manual de muchos componentes pequeños puede resultar costosa.
y propenso a errores

Otra desventaja (aunque no siempre obvia al principio) de descomponer una aplicación en un grupo de componentes autónomos es que forman un sistema distribuido. Los sistemas distribuidos son , por naturaleza, muy difíciles de gestionar. Esto se sabe desde hace muchos años (pero en muchos casos se pasó por alto hasta que se demostró lo contrario). Mi cita favorita para demostrarlo es la de Peter Deutsch, quien, en 1994, afirmó lo siguiente:



Las 8 falacias de la computación distribuida: Básicamente, todos, al crear una aplicación distribuida por primera vez, hacen las siguientes ocho suposiciones. Todas resultan ser falsas a largo plazo y todas causan grandes problemas y experiencias de aprendizaje difíciles:

1. La red es confiable
2. La latencia es cero
3. El ancho de banda es infinito
4. La red es segura
5. La topología no cambia
6. Hay un administrador



7. El coste del transporte es cero

8. La red es homogénea

(Peter Deutsch, 1994)

En general, crear microservicios basándose en estas suposiciones falsas conduce a soluciones que son propensas tanto a fallas temporales de la red como a problemas que ocurren en otras instancias de microservicios.

Cuando aumenta el número de microservicios en un entorno de sistemas, también aumenta la probabilidad de problemas.

Una buena regla general es diseñar la arquitectura de microservicios partiendo de la base de que siempre hay algún problema en el entorno de sistemas. La arquitectura de microservicios debe estar diseñada para gestionar esto, detectando problemas y reiniciando los componentes fallidos. Además, en el lado del cliente, asegúrese de que las solicitudes no se envíen a instancias de microservicio fallidas. Cuando se corrijan los problemas, se deben reanudar las solicitudes al microservicio que falló previamente; es decir, los clientes de microservicios deben ser resilientes. Todo esto, por supuesto, debe estar completamente automatizado. Con un gran número de microservicios, no es viable que los operadores lo gestionen manualmente.

El alcance de esto es amplio, pero nos limitaremos por ahora y avanzaremos para aprender sobre patrones de diseño para microservicios.

Patrones de diseño para microservicios

Este tema abordará el uso de patrones de diseño para mitigar los desafíos de los microservicios, como se describió en la sección anterior. Más adelante en este libro, veremos cómo implementar estos patrones de diseño con Spring Boot, Spring Cloud, Kubernetes e Istio.

El concepto de patrones de diseño es bastante antiguo; fue inventado por Christopher Alexander en 1977. En esencia, un patrón de diseño describe una solución reutilizable a un problema en un contexto específico. Usar una solución probada de un patrón de diseño puede ahorrar mucho tiempo y mejorar la calidad de la implementación, en comparación con dedicar tiempo a inventar.

La solución nosotros mismos.

Los patrones de diseño que cubriremos son los siguientes:

- Descubrimiento de servicios
- Servidor de borde
- Microservicios reactivos
- Configuración central
- Análisis de registros centralizado
- Rastreo distribuido
- Cortacircuitos
- Bucle de control
- Monitoreo centralizado y alarmas



Esta lista no pretende ser exhaustiva; en cambio, es una lista mínima de patrones de diseño necesarios para abordar los desafíos que describimos anteriormente.

Utilizaremos un enfoque ligero para describir patrones de diseño y nos centraremos en lo siguiente:

- El problema
- Una solución
- Requisitos para la solución

A lo largo de este libro, profundizaremos en la aplicación de estos patrones de diseño. El contexto de estos patrones de diseño es un entorno de sistemas de microservicios cooperativos donde estos se comunican entre sí mediante solicitudes síncronas (por ejemplo, HTTP) o mediante el envío de mensajes asíncronos (por ejemplo, mediante un intermediario de mensajes).

Descubrimiento de servicios

El patrón de descubrimiento de servicios tiene los siguientes problemas, soluciones y requisitos de solución.

Problema

¿Cómo pueden los clientes encontrar microservicios y sus instancias?

Las instancias de microservicios suelen recibir direcciones IP asignadas dinámicamente al iniciarse , por ejemplo, al ejecutarse en contenedores. Esto dificulta que un cliente realice una solicitud a un microservicio que, por ejemplo, exponga una API REST mediante HTTP. Considere el siguiente diagrama:

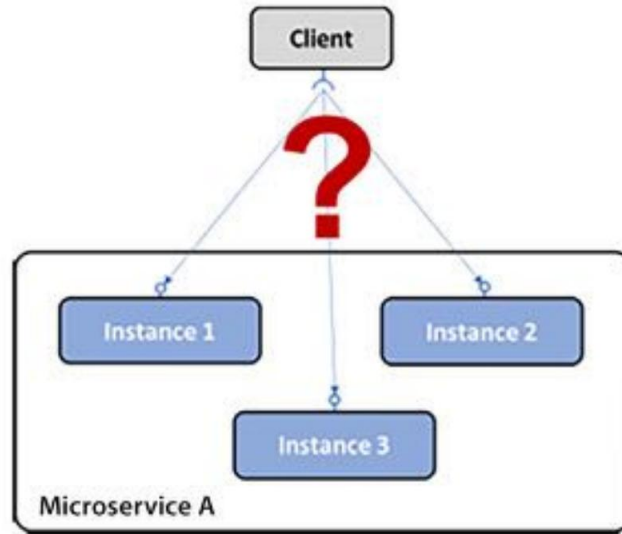


Figura 1.12: El problema del descubrimiento del servicio

Solución

Agregue un nuevo componente (un servicio de descubrimiento de servicios) al panorama del sistema, que realiza un seguimiento de los microservicios actualmente disponibles y las direcciones IP de sus instancias.

Requisitos de la solución

Algunos requisitos de la solución son los siguientes:

- Registrar o cancelar automáticamente el registro de microservicios y sus instancias a medida que entran y salen.
- El cliente debe poder realizar una solicitud a un punto final lógico para el microservicio. El
La solicitud se enrutará a una de las instancias de microservicio disponibles.
- Las solicitudes a un microservicio deben tener una carga equilibrada entre las instancias disponibles.
- Debemos ser capaces de detectar instancias que actualmente no están en buen estado para que las solicitudes se realicen.
no ser enrutado hacia ellos.

Notas de implementación: Como veremos en el Capítulo 9, Agregar descubrimiento de servicios mediante Netflix Eureka, Capítulo 15, Introducción a Kubernetes, y Capítulo 16, Implementación de nuestros microservicios en Kubernetes, este patrón de diseño se puede implementar utilizando dos estrategias diferentes:

- Enrutamiento del lado del cliente: el cliente utiliza una biblioteca que se comunica con el descubrimiento de servicios. servicio para averiguar las instancias adecuadas a las que enviar las solicitudes.

Enrutamiento del lado del servidor: La infraestructura del servicio de descubrimiento de servicios también expone un proxy inverso al que se envían todas las solicitudes. El proxy inverso reenvía las solicitudes a una instancia de microservicio adecuada en nombre del cliente.

Servidor de borde

El patrón de servidor de borde tiene el siguiente problema, solución y requisitos de solución.

Problema

En un entorno de sistemas de microservicios, en muchos casos es deseable exponer algunos de ellos al exterior y ocultar el resto del acceso externo. Los microservicios expuestos deben estar protegidos contra solicitudes de clientes maliciosos.

Solución

Agregue un nuevo componente, un servidor perimetral, al panorama del sistema por donde pasarán todas las solicitudes entrantes:

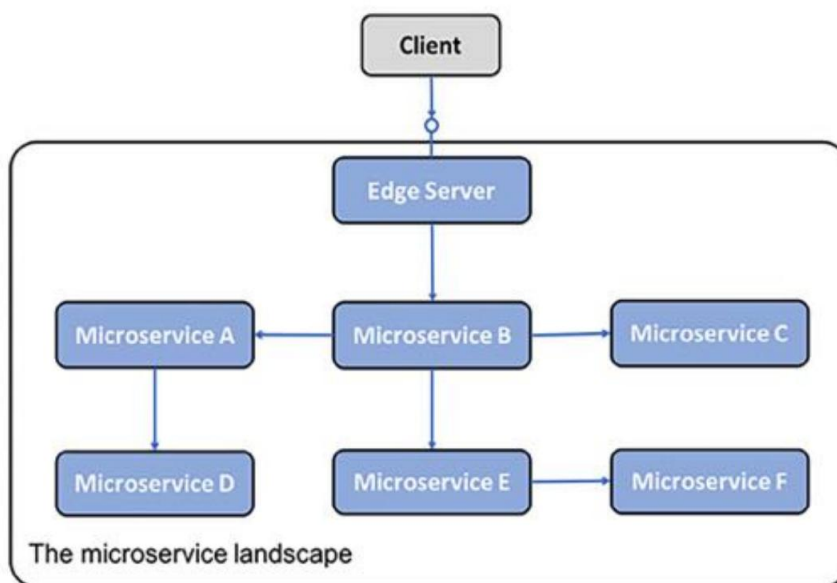


Figura 1.13: El patrón de diseño del servidor de borde

Notas de implementación: Un servidor perimetral normalmente se comporta como un proxy inverso y puede integrarse con un servicio de descubrimiento para proporcionar capacidades de equilibrio de carga dinámico.

Requisitos de la solución

Algunos requisitos de la solución son los siguientes:

- Ocultar los servicios internos que no deben exponerse fuera de su contexto; es decir, solo enrutar solicitudes a microservicios que estén configurados para permitir solicitudes externas
- Exponer servicios externos y protegerlos de solicitudes maliciosas; es decir, utilizar protocolos estándar y mejores prácticas como OAuth, OIDC, tokens JWT y claves API para garantizar que los clientes sean confiables.

Microservicios reactivos

El patrón de microservicios reactivos tiene los siguientes problemas, soluciones y requisitos de solución.

Problema

Tradicionalmente, como desarrolladores de Java, estamos acostumbrados a implementar la comunicación síncrona mediante E/S de bloqueo, por ejemplo, una API JSON RESTful sobre HTTP. El uso de E/S de bloqueo implica que se asigna un hilo del sistema operativo durante la duración de la solicitud. Si el número de solicitudes simultáneas aumenta, un servidor podría agotar los hilos disponibles en el sistema operativo, lo que causa problemas que van desde tiempos de respuesta más largos hasta fallos del servidor. El uso de una arquitectura de microservicios suele agravar este problema, ya que se utiliza una cadena de microservicios que cooperan para atender una solicitud. Cuantos más microservicios participen en la atención de una solicitud, cuanto más rápido se agotarán los hilos disponibles.

Solución

Utilice E/S sin bloqueo para garantizar que no se asignen subprocesos del sistema operativo mientras se espera que se produzca el procesamiento en otro servicio, es decir, una base de datos u otro microservicio, por ejemplo, utilizando un modelo de programación reactiva como Project Reactor o utilizando subprocesos virtuales.

Requisitos de la solución

Algunos requisitos de la solución son los siguientes:

- Siempre que sea posible, utilice un modelo de programación asíncrona, enviando mensajes sin esperando que el receptor los procese.
- Si se prefiere un modelo de programación síncrona, utilice marcos reactivos que puedan ejecutar solicitudes síncronas mediante E/S sin bloqueo, sin asignar un hilo mientras se espera una respuesta. Esto facilitará la escalabilidad de los microservicios para su gestión. una mayor carga de trabajo.

Los microservicios también deben diseñarse para ser resilientes y autocurativos. Resiliente significa ser capaz de generar una respuesta incluso si falla uno de los servicios de los que depende; autocurativo significa que, una vez que el servicio fallido vuelva a estar operativo, el microservicio debe poder reanudar su uso.

En 2013, se establecieron los principios clave para el diseño de sistemas reactivos en el Manifiesto Reactivo (<https://www.reactivemanifesto.org/>).



Según el manifiesto, la base de los sistemas reactivos es que se basan en mensajes y utilizan comunicación asíncrona. Esto les permite ser elásticos (es decir, escalables) y resilientes (es decir, tolerantes a fallos). La elasticidad y la resiliencia, en conjunto, permiten que un sistema reactivo responda siempre de forma oportuna.

Configuración central

El patrón de configuración central tiene el siguiente problema, solución y requisitos de solución.

Problema

Tradicionalmente, una aplicación se implementa junto con su configuración; por ejemplo, un conjunto de variables de entorno o archivos que contienen información de configuración. Dado un entorno de sistema basado en una arquitectura de microservicios, es decir, con un gran número de instancias de microservicios implementadas, surgen algunas preguntas:

- ¿Cómo puedo obtener una imagen completa de la configuración existente para todos los equipos en ejecución?
¿instancias de microservicio?
- ¿Cómo actualizo la configuración y me aseguro de que todos los microservicios afectados estén incluidos?
¿Las instancias se actualizan correctamente?

Solución

Agregue un nuevo componente, un servidor de configuración, al panorama del sistema para almacenar la configuración de todos los microservicios, como lo ilustra el siguiente diagrama:

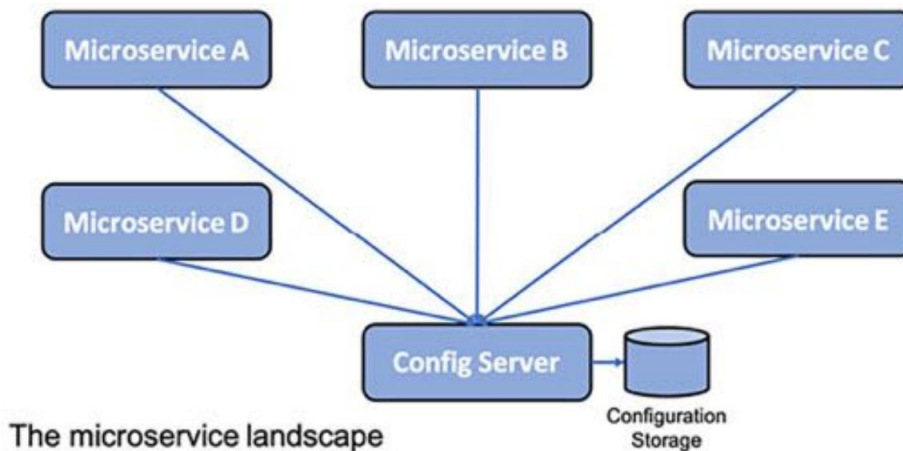


Figura 1.14: El patrón de diseño de configuración central

Requisitos de la solución

Posibilite almacenar información de configuración para un grupo de microservicios en un solo lugar, con diferentes configuraciones para distintos entornos (por ejemplo, desarrollo, prueba, control de calidad y producción).

Análisis de registros centralizado

El análisis de registros centralizado tiene los siguientes problemas, soluciones y requisitos de solución.

Problema

Tradicionalmente, una aplicación escribía eventos de registro en archivos de registro almacenados en el sistema de archivos local del servidor donde se ejecuta. Dado un entorno de sistema basado en una arquitectura de microservicios, es decir, con un gran número de instancias de microservicios implementadas en un gran número de servidores más pequeños, podemos plantearnos las siguientes preguntas:

- ¿Cómo puedo obtener una descripción general de lo que está sucediendo en el panorama del sistema cuando cada micro-
 - ¿La instancia de servicio escribe en su propio archivo de registro local?
- ¿Cómo puedo saber si alguna de las instancias de microservicio tiene problemas y comienza a escribir?
 - ¿Mensajes de error en sus archivos de registro?

- Si los usuarios finales empiezan a reportar problemas, ¿cómo puedo encontrar los mensajes de registro relacionados? Es decir, ¿cómo puedo identificar qué instancia del microservicio es la causa raíz del problema? El siguiente diagrama ilustra el problema:

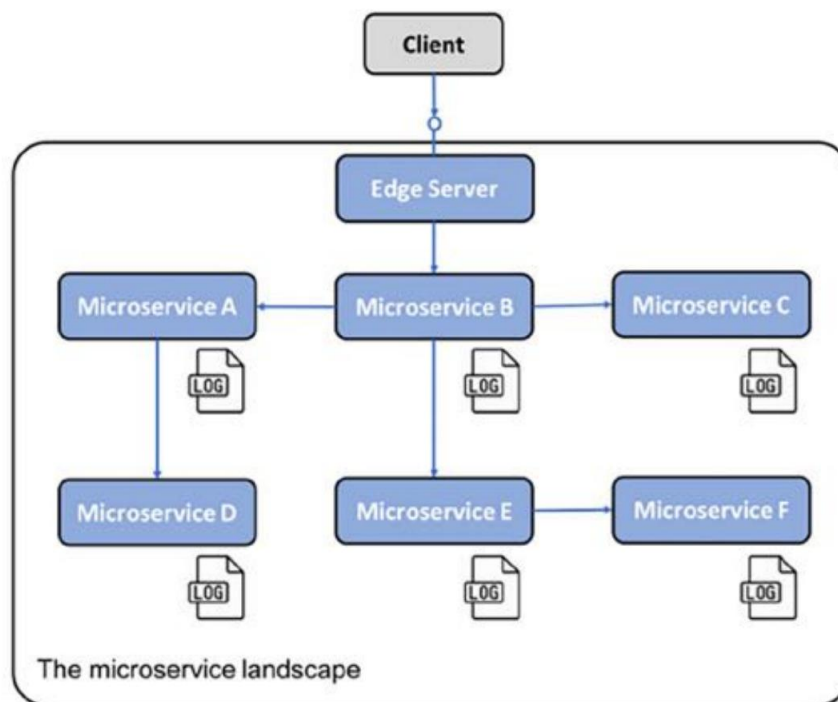


Figura 1.15: Los microservicios escriben archivos de registro en su sistema de archivos local

Solución

Agregue un nuevo componente que pueda administrar el registro centralizado y sea capaz de lo siguiente:

- Detectar nuevas instancias de microservicios y recopilar eventos de registro de ellas
- Interpretar y almacenar eventos de registro de forma estructurada y con capacidad de búsqueda en una base de datos central
- Proporcionar API y herramientas gráficas para consultar y analizar eventos de registro

Requisitos de la solución

Algunos requisitos de la solución son los siguientes:

Los microservicios transmiten los eventos de registro a la salida estándar del sistema (stdout). Esto facilita que un recopilador de registros los encuentre, a diferencia de cuando se escriben en archivos de registro específicos del microservicio.

- Los microservicios etiquetan los eventos de registro con el ID de correlación que se describe en la siguiente sección sobre el patrón de diseño de rastreo distribuido.

Se define un formato de registro canónico para que los recopiladores de registros puedan transformar los eventos de registro recopilados de los microservicios a un formato de registro canónico antes de que se almacenen en la base de datos central. El almacenamiento de eventos de registro en un formato de registro canónico es necesario para poder consultar y analizar los eventos de registro recopilados.

Rastreo distribuido

El rastreo distribuido tiene el siguiente problema, solución y requisitos de solución.

Problema

Debe ser posible rastrear las solicitudes y los mensajes que fluyen entre microservicios mientras se procesa una solicitud externa al panorama del sistema.

Algunos ejemplos de escenarios de falla son los siguientes:

- Si los usuarios finales comienzan a presentar casos de soporte con respecto a una falla específica, ¿cómo podemos identificar el microservicio que causó el problema, es decir, la causa raíz?
- Si un caso de soporte menciona problemas relacionados con una entidad específica, por ejemplo, un número de pedido específico, ¿cómo podemos encontrar mensajes de registro relacionados con el procesamiento de este pedido específico (por ejemplo, mensajes de registro de todos los microservicios que participaron en su procesamiento)?
- Si los usuarios finales comienzan a presentar casos de soporte debido a un tiempo de respuesta inaceptablemente largo, ¿cómo podemos identificar qué microservicio en una cadena de llamadas está causando la demora?

El siguiente diagrama lo representa:

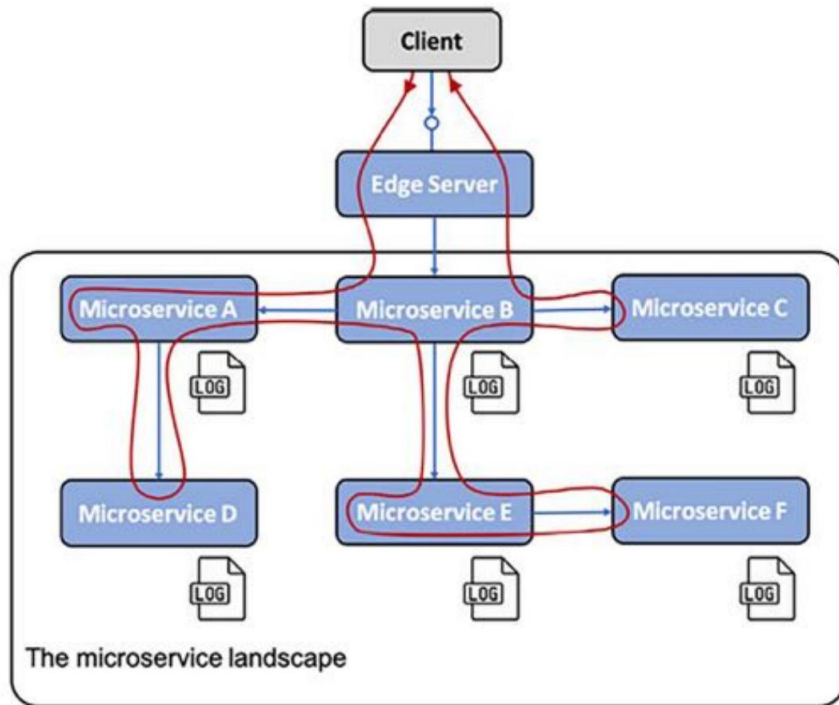


Figura 1.16: El problema del rastreo distribuido

Solución

Para rastrear el procesamiento entre microservicios que cooperan, debemos asegurarnos de que todas las solicitudes y mensajes relacionados estén marcados con un ID de correlación común y que este forme parte de todos los eventos de registro. Con base en un ID de correlación, podemos usar el servicio de registro centralizado para encontrar todos los eventos de registro relacionados. Si uno de los eventos de registro también incluye información sobre un identificador empresarial, por ejemplo, el ID de un cliente, producto o pedido, podemos encontrar todos los eventos de registro relacionados con ese identificador empresarial mediante el ID de correlación.

Para poder analizar los retrasos en una cadena de llamadas de microservicios que cooperan, debemos poder recopilar marcas de tiempo de cuándo las solicitudes, respuestas y mensajes ingresan y salen de cada microservicio.

Requisitos de la solución

Los requisitos de la solución son los siguientes:

- Asignar identificadores de correlación únicos a todas las solicitudes y eventos entrantes o nuevos de una manera conocida. lugar, como un encabezado con un nombre estandarizado
- Cuando un microservicio realiza una solicitud saliente o envía un mensaje, debe agregar el ID de correlación a la solicitud y al mensaje.
- Todos los eventos de registro deben incluir el ID de correlación en un formato predefinido para que el servicio de registro centralizado pueda extraer el ID de correlación del evento de registro y permitir su búsqueda.
- Se deben crear registros de seguimiento para cuando las solicitudes, las respuestas y los mensajes ingresan o salir de una instancia de microservicio

Cortacircuitos

El patrón del disyuntor tiene los siguientes problemas, soluciones y requisitos de solución.

Problema

Un entorno de sistemas de microservicios que utiliza intercomunicación síncrona puede estar expuesto a una cadena de fallos. Si un microservicio deja de responder, sus clientes también podrían experimentar problemas y dejar de responder a sus solicitudes. El problema puede propagarse recursivamente por todo el entorno de sistemas y afectar a partes importantes del mismo.

Esto es especialmente común en casos en los que se ejecutan solicitudes sincrónicas utilizando bloqueo de E/S, es decir, bloqueando un hilo del sistema operativo subyacente mientras se procesa una solicitud.

En combinación con un gran número de solicitudes simultáneas y un servicio que empieza a responder con una lentitud inesperada, los grupos de subprocesos pueden agotarse rápidamente, provocando que el usuario que realiza la llamada se cuelgue o se bloquee. Este fallo puede propagarse rápidamente y de forma desagradable al usuario que realiza la llamada, y así sucesivamente.

Solución

Agregue un disyuntor que evite nuevas solicitudes salientes de una persona que llama si detecta un problema con el servicio que llama.

Requisitos de la solución

Los requisitos de la solución son los siguientes:

- Abrir el circuito y fallar rápidamente (sin esperar un tiempo de espera) si hay problemas con el servicio se detectan.

- Sonda de corrección de fallos (también conocida como circuito semiabierto); es decir, permitir que una única solicitud pase de manera regular para ver si el servicio vuelve a funcionar con normalidad.
- Cierre el circuito si la sonda detecta que el servicio vuelve a funcionar con normalidad. Esta capacidad es fundamental, ya que hace que el sistema sea resistente a este tipo de problemas; en otras palabras, se autorrepara.

El siguiente diagrama ilustra un escenario donde toda la comunicación síncrona dentro del entorno del sistema de microservicios se realiza mediante interruptores automáticos. Todos los interruptores automáticos están cerrados y permiten el tráfico, excepto uno (para el microservicio E) que ha detectado problemas en el servicio al que se dirigen las solicitudes. Por lo tanto, este interruptor automático está abierto y utiliza lógica de fallo rápido; es decir, no llama al servicio que falla y espera a que se agote el tiempo de espera. En su lugar, el microservicio E puede devolver una respuesta inmediatamente, aplicando opcionalmente alguna lógica de respaldo antes de responder:

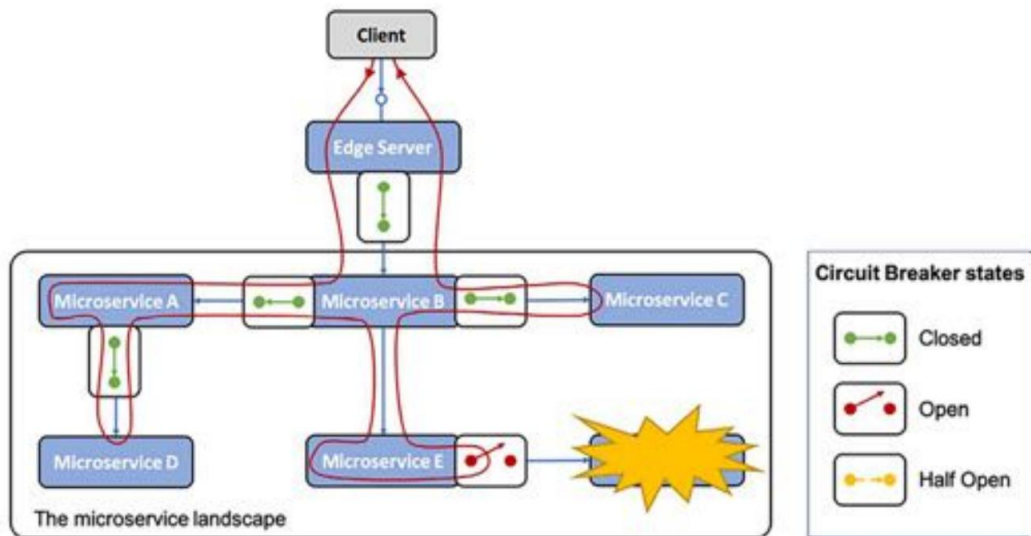


Figura 1.17: Patrón de diseño del disyuntor

Bucle de control

El patrón de bucle de control tiene el siguiente problema, solución y requisitos de solución.

Problema

En un panorama de sistemas con una gran cantidad de instancias de microservicios distribuidas en varios servidores, es muy difícil detectar y corregir manualmente problemas como fallas o bloqueos. instancias de microservicio.

Solución

Agregue un nuevo componente, un bucle de control, al panorama del sistema. Este proceso se ilustra como Sigue:

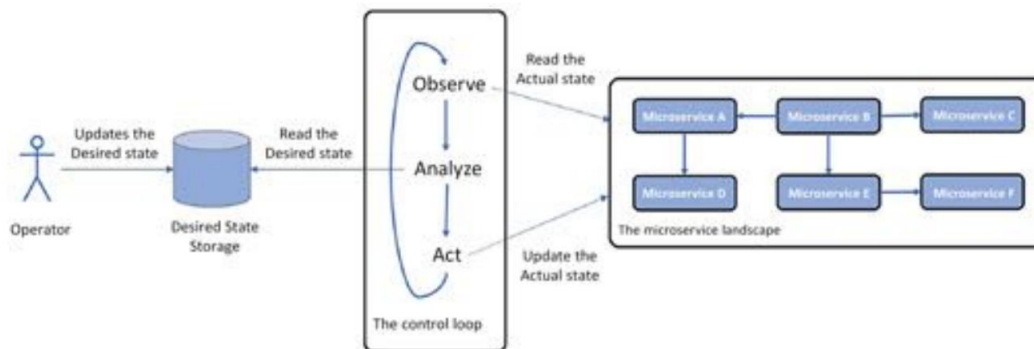


Figura 1.18: El patrón de diseño del bucle de control

Requisitos de la solución

El bucle de control observará constantemente el estado real del sistema, comparándolo con el estado deseado, según lo especificado por los operadores. Si ambos estados difieren, tomará medidas para que el estado real sea igual al deseado.

Notas de implementación: En el mundo de los contenedores, se suele utilizar un orquestador de contenedores como Kubernetes para implementar este patrón. Aprenderemos más sobre Kubernetes en el capítulo 15. Introducción a Kubernetes.

Monitoreo centralizado y alarmas

Para este patrón, tenemos el siguiente problema, solución y requisitos de solución.

Problema

Si los tiempos de respuesta observados o el uso de recursos de hardware se vuelven inaceptablemente altos, puede ser muy difícil descubrir la causa raíz del problema. Por ejemplo, necesitamos poder analizar el consumo de recursos de hardware por microservicio.

Solución

Para frenar esto, agregamos un nuevo componente, un servicio de monitorización, al panorama del sistema, que es capaz de recopilar métricas sobre el uso de recursos de hardware para cada nivel de instancia de microservicio.

Requisitos de la solución

Los requisitos de la solución son los siguientes:

- Debe poder recopilar métricas de todos los servidores que utiliza el panorama del sistema, lo que incluye servidores de escalado automático.
- Debe ser capaz de detectar nuevas instancias de microservicios a medida que se lanzan en los servidores disponibles y comenzar a recopilar métricas de ellas.
- Debe ser capaz de proporcionar API y herramientas gráficas para consultar y analizar la colección.
métricas seleccionadas
- Debe ser posible definir alertas que se activen cuando una métrica específica supere un valor de umbral específico.

La siguiente captura de pantalla muestra Grafana, que visualiza métricas de Prometheus, una herramienta de monitoreo que veremos en el Capítulo 20, Monitoreo de microservicios:

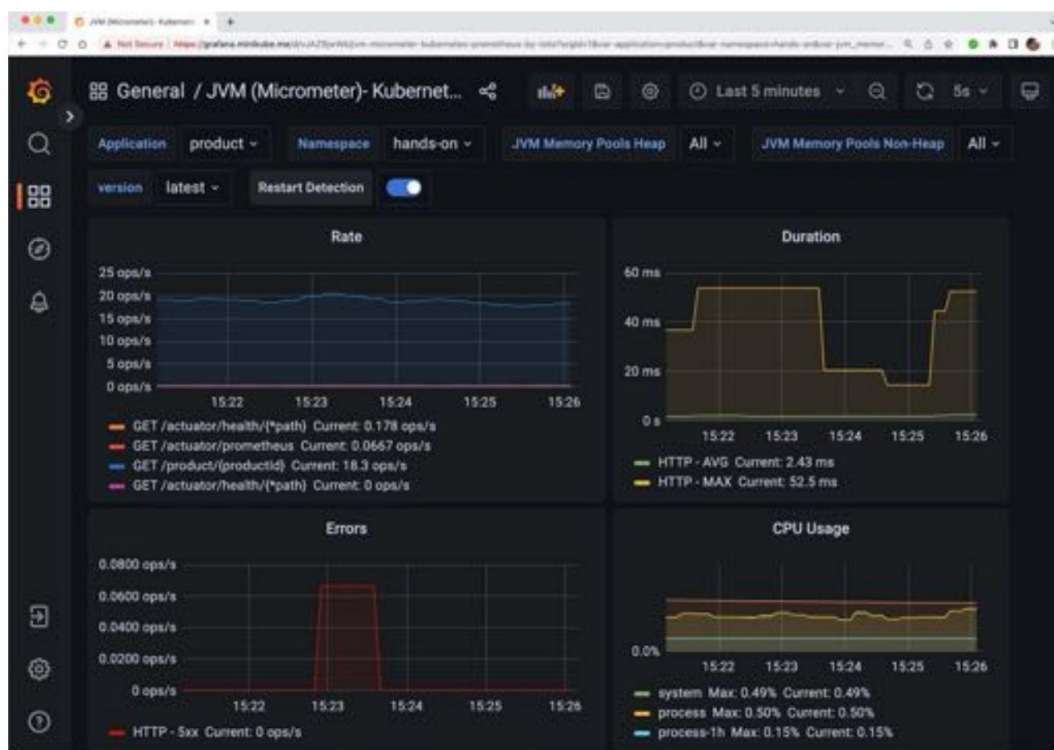


Figura 1.19: Monitoreo con Grafana

¡Qué lista tan extensa! Seguro que estos patrones de diseño te ayudaron a comprender mejor los desafíos de los microservicios. A continuación, aprenderemos sobre los facilitadores de software.

Habilitadores de software

Como ya hemos mencionado, tenemos una serie de muy buenas herramientas de código abierto que pueden ayudarnos a cumplir nuestras expectativas de microservicios y, lo más importante, abordar los nuevos desafíos. que vienen con ellos:

- Spring Boot, un marco de aplicación
- Spring Cloud/Netflix OSS, una combinación de marco de aplicación y servicios listos para usar
- Docker, una herramienta para ejecutar contenedores en un solo servidor
- Kubernetes, un orquestador de contenedores que administra un clúster de servidores que ejecutan contenedores
- Istio, una implementación de malla de servicios

La siguiente tabla muestra los patrones de diseño que necesitaremos para abordar estos desafíos, junto con la herramienta de código abierto correspondiente que se utilizará en este libro para implementar los patrones de diseño:

Patrón de diseño de bota de primavera	Nube de primavera	Kubernetes	Istio
Servicio descubrimiento	Netflix Eureka y Nube de primavera Balanceador de carga	Kubernetes kube-proxy y recursos de servicio	
Servidor de borde	Nube de primavera Puerta de enlace y Seguridad de primavera OAuth	Entrada de Kubernetes controlador	Entrada de Istio puerta
Reactivo microservicios	Proyecto Reactor y primavera WebFlux		
Central configuración	Configuración de primavera Servidor	Mapas de configuración de Kubernetes y secretos	

Análisis de registros centralizado			Elasticsearch, Fluentd, y Kibana. Nota: En realidad no es parte de Kubernetes, pero se puede implementar y configurar fácilmente junto con Kubernetes.	
Repartido rastreo	Micrómetro Rastreo y Zipkin			Jaeger
Cortacircuitos		Resiliencia4j		Parte aislada detección
Bucle de control			Controlador de Kubernetes gerentes	
Centralizado Monitoreo y alarmas				Kiali, Grafana, y Prometeo

Figura 1.17: Asignación de patrones de diseño a herramientas de código abierto

Tenga en cuenta que Spring Cloud, Kubernetes o Istio pueden usarse para implementar algunos patrones de diseño, como el descubrimiento de servicios, el servidor perimetral y la configuración central. Analizaremos las ventajas y desventajas de usar estas opciones más adelante en este libro.

Con los patrones de diseño y las herramientas que utilizaremos en el libro presentados, finalizaremos este capítulo repasando algunas áreas relacionadas que también son importantes, pero que no se cubren en este libro.

Otras consideraciones importantes

Para tener éxito al implementar una arquitectura de microservicios, hay varias áreas relacionadas que considerar. No las cubriré en este libro; en su lugar, las describiré brevemente.

Menciónelos aquí de la siguiente manera:

- **Importancia de DevOps:** Uno de los beneficios de una arquitectura de microservicios es que permite tiempos de entrega más cortos y, en casos extremos, permite la entrega continua de nuevas versiones.

Para poder entregar tan rápido, es necesario establecer una organización donde los equipos de desarrollo y operaciones colaboren bajo el lema « tú lo creas, tú lo gestionas». Esto significa que los desarrolladores ya no pueden simplemente pasar nuevas versiones del software al equipo de operaciones.

En cambio, los equipos de desarrollo y operaciones necesitan colaborar de forma mucho más estrecha, organizándose en equipos con plena responsabilidad sobre el ciclo de vida completo de un microservicio (o un grupo de microservicios relacionados). Además de la parte organizativa de desarrollo y operaciones, los equipos también necesitan automatizar la cadena de entrega, es decir, los pasos para crear, probar, empaquetar e implementar los microservicios en los distintos entornos de implementación. Esto se conoce como configurar un pipeline de entrega.

- Aspectos organizacionales y la ley de Conway: Otro aspecto interesante de cómo una arquitectura de microservicios podría afectar a la organización es la ley de Conway, que establece lo siguiente:



“Cualquier organización que diseñe un sistema (definido en sentido amplio) producirá un diseño cuya estructura es una copia de la estructura de comunicación de la organización”.

(Melvyn Conway, 1967)

Esto significa que el enfoque tradicional de organizar equipos de TI para aplicaciones grandes, basándose en su experiencia tecnológica (por ejemplo, equipos de UX, lógica de negocio y bases de datos), dará lugar a una gran aplicación de tres niveles: típicamente, una gran aplicación monolítica con una unidad desplegable independiente para la interfaz de usuario (IU), otra para procesar la lógica de negocio y otra para la gran base de datos. Para implementar con éxito una aplicación basada en una arquitectura de microservicios, la organización debe transformarse en equipos que trabajen con uno o varios microservicios relacionados. El equipo debe poseer las habilidades necesarias para dichos microservicios, por ejemplo, lenguajes y marcos para la lógica de negocio y tecnologías de bases de datos para la persistencia de los datos.

Descomponer una aplicación monolítica en microservicios: Una de las decisiones más difíciles (y costosas si se hace mal) es cómo descomponer una aplicación monolítica en un conjunto de microservicios cooperativos. Si esto se hace mal, se producirán problemas como los siguientes:

- Entrega lenta: los cambios en los requisitos comerciales afectarán a muchos microservicios, lo que resultará en trabajo adicional.
- Mal rendimiento: para poder realizar una función comercial específica, se deben pasar muchas solicitudes entre varios microservicios, lo que genera tiempos de respuesta prolongados.

- Datos inconsistentes: dado que los datos relacionados se separan en diferentes microservicios, pueden aparecer inconsistencias con el tiempo en los datos administrados por diferentes microservicios.

Un buen enfoque para definir los límites adecuados para los microservicios es aplicar el diseño orientado al dominio y su concepto de contextos delimitados. Según Eric Evans, un contexto delimitado es:



“Una descripción de un límite (normalmente un subsistema o el trabajo de un equipo en particular) dentro del cual se define y es aplicable un modelo particular”.

Esto significa que un microservicio definido por un contexto delimitado tendrá un contexto bien definido. modelo de sus propios datos.

- Importancia del diseño de API: si un grupo de microservicios expone una API común disponible externamente, es importante que la API sea fácil de entender y cumpla con las siguientes pautas:
 - Si se utiliza el mismo concepto en varias API, debe tener la misma descripción en términos de nombres y tipos de datos utilizados.
 - Es fundamental que las API puedan evolucionar de forma independiente, pero controlada. Esto suele requerir la aplicación de un esquema de control de versiones adecuado para las API, por ejemplo, <https://semver.org/>. Esto implica soportar múltiples versiones principales de una API durante un período de tiempo específico, permitiendo a los clientes de la API migrar a nuevas versiones principales a su propio ritmo.
- Rutas de migración desde las instalaciones locales a la nube: hoy en día, muchas empresas ejecutan sus cargas de trabajo en las instalaciones, pero están buscando formas de trasladar partes de su carga de trabajo a la nube. Dado que hoy en día la mayoría de los proveedores de nube ofrecen Kubernetes como servicio, un enfoque de migración atractivo puede ser primero mover la carga de trabajo a Kubernetes local (como microservicios o no) y luego volver a implementarla en una oferta de Kubernetes como servicio proporcionada por un proveedor de nube preferido.
- Buenos principios de diseño para microservicios: la aplicación de 12 factores: La aplicación de 12 factores (<https://12factor.net>) Es un conjunto de principios de diseño para desarrollar software que se puede implementar en la nube. La mayoría de estos principios de diseño son aplicables a la creación de microservicios, independientemente de dónde y cómo se implementen, es decir, en la nube o en las instalaciones. Algunos de estos principios se abordarán en este libro, como la configuración, los procesos y los registros, pero no todos.

¡Eso es todo por el primer capítulo! Espero que les haya dado una buena idea básica de los microservicios y los desafíos que conllevan, así como una visión general de lo que cubriremos en este libro.

Resumen

En este capítulo introductorio, describí mi propia experiencia con los microservicios y profundicé un poco en su historia. Definimos qué es un microservicio: un tipo de componente distribuido autónomo con requisitos específicos. También repasamos los aspectos positivos y negativos de la arquitectura basada en microservicios.

Para abordar estos desafíos, definimos un conjunto de patrones de diseño y mapeamos brevemente las capacidades de productos de código abierto como Spring Boot, Spring Cloud, Kubernetes e Istio con los patrones de diseño.

¿Estás deseando desarrollar tu primer microservicio? En el siguiente capítulo, te presentaremos Spring Boot y las herramientas complementarias de código abierto que usaremos para desarrollar nuestro primer microservicio.

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



2

Introducción a Spring Boot

En este capítulo, aprenderá a crear un conjunto de microservicios cooperativos con Spring Boot, centrándose en el desarrollo de funcionalidades que aporten valor al negocio. Los desafíos de los microservicios que mencionamos en el capítulo anterior se abordarán solo parcialmente, pero se abordarán en profundidad en capítulos posteriores.

Desarrollaremos microservicios que contengan lógica de negocio basada en beans Spring y expondremos API REST mediante Spring WebFlux. Las API se documentarán según la especificación OpenAPI mediante springdoc-openapi. Para que los datos procesados por los microservicios sean persistentes, utilizaremos Spring Data para almacenarlos en bases de datos SQL y NoSQL.

Desde el lanzamiento de Spring Boot v2.0 en marzo de 2018, es mucho más fácil desarrollar microservicios reactivos, incluyendo API REST síncronas no bloqueantes. Para desarrollar servicios asíncronos basados en mensajes, utilizaremos Spring Cloud Stream. Para más información, consulte el capítulo 1, Introducción a los microservicios, sección Microservicios reactivos.

Spring Boot 3.0 se lanzó en noviembre de 2022. Se basa en Spring Framework 6.0 y Jakarta EE 9, y es compatible con Jakarta EE 10. Se requiere Java 17, la versión actual de soporte a largo plazo (LTS), como versión mínima de Java. Se han publicado cinco versiones menores, 3.1 a 3.5, desde Spring Boot 3.0. Este libro se basa en Spring Boot 3.5, publicado en mayo de 2025.

Finalmente, usaremos Docker para ejecutar nuestros microservicios como contenedores. Esto nos permitirá iniciar y detener nuestro entorno de microservicios, incluyendo servidores de bases de datos y un agente de mensajes, con un solo comando.

Son muchas tecnologías y marcos, así que repasemos cada uno de ellos brevemente para ver de qué se tratan.

En este capítulo, presentaremos los siguientes proyectos de código abierto:

- Spring Boot: esta sección también incluye una descripción general de las novedades en las versiones 3.0 a 3.5 y cómo migrar aplicaciones v2
- WebFlux de primavera
- springdoc-openapi
- Datos de primavera
- Arroyo de nubes de primavera
- Docker



Se proporcionarán más detalles sobre cada producto en los próximos capítulos.

Requisitos técnicos

Este capítulo no contiene ningún código fuente que se pueda descargar ni requiere la instalación de ninguna herramienta.

Bota de primavera

Spring Boot, y el marco Spring en el que se basa Spring Boot, es un excelente marco para desarrollar microservicios en Java.

Cuando se lanzó Spring Framework v1.0 en 2004, uno de sus principales objetivos era abordar el complejo estándar J2EE (Java 2 Platform, Enterprise Edition) con sus infames y pesados descriptores de implementación. Spring Framework proporcionó un modelo de desarrollo mucho más ligero basado en el concepto de inyección de dependencias. Además, Spring Framework utilizaba archivos de configuración XML mucho más ligeros en comparación con los descriptores de implementación de J2EE.



Para empeorar las cosas aún más, con el estándar J2EE, los descriptores de implementación pesados en realidad venían en dos tipos:

- Descriptores de implementación estándar, que describen la configuración de forma estándar.
forma tradicional
- Descriptores de implementación específicos del proveedor, que asignan la configuración a las características específicas del proveedor en el servidor de aplicaciones del proveedor

En 2006, J2EE pasó a llamarse Java EE, abreviatura de Java Platform, Enterprise Edition. En 2017, Oracle presentó Java EE a la Fundación Eclipse. En febrero de 2018, Java EE pasó a llamarse Jakarta EE. El nuevo nombre, Jakarta EE, también afecta a los nombres de los paquetes Java definidos por el estándar, lo que obliga a los desarrolladores a renombrarlos al actualizar a Jakarta EE, como se describe en la sección " Migración de una aplicación Spring Boot 2" más adelante en este capítulo. Con el paso de los años, a medida que Spring Framework ganaba popularidad, su funcionalidad se expandió significativamente. Poco a poco, la carga de configurar una aplicación Spring utilizando el ya no tan ligero archivo de configuración XML se convirtió en un problema.

¡En 2014, se lanzó Spring Boot v1.0, que solucionó estos problemas!

Convención sobre configuración y archivos JAR pesados

Spring Boot se centra en el desarrollo rápido de aplicaciones Spring listas para producción, con una sólida base de conocimientos sobre cómo configurar tanto los módulos principales de Spring Framework como los productos de terceros, como las bibliotecas utilizadas para el registro o la conexión a bases de datos. Spring Boot logra esto aplicando una serie de convenciones por defecto, minimizando la necesidad de configuración. Siempre que sea necesario, cada convención puede sobrescribirse escribiendo una configuración caso por caso. Este patrón de diseño se conoce como convención sobre configuración y minimiza la necesidad de configuración inicial.



En mi opinión, la configuración, cuando es necesaria, se escribe mejor con Java y anotaciones . Los archivos de configuración basados en XML aún se pueden usar, aunque son considerablemente más pequeños que antes de la introducción de Spring Boot.

Además de priorizar la convención sobre la configuración, Spring Boot también prioriza un modelo de ejecución basado en un archivo JAR independiente, también conocido como archivo JAR FAT. Antes de Spring Boot, la forma más común de ejecutar una aplicación Spring era implementarla como un archivo WAR en un servidor web Java EE, como Apache Tomcat. Spring Boot aún admite la implementación de archivos WAR.



Un archivo JAR fat contiene no solo las clases y los archivos de recursos de la aplicación, sino también todos los archivos JAR de los que depende. Esto significa que el archivo JAR fat es el único archivo JAR necesario para ejecutar la aplicación; es decir, solo necesitamos transferir un archivo JAR al entorno donde queremos ejecutar la aplicación, en lugar de transferir el archivo JAR de la aplicación junto con todos los archivos JAR de los que depende.

Para iniciar un JAR fat no es necesario instalar por separado un servidor web Java EE, como Apache Tomcat. En cambio, se puede iniciar con un comando simple como `java -jar app.jar`, lo que lo convierte en la opción perfecta para ejecutarse en un contenedor Docker. Si la aplicación Spring Boot, por ejemplo, usa HTTP para exponer una API REST, también contendrá un servidor web integrado.

Ejemplos de código para configurar una aplicación Spring Boot

Para entender mejor qué significa esto, veamos algunos ejemplos de código fuente.



Aquí sólo veremos algunos pequeños fragmentos de código para señalar las características principales.
¡Para ver un ejemplo que funcione completamente, tendrás que esperar hasta el próximo capítulo!

La anotación mágica @SpringBootApplication

El mecanismo de autoconfiguración basado en convenciones se puede iniciar anotando la clase de aplicación (es decir, la clase que contiene el método principal estático) con `@SpringBootApplication` Anotación. El siguiente código lo muestra:

```
@SpringBootApplication
clase pública MiAplicacion {

    público estático void main(String[] args) {
        SpringApplication.run(MiAplicacion.clase, argumentos);
    }
}
```

💡 Consejo rápido: Mejora tu experiencia de programación con las funciones AI Code Explainer y Quick Copy . Abre este libro en el Packt Reader de última generación. Haz clic en el botón Copiar.

(1) para copiar rápidamente el código en su entorno de codificación, o haga clic en el botón Explicar

(2) para que el asistente de IA le explique un bloque de código.



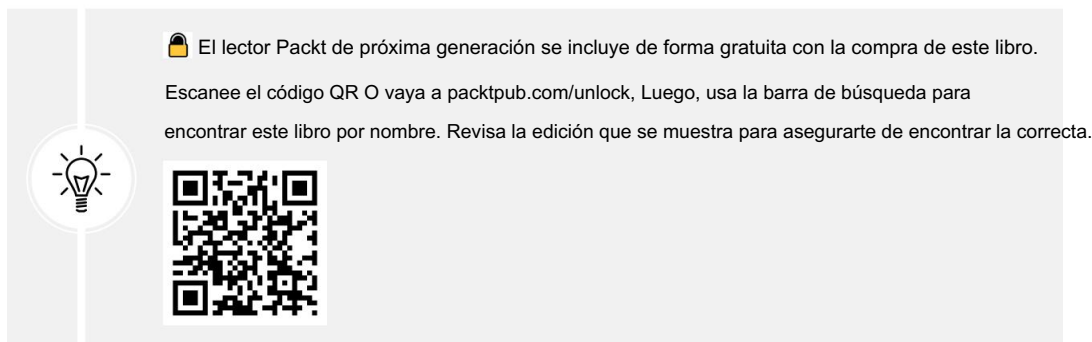
```
function calculate(a, b) {
    return {sum: a + b};
};
```

Copy

Explain

1

2



Esta anotación proporcionará la siguiente funcionalidad:

- Permite el escaneo de componentes, es decir, buscar componentes Spring y clases de configuración en el paquete de la clase de aplicación y todos sus subpaquetes.
- La clase de aplicación en sí se convierte en una clase de configuración.
- Permite la autoconfiguración, donde Spring Boot busca archivos JAR en la ruta de clases que pueda configurar automáticamente. Por ejemplo, si tiene Tomcat en la ruta de clases, Spring Boot lo configurará automáticamente como servidor web integrado.

Escaneo de componentes

Supongamos que tenemos el siguiente componente Spring en el paquete de la clase de aplicación (o en uno de sus subpaquetes):

```
@Component
clase pública MyComponentImpl implementa MyComponent { ...
```

Otro componente de la aplicación puede inyectar este componente automáticamente, también conocido como cableado automático, mediante la anotación `@Autowired` :

```
clase pública OtroComponente {

    privado final MiComponente miComponente;

    @Autowired
    público OtroComponente( MiComponente miComponente) {
        este.myComponent = myComponent;
    }
}
```

Si queremos utilizar componentes que están declarados en un paquete externo al paquete de la aplicación, por ejemplo, un componente de utilidad compartido por múltiples aplicaciones Spring Boot, podemos complementar el Anotación `@SpringBootApplication` en la clase de aplicación con una anotación `@ComponentScan` :

```
paquete se.magnus.myapp;

@SpringBootApplication
@ComponentScan({"se.magnus.myapp", "se.magnus.util" })
clase pública MiAplicacion {
```

Ahora podemos cablear automáticamente componentes del paquete `se.magnus.util` en el código de la aplicación, por ejemplo, un componente de utilidad llamado `MyUtility`, de la siguiente manera:

```
paquete se.magnus.util;

@Componente
clase pública MyUtility { ...
```

Este componente de utilidad se puede cablear automáticamente en un componente de aplicación de la siguiente manera:

```
paquete se.magnus.myapp.services;

clase pública OtroComponente {

    privado final MiUtilidad miUtilidad;

    @Autowired
    público OtroComponente( MiUtilidad miUtilidad) {
        esto.miUtilidad = miUtilidad;
    }
}
```



Si una clase solo define un constructor, no se requiere la anotación `@Autowired` .
En el código fuente del libro, no se utilizan anotaciones `@Autowired` en estos casos.

Configuración basada en Java

Si queremos anular la configuración predeterminada de Spring Boot, o queremos agregar nuestra propia configuración , podemos simplemente anotar una clase con `@Configuration` y será detectada por el mecanismo de escaneo de componentes que describimos anteriormente.

Por ejemplo, si queremos configurar un filtro en el procesamiento de solicitudes HTTP (manejado por Spring WebFlux, que se describe en la siguiente sección) que escribe un mensaje de registro al inicio y al final del procesamiento, podemos configurar un filtro de registro, de la siguiente manera:

```
@Configuración
clase pública AplicaciónSuscriptora {

    @Frijol
    público Filtro logFilter() {
        CommonsRequestLoggingFilter filtro = nuevo
            CommonsRequestLoggingFilter();
        filtro.setIncludeQueryString(verdadero);
        filtro.setIncludePayload(verdadero);
        filtro.setMaxPayloadLength(5120);
        filtro de retorno ;
    }
}
```



También podemos colocar la configuración directamente en la clase de aplicación ya que el @ La anotación SpringApplication implica la anotación @Configuration .

Eso es todo por ahora sobre Spring Boot, pero antes de pasar al siguiente componente, veamos qué hay de nuevo en las versiones de Spring Boot 3.x y cómo migrar una aplicación Spring Boot 2.

¿Qué novedades hay en Spring Boot 3.0 a 3.5?

Para los propósitos de este libro, las novedades más importantes de Spring Boot 3.0 son las siguientes:

- Observabilidad

Spring Boot 3.0 incluye un soporte mejorado para la observabilidad, con soporte integrado para el rastreo distribuido, que se suma al soporte ya existente para métricas y registro en versiones anteriores de Spring Boot. Este nuevo soporte para el rastreo distribuido se basa en una nueva API de Observabilidad en Spring Framework 6.0 y un nuevo módulo llamado Micrometer Tracing.

El rastreo micrométrico se basa en Spring Cloud Sleuth, que ya no se utiliza. El capítulo 14, «Comprender el rastreo distribuido», explica cómo usar la nueva compatibilidad con la observabilidad y el rastreo distribuido.

- **Compilación nativa**

Spring Boot 3.0 también permite compilar aplicaciones Spring Boot en imágenes nativas mediante GraalVM, que son archivos ejecutables independientes. Una aplicación Spring Boot compilada de forma nativa se inicia mucho más rápido y consume menos memoria. El capítulo 23, «Microservicios Java compilados de forma nativa», describe cómo compilar microservicios de forma nativa basados en Spring Boot.

- **Hilos virtuales y concurrencia estructurada**

Finalmente, Spring Boot 3.0 viene con soporte para subprocesos livianos llamados subprocesos virtuales. Del Proyecto Loom de OpenJDK. Se espera que los hilos virtuales simplifiquen el modelo de programación para el desarrollo de microservicios reactivos no bloqueantes, por ejemplo, en comparación con el modelo de programación utilizado en el Proyecto Reactor y varios componentes de Spring. Los hilos virtuales están disponibles desde Java 21. Para desarrollar microservicios que realicen actividades concurrentes, por ejemplo, agregar información de otros microservicios simultáneamente, también necesitamos compatibilidad con funciones de componibilidad. Se espera que esto sea compatible con la concurrencia estructurada, actualmente en versión preliminar; consulte <https://openjdk.org/jeps/8340343>. Dado que la concurrencia estructurada aún no está finalizada, este libro no abordará los hilos virtuales ni la concurrencia estructurada. Capítulo 7, Desarrollo de microservicios reactivos, Cubre la implementación de microservicios reactivos utilizando Project Reactor y Spring WebFlux.

En las versiones menores, Spring Boot 3.1–3.5, las siguientes características nuevas son las más interesantes:

- **Uso simplificado de Testcontainers**

Testcontainers simplifica las pruebas de integración automatizadas con las mismas bases de datos, intermediarios de mensajes y otros servicios utilizados en producción. Un desafío al configurar Testcontainers es determinar qué propiedades de Spring deben asignarse a sus propiedades. Spring Boot 3.1 gestiona esto automáticamente para muchos servicios mediante el nuevo concepto de Conexión de Servicio. Testcontainers se presentará en el Capítulo 6, "Añadiendo". Persistencia.

- **Uso simplificado de paquetes SSL**

Establecer una comunicación segura con SSL y TLS es inherentemente complejo. Esta complejidad se agrava aún más por el hecho de que diferentes bibliotecas, e incluso versiones principales de la misma, suelen requerir enfoques de configuración únicos. Para simplificar este proceso, Spring Boot 3.1 introduce los paquetes SSL, un nuevo concepto diseñado para optimizar y centralizar la configuración de SSL. Los paquetes SSL se presentarán en el Capítulo 11, "Securing". Acceso a APIs.

- Configuración automática del servidor de autorización Spring

Spring Boot 3.1 también admite la configuración automática del Servidor de Autorización de Spring, lo que simplifica la declaración de dependencias. El Servidor de Autorización de Spring se presentará en el Capítulo 11, "Asegurando el acceso a las API".

- Inicio más rápido usando CRaC

Desde Spring Boot 3.2, la Restauración Coordinada en el Punto de Control (CRaC) se puede usar para acortar el tiempo de inicio de una aplicación Spring Boot. Su concepto se basa en la idea de escribir la memoria del proceso Java de una aplicación Spring Boot iniciada en el disco durante una fase de entrenamiento. Posteriormente, la aplicación Spring Boot se puede reiniciar rápidamente cargando la memoria en un nuevo proceso desde el disco. Usar CRaC generalmente es más fácil de configurar que usar Native Compile con GraalVM. Desafortunadamente, no todas las bibliotecas utilizadas en este libro son compatibles con CRaC, por lo que no se cubrirá. Por ejemplo, el cliente Java de MongoDB aún no admite la suspensión y reanudación de MongoClient como requiere CRaC; consulte <https://jira.mongodb.org/browse/JAVA-5034> Para más detalles.

- Inicio más rápido usando AppCDS

Desde Spring Boot 3.3, AppCDS permite acortar el tiempo de inicio de una aplicación Spring Boot. AppCDS es una extensión de Class-Data Sharing (CDS), una función de la JVM que permite compartir metadatos de clase preprocesados entre múltiples instancias de la JVM. Inicialmente, CDS solo funcionaba con las clases principales de Java del JDK, pero AppCDS lo amplió para incluir clases de aplicación y bibliotecas de terceros. AppCDS es muy fácil de usar, pero actualmente, la mejora en los tiempos de inicio es relativamente modesta; como máximo, se pueden esperar tiempos de inicio el doble de rápidos. Dado que Native Compile con GraalVM puede ofrecer tiempos de inicio de 10 a 20 veces más rápidos, en este libro solo se tratará Native Compile; consulte el capítulo 23, Microservicios Java compilados de forma nativa.

- Registro estructurado

Spring Boot 3.4 admite el registro estructurado para facilitar la ingesta de la salida de registros de las aplicaciones Spring Boot en motores de búsqueda y análisis como Elasticsearch. Para obtener más información sobre cómo se recopila y analiza la salida de registros de las aplicaciones Spring Boot mediante la pila Elasticsearch, Fluentd y Kibana (EFK), consulte el Capítulo 19, Registro centralizado con la pila EFK.

- Soporte para Java 24

A partir de Spring Boot 3.4, se admite Java 24.

- Desuso del código

Varias clases y métodos quedaron obsoletos en Spring Boot 3.1–3.5 y se eliminarán en Spring Boot 4.0. Todas las clases y métodos marcados como obsoletos que se usaron en este libro se han reemplazado por las alternativas recomendadas. Esto garantiza que los ejemplos de código que se proporcionan en este libro sean compatibles con el futuro; es decir, deberían ejecutarse en Spring Boot 4.0 sin problemas.

Para obtener información completa sobre las novedades de cada versión 3.x, consulte <https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.{MINOR}-Release-Notes> . Reemplace {MINOR} en la URL con la versión secundaria que le interese, de la 0 a la 5.

Migración de una aplicación Spring Boot 2

Si ya tiene aplicaciones basadas en Spring Boot 2, le interesará saber qué se necesita para migrar a Spring Boot 3.0. Aquí tiene una lista de pasos a seguir:

- Pivotal recomienda primero actualizar las aplicaciones Spring Boot 2 a la última versión v2.7.x ya que su guía de migración asume que está en la versión v2.7.

Asegúrese de tener instalado Java 17 o posterior, tanto en su entorno de desarrollo como en el de ejecución. Si sus aplicaciones Spring Boot se implementan como contenedores Docker, debe asegurarse de que su empresa apruebe el uso de imágenes Docker basadas en Java 17 o posterior. nuevos lanzamientos.

- Elimine las llamadas a métodos obsoletos en Spring Boot 2.x. Todos los métodos obsoletos se eliminaron en Spring Boot 3.0, por lo que debe asegurarse de que su aplicación no los llame. Para ver exactamente dónde se realizan las llamadas en su aplicación, puede habilitar el indicador lint:deprecation en el compilador de Java usando lo siguiente (suponiendo que usa Gradle):

```
tareas.withType(JavaCompile) {  
    opciones.compilerArgs += ['-Xlint:deprecation']  
}
```

- Cambie el nombre de todas las importaciones de paquetes javax que ahora forman parte de Jakarta EE a jakarta.
- Para las bibliotecas que no son administradas por Spring, debes asegurarte de estar usando la versión siones que sean compatibles con Yakarta, es decir, que utilicen paquetes de Yakarta .
- Para conocer los cambios más importantes y otra información importante sobre la migración , lea lo siguiente: <https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0-Migration-Guide> y <https://docs.spring.io/spring-security/reference/migración/index.html>.

Asegúrese de contar con pruebas de caja negra integrales que verifiquen la funcionalidad de su aplicación. Ejecútelas antes y después de la migración para garantizar que la funcionalidad de la aplicación no se haya visto afectada por la migración.

Al migrar el código fuente de la edición anterior de este libro a Spring Boot 3.0, la parte más laboriosa fue determinar cómo gestionar los cambios importantes en la configuración de Spring Security; consulte el Capítulo 11, "Asegurando el acceso a las API", para obtener más información. Por ejemplo, fue necesario actualizar la siguiente configuración del servidor de autorización de la edición anterior:

```
@Frijol
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) lanza
Excepción {
    http
        .authorizeRequests(autorizarSolicitudes -> autorizarSolicitudes
            .antMatchers("/actuador/**").permitAll())
```

Esta configuración se ve así con Spring Boot 3.0:

```
@Frijol
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) lanza una excepción {

    http
        .authorizeHttpRequests(autorizarSolicitudes -> autorizarSolicitudes
            .requestMatchers("/actuador/**").permitAll())
```

El script de prueba de extremo a extremo, test-em-all.bash, que viene con cada capítulo resultó indispensable para verificar que la funcionalidad no se viera afectada después de la migración de cada capítulo.

Ahora que hemos aprendido sobre Spring Boot, hablemos de Spring WebFlux.

WebFlux de primavera

Spring Boot 3.5 se basa en Spring Framework 6.2, que incorpora soporte para el desarrollo de aplicaciones reactivas. Spring Framework utiliza Project Reactor como base para su compatibilidad reactiva y también incluye un nuevo framework web, Spring WebFlux, que facilita el desarrollo de clientes y servicios HTTP reactivos (es decir, no bloqueantes).

Spring WebFlux admite dos modelos de programación diferentes:

- Un estilo imperativo basado en anotaciones, similar al marco web ya existente, Spring Web MVC, pero con soporte para servicios reactivos
- Un nuevo modelo orientado a funciones basado en enrutadores y controladores

En este libro, utilizaremos el estilo imperativo basado en anotaciones para demostrar lo fácil que es trasladar servicios REST de Spring Web MVC a Spring WebFlux y luego comenzar a refactorizar los servicios para que se vuelvan completamente reactivos.

Spring WebFlux también proporciona un cliente HTTP totalmente reactivo, `WebClient`, como complemento al cliente `RestTemplate` existente .

Spring WebFlux admite la ejecución en un contenedor de servlets basado en la especificación Jakarta Servlet v5.0 o superior, como Apache Tomcat, pero también admite servidores web integrados reactivos no basados en servlets, como Netty (<https://netty.io/>).



La especificación Servlet es una especificación en la plataforma Java EE que estandariza cómo desarrollar aplicaciones Java que se comunican utilizando protocolos web como HTTP.

Ejemplos de código de configuración de un servicio REST

Antes de crear un servicio REST basado en Spring WebFlux, debemos agregar Spring WebFlux (y las dependencias que requiere) a la ruta de clases para que Spring Boot se detecte y configure durante el inicio. Spring Boot ofrece una gran cantidad de prácticas dependencias de inicio que incorporan una característica específica, junto con las dependencias que cada característica normalmente requiere. Por lo tanto, usemos la dependencia de inicio para Spring WebFlux y veamos cómo se ve un servicio REST simple.

Dependencias de inicio

En este libro, usaremos Gradle como herramienta de compilación, por lo que la dependencia de inicio de Spring WebFlux se añadirá al archivo `build.gradle` . Su aspecto es el siguiente:

```
implementación('org.springframework.boot:spring-boot-starter-webflux')
```



Quizás se pregunte por qué no especificamos un número de versión. Hablaremos de ello cuando veamos un ejemplo completo en el Capítulo 3, "Creación de un conjunto de componentes cooperantes". ¡Microservicios!

Al iniciar el microservicio, Spring Boot detectará Spring WebFlux en la ruta de clases y lo configurará, además de otras funciones, como el inicio de un servidor web integrado. Spring WebFlux usa Netty por defecto, como se puede ver en la salida del registro:

```
2025-03-09 15:23:43.592 INFO 17429 --- [ principal] osbweb.embedded.netty.  
NettyWebServer: Netty se inició en el puerto(s): 8080
```

Si queremos cambiar de Netty a Tomcat como nuestro servidor web integrado, podemos anular la configuración predeterminada excluyendo Netty de la dependencia de inicio y agregando la dependencia de inicio para Tomcat:

```
implementación('org.springframework.boot:spring-boot-starter-webflux')  
{  
  excluir grupo: 'org.springframework.boot', módulo: 'spring-boot-  
  reactor de arranque-netty'  
}  
implementación('org.springframework.boot:spring-boot-starter-tomcat')
```

Después de reiniciar el microservicio, podemos ver que Spring Boot eligió Tomcat en su lugar:

```
2025-03-09 18:23:44.182 INFO 17648 --- [ principal] osbwembdded.tomcat.  
TomcatWebServer: Tomcat inicializado con puerto(s): 8080 (http)
```

Archivos de propiedad

Como puede ver en los ejemplos anteriores, el servidor web se inicia usando el puerto 8080. Si desea cambiar el puerto, puede sobrescribir el valor predeterminado mediante un archivo de propiedades. Los archivos de propiedades de la aplicación Spring Boot pueden ser archivos .properties o YAML. Por defecto, se llaman application.properties y application.yml, respectivamente.

En este libro, usaremos archivos YAML para cambiar el puerto HTTP del servidor web integrado a, por ejemplo, 7001. De esta forma, evitamos colisiones de puertos con otros microservicios que se ejecutan en el mismo servidor. Para ello, podemos añadir la siguiente línea al archivo application.yml :

```
puerto del servidor: 7001
```



Cuando comencemos a desarrollar nuestros microservicios como contenedores en el Capítulo 4, Implementación de nuestros microservicios con Docker, las colisiones de puertos dejarán de ser un problema. Cada contenedor tiene su propio nombre de host y rango de puertos, por lo que todos los microservicios pueden usar, por ejemplo, el puerto 8080 sin colisionar entre sí.

Ejemplo RestController

Ahora, con Spring WebFlux y un servidor web integrado de nuestra elección, podemos escribir un Servicio REST de la misma manera que cuando se utiliza Spring MVC, es decir, como un RestController:

```
@RestController
clase pública MyRestService {

    @GetMapping(valor = "/mi-recurso", produce = "aplicación/json")
    Lista<Recurso> listaRecursos() {
        ...
    }
}
```

La anotación `@GetMapping` del método `listResources()` asigna el método Java a una API HTTP GET en la URL `host:8080/myResource`. El valor de retorno del tipo `List<Resource>` se convierte a JSON.

Ahora que hemos hablado de Spring WebFlux, veamos cómo podemos documentar las API que desarrollamos usando Spring WebFlux.

springdoc-openapi

Un aspecto fundamental del desarrollo de API, por ejemplo, de servicios RESTful, es cómo documentarlas para que sean fáciles de usar. La especificación Swagger de SmartBear Software es una de las formas más utilizadas para documentar servicios RESTful. Muchas de las principales pasarelas de API ofrecen soporte nativo para exponer la documentación de servicios RESTful mediante la especificación Swagger.

En 2015, SmartBear Software donó la especificación Swagger a la Fundación Linux bajo la Iniciativa OpenAPI y creó la Especificación OpenAPI. El nombre Swagger aún se utiliza para las herramientas proporcionadas por SmartBear Software.

springdoc-openapi es un proyecto de código abierto, independiente de Spring Framework, que permite crear documentación de API basada en OpenAPI en tiempo de ejecución. Esto se logra examinando la aplicación, por ejemplo, inspeccionando anotaciones basadas en WebFlux y Swagger.

Veremos ejemplos de código fuente completo en los próximos capítulos, pero por ahora, la siguiente captura de pantalla condensada (las partes eliminadas están marcadas con ...) de una documentación de API de muestra será suficiente:

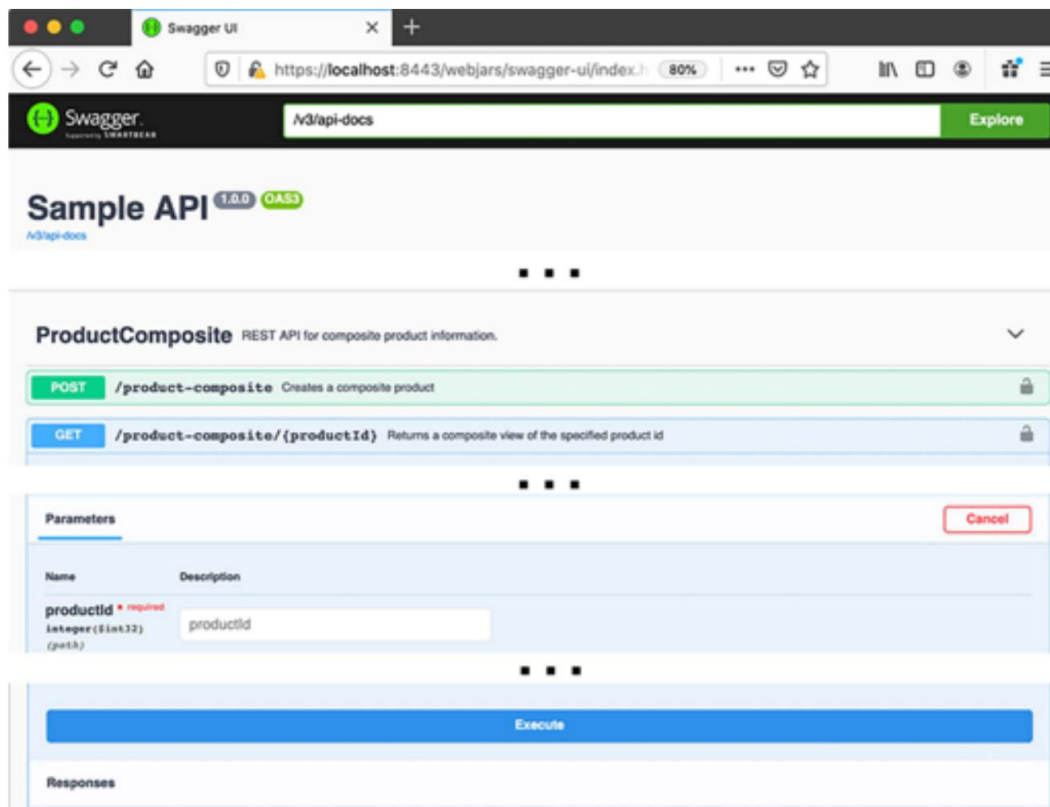


Figura 2.1: Documentación de API de muestra visualizada utilizando Swagger UI

Consejo rápido: ¿Necesitas ver una versión en alta resolución de esta imagen? Abre este libro. en el Packt Reader de próxima generación o vértelo en copia PDF/ePub.

El lector Packt de última generación y una copia gratuita en PDF/ePub de este libro se incluyen con la compra. Escanea el código QR o visita packtpub.com/unlock. Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.





¡Tenga en cuenta el gran botón Ejecutar , que puede usarse para probar la API, no solo para leer su documentación!

springdoc-openapi nos ayuda a documentar las API expuestas por nuestros microservicios. Ahora, pasemos a Spring Data.

Datos de primavera

Spring Data viene con un modelo de programación común para persistir datos en varios tipos de motores de bases de datos, que van desde bases de datos relacionales tradicionales (bases de datos SQL) hasta varios tipos de motores de bases de datos NoSQL, como bases de datos de documentos (por ejemplo, MongoDB), bases de datos clave-valor (por ejemplo, Redis) y bases de datos de gráficos (por ejemplo, Neo4j).

El proyecto Spring Data se divide en varios subproyectos y en este libro utilizaremos subproyectos de Spring Data para MongoDB y JPA que se han mapeado a una base de datos MySQL.



JPA significa API de Persistencia de Jakarta y es una especificación de Java sobre cómo gestionar datos relacionales. Visite <https://jakarta.ee/specifications/persistencia/> Para conocer las especificaciones más recientes. Jakarta EE 9 se basa en Jakarta Persistence 3.0.

Los dos conceptos centrales del modelo de programación en Spring Data son entidades y repositorios.

Las entidades y los repositorios generalizan cómo se almacenan y se accede a los datos desde los distintos tipos de bases de datos. Ofrecen una abstracción común, pero permiten añadir comportamiento específico de la base de datos a las entidades y repositorios. Estos dos conceptos fundamentales se explican brevemente, junto con algunos ejemplos de código ilustrativos, a medida que avanzamos en este capítulo. ¡Recuerde que se proporcionarán más detalles en los próximos capítulos!



Si bien Spring Data proporciona un modelo de programación común para diferentes tipos de bases de datos, esto no significa que podrá escribir código fuente portable.

Por ejemplo, cambiar la tecnología de base de datos de una base de datos SQL a una base de datos NoSQL, en general, no será posible sin algunos cambios en el código fuente.

Entidad:

Una entidad describe los datos que Spring Data almacenará. Las clases de entidad suelen estar anotadas con una combinación de anotaciones genéricas de Spring Data y anotaciones específicas de cada tecnología de base de datos.

Por ejemplo, una entidad que se almacenará en una base de datos relacional se puede anotar con anotaciones JPA como las siguientes:

```
import jakarta.persistence.Entity; import
jakarta.persistence.Id; import
jakarta.persistence.IdClass; import
jakarta.persistence.Table;

@Entity
@IdClass(ReviewEntityPK.class)
@Table(name = "review")
clase pública ReviewEntity { @Id
    int privado productId; @Id int
    privado reviewId; cadena privada
    autor; cadena privada
    asunto; cadena privada
    contenido;
```

Si una entidad se va a almacenar en una base de datos MongoDB, se pueden usar las anotaciones del subproyecto Spring Data MongoDB junto con las anotaciones genéricas de Spring Data. Por ejemplo, considere el siguiente código:

```
import org.springframework.data.annotation.Id; import
org.springframework.data.annotation.Version; import
org.springframework.data.mongodb.core.mapping.Document;

@Document
clase pública RecomendaciónEntidad {

    @Id
    cadena privada id;

    @Version
    int privado versión;
```



```

privado int productId;
int privado recomendaciónId;
cadena privada autor;
tasa int privada;
contenido de cadena privada ;

```

Las anotaciones `@Id` y `@Version` son anotaciones genéricas, mientras que la anotación `@Document` es específica del subproyecto Spring Data MongoDB.



Esto se puede revelar estudiando las declaraciones de importación ; las declaraciones de importación que contienen mongodb provienen del subproyecto Spring Data MongoDB.

Repositorios

Los repositorios se utilizan para almacenar y acceder a datos de diferentes tipos de bases de datos. En su forma más básica, un repositorio puede declararse como una interfaz Java, y Spring Data generará su implementación sobre la marcha utilizando convenciones establecidas. Estas convenciones pueden sobrescribirse y/o modificarse, o complementarse con configuración adicional y, si es necesario, código Java. Spring Data también incluye interfaces Java básicas, como `CrudRepository`, para simplificar aún más la definición de un repositorio. Esta interfaz básica proporciona métodos estándar para crear, leer, actualizar y eliminar operaciones.

Para especificar un repositorio para manejar la entidad JPA `ReviewEntity`, solo necesitamos declarar lo siguiente:

```

import org.springframework.data.repository.CrudRepository;

```

La interfaz pública `ReviewRepository` se extiende

```

CrudRepository<ReviewEntity, ReviewEntityPK> {

    Colección<ReviewEntity> findByProductId(int productId);
}

```

En este ejemplo, usamos una clase, `ReviewEntityPK`, para describir una clave primaria compuesta. Parece como sigue:

```

la clase pública ReviewEntityPK implementa Serializable {
    público int productId;
    público int reviewId;
}

```

También hemos añadido un método adicional, `findByProductId`, que permite buscar entidades de reseñas según el ID de producto, un campo que forma parte de la clave principal. El método sigue una convención de nomenclatura definida por Spring Data, lo que permite a Spring Data generar la implementación de este método sobre la marcha.

Si queremos utilizar el repositorio, simplemente podemos inyectarlo en el constructor y luego comenzar a usarlo, como en el siguiente ejemplo:

```
repositorio privado final ReviewRepository ;

public ReviewService(ReviewRepository repositorio) {
    este.repositorio = repositorio;
}

público void algúnMétodo() {
    repositorio.save(entidad);
    repositorio.delete(entidad);
    repositorio.findByProductId(productId);
}
```

Además de la interfaz `CrudRepository`, Spring Data también proporciona una interfaz base reactiva, `ReactiveCrudRepository`, que habilita repositorios reactivos. Los métodos de esta interfaz no devuelven objetos ni colecciones de objetos; en su lugar, devuelven objetos Mono y Flux. Mono Los objetos Flux son, como veremos en el Capítulo 7, Desarrollo de Microservicios Reactivos, flujos reactivos capaces de devolver 0...1 o 0...m entidades a medida que estén disponibles en el flujo. La interfaz reactiva solo puede ser utilizada por subproyectos de Spring Data que admitan controladores de bases de datos reactivos; es decir, que se basen en E/S sin bloqueo. El subproyecto Spring Data MongoDB admite repositorios reactivos, mientras que Spring Data JPA no.

La especificación de un repositorio reactivo para manejar la entidad MongoDB, `RecommendationEntity`, como se describió anteriormente, podría verse así:

```
import org.springframework.data.repository.reactive.
Repositorio ReactivoCrud;
import reactor.core.publisher.Flux;

La interfaz pública RecommendationRepository se extiende
RepositorioReactivoCrud<EntidadDeRecomendación, Cadena> {
    Flux<EntidadDeRecomendación> findByProductId(int IdProducto);
}
```

Con esto concluye la sección sobre Spring Data. Ahora, veamos cómo podemos usar Spring Cloud Stream para desarrollar servicios asincrónicos basados en mensajes.

Corriente de nubes de primavera

No nos centraremos en Spring Cloud en esta parte; lo haremos en la Parte 2 del libro, desde el Capítulo 8, Introducción a Spring Cloud, hasta el Capítulo 14, Comprensión del Rastreo Distribuido. Sin embargo, abordaremos uno de los módulos de Spring Cloud: Spring Cloud Stream. Spring Cloud Stream proporciona una abstracción de streaming sobre mensajería, basada en la publicación y suscripción.

Patrón de integración. Spring Cloud Stream actualmente incluye compatibilidad con Apache Kafka y RabbitMQ. Existen varios proyectos independientes que ofrecen integración con otros sistemas de mensajería populares. Consulte <https://github.com/spring-cloud?q=binder> Para más detalles.

Los conceptos centrales de Spring Cloud Stream son los siguientes:

- Mensaje: Una estructura de datos que se utiliza para describir los datos enviados y recibidos desde un mensaje.

sistema de envejecimiento.

- Editor: envía mensajes al sistema de mensajería, también conocido como proveedor.
- Suscriptor: Recibe mensajes del sistema de mensajería, también conocido como consumidor.

Destino: Se utiliza para comunicarse con el sistema de mensajería. Los publicadores utilizan destinos de salida y los suscriptores, destinos de entrada. Los destinos se asignan mediante los enlazadores específicos a las colas y temas del sistema de mensajería subyacente .

- Binder: Un binder proporciona la integración real con un sistema de mensajería específico, similar a lo que hace un controlador JDBC para un tipo específico de base de datos.

El sistema de mensajería que se utilizará se determina en tiempo de ejecución, según lo que se encuentre en la ruta de clases. Spring Cloud Stream incluye convenciones establecidas sobre cómo gestionar la mensajería. Estas convenciones se pueden anular especificando una configuración para funciones de mensajería como grupos de consumidores, particionamiento, persistencia, durabilidad y gestión de errores; por ejemplo, reintentos y gestión de colas de mensajes fallidos.

Ejemplos de código para enviar y recibir mensajes

Para entender mejor cómo encaja todo esto, veamos algunos ejemplos de código fuente.

Spring Cloud Stream viene con dos modelos de programación: un modelo más antiguo y actualmente obsoleto basado en el uso de anotaciones (por ejemplo, `@EnableBinding`, `@Output` y `@StreamListener`) y un modelo más reciente basado en la escritura de funciones. En este libro, utilizaremos implementaciones funcionales.

Para implementar un publicador, solo necesitamos implementar la interfaz funcional `java.util.function.Supplier` como un bean de Spring. Por ejemplo, el siguiente es un publicador que publica mensajes como una cadena:

```
@Frijol
Proveedor público <String> myPublisher() {
    retorna () -> nueva Fecha().toString();
}
```

Un suscriptor se implementa como un bean Spring que implementa `java.util.function.Consumer` Interfaz funcional. Por ejemplo, el siguiente es un suscriptor que consume mensajes como cadenas:

```
@Frijol
público Consumidor<Cadena> miSuscriptor() {
    devolver s -> System.out.println("ML RECIBIDO: " + s);
}
```

También es posible definir un bean Spring que procese mensajes, es decir, que los consuma y publique. Esto se puede lograr implementando `java.util.function.Function` interfaz funcional – por ejemplo, un bean Spring que consume mensajes entrantes y publica un nuevo mensaje después de algún procesamiento (ambos mensajes son cadenas en este ejemplo):

```
@Frijol
Función pública <Cadena, Cadena> miProcesador() {
    devolver s -> "ML PROCESADO: " + s;
}
```

Para que Spring Cloud Stream conozca estas funciones, debemos declararlas mediante spring. Propiedad de configuración `cloud.function.definition`. Por ejemplo, para las tres funciones definidas anteriormente, esto se vería así:

```
spring.cloud.función:
    definición: miEditor;miProcesador;miSuscriptor
```

Finalmente, necesitamos indicar a Spring Cloud Stream el destino que debe usar cada función. Para conectar las tres funciones de modo que el procesador consuma mensajes del publicador y el suscriptor los del procesador, podemos proporcionar la siguiente configuración:

```
spring.cloud.stream.bindings:
    myPublisher-out-0:
        destino: myProcessor-in
    miProcesador-en-0:
```

```

destino: myProcessor-in
miProcesador-salida-0:
destino: myProcessor-out
miSuscriptor-en-0:
destino: myProcessor-out

```

Esto dará como resultado el siguiente flujo de mensajes:

```
miEditor → miProcesador → miSuscriptor
```

Spring Cloud Stream activa un proveedor cada segundo de forma predeterminada, por lo que podríamos esperar un resultado como el siguiente si iniciamos una aplicación Spring Boot que incluya las funciones y la configuración descritas anteriormente:

```

ML RECIBIDO: ML PROCESADO: Mié 09 Mar 16:28:30 CET 2021
ML RECIBIDO: ML PROCESADO: Mié 09 Mar 16:28:31 CET 2021
ML RECIBIDO: ML PROCESADO: Mié 09 Mar 16:28:32 CET 2021
ML RECIBIDO: ML PROCESADO: Mié 09 Mar 16:28:33 CET 2021

```

En los casos en que el proveedor deba activarse mediante un evento externo en lugar de usar un temporizador, se puede usar la clase auxiliar `StreamBridge`. Por ejemplo, si se debe publicar un mensaje en el procesador al llamar a una API REST, `sampleCreateAPI`, el código podría ser similar al siguiente:

```

@Autowired
StreamBridge privado streamBridge;

@PostMapping
void sampleCreateAPI(@RequestBody String cuerpo) {
    streamBridge.send("miProcesador-en-0", cuerpo);
}

```

Ahora que entendemos las distintas API de Spring, aprendamos un poco sobre Docker y los contenedores en la siguiente sección.

Estibador

Supongo que Docker y el concepto de contenedores no necesitan una presentación detallada. Docker popularizó el concepto de contenedores como una alternativa ligera a las máquinas virtuales en 2013.

Un contenedor es un proceso en un host Linux que utiliza espacios de nombres de Linux para aislar diferentes contenedores, en cuanto al uso de recursos globales del sistema, como usuarios, procesos, sistemas de archivos y redes. Los grupos de control de Linux (también conocidos como cgroups) se utilizan para limitar la cantidad de CPU y memoria que un contenedor puede consumir.

En comparación con una máquina virtual que utiliza un hipervisor para ejecutar una copia completa del sistema operativo en cada máquina virtual, la sobrecarga en un contenedor es una fracción de la de una máquina virtual tradicional. Esto se traduce en tiempos de arranque mucho más rápidos y una sobrecarga significativamente menor en términos de uso de CPU y memoria.

Sin embargo, el aislamiento proporcionado para un contenedor no se considera tan seguro como el de una máquina virtual. Con el lanzamiento de Windows Server 2016, Microsoft admite el uso de Docker en servidores Windows.



En los últimos años, se ha desarrollado una forma ligera de máquinas virtuales. Esta combina lo mejor de las máquinas virtuales tradicionales y los contenedores, proporcionando máquinas virtuales con un espacio de almacenamiento y un tiempo de inicio similares a los de los contenedores, y con el mismo nivel de aislamiento seguro que ofrecen las máquinas virtuales tradicionales. Algunos ejemplos son Amazon Firecracker y el Subsistema de Microsoft Windows para Linux v2 (WSL2). Para más información, consulte <https://firecracker-microvm.github.io/> y <https://docs.microsoft.com/es-es/windows/wsl/>.

Los contenedores son muy útiles tanto durante el desarrollo como durante las pruebas. Poder iniciar un entorno de sistema completo de microservicios y administradores de recursos cooperativos (por ejemplo, servidores de bases de datos, intermediarios de mensajería, etc.) con un solo comando para realizar pruebas es simplemente asombroso.

Por ejemplo, podemos escribir scripts para automatizar las pruebas integrales de nuestro entorno de microservicios. Un script de prueba puede iniciar el entorno de microservicios, ejecutar pruebas utilizando las API expuestas y desmantelarlo. Este tipo de script de prueba automatizado es muy útil, tanto para ejecutarse localmente en el equipo del desarrollador antes de subir el código a un repositorio de código fuente, como para ejecutarse como un paso en una canalización de entrega. Un servidor de compilación puede ejecutar este tipo de pruebas en su proceso de integración e implementación continuas cada vez que un desarrollador sube código al repositorio de código fuente.



Para uso en producción, necesitamos un orquestador de contenedores como Kubernetes. Volveremos a hablar de los orquestadores de contenedores y Kubernetes más adelante en este libro.

Para la mayoría de los microservicios que veremos en este libro, un Dockerfile como el siguiente es todo lo que se requiere para ejecutar el microservicio como un contenedor Docker:

DESDE `openjdk:24`

MANTENEDOR `Magnus Larsson` <magnus.larsson.ml@gmail.com>

EXPONER 8080

AGREGAR `./build/libs/*.jar app.jar`

PUNTO DE ENTRADA `["java", "-jar", "/app.jar"]`

Si queremos iniciar y detener varios contenedores con un solo comando, Docker Compose es la herramienta perfecta. Docker Compose utiliza un archivo YAML para describir los contenedores que se van a gestionar. Para nuestros microservicios, podría ser similar a lo siguiente:

```
producto:
  construir: microservicios/producto-servicio

recomendación:
  compilación: microservicios/servicio de recomendación

revisar:
  compilación: microservicios/servicio de revisión

compuesto:
  compilación: microservicios/servicio-compuesto-de-producto

puertos:
  - "8080:8080"
```

Permítanme explicar un poco el código fuente anterior:

La directiva de compilación se utiliza para especificar qué Dockerfile se usará para cada microservicio.

Docker Compose la usará para crear una imagen de Docker y, a continuación, lanzar un contenedor basado en ella.

La directiva de puertos del servicio compuesto se utiliza para exponer el puerto 8080 en el servidor donde se ejecuta

Docker. En el equipo de un desarrollador, esto significa que se puede acceder al puerto del servicio compuesto simplemente usando localhost:8080.

Todos los contenedores en los archivos YAML se pueden administrar con comandos simples como los siguientes:

- `docker compose up -d`: inicia todos los contenedores. `-d` significa que los contenedores se ejecutan en el fondo, no bloqueando la terminal desde donde se ejecutó el comando.
- `docker compose down`: detiene y elimina todos los contenedores.
- `docker compose logs -f --tail=0`: imprime mensajes de registro de todos los contenedores. `-f` significa que el comando no se completará y, en su lugar, esperará nuevos mensajes de registro. `--tail=0` significa que no queremos ver ningún mensaje de registro anterior, solo los nuevos.

Para obtener una lista completa de los comandos de Docker Compose, consulte <https://docs.docker.com/compose/reference/>.

Esta fue una breve introducción a Docker. Profundizaremos en su funcionamiento a partir del Capítulo 4, "Implementación de nuestros microservicios con Docker".

Resumen

En este capítulo, presentamos Spring Boot y herramientas complementarias de código abierto que se pueden utilizar para crear microservicios cooperativos.

Spring Boot se utiliza para simplificar el desarrollo de aplicaciones basadas en Spring y listas para producción, como los microservicios. Tiene una postura firme respecto a cómo configurar tanto los módulos principales de Spring Framework como las herramientas de terceros. Con Spring WebFlux, podemos desarrollar microservicios que exponen servicios REST reactivos, es decir, no bloqueantes. Para documentar estos servicios REST, podemos usar springdoc-openapi para crear documentación basada en OpenAPI para las API. Si necesitamos persistir los datos utilizados por los microservicios, podemos usar Spring Data, que proporciona una elegante abstracción para acceder y manipular datos persistentes mediante entidades y repositorios. El modelo de programación de Spring Data es similar, pero no es totalmente portable entre diferentes tipos de bases de datos, por ejemplo, bases de datos relacionales, de documentos, de clave-valor y de grafos.

Si preferimos enviar mensajes de forma asíncrona entre nuestros microservicios, podemos usar Spring Cloud Stream, que ofrece una abstracción de streaming en lugar de mensajería. Spring Cloud Stream incluye compatibilidad inmediata con Apache Kafka y RabbitMQ, pero puede ampliarse para admitir otros agentes de mensajería mediante enlazadores personalizados. Finalmente, Docker facilita el uso de contenedores como una alternativa ligera a las máquinas virtuales. Basados en espacios de nombres y grupos de control de Linux, los contenedores proporcionan un aislamiento similar al de las máquinas virtuales tradicionales, pero con una sobrecarga significativamente menor en términos de uso de CPU y memoria.

En el próximo capítulo, daremos nuestros primeros pequeños pasos, creando microservicios con funcionalidad minimalista utilizando Spring Boot y Spring WebFlux.

Preguntas

1. ¿Cuál es el propósito de la anotación `@SpringBootApplication` ?
2. ¿Cuáles son las principales diferencias entre el componente Spring más antiguo para desarrollar REST? servicios, Spring Web MVC y el nuevo Spring WebFlux?
3. ¿Cómo ayuda springdoc-openapi a un desarrollador a documentar las API REST?
4. ¿Cuál es la función de un repositorio en Spring Data y cuál es la forma más simple posible?
¿Implementación de un repositorio?

5. ¿Cuál es el propósito de un enlazador en Spring Cloud Stream?
6. ¿Cuál es el propósito de Docker Compose?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



3

Creación de un conjunto de cooperación Microservicios

En este capítulo, crearemos nuestros primeros microservicios. Aprenderemos a crear microservicios cooperativos con funcionalidad minimalista. En próximos capítulos, añadiremos más funcionalidad a estos microservicios. Al final de este capítulo, tendremos una API RESTful expuesta por un microservicio compuesto. Este microservicio compuesto llamará a otros tres microservicios usando sus API RESTful para crear una respuesta agregada.

En este capítulo se tratarán los siguientes temas:

- Introducción al panorama de microservicios
- Generación de microservicios esqueléticos
- Agregar API RESTful
- Agregar un microservicio compuesto
- Agregar gestión de errores
- Probar las API manualmente
- Agregar pruebas automatizadas de microservicios de forma aislada
- Agregar pruebas semiautomatizadas a un entorno de microservicios

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS

- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter03`.

Con las herramientas y el código fuente en su lugar, podemos comenzar a aprender sobre el panorama del sistema de microservicios que crearemos en este capítulo.

Presentamos el panorama de microservicios

En el Capítulo 1, Introducción a los microservicios, presentamos brevemente el panorama del sistema basado en microservicios que utilizaremos a lo largo de este libro:

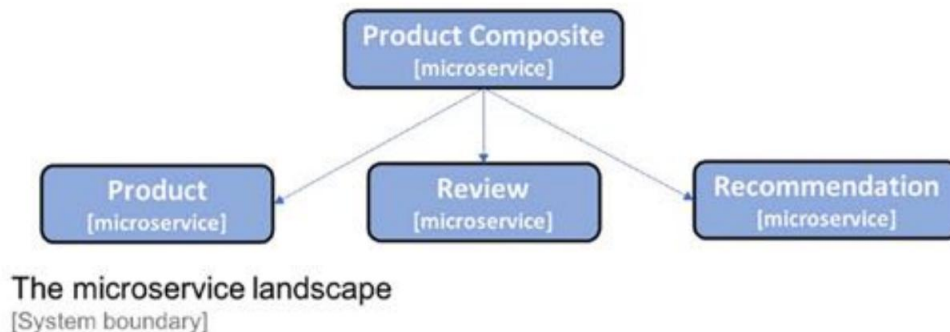


Figura 3.1: El panorama de microservicios

Consta de tres microservicios principales: los servicios de Producto, Revisión y Recomendación, todos ellos relacionados con un tipo de recurso, y un microservicio compuesto denominado servicio compuesto de Producto, que agrega información de los tres servicios principales.

Información manejada por los microservicios

Para facilitar la comprensión de los ejemplos de código fuente de este libro, se minimiza la lógica de negocio.

El modelo de información de los objetos de negocio que procesan se mantiene mínimo por la misma razón. En esta sección, analizaremos la información que gestiona cada microservicio, incluyendo la información relacionada con la infraestructura.

El servicio del producto

El servicio de producto administra la información del producto y describe cada producto con los siguientes atributos:

- Identificación del producto
- Nombre
- Peso

El servicio de revisión

El servicio de revisión administra las revisiones de productos y almacena la siguiente información sobre cada uno revisar:

- Identificación del producto
- Identificación de la revisión
- Autor
- Sujeto
- Contenido

El servicio de recomendaciones

El servicio de recomendaciones gestiona las recomendaciones de productos y almacena la siguiente información: Información sobre cada recomendación:

- Identificación del producto
- ID de recomendación
- Autor
- Tasa
- Contenido

El servicio compuesto de productos

El servicio compuesto de producto agrega información de los tres servicios principales y presenta información sobre un producto de la siguiente manera:

- Información del producto, tal como se describe en el servicio del producto.
- Una lista de reseñas de productos para el producto especificado, tal como se describe en el servicio de reseñas
- Una lista de recomendaciones de productos para el producto especificado, tal como se describe en la recomendación.
servicio de recomendación

Información relacionada con la infraestructura

Una vez que empecemos a ejecutar nuestros microservicios como contenedores administrados por la infraestructura (primero Docker y posteriormente Kubernetes), será interesante rastrear qué contenedores respondieron realmente a nuestras solicitudes. Como solución sencilla, se ha añadido un atributo `serviceAddress` a todas las respuestas, con el formato `nombre de host/dirección IP:puerto`.



En el Capítulo 18, Uso de una malla de servicios para mejorar la observabilidad y la administración, y el Capítulo 19, Registro centralizado con la pila EFK, aprenderemos acerca de soluciones más poderosas para rastrear las solicitudes que procesan los microservicios.

Reemplazo temporal del descubrimiento de servicios

Dado que en esta etapa no contamos con ningún mecanismo de descubrimiento de servicios, ejecutaremos todos los microservicios en el host local y usaremos números de puerto predefinidos para cada uno. Usaremos los siguientes puertos:

- El servicio compuesto del producto: 7001
- El servicio del producto: 7002
- El servicio de recomendaciones: 7003
- El servicio de revisión: 7004

¡Nos desharemos de los puertos codificados más tarde cuando comencemos a usar Docker y Kubernetes!

En esta sección, presentamos los microservicios que vamos a crear y la información que manejarán. En la siguiente sección, usaremos Spring Initialize para crear el código esqueleto. los microservicios.

Generando microservicios esqueléticos

Ahora es el momento de ver cómo podemos crear proyectos para nuestros microservicios. El resultado final de este tema se encuentra en la carpeta `$BOOK_HOME/Chapter03/1-spring-init`. Para simplificar la configuración de los proyectos, usaremos Spring Initializr para generar un proyecto esqueleto para cada microservicio. Un proyecto esqueleto contiene los archivos necesarios para compilarlo, junto con un archivo principal vacío.

Clase y clase de prueba para el microservicio. Después, veremos cómo podemos compilar todos nuestros microservicios con un solo comando mediante compilaciones multiproyecto en Gradle, la herramienta de compilación que usaremos.

Uso de Spring Initializr para generar código esqueleto

Para comenzar a desarrollar nuestros microservicios, usaremos una herramienta llamada Spring Initializr para generar el código esqueleto. Spring Initializr es proporcionada por el equipo de Spring y permite configurar y generar nuevas aplicaciones Spring Boot. Esta herramienta ayuda a los desarrolladores a seleccionar módulos Spring adicionales para la aplicación y garantiza que las dependencias estén configuradas para usar versiones compatibles de los módulos seleccionados. La herramienta admite el uso de Maven o Gradle como sistema de compilación y puede generar código fuente para Java, Kotlin o Groovy.

Se puede invocar desde un navegador web utilizando la URL <https://start.spring.io/> o usando una herramienta de línea de comandos, Spring Init. Para facilitar la reproducción de la creación de los microservicios,

Utilizaremos la herramienta de línea de comandos.

Para cada microservicio, crearemos un proyecto Spring Boot que hará lo siguiente:

- Utiliza Gradle como herramienta de construcción
- Utiliza Java 24 para compilar y ejecutar el microservicio.
- Empaqueta el proyecto como un archivo JAR pesado
- Incorpora dependencias para los módulos Actuator y WebFlux Spring
- Se basa en Spring Boot v3.5.0 (que depende de Spring Framework v6.2.6)



Spring Boot Actuator habilita varios puntos finales valiosos para la gestión y la monitorización. Los veremos en acción más adelante. Aquí se utilizará Spring WebFlux . para crear nuestras API RESTful.

Para crear el código esqueleto para nuestros microservicios, necesitamos ejecutar el siguiente comando para producto-servicio:

```
inicio de primavera \  
--versión de arranque=3.5.0 \  
--type=proyecto-gradle \  
--java-version=24 \  
--packaging=jar \  
--nombre=producto-servicio\  
--nombre-del-paquete=se.magnus.microservices.core.product \  
--groupId=se.magnus.microservices.core.product \  
--dependencias=actuador,webflux \  
--versión=1.0.0-INSTANTÁNEA \  
producto-servicio
```



Si desea obtener más información sobre la CLI de Spring Init , puede ejecutar el comando `spring help init` . Para ver qué dependencias puede agregar, ejecute `spring init`. Comando `--list` .

Si desea crear los cuatro proyectos por su cuenta en lugar de usar el código fuente en el repositorio de GitHub de este libro, pruebe `$BOOK_HOME/Chapter03/1-spring-init/create-projects.bash`, como se muestra a continuación.

Sigue:

```
mkdir alguna carpeta temporal
cd alguna-carpeta-temporal
$BOOK_HOME/Capítulo03/1-spring-init/crear-proyectos.bash
```

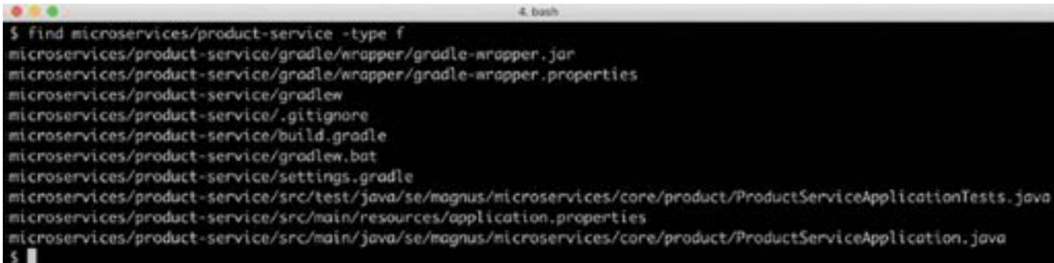
Después de crear nuestros cuatro proyectos usando `create-projects.bash`, tendremos el siguiente archivo estructura:

```
microservicios/
├── producto-compuesto-servicio
├── producto-servicio
├── servicio de recomendaciones
└── servicio de revisión
```

Para cada proyecto, podemos listar los archivos creados. Hagamos lo mismo para el proyecto de `producto-servicio` :

```
buscar microservicios/producto-servicio-tipo f
```

Recibiremos el siguiente resultado:



```
$ find microservicios/product-service -type f
microservicios/product-service/gradle/wrapper/gradle-wrapper.jar
microservicios/product-service/gradle/wrapper/gradle-wrapper.properties
microservicios/product-service/gradlew
microservicios/product-service/.gitignore
microservicios/product-service/build.gradle
microservicios/product-service/gradlew.bat
microservicios/product-service/settings.gradle
microservicios/product-service/src/test/java/se/magnus/microservices/core/product/ProductServiceApplicationTests.java
microservicios/product-service/src/main/resources/application.properties
microservicios/product-service/src/main/java/se/magnus/microservices/core/product/ProductServiceApplication.java
$
```

Figura 3.2: Listado de los archivos que creamos para el `producto-servicio`

Spring Initializr creó una serie de archivos para Gradle: un archivo `.gitignore` y tres archivos Spring Boot:

- `ProductServiceApplication.java`, nuestra clase de aplicación principal
- `application.properties`, un archivo de propiedades vacío
- `ProductServiceApplicationTests.java`, una clase de prueba que se ha configurado para ejecutar pruebas en nuestra aplicación Spring Boot usando JUnit

La clase de aplicación principal, `ProductServiceApplication.java`, se ve como esperaríamos según la sección de anotación mágica `@SpringBootApplication` del capítulo anterior:

```
paquete se.magnus.microservices.core.product;

@SpringBootApplication
clase pública ProductServiceApplication {

    público estático void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

La clase de prueba se ve así:

```
paquete se.magnus.microservices.core.product;

@SpringBootTest
clase ProductServiceApplicationTests {

    @Test
    void contextLoads() {
    }
}
```

La anotación `@SpringBootTest` inicializará nuestra aplicación de la misma manera que lo hace `@SpringBootApplication` cuando ejecuta la aplicación; es decir, el contexto de la aplicación Spring se configurará antes de que se ejecuten las pruebas mediante el escaneo de componentes y la configuración automática, como se describe en el capítulo anterior.

Analicemos también el archivo más importante de Gradle: `build.gradle`. Este archivo describe cómo compilar el proyecto; por ejemplo, cómo resolver dependencias y compilar, probar y empaquetar el código fuente. El archivo Gradle comienza enumerando los complementos que se deben aplicar:

```
complementos { id 'java'
    id 'org.springframework.boot' versión '3.5.0' id
    'io.spring.dependency-management' versión '1.1.7'
}
```

Los complementos declarados se utilizan de la siguiente manera:

- El complemento Java agrega el compilador Java al proyecto.

Se declaran los plugins `org.springframework.boot` e `io.spring.dependency-management`, lo que garantiza que Gradle genere un archivo JAR robusto y que no sea necesario especificar números de versión explícitos en las dependencias de inicio de Spring Boot. En cambio, se deducen de la versión del plugin `org.springframework.boot`, es decir, la 3.5.0.

En el resto del archivo de compilación, básicamente declaramos un nombre de grupo y una versión para nuestro proyecto, la versión de Java y sus dependencias:

```
grupo = 'se.magnus.microservices.composite.product'
versión = '1.0.0-SNAPSHOT'

Java {
    cadena de herramientas {
        versiónidioma = JavaLanguageVersion.of(24)
    }
}

repositorios {
    mavenCentral()
}

dependencias {
    implementación 'org.springframework.boot:spring-boot-starter-actuator'
    Implementación 'org.springframework.boot:spring-boot-starter-webflux'
    Implementación de prueba 'org.springframework.boot:spring-boot-starter-test'
    Implementación de prueba 'io.projectreactor:reactor-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tareadas.named('prueba') {
    useJUnitPlatform()
}
```

A continuación se presentan algunas notas sobre las dependencias utilizadas y la declaración de prueba final:

- Las dependencias se obtienen, al igual que con los complementos anteriores, del repositorio central de Maven.
- Las dependencias se configuran como se especifica en los módulos Actuator y WebFlux, junto con un par de dependencias de prueba útiles
- Finalmente, JUnit está configurado para usarse para ejecutar nuestras pruebas en las compilaciones de Gradle.

Podemos construir cada microservicio por separado con el siguiente comando:

```
cd microservicios/servicio-compuesto-de-producto; ./gradlew compilación; cd -; \  
cd microservicios/producto-servicio; ./gradlew compilación; cd -; \  
cd microservicios/servicio-de-recomendación; ./gradlew compilación; cd -; \  
cd microservicios/revisión-servicio; ./gradlew compilación; cd -;
```



Tenga en cuenta cómo utilizamos los ejecutables gradlew creados por Spring Initializr; es decir, ¡no necesitamos tener Gradle instalado!

La primera vez que ejecutemos un comando con gradlew, se descargará Gradle automáticamente. La versión de Gradle utilizada se determina mediante la propiedad `distributionUrl` en los archivos `gradle/wrapper/gradle-wrapper.properties`.

Configuración de compilaciones multiproyecto en Gradle

Para simplificar la compilación de todos los microservicios con un solo comando, podemos configurar una compilación multiproyecto en Gradle. Los pasos son los siguientes:

1. Primero, creamos el archivo `settings.gradle`, que describe qué proyectos debe ejecutar Gradle.

construir:

```
gato <<EOF > configuraciones.gradle  
incluir ':microservicios:producto-servicio'  
incluir ':microservicios:revisión-servicio'  
incluir ':microservicios:servicio-de-recomendación'  
incluir ':microservicios:producto-servicio-compuesto'  
EOF
```

2. A continuación, copiamos los archivos ejecutables de Gradle que se generaron desde uno de los proyectos para que podemos reutilizarlos para las compilaciones de múltiples proyectos:

```
cp -r microservicios/producto-servicio/gradle .  
cp microservicios/producto-servicio/gradlew .  
cp microservicios/producto-servicio/gradlew.bat .  
cp microservicios/producto-servicio/.gitignore .
```

3. Ya no necesitamos los archivos ejecutables de Gradle generados en cada proyecto, por lo que podemos eliminarlos con los siguientes comandos:

```
buscar microservicios -depth -name "gradle" -exec rm -rfv "{}" \  
buscar microservicios -depth -name "gradlew*" -exec rm -fv "{}" \  

```

El resultado debería ser similar al código que puedes encontrar en `$BOOK_HOME/Chapter03/1-carpeta spring-init`.

4. Ahora, podemos construir todos los microservicios con un solo comando:

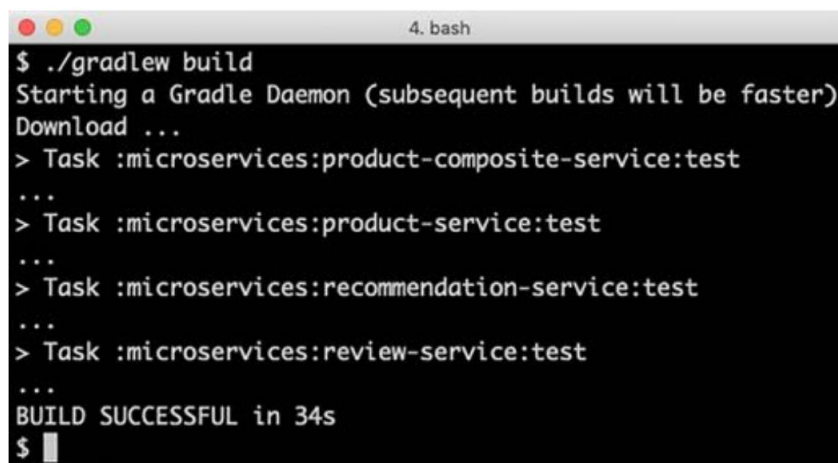
```
./gradlew compilación
```

Si no ha ejecutado los comandos anteriores, puede simplemente ir al código fuente del libro y compilarlo desde allí:

```
cd $BOOK_HOME/Capítulo03/1-spring-init
```

```
./gradlew compilación
```

Esto debería generar el siguiente resultado:



```
4. bash
$ ./gradlew build
Starting a Gradle Daemon (subsequent builds will be faster)
Download ...
> Task :microservices:product-composite-service:test
...
> Task :microservices:product-service:test
...
> Task :microservices:recommendation-service:test
...
> Task :microservices:review-service:test
...
BUILD SUCCESSFUL in 34s
$
```

Figura 3.3: Salida tras una compilación exitosa

Con proyectos esqueléticos para los microservicios creados con Spring Initializr y contruidos exitosamente con Gradle, estamos listos para agregar algo de código a los microservicios en la siguiente sección.



Desde una perspectiva DevOps, una configuración multiproyecto podría no ser la más recomendable. En cambio, para que cada microservicio tenga su propio ciclo de compilación y lanzamiento, probablemente sería preferible configurar una canalización de compilación independiente para cada proyecto de microservicio. Sin embargo, para los propósitos de este libro, utilizaremos la configuración de múltiples proyectos para facilitar la construcción e implementación de todo el panorama del sistema con un solo comando.

Agregar API RESTful

Ahora que tenemos proyectos configurados para nuestros microservicios, ¡agreguemos algunas API RESTful a nuestros tres microservicios principales!

El resultado final de este y los temas restantes de este capítulo se puede encontrar en `$BOOK_HOME/`

`Capítulo03/2-carpeta-servicios-descanso-basicos` .

Primero, agregaremos dos proyectos (api y util) que contendrán código compartido por los proyectos de microservicios y luego implementaremos las API RESTful.

Agregar una API y un proyecto de utilidad

Para agregar un proyecto api , necesitamos hacer lo siguiente:

1. Primero, configuraremos un proyecto Gradle independiente donde colocaremos nuestras definiciones de API. Usaremos interfaces Java para describir nuestras API RESTful y clases de modelo para describir los datos que la API utiliza en sus solicitudes y respuestas. Para describir los tipos de errores que puede devolver la API, también se definen varias clases de excepción. Describir una API RESTful en una interfaz Java, en lugar de directamente en la clase Java, es, en mi opinión, una buena manera de separar la definición de la API de su implementación. Ampliaremos este patrón más adelante en este libro, cuando agreguemos más información de la API en las interfaces Java que se expondrán en una especificación OpenAPI. Consulte el Capítulo 5, "Añadir una descripción de API mediante OpenAPI".

Más información.



Es discutible si es una buena práctica almacenar las definiciones de API para un grupo de microservicios en un módulo de API común. Esto podría causar dependencias no deseadas entre los microservicios, lo que resultaría en características monolíticas, por ejemplo, y un proceso de desarrollo más complejo y lento . En mi opinión, es una buena opción para microservicios que forman parte de la misma organización de entrega, es decir, cuyas versiones son regidas por la misma organización (compárese esto con un contexto delimitado en el diseño impulsado por el dominio, donde nuestros microservicios se ubican en un único contexto delimitado). Como ya se mencionó en el Capítulo 1, Introducción a los Microservicios, los microservicios dentro El mismo contexto delimitado necesita tener definiciones de API que se basen en un modelo de información común, por lo que almacenar estas definiciones de API en el mismo módulo de API no agrega ninguna dependencia no deseada.

2. A continuación, crearemos un proyecto de utilidad que pueda contener algunas clases auxiliares compartidas por nuestros microservicios, por ejemplo, para gestionar errores de manera uniforme.



Nuevamente, desde una perspectiva DevOps, sería preferible compilar todos los proyectos en su propia canalización de compilación y tener dependencias con control de versiones para los proyectos de API y Util en los proyectos de microservicios. De esta manera, cada microservicio puede elegir qué versiones de los proyectos de API y Util usar. Sin embargo, para simplificar los pasos de compilación e implementación en el contexto de este libro, haremos que los proyectos de API y Util formen parte de la compilación multiproyecto.

El proyecto API

El proyecto API se empaquetará como una biblioteca; es decir, no tendrá su propia clase de aplicación principal. Lamentablemente, Spring Initializr no permite la creación de proyectos de biblioteca. En su lugar, es necesario crear un proyecto de biblioteca manualmente desde cero. El código fuente del proyecto de API está disponible en `$BOOK_HOME/Chapter03/2-basic-rest-services/api`.

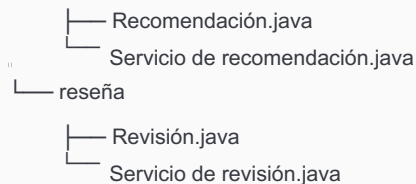
La estructura de un proyecto de biblioteca es la misma que la de un proyecto de aplicación, salvo que ya no se incluye la clase principal de la aplicación, además de algunas pequeñas diferencias en el archivo `build.gradle`. El complemento de Gradle `org.springframework.boot` se reemplaza por una sección de plataforma de implementación :

```
extensión {
    springBootVersion = '3.5.0'
}
dependencias {
    plataforma de implementación ("org.springframework.boot:spring-boot-dependencies:
    ${springBootVersion}")
}
```

Esto nos permite conservar la gestión de dependencias de Spring Boot mientras reemplazamos la construcción de un JAR pesado en el paso de compilación con la creación de un archivo JAR normal que solo contiene las propias clases y archivos de propiedades del proyecto.

Los archivos Java en el proyecto de API para nuestros tres microservicios principales son los siguientes:

```
$BOOK_HOME/Capítulo03/2-servicios-descansos-básicos/api/src/main/java/se/magnus/
API/núcleo
| — producto
|   | — Producto.java
|   |   | — ProductoServicio.java
| — recomendación
```



La estructura de las clases Java es muy similar para los tres microservicios principales, por lo que solo revisaremos el código fuente del servicio del producto.

Primero, veremos la interfaz Java ProductService.java , como se muestra en el siguiente código:

```

paquete se.magnus.api.core.product;

interfaz pública ProductService {

    @GetMapping(
        valor      = "/producto/{productId}",
        produce = "aplicación/json")
    Producto getProduct(@PathVariable int productId);
}

```

La declaración de la interfaz Java funciona de la siguiente manera:

- El servicio del producto solo expone un método API, getProduct() (ampliaremos la API más adelante en este libro, en el Capítulo 6, Añadiendo persistencia).
- Para mapear el método a una solicitud HTTP GET , usamos la anotación Spring @GetMapping , donde especificamos a qué ruta URL se mapeará el método (/product/{productId}) y en qué formato estará la respuesta, en este caso, JSON.
- La parte {productId} de la ruta se asigna a una variable de ruta llamada productId.

El parámetro del método productId se anota con @PathVariable, lo que asigna el valor pasado en la solicitud HTTP al parámetro. Por ejemplo, un HTTP GET

La solicitud a /product/123 dará como resultado que se llame al método getProduct() con el parámetro productId establecido en 123.

El método devuelve un objeto Producto , una clase de modelo simple basada en POJO con las variables miembro correspondientes a los atributos de Producto, como se describe al comienzo de este capítulo. Product.java Se ve así (con constructores y métodos getter excluidos):

```

clase pública Producto {
    privado final int productId;
}

```

```
cadena final privada nombre;
peso int final privado;
servicio de cadena final privado ;
}
```



Este tipo de clase POJO también se conoce como Objeto de Transferencia de Datos (DTO), ya que se utiliza para transferir datos entre la implementación de la API y el usuario que la llama. En el Capítulo 6, "Añadiendo Persistencia", veremos otro tipo de POJO que se puede usar para describir cómo se almacenan los datos en las bases de datos, también conocidos como objetos de entidad.

El proyecto API también contiene las excepciones `InvalidInputException` y `NotFoundException` clases.

El proyecto util

El proyecto util se empaquetará como biblioteca, al igual que el proyecto api . El código fuente del proyecto util está disponible en `$BOOK_HOME/Capítulo 03/2-servicios-básicos-rest/` util. El proyecto contiene las siguientes clases de utilidad: `GlobalControllerExceptionHandler`, `HttpErrorInfo` y `ServiceUtil`.

A excepción del código en `ServiceUtil.java`, estas clases son clases de utilidad reutilizables que podemos usar para asignar excepciones de Java a códigos de estado HTTP adecuados, como se describe en `Agregar manejo de errores`.

Sección. El objetivo principal de `ServiceUtil.java` es averiguar el nombre de host, la dirección IP y el puerto que utiliza el microservicio. La clase expone un método, `getServiceAddress()`, que los microservicios pueden usar para encontrar su nombre de host, dirección IP y puerto, como se describe en la sección anterior, `Información relacionada con la infraestructura`.

Implementando nuestra API

¡Ahora podemos comenzar a implementar nuestras API en los microservicios principales!

La implementación es muy similar para los tres microservicios principales, por lo que solo revisaremos el código fuente del servicio del producto. Puede encontrar los demás archivos en `$BOOK_HOME/Chapter03/2-Servicios básicos de descanso/microservicios`. Veamos cómo lo hacemos:

1. Necesitamos agregar los proyectos api y util como dependencias a nuestro archivo `build.gradle` , en

El proyecto `producto-servicio` :

```
dependencias {
    proyecto de implementación ('api')
    proyecto de implementación ('util')
```

2. Para habilitar la función de autoconfiguración de Spring Boot para detectar beans Spring en los proyectos `api` y `util`, también necesitamos agregar una anotación `@ComponentScan` a la clase de aplicación principal, que incluye los paquetes de los proyectos `api` y `util`:

```
@SpringBootApplication
@ComponentScan("se.magnus")
clase pública ProductoServicioAplicación {
```

3. A continuación, creamos nuestro archivo de implementación de servicio, `ProductServiceImpl.java`, para implementar la interfaz Java, `ProductService`, desde el proyecto de la API y anotamos la clase con `@RestController` para que Spring llame a los métodos en esta clase según las asignaciones especificadas en la clase Interface:

```
paquete se.magnus.microservices.core.product.services;
@RestController
clase pública ProductServiceImpl implementa ProductService {
}
```

4. Para poder utilizar la clase `ServiceUtil` del proyecto `util`, la inyectaremos en el constructor, de la siguiente manera:

```
servicioUtil privado final serviceUtil;

público ProductServiceImpl(ServiceUtil serviceUtil) {
    este.serviceUtil = serviceUtil;
}
```

5. Ahora, podemos implementar la API anulando el método `getProduct()` desde la interfaz. cara en el proyecto `api`:

```
@Anular
público Producto getProduct(int productId) {

    devolver nuevo Producto(productId, "nombre-" + productId, 123,
    serviceUtil.getServiceAddress());
}
```

Dado que actualmente no utilizamos una base de datos, simplemente devolvemos una respuesta codificada basada en la entrada de `productId`, junto con la dirección de servicio proporcionada por `ServiceUtil` clase.

Para obtener el resultado final, incluido el registro y el manejo de errores, consulte `ProductServiceImpl.java`.

6. Por último, también necesitamos configurar algunas propiedades de tiempo de ejecución: qué puerto usar y el puerto deseado.

Nivel de registro. Esto se añade al archivo de propiedades application.yml :

```
puerto del servidor: 7002

explotación florestal:
  nivel:
    raíz: INFO
  se.magnus.microservices: DEPURACIÓN
```



Tenga en cuenta que el archivo vacío application.properties generado por Spring Initializr se ha reemplazado por un archivo YAML, application.yml. Los archivos YAML ofrecen una mejor compatibilidad para agrupar propiedades relacionadas que los archivos .properties. Consulte la configuración del nivel de registro anterior como ejemplo.

7. Podemos probar el servicio del producto por sí solo. Construir e iniciar el microservicio con el siguientes comandos:

```
cd $BOOK_HOME/Capítulo03/2-servicios-de-descanso-básicos
./gradlew compilación
java -jar microservicios/producto-servicio/build/libs/*.jar &
```

Espere hasta que se imprima lo siguiente en la terminal:

```
bash
$ java -jar microservicios/product-servicio/build/libs/*.jar &

  ____ _
 / __ \| | | |
( (  / \| | | |
 \|  / _ \| | | |
  / _ \| | | |
 / ___ \| | | |
/_____\|_|_|_|

:: Spring Boot ::                (v3.5.0)

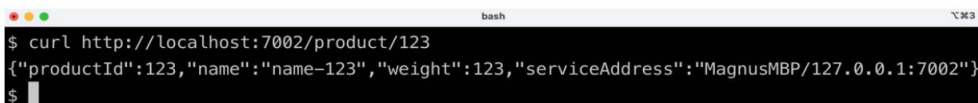
...Started ProductServiceApplication in 1.305 seconds (process running for 1.563)
$
```

Figura 3.4: Inicio de ProductServiceApplication

8. Realice una llamada de prueba al servicio de atención al cliente:

```
rizo http://localhost:7002/producto/123
```

Debería responder con algo similar a lo siguiente:



```
bash
$ curl http://localhost:7002/product/123
{"productId":123,"name":"name-123","weight":123,"serviceAddress":"MagnusMBP/127.0.0.1:7002"}
$
```

Figura 3.5: Respuesta esperada a la llamada de prueba

9. Finalmente, detenga el servicio del producto:

```
matar $(trabajos -p)
```

Ya hemos creado, ejecutado y probado nuestro primer microservicio. En la siguiente sección, implementaremos el microservicio compuesto que utilizará los tres microservicios principales que hemos creado hasta ahora.



A partir de Spring Boot v2.5.0, se crean dos archivos JAR al ejecutar el comando de compilación `./gradlew` : el archivo JAR normal, más un archivo JAR simple que contiene solo los archivos de clase resultantes de la compilación de los archivos Java en la aplicación Spring Boot. Como no necesitamos el nuevo archivo JAR simple, se ha deshabilitado su creación para hacer posible hacer referencia al archivo JAR ordinario usando un comodín al ejecutar la aplicación Spring Boot, como en este ejemplo:

```
java -jar microservicios/producto-servicio/compilación/libs/*.jar
```

Se ha deshabilitado la creación del nuevo archivo JAR simple agregando las siguientes líneas al archivo `build.gradle` para cada microservicio:

```
frasco {
    habilitado = falso
}
```

Para obtener más detalles, consulte <https://docs.spring.io/spring-boot/3.5/gradle-plugin/packaging.html#packaging-executable.and-plain-archives> .

Agregar un microservicio compuesto

Ahora, es el momento de unir las cosas agregando el servicio compuesto que llamará a los tres núcleos.

¡servicios!

La implementación de los servicios compuestos se divide en dos partes: un componente de integración que gestiona las solicitudes HTTP salientes a los servicios principales y la propia implementación del servicio compuesto. La razón principal de esta división de responsabilidades es que simplifica la automatización de las pruebas unitarias y de integración; podemos probar la implementación del servicio de forma aislada reemplazando el componente de integración con una simulación.



Como veremos más adelante en este libro, esta división de responsabilidades también facilitará la introducción de un disyuntor.

Antes de analizar el código fuente de los dos componentes, debemos echar un vistazo a las clases de API que usarán los microservicios compuestos y también aprender sobre cómo se ejecutan las propiedades en tiempo de ejecución. Se utilizan para almacenar información de direcciones para los microservicios principales.

La implementación completa tanto del componente de integración como de la implementación del servicio compuesto se pueden encontrar en `se.magnus.microservices.composite.product.services`.

Paquete Java.

Clases de API

En esta sección, veremos las clases que describen la API del componente compuesto.

Se pueden encontrar en `$BOOK_HOME/Capítulo03/2-servicios-rest-básicos/api`. Los siguientes son

Las clases API:

```
$BOOK_HOME/Capítulo 03/2-servicios-descansos-básicos/api
├── src/main/java/se/magnus/api/composite
│   └── producto
│       ├── ProductoAgregado.java
│       ├── ProductoCompuestoService.java
│       ├── Resumen de recomendaciones.java
│       ├── Resumen de la revisión.java
│       └── Direcciones de servicio.java
```

La clase de interfaz Java, `ProductCompositeService.java`, sigue el mismo patrón que utilizan los servicios principales y tiene el siguiente aspecto:

```
paquete se.magnus.api.composite.product;

interfaz pública ProductCompositeService {

    @GetMapping(
        valor      = "/producto-compuesto/{productId}",
        produce = "aplicación/json")
    ProductAggregate getProduct(@PathVariable int productId);

}
```

La clase de modelo, `ProductAggregate.java`, es un poco más compleja que los modelos principales, ya que contiene campos para listas de recomendaciones y reseñas:

```
paquete se.magnus.api.composite.product;

clase pública ProductoAgregado {
    privado final int productId; privado final
    String nombre;
    int final privado peso; lista final
    privada List<RecommendationSummary> recomendaciones;
    Lista privada final <ReviewSummary> revisiones; Direcciones
    de servicio privadas finales serviceAddresses;
```

Las clases de API restantes son objetos de modelo simples basados en POJO y tienen la misma estructura que los objetos de modelo para las API principales.

Propiedades. Para

evitar la codificación rígida de la información de dirección de los servicios principales en el código fuente del microservicio compuesto, este utiliza un archivo de propiedades que almacena información sobre cómo encontrarlos. El archivo de propiedades, `application.yml`, tiene el siguiente aspecto:

```
puerto del servidor: 7001

aplicación:
  producto-servicio:
    anfitrión: localhost
    puerto: 7002
  servicio de recomendación:
    anfitrión: localhost
    puerto: 7003
  servicio de revisión:
    anfitrión: localhost
    puerto: 7004
```

Esta configuración, como ya se señaló, será reemplazada más adelante por un mecanismo de descubrimiento de servicios. en este libro.

El componente de integración

Analicemos la primera parte de la implementación del microservicio compuesto, el componente de integración `ProductCompositeIntegration.java`. Se declara como un bean de Spring mediante la anotación `@Component` e implementa las API de los tres servicios principales:

```
paquete se.magnus.microservices.composite.product.services;  
  
@Component  
La clase pública ProductCompositeIntegration implementa ProductService,  
Servicio de recomendación, Servicio de revisión {
```

El componente de integración utiliza una clase auxiliar de Spring Framework, `RestTemplate`, para ejecutar las solicitudes HTTP a los microservicios principales. Antes de poder inyectarla en el componente de integración, es necesario configurarla. Esto se realiza en la clase principal de la aplicación, `ProductCompositeServiceApplication.java`, como sigue:

```
@Frijol  
RestTemplate restTemplate() {  
    devolver nueva RestTemplate();  
}
```

Un objeto `RestTemplate` es altamente configurable, pero lo dejamos con sus valores predeterminados por ahora.



En la sección Spring WebFlux del Capítulo 2, Introducción a Spring Boot, presentamos el cliente HTTP reactivo, `WebClient`. Usar `WebClient` en lugar de `RestTemplate` en este capítulo requeriría que todo el código fuente donde se usa `WebClient` también sea reactivo, incluyendo la declaración de la API RESTful en el proyecto de API y el código fuente en el microservicio compuesto. En el Capítulo 7, Desarrollo de Microservicios Reactivos, aprenderemos a modificar la implementación de nuestros microservicios para que sigan un modelo de programación reactiva. Como parte de esta actualización, reemplazaremos la clase auxiliar `RestTemplate` por la clase `WebClient`. Sin embargo, hasta que aprendamos sobre el desarrollo reactivo en Spring, usaremos la clase `RestTemplate`.

Ahora podemos inyectar `RestTemplate`, junto con un mapeador JSON, que se utiliza para acceder a los mensajes de error en caso de errores, y los valores de configuración que hemos establecido en el archivo de propiedades. Veamos cómo se hace esto:

1. Los objetos y los valores de configuración se inyectan en el constructor de la siguiente manera:

```

RestTemplate final privado restTemplate; asignador final
privado de ObjectMapper ;

cadena privada final productServiceUrl; cadena privada
final RecommendationServiceUrl; cadena privada final
ReviewServiceUrl;

público ProductoCompuestoIntegración(
    RestTemplate restTemplate,
    Mapeador ObjectMapper ,

    @Value("${app.product-service.host}")
    Cadena productServiceHost,

    @Value("${app.product-service.port}") int
    puertoDeServicioProducto,

    @Value("${app.recommendation-service.host}")
    Cadena recomendaciónServiceHost,

    @Value("${app.recommendation-service.port}") int
    puertoDeServicioDeRecomendación,

    @Value("${app.review-service.host}")
    Revisión de cadenaServiceHost ,

    @Value("${app.review-service.port}")
    int puertoDeServicioDeRevisión

)

```

2. El cuerpo del constructor almacena los objetos inyectados y construye las URL en función de ellos. valores inyectados, como sigue:

```

{
    este.restTemplate = restTemplate; este.mapper
    = mapper;

    productServiceUrl = "http://" + productServiceHost + ":" + productServicePort + "/"
    product/"; RecommendationServiceUrl =
    "http://" + RecommendationServiceHost

```

```

+ ":" + RecommendationServicePort + "/recomendación?
productId="; reviewServiceUrl = "http://" + reviewServiceHost + ":" + reviewServicePort
+ "/review?productId=";
}

```

3. Finalmente, el componente de integración implementa los métodos API para los tres servicios principales. utilizando RestTemplate para realizar las llamadas salientes reales:

```

público Producto getProduct(int productId) {
    Cadena url = productServiceUrl + productId; Producto
    producto = restTemplate.getForObject(url,
        Producto.clase);
    devolver producto;
}

Lista pública <Recomendación> obtenerRecomendaciones(int IdProducto) {
    Cadena url = recomendaciónServiceUrl + productId;
    Lista< Recomendación> recomendaciones =
    restTemplate.exchange(url, GET, nulo, nuevo
    ParameterizedTypeReference<Lista<Recomendación>>())
    {}).getBody();
    recomendaciones de devolución ;
}

Lista pública <Revisión> obtenerReseñas(int IdProducto) {
    Cadena url = reviewServiceUrl + productId;
    Lista<Revisión> revisiones = restTemplate.exchange(url, GET,
    nulo,
    nueva ParameterizedTypeReference<Lista<Revisión>>())
    {}).getBody();
    devolver reseñas;
}

```

Algunas notas interesantes sobre la implementación de los métodos:

Para la implementación de `getProduct()`, se puede usar el método `getForObject()` en `RestTemplate`. La respuesta esperada es un objeto `Product`. Esto se puede expresar en la llamada a `getForObject()` especificando la clase `Product.class` a la que `RestTemplate` asignará la respuesta JSON.

Para las llamadas a `getRecommendations()` y `getReviews()`, se debe usar un método más avanzado, `exchange()`. Esto se debe a la asignación automática de una respuesta JSON a una clase de modelo que `RestTemplate` realiza. Los métodos `getRecommendations()` y `getReviews()` esperan una lista genérica en las respuestas, es decir, `List<Recommendation>`, y `List<Review>`. Dado que los genéricos no almacenan ningún tipo de información en tiempo de ejecución, no podemos especificar que los métodos esperen una lista genérica en sus respuestas. En su lugar, podemos usar una clase auxiliar de Spring Framework, `ParameterizedTypeReference`, diseñada para resolver este problema al almacenar la información de tipo en tiempo de ejecución. Esto significa que `RestTemplate` puede determinar a qué clase asignar las respuestas JSON. Para utilizar esta clase auxiliar, debemos usar el método `exchange()`, más complejo, en lugar del método `getForObject()`, más sencillo, de `RestTemplate`.

Implementación de API compuesta

Finalmente, analizaremos la última parte de la implementación del microservicio compuesto: la clase de implementación de la API `ProductCompositeServiceImpl.java`. Analicémosla paso a paso:

1. De la misma manera que lo hicimos para los servicios principales, el servicio compuesto implementa su interfaz API, `ProductCompositeService`, y se anota con `@RestController` para marcarlo como un servicio REST:

```
paquete se.magnus.microservices.composite.product.services;

@RestController
la clase pública ProductCompositeServiceImpl implementa
ServicioCompuesto de Producto {
```

2. La clase de implementación requiere el bean `ServiceUtil` y su propia integración. componente, por lo que se inyectan en su constructor:

```
servicioUtil privado final serviceUtil;
integración privada ProductCompositeIntegration ;

público ProductCompositeServiceImpl(ServiceUtil serviceUtil,
Integración de ProductCompositeIntegration ) {
    este.serviceUtil = serviceUtil;
    esto.integracion = integración;
}
```


3. Finalmente, el método API se implementa de la siguiente manera:

```
@Anular
público ProductAggregate obtenerProducto(int productId) {

    Producto producto = integración.getProduct(productId);
    Lista< Recomendación> recomendaciones =
    integración.getRecommendations(productId);
    Lista<Reseña> reseñas = integración.getReviews(productId);

    devolver createProductAggregate(producto, recomendaciones,
    reseñas, serviceUtil.getServiceAddress());
}
```

El componente de integración se utiliza para llamar a los tres servicios principales, y se utiliza un método auxiliar, create- ProductAggregate(), para crear un objeto de respuesta del tipo ProductAggregate basado en las respuestas de las llamadas al componente de integración.

La implementación del método auxiliar, createProductAggregate(), es bastante extensa y no muy importante, por lo que se ha omitido de este capítulo; sin embargo, se puede encontrar en este código fuente del libro.

La implementación completa tanto del componente de integración como del servicio compuesto se puede encontrar en el paquete Java se.magnus.microservices.composite.product.services .

Con esto, se completa la implementación del microservicio compuesto desde un punto de vista funcional. En la siguiente sección, veremos cómo gestionamos los errores.

Añadiendo gestión de errores

Gestionar errores de forma estructurada y bien pensada es esencial en un entorno de microservicios donde un gran número de ellos se comunican entre sí mediante API síncronas, por ejemplo, HTTP y JSON. También es importante separar la gestión de errores específica del protocolo, como los códigos de estado HTTP, de la lógica de negocio.



Se podría argumentar que debería añadirse una capa independiente para la lógica de negocio al implementar los microservicios. Esto garantizaría que la lógica de negocio esté separada del código específico del protocolo, facilitando así tanto las pruebas como la reutilización. Para evitar una complejidad innecesaria en los ejemplos de este libro, hemos omitido una capa independiente para la lógica de negocio, de modo que los microservicios implementen su lógica de negocio directamente en los componentes `@RestController`.

He creado un conjunto de excepciones de Java en el proyecto `util`, que son utilizadas tanto por las implementaciones de la API como por los clientes de la API; inicialmente, `InvalidInputException` y `NotFoundException`. Para más detalles, consulte el paquete de Java `se.magnus.util.exceptions`.

El manejador de excepciones del controlador REST global

Para separar el manejo de errores específicos del protocolo de la lógica empresarial en los controladores REST, es decir, las implementaciones de API, he creado una clase de utilidad, `GlobalControllerExceptionHandler`. java, en el proyecto `util`, que está anotado como `@RestControllerAdvice`.

Para cada excepción de Java que las implementaciones de API lanzan, la clase de utilidad tiene un método controlador de excepciones que asigna la excepción de Java a una respuesta HTTP adecuada, es decir, con un estado HTTP y un cuerpo de respuesta HTTP adecuados.

Por ejemplo, si una clase de implementación de API genera una excepción `InvalidInputException`, la clase de utilidad la asignará a una respuesta HTTP con el código de estado 422 (`UNPROCESSABLE_ENTITY`). El siguiente código lo muestra:

```
@ResponseStatus(ENTIDAD NO PROCESABLE)
@ExceptionHandler(InvalidInputException.class)
público @ResponseBody HttpErrorInfo handleInvalidInputException(
    Solicitud ServerHttpRequest, InvalidInputException ej.) {

    devolver createHttpErrorInfo(ENTIDAD_INPROCESABLE, solicitud, ej);
}
```

De la misma manera, `NotFoundException` se asigna a un código de estado HTTP 404 (`NOT_FOUND`).

Siempre que un controlador REST lanza cualquiera de estas excepciones, Spring usará la clase de utilidad para crear una respuesta HTTP.



Tenga en cuenta que Spring en sí mismo devuelve el código de estado HTTP 400 (BAD_REQUEST) cuando detecta una solicitud no válida, por ejemplo, si la solicitud contiene un ID de producto no numérico (productId se especifica como un entero en la declaración de API).

Para obtener el código fuente completo de la clase de utilidad, consulte `GlobalControllerExceptionHandler.java`.

Manejo de errores en implementaciones de API

Las implementaciones de API utilizan las excepciones del proyecto util para señalar errores. Estas se reportan al cliente REST como códigos de estado HTTPS que indican el error. Por ejemplo, la clase de implementación del microservicio Product, `ProductServiceImpl.java`, utiliza la excepción `InvalidInputException` para devolver un error que indica una entrada no válida, así como la excepción `NotFoundException` para indicar que el producto solicitado no existe. El código se ve así:

```
si (productId < 1) arrojar nueva InvalidInputException(
    "ProductId no válido: " + productId);
(productId == 13) arroja una nueva NotFoundException(
    "No se encontró ningún producto para el ID del producto: " + productId);
```



Como actualmente no estamos usando una base de datos, tenemos que simular cuándo lanzar `NotFoundException`.

Manejo de errores en el cliente API

El cliente de API, es decir, el componente de integración del microservicio Composite, realiza el proceso inverso: asigna el código de estado HTTP 422 (UNPROCESSABLE_ENTITY) a `InvalidInputException` y el código de estado HTTP 404 (NOT_FOUND) a `NotFoundException`. Consulte el método `getProduct()` en `ProductCompositeIntegration.java` para ver la implementación de esta lógica de gestión de errores. El código fuente se ve así:

```
captura (HttpClientErrorException ex) {

    cambiar (HttpStatus.resolve(ex.getStatusCode().value())) {

        caso NO_ENCONTRADO:
            lanzar nueva NotFoundException(getErrorMessage(ex));
```

```
caso ENTIDAD_INPROCESABLE:
    lanzar nueva InvalidInputException(getErrorMessage(ex));

por defecto:
    LOG.warn("Se produjo un error HTTP inesperado: {}, lo volveré a generar",
ej.getStatusCode());
    LOG.warn(" Cuerpo del error: {}", ej.getResponseBodyAsString());
    lanzar ex;
}
}
```

El manejo de errores para `getRecommendations()` y `getReviews()` en el componente de integración es un poco más relajado: se clasifica como de máximo esfuerzo, lo que significa que si logra obtener información del producto pero no obtiene recomendaciones ni reseñas, aún así se considera correcto. Sin embargo, se escribe una advertencia en el registro.

Para obtener más detalles, consulte `ProductCompositeIntegration.java`.

Con esto, se completa la implementación tanto del código como de los microservicios compuestos. En la siguiente sección, probaremos los microservicios y la API que exponen.

Probar las API manualmente

Con esto concluimos la implementación de nuestros microservicios. Probémoslos con los siguientes pasos:

1. Construya e inicie los microservicios como procesos en segundo plano.
2. Utilice curl para llamar a la API compuesta.
3. Detenerlos.

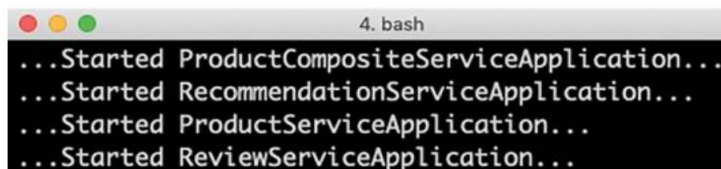
Primero, construya e inicie cada microservicio como un proceso en segundo plano, de la siguiente manera:

```
cd $BOOK_HOME/Capítulo03/2-servicios-de-descanso-básicos/
./gradlew compilación
```

Una vez que se completa la compilación, podemos iniciar nuestros microservicios como procesos en segundo plano para el proceso de terminal con el siguiente código:

```
java -jar microservicios/producto-servicio-compuesto/build/libs/*.jar &
java -jar microservicios/producto-servicio/build/libs/*.jar &
java -jar microservicios/servicio-de-recomendación/build/libs/*.jar &
java -jar microservicios/servicio-de-revisión/build/libs/*.jar &
```

Se escribirán muchos mensajes de registro en la terminal, pero después de unos segundos, las cosas se calmarán y encontraremos los siguientes mensajes escritos en el registro:

A terminal window titled '4. bash' with a dark background. It displays four lines of log output: '...Started ProductCompositeServiceApplication...', '...Started RecommendationServiceApplication...', '...Started ProductServiceApplication...', and '...Started ReviewServiceApplication...'. Each line is preceded by a series of dots, indicating a sequence of events.

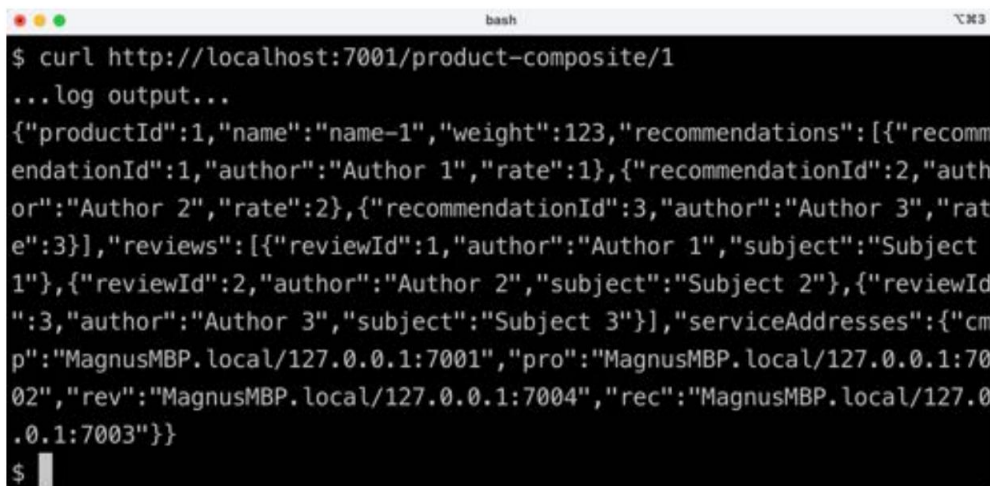
```
4. bash
...Started ProductCompositeServiceApplication...
...Started RecommendationServiceApplication...
...Started ProductServiceApplication...
...Started ReviewServiceApplication...
```

Figura 3.6: Mensajes de registro después del inicio de las aplicaciones

Esto significa que todos están listos para recibir solicitudes. Pruebe esto con el siguiente código:

```
rizo http://localhost:7001/producto-compuesto/1
```

Después de obtener una salida de registro, obtendremos una respuesta JSON que se parecerá a lo siguiente:

A terminal window titled 'bash' with a dark background. It shows a curl command being executed: '\$ curl http://localhost:7001/product-composite/1'. Below the command, it shows the output: '...log output...' followed by a large JSON object. The JSON object contains fields for 'productId', 'name', 'weight', 'recommendations' (an array of recommendation objects), 'reviews' (an array of review objects), and 'serviceAddresses' (an object with 'cmp', 'pro', 'rev', and 'rec' fields).

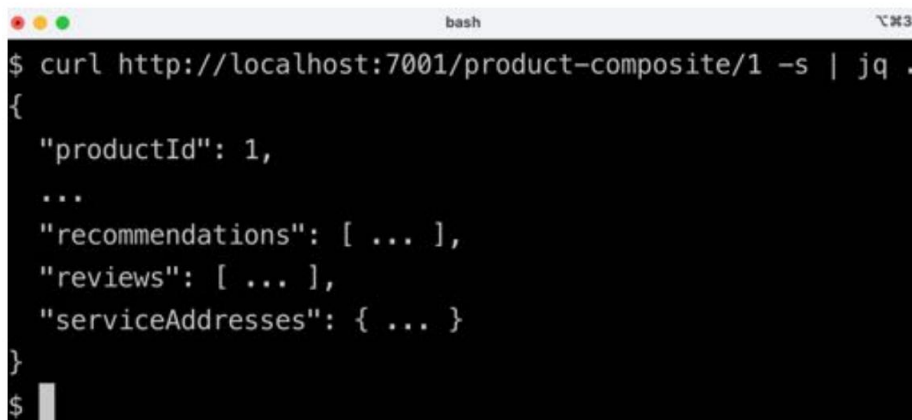
```
bash
$ curl http://localhost:7001/product-composite/1
...log output...
{"productId":1,"name":"name-1","weight":123,"recommendations":[{"recomm
endationId":1,"author":"Author 1","rate":1},{"recommendationId":2,"auth
or":"Author 2","rate":2},{"recommendationId":3,"author":"Author 3","rat
e":3}], "reviews":[{"reviewId":1,"author":"Author 1","subject":"Subject
1"}, {"reviewId":2,"author":"Author 2","subject":"Subject 2"}, {"reviewId
":3,"author":"Author 3","subject":"Subject 3"}], "serviceAddresses":{"cm
p":"MagnusMBP.local/127.0.0.1:7001", "pro":"MagnusMBP.local/127.0.0.1:70
02", "rev":"MagnusMBP.local/127.0.0.1:7004", "rec":"MagnusMBP.local/127.0
.0.1:7003"}}
$
```

Figura 3.7: Respuesta JSON después de la solicitud

Para obtener la respuesta JSON impresa correctamente, puede utilizar la herramienta jq:

```
curl http://localhost:7001/producto-compuesto/1 -s | jq .
```

Esto da como resultado el siguiente resultado (algunos detalles se han reemplazado por ... para mejorar la legibilidad):

A terminal window with a dark background and light text. The title bar shows 'bash' and a window icon. The command '\$ curl http://localhost:7001/product-composite/1 -s | jq .' is entered. The output is a JSON object with fields 'productId', '...', 'recommendations', 'reviews', and 'serviceAddresses', all containing values or '...' indicating they have been truncated for readability. The prompt '\$' is visible at the bottom left.

```
$ curl http://localhost:7001/product-composite/1 -s | jq .
{
  "productId": 1,
  ...,
  "recommendations": [ ... ],
  "reviews": [ ... ],
  "serviceAddresses": { ... }
}
$
```

Figura 3.8: Respuesta JSON impresa de forma atractiva

Si lo desea, también puede probar los siguientes comandos para verificar que el manejo de errores funciona como se espera:

```
# Verificar que se devuelva un error 404 (No encontrado) para un productId inexistente (13)
```

```
curl http://localhost:7001/producto-compuesto/13 -i
```

```
# Verificar que no se devuelvan recomendaciones para el productId 113
```

```
curl http://localhost:7001/producto-compuesto/113 -s | jq .
```

```
# Verificar que no se devuelvan reseñas para el productId 213
```

```
curl http://localhost:7001/producto-compuesto/213 -s | jq .
```

```
# Verifique que se devuelva un error 422 (Entidad no procesable) para un productId que
esté fuera del rango (-1)
```

```
curl http://localhost:7001/producto-compuesto/-1 -i
```

```
# Verifique que se devuelva un error 400 (Solicitud incorrecta) para un productId que no sea un
número, es decir, un formato no válido
```

```
curl http://localhost:7001/producto-compuesto/invalidProductId -i
```

Finalmente, puedes apagar los microservicios con el siguiente comando:

```
matar $(trabajos -p)
```

Si usa un IDE como Visual Studio Code con Spring Tool Suite, puede aprovechar su compatibilidad con Spring Boot Dashboard para iniciar y detener sus microservicios con un solo clic. Para obtener instrucciones sobre cómo instalar Spring Tool Suite, consulte <https://github.com/spring-projects/sts4/wiki/Instalación>.

La siguiente captura de pantalla muestra el uso de Spring Boot Dashboard en Visual Studio Code:

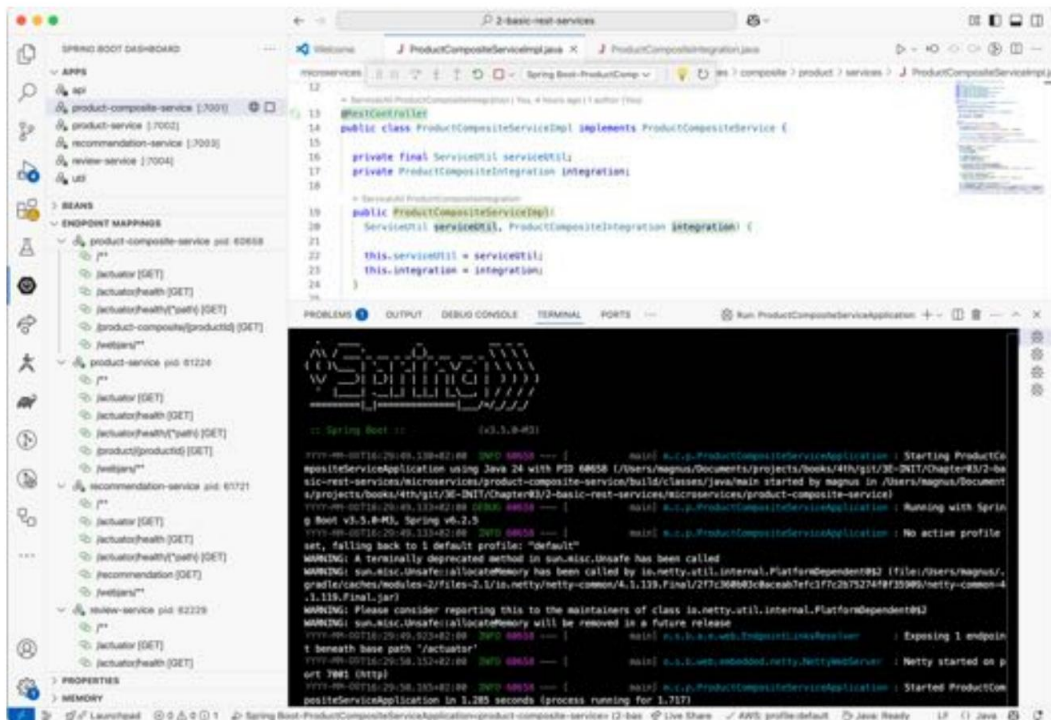


Figura 3.9: Panel de Spring Boot en Visual Studio Code

En esta sección, hemos aprendido a iniciar, probar y detener manualmente el entorno del sistema de microservicios cooperativos. Este tipo de pruebas consume mucho tiempo, por lo que es evidente que deben automatizarse. En las dos siguientes secciones, daremos los primeros pasos para aprender a automatizar las pruebas, tanto para un único microservicio de forma aislada como para todo el entorno del sistema de microservicios cooperativos. A lo largo de este libro, mejoraremos la forma en que probamos nuestros microservicios.

Agregar pruebas de microservicios automatizadas de forma aislada

Antes de finalizar la implementación, también necesitamos escribir algunas pruebas automatizadas.

No tenemos mucha lógica de negocios para probar en este momento, por lo que no necesitamos escribir ninguna prueba unitaria. En su lugar, nos centraremos en probar las API que exponen nuestros microservicios; es decir, las iniciaremos en pruebas de integración con su servidor web integrado y luego usaremos un cliente de prueba para realizar solicitudes HTTP y validar las respuestas. Spring WebFlux incluye un cliente de prueba, `WebTestClient`, que proporciona una API fluida para realizar una solicitud y aplicar aserciones a su resultado.

El siguiente es un ejemplo en el que probamos la API del producto compuesto realizando las siguientes pruebas:

- Enviar el ID del producto para un producto existente y afirmar que recibiremos 200 como retorno
Código de respuesta HTTP y una respuesta JSON que contiene el `productId` solicitado junto
Con una recomendación y una reseña
- Enviar un `productId` faltante y afirmar que recibiremos 404 como respuesta HTTP
código y una respuesta JSON que contiene información de error relevante

La implementación de estas dos pruebas se muestra en el siguiente código. La primera prueba se ve así:

```
@Autowired
cliente WebTestClient privado ;

@Prueba
void obtenerProductoPorId() {
    cliente.get()
        .uri("/producto-compuesto/" + PRODUCT_ID_OK)
        .aceptar(APLICACIÓN_JSON_UTF8)
        .intercambio()
        .expectStatus().isOk()
        .expectHeader().contentType(APLICACIÓN_JSON_UTF8)
        .expectBody()
        .jsonPath("$.productId").esIgualA(PRODUCT_ID_OK)
        .jsonPath("$.recomendaciones.length()").isEqualTo(1)
        .jsonPath("$.reviews.length()").isEqualTo(1);
}
```

El código de prueba funciona así:

- La prueba utiliza la API fluida `WebTestClient` para configurar la URL para llamar a `"/product-composite/" + PRODUCT_ID_OK` y especifique el formato de respuesta aceptado, JSON.
- Después de ejecutar la solicitud utilizando el método `exchange()`, la prueba verifica que el estado de la respuesta sea OK (200) y que el formato de la respuesta sea realmente JSON (como se solicitó).

- Finalmente, la prueba inspecciona el cuerpo de la respuesta y verifica que contenga la información esperada en términos de productId y la cantidad de recomendaciones y reseñas.

La segunda prueba se ve así:

```
@Prueba
público void obtenerProductoNoEncontrado() {
    cliente.get()
        .uri("/producto-compuesto/" + ID_DE_PRODUCTO_NO_ENCONTRADO)
        .aceptar(APLICACIÓN_JSON_UTF8)
        .intercambio()
        .expectStatus().isNotFound()
        .expectHeader().contentType(APLICACIÓN_JSON_UTF8)
        .expectBody()
        .jsonPath("$.path").isEqualTo("/producto-compuesto/" +
            ID_DE_PRODUCTO_NO_ENCONTRADO)
        .jsonPath("$.message").isEqualTo("NO ENCONTRADO: " +
            ID_DE_PRODUCTO_NO_ENCONTRADO);
}
```

Una nota importante con respecto a este código de prueba es que esta prueba negativa es muy similar a la prueba anterior en términos de su estructura; la principal diferencia es que verifica que recibió un código de estado de error, No encontrado (404), y que el cuerpo de la respuesta contiene el mensaje de error esperado.

Para probar la API del producto compuesto de forma aislada, necesitamos simular sus dependencias, es decir, las solicitudes a los otros tres microservicios realizadas por el componente de integración ProductCompositeIntegration. Para ello, utilizamos Mockito, como se indica a continuación:

```
privado estático final int PRODUCT_ID_OK = 1;
privado estático final int PRODUCTO_NO_ENCONTRADO = 2;
privado estático final int PRODUCTO_ID_INVÁLIDO = 3;

@MockitoBean
privado ProductCompositeIntegration compositeIntegration;

@AntesDeCada
void setUp() {

    cuando(compositeIntegration.getProduct(PRODUCT_ID_OK)).
        thenReturn(new Producto(PRODUCT_ID_OK, "nombre", 1, "dirección simulada"));
    cuando(compositeIntegration.getRecommendations(PRODUCT_ID_OK)).
```

```

        luegoReturn(singletonList(nueva Recomendación(PRODUCT_ID_OK, 1,
            "autor", 1, "contenido", "dirección simulada")));

        cuando(compositeIntegration.getReviews(PRODUCT_ID_OK)).
            luegoReturn(singletonList(nueva revisión(PRODUCT_ID_OK, 1, "autor",
                "asunto", "contenido", "dirección simulada")));

        cuando(compositeIntegration.getProduct(PRODUCT_ID_NO_ENCONTRADO)).
            thenThrow(new NotFoundException("NO ENCONTRADO: " +
                ID_DE_PRODUCTO_NO_ENCONTRADO));

        cuando (compositeIntegration.getProduct(PRODUCT_ID_INVALID)).
            thenThrow(new InvalidInputException("INVÁLIDO: " +
                ID_DE_PRODUCTO_INVÁLIDO));
    }

```

La implementación simulada funciona de la siguiente manera:

1. Primero, declaramos tres constantes que se utilizan en la clase de prueba: `PRODUCT_ID_OK`, `PRODUCT_ID_NO_ENCONTRADO` y `ID_DE_PRODUCTO_INVÁLIDO`.
2. A continuación, se utiliza la anotación `@MockitoBean` para configurar Mockito a fin de configurar una simulación para la interfaz `ProductCompositeIntegration`.
3. Si se llaman los métodos `getProduct()`, `getRecommendations()` y `getReviews()` en el componente de integración y `productId` está configurado en `PRODUCT_ID_OK`, la simulación devolverá una respuesta normal.
4. Si se llama al método `getProduct()` con `productId` establecido en `PRODUCT_ID_NOT_FOUND`, la simulación lanzará `NotFoundException`.
5. Si se llama al método `getProduct()` con `productId` establecido en `PRODUCT_ID_INVALID`, `mock` lanzará `InvalidInputException`.

El código fuente completo para las pruebas de integración automatizadas en la API de producto compuesto se puede encontrar en la clase de prueba `ProductCompositeServiceApplicationTests.java`.

Las pruebas de integración automatizadas en la API expuesta por los tres microservicios principales son similares, pero más sencillas, ya que no necesitan simular nada. El código fuente de las pruebas se puede encontrar aquí.
en la carpeta de prueba de cada microservicio.

Gradle ejecuta automáticamente las pruebas al realizar una compilación:

```
./gradlew compilación
```

Sin embargo, puedes especificar que solo deseas ejecutar las pruebas (y no el resto de la compilación):

```
Prueba ./gradlew
```

Esta fue una introducción a cómo escribir pruebas automatizadas para microservicios de forma aislada. En la siguiente sección, aprenderemos cómo escribir pruebas que prueben automáticamente un entorno de microservicios. En este capítulo, estas pruebas solo estarán semiautomatizadas. En los próximos capítulos, estarán completamente automatizadas, lo cual representa una mejora significativa.

Agregar pruebas semiautomatizadas de un entorno de microservicios

Poder ejecutar automáticamente pruebas unitarias y de integración para cada microservicio de forma aislada usando Java, JUnit y Gradle es muy útil durante el desarrollo, pero insuficiente al pasar a la fase operativa. Durante la operación, también necesitamos una forma de verificar automáticamente que un entorno de microservicios cooperativos ofrezca lo esperado. Poder ejecutar, en cualquier momento, un script que verifique que varios microservicios cooperantes funcionen correctamente es muy valioso: cuantos más microservicios haya, mayor será el valor de dicho script de verificación.

Por esta razón, he escrito un script Bash simple que puede verificar la funcionalidad de un entorno de sistema implementado mediante llamadas a las API RESTful expuestas por los microservicios. Se basa en los comandos curl que aprendimos y usamos previamente. El script verifica los códigos de retorno y partes de las respuestas JSON usando jq. El script contiene dos funciones auxiliares: `assertCurl()` y `assertEqual()`, para hacer que el código de prueba sea compacto y fácil de leer.

Por ejemplo, realizar una solicitud normal y esperar 200 como código de estado, además de afirmar que recibiremos una respuesta JSON que devuelve el `productId` solicitado junto con tres recomendaciones y tres reseñas, se ve así:

```
# Verifique que una solicitud normal funcione, espere tres recomendaciones y
tres reseñas

assertCurl 200 "curl http://$HOST:${PORT}/producto-compuesto/1 -s"
assertEqual 1 $(echo $RESPONSE | jq .productId)
assertEqual 3 $(echo $RESPONSE | jq ".recommendations | length")
assertEqual 3 $(echo $RESPONSE | jq ".reviews | length")
```

Para verificar que obtenemos 404 (No encontrado) como código de respuesta HTTP (cuando intentamos buscar un producto que no existe), se ve de la siguiente manera:

```
# Verificar que se devuelva un error 404 (No encontrado) para un productId inexistente (13)
```

```
assertCurl 404 "curl http://$HOST:$PORT/producto-compuesto/13 -s"
```

El script de prueba, `test-em-all.bash`, implementa las pruebas manuales descritas en la sección "Probar las API manualmente" y se encuentra en la carpeta principal `$BOOK_HOME/Chapter03/2-basic-rest-services`. Ampliaremos la funcionalidad del script de prueba a medida que agreguemos más funciones al sistema en capítulos posteriores.



En el Capítulo 20, Monitoreo de Microservicios, aprenderemos técnicas complementarias para supervisar automáticamente el entorno de un sistema en funcionamiento. Aquí, aprenderemos sobre una herramienta de monitoreo que monitorea continuamente el estado de los microservicios implementados y cómo se pueden generar alarmas si las métricas recopiladas superan los umbrales configurados, como el uso excesivo de CPU o memoria.

Probando el script de prueba

Para probar el script de prueba, realice los siguientes pasos:

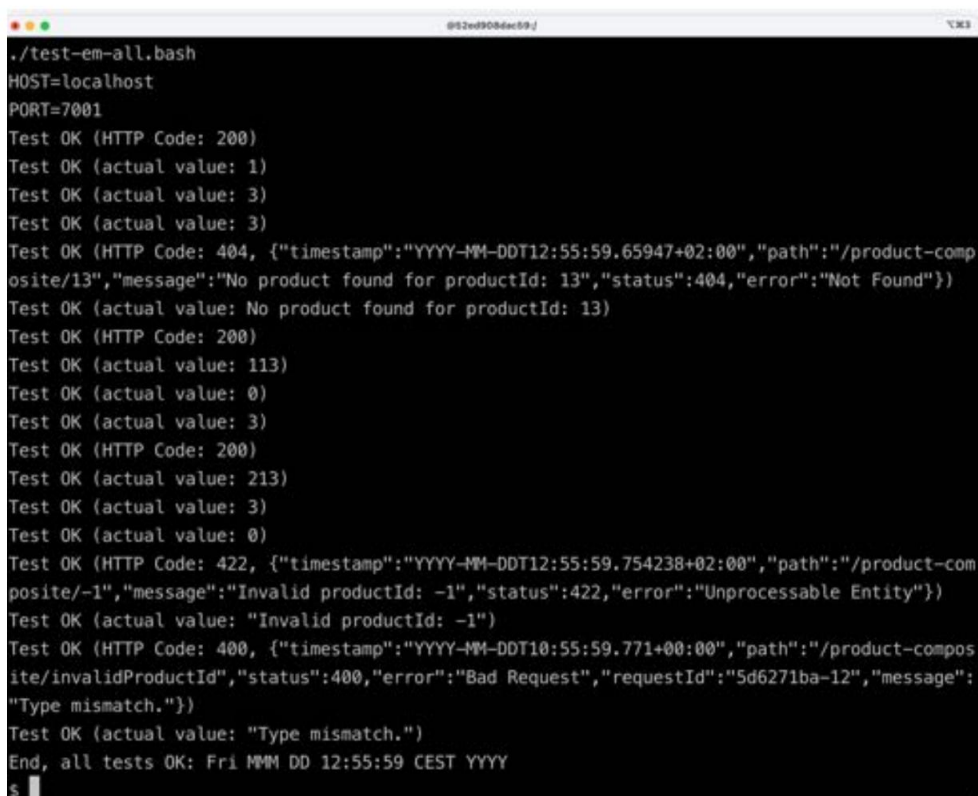
1. Primero, inicie los microservicios, como hicimos anteriormente:

```
cd $BOOK_HOME/Capítulo03/2-servicios-de-descanso-básicos
java -jar microservicios/producto-servicio-compuesto/build/libs/*.jar &
java -jar microservicios/producto-servicio/build/libs/*.jar &
java -jar microservicios/servicio-de-recomendación/build/libs/*.jar &
java -jar microservicios/servicio-de-revisión/build/libs/*.jar &
```

2. Una vez que todos se hayan iniciado, ejecute el script de prueba:

```
./pruébalos-a-todos.bash
```

Espere que el resultado sea similar al siguiente:

A terminal window with a dark background and light text. The window title is '@52ed908dac59-j'. The output shows a series of test results for a script named 'test-em-all.bash'. The tests check for HTTP status codes and JSON responses. The final line indicates that all tests passed.

```
./test-em-all.bash
HOST=localhost
PORT=7001
Test OK (HTTP Code: 200)
Test OK (actual value: 1)
Test OK (actual value: 3)
Test OK (actual value: 3)
Test OK (HTTP Code: 404, {"timestamp":"YYYY-MM-DDT12:55:59.65947+02:00","path":"/product-composite/13","message":"No product found for productId: 13","status":404,"error":"Not Found"})
Test OK (actual value: No product found for productId: 13)
Test OK (HTTP Code: 200)
Test OK (actual value: 113)
Test OK (actual value: 0)
Test OK (actual value: 3)
Test OK (HTTP Code: 200)
Test OK (actual value: 213)
Test OK (actual value: 3)
Test OK (actual value: 0)
Test OK (HTTP Code: 422, {"timestamp":"YYYY-MM-DDT12:55:59.754238+02:00","path":"/product-composite/~1","message":"Invalid productId: -1","status":422,"error":"Unprocessable Entity"})
Test OK (actual value: "Invalid productId: -1")
Test OK (HTTP Code: 400, {"timestamp":"YYYY-MM-DDT10:55:59.771+00:00","path":"/product-composite/invalidProductId","status":400,"error":"Bad Request","requestId":"5d6271ba-12","message":"Type mismatch."})
Test OK (actual value: "Type mismatch.")
End, all tests OK: Fri MMM DD 12:55:59 CEST YYYY
$
```

Figura 3.10: Salida después de ejecutar el script de prueba

3. Termine esto apagando los microservicios con el siguiente comando:

```
matar $(trabajos -p)
```

En esta sección, hemos dado los primeros pasos hacia la automatización de las pruebas para un panorama de sistemas de microservicios cooperativos, todo lo cual se mejorará en los próximos capítulos.

Resumen

Ya hemos creado nuestros primeros microservicios con Spring Boot. Tras conocer el entorno de microservicios que utilizaremos a lo largo de este libro, aprendimos a usar Spring Initializr para crear proyectos esqueleto para cada microservicio.

A continuación, aprendimos a añadir API mediante Spring WebFlux para los tres servicios principales e implementamos un servicio compuesto que utiliza las API de estos tres servicios para crear una vista agregada de su información. El servicio compuesto utiliza la clase `RestTemplate` de Spring Framework para realizar solicitudes HTTP a las API expuestas por los servicios principales. Tras añadir lógica para la gestión de errores a los servicios, realizamos pruebas manuales en el entorno de microservicios.

Concluimos este capítulo aprendiendo a agregar pruebas para microservicios de forma aislada y cuando funcionan juntos como un entorno de sistema. Para proporcionar un aislamiento controlado para el servicio compuesto, simulamos sus dependencias con los servicios principales usando Mockito. Las pruebas de todo el entorno de sistema se realizan mediante un script de Bash que usa curl para realizar llamadas a la API del servicio compuesto.

Con estas habilidades, estamos listos para dar el siguiente paso: ¡adentrarnos en el mundo de Docker y los contenedores en el próximo capítulo! Entre otras cosas, aprenderemos a usar Docker para automatizar completamente las pruebas de un entorno de sistemas de microservicios cooperativos.

Preguntas

1. ¿Cuál es el comando que enumera las dependencias disponibles cuando se crea un nuevo Spring?
¿Arrancar el proyecto usando la herramienta CLI Spring Initializr de Spring init ?
2. ¿Cómo puedes configurar Gradle para crear múltiples proyectos relacionados con un solo comando?
3. ¿ Para qué se utilizan las anotaciones `@PathVariable` y `@RequestParam` ?
4. ¿Cómo se puede separar el manejo de errores específicos del protocolo de la lógica empresarial en una API?
¿clase de implementación?
5. ¿Para qué se utiliza Mockito?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



4

Implementando nuestros microservicios Usando Docker

En este capítulo, comenzaremos a usar Docker y colocaremos nuestros microservicios en contenedores.

Al finalizar este capítulo, habremos ejecutado pruebas totalmente automatizadas de nuestro entorno de microservicios , que inician todos nuestros microservicios como contenedores Docker, sin requerir más infraestructura que un motor Docker. También habremos realizado varias pruebas para verificar que los microservicios funcionen correctamente y, finalmente, los habremos apagado, sin dejar rastros de las pruebas.

Nosotros ejecutamos.

Poder probar varios microservicios que cooperan de esta manera es muy útil. Como desarrolladores, podemos verificar que los microservicios funcionen en nuestras máquinas locales. También podemos ejecutar exactamente las mismas pruebas en un servidor de compilación para verificar automáticamente que los cambios en el código fuente no afecten las pruebas a nivel de sistema. Además, no necesitamos una infraestructura dedicada para ejecutar este tipo de pruebas. En los próximos capítulos, veremos cómo podemos agregar bases de datos y gestores de colas a nuestro entorno de pruebas, todos los cuales se ejecutarán como contenedores Docker.



Sin embargo, esto no reemplaza la necesidad de pruebas unitarias y de integración automatizadas, que evalúan microservicios individuales de forma aislada. Estas pruebas son tan importantes como siempre.

Para uso en producción, como mencionamos anteriormente en este libro, necesitamos un orquestador de contenedores como Kubernetes. Volveremos a hablar de los orquestadores de contenedores y Kubernetes más adelante.

En este capítulo se tratarán los siguientes temas:

- Introducción a Docker
- Docker y Java: históricamente, Java no ha sido muy amigable con los contenedores, pero eso cambió. con Java 10, así que veamos cómo encajan Docker y Java
- Uso de Docker con un microservicio
- Gestión de un paisaje de microservicios mediante Docker Compose
- Automatizar pruebas de microservicios cooperativos

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte los siguientes capítulos:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter04`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, el proceso para añadir compatibilidad con Docker, puede compararlo con el código fuente del Capítulo 3, "Creación de un conjunto de microservicios cooperativos". Puede usar su herramienta de comparación preferida y comparar las dos carpetas: `$BOOK_HOME/Chapter03/2-basic-rest-services` y `$BOOK_HOME/Chapter04`.

Introducción a Docker

Como ya mencionamos en el Capítulo 2, Introducción a Spring Boot, Docker hizo que el concepto de contenedores como una alternativa liviana a las máquinas virtuales fuera muy popular en 2013. Para recapitular rápidamente: los contenedores en realidad se procesan en un host Linux que usa espacios de nombres de Linux para proporcionar aislamiento entre contenedores, y los grupos de control de Linux (cgroups) se utilizan para limitar la cantidad de CPU y memoria que un contenedor puede consumir.

En comparación con una máquina virtual que utiliza un hipervisor para ejecutar una copia completa del sistema operativo en cada máquina virtual, la sobrecarga en un contenedor es una fracción de la de una máquina virtual. Esto se traduce en tiempos de arranque mucho más rápidos y un espacio ocupado significativamente menor. Sin embargo, los contenedores no se consideran tan seguros como las máquinas virtuales. Observe el siguiente diagrama:

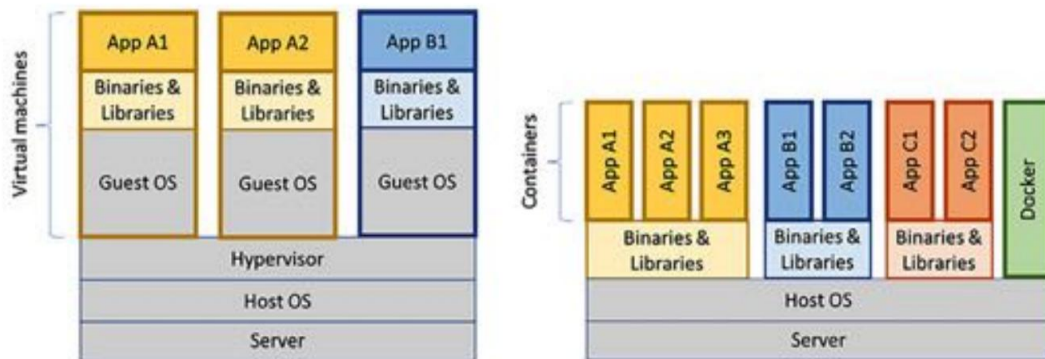


Figura 4.1: Máquinas virtuales versus contenedores

El diagrama ilustra la diferencia entre el uso de recursos de las máquinas virtuales y los contenedores, demostrando que un mismo tipo de servidor puede ejecutar muchos más contenedores que máquinas virtuales. La principal ventaja es que un contenedor no necesita ejecutar su propia instancia de un sistema operativo como sí lo hace una máquina virtual.

Ejecutando nuestros primeros comandos Docker

Intentemos iniciar un contenedor lanzando un servidor Ubuntu usando el comando de ejecución de Docker :

```
docker run -it --rm ubuntu
```

Con el comando anterior, solicitamos a Docker que cree un contenedor que ejecute Ubuntu, basado en la última versión disponible de la imagen oficial de Docker para Ubuntu. La opción `-it` permite interactuar con el contenedor mediante la Terminal, y la opción `--rm` indica a Docker que lo elimine al salir de la Terminal; de lo contrario, permanecerá en el motor de Docker con el estado "Salido" .

La primera vez que usamos una imagen de Docker que no hemos creado nosotros mismos, Docker la descargará desde un registro de Docker, que es Docker Hub de forma predeterminada (<https://hub.docker.com>). Esto tomará algún tiempo, pero para el uso posterior de esa imagen Docker, el contenedor se iniciará en solo unos segundos.

Una vez que se haya descargado la imagen de Docker y se haya iniciado el contenedor, el servidor Ubuntu debería responder con un mensaje como el siguiente:

```
4. root@435dc56d39f6: / (docker)
$ docker run -it --rm ubuntu
root@435dc56d39f6: /#
```

Figura 4.2: Respuesta del servidor Ubuntu

Podemos probar el contenedor, por ejemplo, preguntando qué versión de Ubuntu ejecuta:

```
cat /etc/os-release | grep 'NOMBRE_BONITO'
```

Debería responder con algo como lo siguiente:



```
root@5a9c0bae8632:/# cat /etc/os-release | grep 'PRETTY_NAME'
PRETTY_NAME="Ubuntu 24.04.1 LTS"
root@5a9c0bae8632:/#
```

Figura 4.3: Respuesta de la versión de Ubuntu

Podemos salir del contenedor con el comando "exit" y verificar que el contenedor de Ubuntu ya no existe con el comando "docker ps -a". Necesitamos usar la opción "-a" para ver los contenedores detenidos; de lo contrario, solo se muestran los contenedores en ejecución.

Si prefieres Fedora sobre Ubuntu, no dudes en probar lo mismo con `docker run --rm -it fedora`

Una vez que el servidor Fedora haya comenzado a ejecutarse en su contenedor, puede, por ejemplo, preguntar qué versión de Fedora se está ejecutando con el comando `cat /etc/os-release | grep 'PRETTY_NAME'` Comando. Debería responder con algo como lo siguiente:



```
$ docker run --rm -it fedora
[root@52ed908dac59 /]# cat /etc/os-release | grep 'PRETTY_NAME'
PRETTY_NAME="Fedora Linux 41 (Container Image)"
[root@52ed908dac59 /]#
```

Figura 4.4: Respuesta de la versión de CentOS

Salga del contenedor con el comando de salida para eliminarlo.



Si en algún momento descubres que tienes muchos contenedores no deseados en Docker Motor y desea obtener una hoja limpia, es decir, deshacerse de todos ellos, puede ejecutar el siguiente comando:

```
docker rm -f $(docker ps -aq)
```

El comando `docker rm -f` detiene y elimina los contenedores cuyos ID se especifican en el comando. El comando `docker ps -aq` lista los ID de todos los contenedores, tanto en ejecución como detenidos, en Docker Engine. La opción `-q` reduce la salida del comando `docker ps` para que solo muestre los ID de los contenedores.

Ahora que hemos entendido qué es Docker, podemos pasar a aprender cómo ejecutar Java en Docker.

Ejecución de Java en Docker

En los últimos años, se han realizado varios intentos para que Java funcione correctamente en Docker. Sin embargo, lo más importante es que, históricamente, Java no ha respetado bien los límites establecidos para los contenedores Docker en cuanto al uso de memoria y CPU.

Anteriormente, la imagen oficial de Docker para Java provenía del proyecto OpenJDK : https://hub.docker.com/_/openjdk/. Desde julio de 2022, no se publican imágenes de Docker en este registro. Utilizaremos una imagen de Docker alternativa del proyecto Eclipse Temurin . Esta contiene los mismos binarios del proyecto OpenJDK y ofrece variantes de las imágenes de Docker que se adaptan mejor a nuestras necesidades que las imágenes de Docker del proyecto OpenJDK.

En esta sección, usaremos una imagen de Docker que contiene el JDK (Java Development Kit) completo con todas sus herramientas. Al comenzar a empaquetar nuestros microservicios en imágenes de Docker (en la sección "Uso de Docker con un microservicio") , usaremos una imagen de Docker más compacta basada en el Entorno de Ejecución de Java (JRE), que contiene únicamente las herramientas Java necesarias en tiempo de ejecución.

Como ya se mencionó, las versiones anteriores de Java no respetaban bien las cuotas especificadas para un contenedor Docker que usaba cgroups de Linux; simplemente ignoraban esta configuración. Por lo tanto, en lugar de asignar memoria dentro de la JVM en función de la memoria disponible en el contenedor, Java asignaba memoria como si tuviera acceso a toda la memoria del host Docker. Al intentar asignar más memoria de la permitida, el host cerraba el contenedor Java con un mensaje de error de "memoria insuficiente". De igual manera, Java asignaba recursos relacionados con la CPU, como grupos de subprocesos, en función del número total de núcleos de CPU disponibles en el host Docker, en lugar del número de núcleos de CPU disponibles para el contenedor en el que se ejecutaba la JVM.

En Java SE 9, se proporcionó soporte inicial para restricciones de memoria y CPU basadas en contenedores, muy mejorado en Java SE 10.

¡Veamos cómo responde Java SE 24 a los límites que establecemos en el contenedor en el que se ejecuta!

En las siguientes pruebas, ejecutaremos Docker Engine dentro de una máquina virtual en una MacBook Pro, que actuará como host de Docker. El host de Docker está configurado para usar 8 núcleos de CPU y 16 GB de memoria.

Comenzaremos viendo cómo podemos limitar la cantidad de CPU disponibles para un contenedor que ejecuta Java. Después, haremos lo mismo con la limitación de memoria.

Limitar las CPU disponibles

Comencemos por determinar cuántos procesadores (es decir, núcleos de CPU) disponibles ve Java sin aplicar restricciones.

Podemos hacerlo enviando la instrucción `Java Runtime.getRuntime().`

`availableProcessors()` a la herramienta CLI de Java `jshell`. Ejecutaremos `jshell` en un contenedor usando la imagen de Docker que contiene el JDK de Java 24 completo. La etiqueta de Docker para esta imagen es `eclipse-temurin:24`. El comando se ve así:

```
echo 'Runtime.getRuntime().availableProcessors()' | docker run --rm -i eclipse-temurin:24
jshell -q
```

Este comando enviará la cadena `Runtime.getRuntime().availableProcessors()` al contenedor Docker, que la procesará mediante `jshell`. Obtendremos la siguiente respuesta:

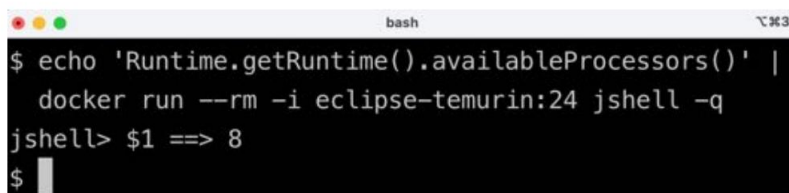
A screenshot of a terminal window with a dark background. The window title is 'bash'. The prompt is '\$'. The user enters the command 'echo 'Runtime.getRuntime().availableProcessors()' | docker run --rm -i eclipse-temurin:24 jshell -q'. The prompt changes to 'jshell>' and the output is '\$1 ==> 8'. The prompt returns to '\$'.

Figura 4.5: Respuesta que muestra la cantidad de núcleos de CPU disponibles

La respuesta de 8 núcleos es la esperada ya que el host Docker se configuró para usar 8 núcleos de CPU. Continuemos y restrinjamus el contenedor Docker para que solo pueda usar tres núcleos de CPU mediante la opción Docker `--cpus 3`, luego preguntemos a la JVM cuántos procesadores disponibles ve:

```
echo 'Runtime.getRuntime().availableProcessors()' | docker run --rm -i --cpus=3 eclipse-temurin:24
jshell -q
```

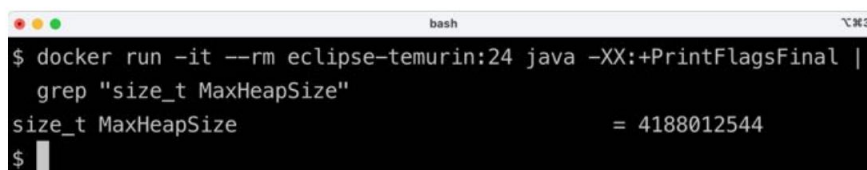
La JVM ahora responde con `$1 ==> 3`; es decir, Java SE 24 respeta las configuraciones en el contenedor y, por lo tanto, podrá configurar correctamente los recursos relacionados con la CPU, como los grupos de subprocesos.

Limitar la memoria disponible

En cuanto a la cantidad de memoria disponible, solicitemos a la JVM el tamaño máximo que cree que puede asignar al montón. Podemos lograrlo solicitando a la JVM información adicional sobre el tiempo de ejecución mediante la opción de Java `-XX:+PrintFlagsFinal` y luego usando el comando `grep` para filtrar el parámetro `MaxHeapSize`, de la siguiente manera:

```
docker run -it --rm eclipse-temurin:24 java -XX:+PrintFlagsFinal | grep "size_t
TamañoMáximoDeMontón"
```

Con 16 GB de memoria asignados al host Docker, obtendremos la siguiente respuesta:



```
bash
$ docker run -it --rm eclipse-temurin:24 java -XX:+PrintFlagsFinal |
  grep "size_t MaxHeapSize"
size_t MaxHeapSize                = 4188012544
$
```

Figura 4.6: Respuesta que muestra MaxHeapSize

Sin restricciones de memoria de la JVM (es decir, sin usar el parámetro `-Xmx` de la JVM), Java asignará una cuarta parte de la memoria disponible para el contenedor a su montón. Por lo tanto, esperamos que asigne hasta 4 GB a su montón. En la captura de pantalla anterior, podemos ver que la respuesta fue de 4 188 012 544 bytes. Esto equivale a $4\,188\,012\,544 / 1024 / 1024 = 3994$ MB, cerca de los 4 GB esperados.

Si limitamos el contenedor Docker a un máximo de 1 GB de memoria mediante la opción `-m=1024M`, esperamos una asignación de memoria máxima menor. Ejecute el siguiente comando:

```
docker run -it --rm -m=1024M eclipse-temurin:24 java -XX:+PrintFlagsFinal | grep "size_t
TamañoMáximoDeMontón"
```

Esto dará como resultado una respuesta de 268.435.456 bytes, lo que equivale a $268.435.456 / 1024 / 1024 = 256$ MB. 256 MB es un cuarto de 1 GB, por lo que, nuevamente, esto es como se esperaba.

Como siempre, podemos configurar el tamaño máximo del montón en la JVM. Por ejemplo, si queremos que la JVM use 600 MB del total de 1 GB que tenemos para su montón, podemos especificarlo con la opción `-Xmx600m` de la JVM, como se muestra a continuación:

```
docker run -it --rm -m=1024M eclipse-temurin:24 java -Xmx600m
-XX:+PrintFlagsFinal -versión | grep "size_t TamañoMáximoDeMontón"
```

La JVM responderá con 629.145.600 bytes = $629.145.600 / 1024 / 1024 = 600$ MB, nuevamente como se esperaba.

Concluiremos con una prueba de "falta de memoria" para asegurarnos de que esto realmente funciona.

Asignaremos algo de memoria usando `jshell` en una JVM que se ejecuta en un contenedor al que se le ha asignado 1 GB de memoria; es decir, tiene un tamaño de montón máximo de 256 MB.

Primero, intente asignar una matriz de bytes de 100 MB:

```
echo 'nuevo byte[100_000_000]' | docker run -i --rm -m=1024M eclipse-temurin:24
jshell -q
```

El comando responderá con `$1 ==>`, ¡lo que significa que funcionó bien!



Normalmente, jshell imprimirá el valor resultante del comando, pero 100 MB de bytes establecidos en cero es demasiado para imprimir, por lo que no obtenemos nada.

Ahora, intentemos asignar una matriz de bytes que sea más grande que el tamaño máximo del montón, por ejemplo, 500 MB:

```
echo 'nuevo byte[500_000_000]' | docker run -i --rm -m=1024M eclipse-temurin:24  
jshell -q
```

La JVM detecta que no puede realizar la acción, ya que respeta la configuración de memoria máxima del contenedor, y responde inmediatamente con la excepción `java.lang.OutOfMemoryError: Java heap space`. ¡Genial!

En resumen, hemos visto cómo Java respeta la configuración de las CPU disponibles y la memoria de su contenedor.

¡Continuemos y construyamos nuestras primeras imágenes de Docker para uno de los microservicios!

Usando Docker con un microservicio

Ahora que entendemos cómo funciona Java en un contenedor, podemos empezar a usar Docker con uno de nuestros microservicios. Antes de ejecutar nuestro microservicio como contenedor Docker, necesitamos empaquetarlo en una imagen Docker. Para crear una imagen Docker, necesitamos un Dockerfile, así que empezaremos con él.

A continuación, necesitamos una configuración específica de Docker para nuestro microservicio. Dado que un microservicio que se ejecuta en un contenedor está aislado de otros microservicios (tiene su propia dirección IP, nombre de host y puertos), necesita una configuración diferente a la que necesita cuando se ejecuta en el mismo host. con otros microservicios.

Por ejemplo, dado que los demás microservicios ya no se ejecutan en el mismo host, no se producirán conflictos de puertos.

Al ejecutar en Docker, podemos usar el puerto predeterminado 8080 para todos nuestros microservicios sin riesgo de conflictos de puertos. Por otro lado, si necesitamos comunicarnos con los demás microservicios, ya no podemos usar localhost como antes.



El código fuente de los microservicios no se verá afectado por la ejecución de los microservicios en contenedores, ¡solo su configuración!

Para gestionar las diferentes configuraciones necesarias al ejecutar localmente sin Docker y al ejecutar los microservicios como contenedores, utilizaremos perfiles de Spring. Desde el capítulo 3, "Creación de un conjunto de microservicios cooperativos", hemos utilizado el perfil de Spring predeterminado para la ejecución local sin Docker.

Ahora, crearemos un nuevo perfil de Spring llamado Docker para ejecutar nuestros microservicios como contenedores en Docker.

Cambios en el código fuente

Comenzaremos con el microservicio del producto , que se puede encontrar en el código fuente en \$BOOK_HOME/Capítulo 04/microservicios/producto-servicio/. En la siguiente sección, aplicaremos esto a También otros microservicios.

Primero, agregamos el perfil Spring para Docker al final del archivo de propiedades, application.yml:

```
spring.config.activate.on-profile: docker
```

```
puerto del servidor: 8080
```



Los perfiles de Spring permiten especificar la configuración específica del entorno, que en este caso solo se utiliza al ejecutar el microservicio en un contenedor Docker. Otros ejemplos son las configuraciones específicas de los entornos de desarrollo, pruebas y producción . Los valores de un perfil anulan los valores del perfil predeterminado. Mediante archivos YAML, se pueden colocar varios perfiles de Spring en el mismo archivo, separados por "--".

El único parámetro que cambiamos por ahora es el puerto que se está utilizando; usaremos el puerto predeterminado 8080 cuando ejecutemos el microservicio en un contenedor.

A continuación, crearemos el Dockerfile que usaremos para construir la imagen de Docker. Como se mencionó en el Capítulo 2, Introducción a Spring Boot, un Dockerfile puede ser tan sencillo como esto:

```
DESDE openjdk:24
```

```
EXPONER 8080
```

```
AGREGAR ./build/libs/*.jar app.jar
```

```
PUNTO DE ENTRADA ["java", "-jar", "/app.jar"]
```

Algunas cosas a tener en cuenta son las siguientes:

- Las imágenes de Docker se basarán en la imagen oficial de Docker para OpenJDK y usarán versión 24
- El puerto 8080 estará expuesto a otros contenedores Docker

- El archivo JAR fat se agregará a la imagen de Docker desde la biblioteca de compilación de Gradle, build/libs
- El comando utilizado por Docker para iniciar un contenedor basado en esta imagen de Docker es `java -jar /app.jar`

Este enfoque simple tiene algunas desventajas:

- Estamos utilizando el JDK completo de Java SE 24, incluidos compiladores y otras herramientas de desarrollo.
Esto hace que las imágenes de Docker sean innecesariamente grandes y, desde una perspectiva de seguridad, no queremos incorporar más herramientas de las necesarias. Por lo tanto, preferimos usar una imagen base para Java SE 24 JRE que solo contenga los programas y bibliotecas necesarios para ejecutar un programa Java. Lamentablemente, el proyecto OpenJDK no proporciona una imagen de Docker para Java SE 24 JRE.
- El archivo JAR pesado tarda en descomprimirse cuando se inicia el contenedor Docker. Una mejor
El enfoque es descomprimir el archivo JAR cuando se crea la imagen de Docker.

El archivo JAR es muy grande; como veremos en breve, unos 20 MB. Si queremos realizar cambios repetibles en el código de la aplicación en las imágenes de Docker durante el desarrollo, esto resultará en un uso subóptimo del comando de compilación de Docker. Dado que las imágenes de Docker se construyen en capas, tendremos una capa muy grande que deberá reemplazarse cada vez, incluso si solo se modifica una clase Java en el código de la aplicación.

Un mejor enfoque consiste en dividir el contenido en diferentes capas, donde los archivos que no cambian con frecuencia se añaden en la primera capa y los que cambian con mayor frecuencia se colocan en la última. Esto permitirá un mejor uso del mecanismo de caché de Docker para las capas. Para las primeras capas estables que no se modifican al modificarse el código de la aplicación, Docker simplemente usará la caché en lugar de reconstruirlas. Esto resultará en compilaciones más rápidas de las imágenes de Docker de los microservicios.

En cuanto a la falta de una imagen de Docker para Java SE 24 JRE del proyecto OpenJDK, existen otros proyectos de código abierto que empaquetan los binarios de OpenJDK en imágenes de Docker. Uno de los proyectos más utilizados es Eclipse Temurin (<https://adoptium.net/temurin/>). El proyecto Temurin proporciona ediciones JDK completas y ediciones JRE minimizadas de sus imágenes Docker.

En cuanto al manejo del empaquetado deficiente de archivos JAR pesados en imágenes de Docker, Spring Boot solucionó este problema en la versión 2.3.0, lo que permite extraer el contenido de un archivo JAR pesado en varias carpetas. Por defecto, Spring Boot crea las siguientes carpetas tras extraer un archivo JAR pesado:

- `dependencies`, que contienen todas las dependencias como archivos JAR
- `spring-boot-loader`, que contiene clases Spring Boot que saben cómo iniciar un Spring

Aplicación de arranque

- dependencias de instantáneas, que contienen dependencias de instantáneas, si las hay
- aplicación, que contiene archivos de clase de aplicación y recursos

La documentación de Spring Boot recomienda crear una capa de Docker para cada carpeta en el orden indicado.

Tras reemplazar la imagen de Docker basada en JDK por una basada en JRE y añadir instrucciones para expandir el archivo JAR en las capas adecuadas de la imagen de Docker, el Dockerfile se ve así:

```
DESDE eclipse-temurin:24_36-jre-noble AS constructor
WORKDIR extraído
AGREGAR ./build/libs/*.jar app.jar
EJECUTAR java -Djarmode=layertools -jar app.jar extract

DESDE eclipse-temurin:24_36-jre-noble
Aplicación WORKDIR
COPIA --from=builder extraído/dependencias/ ./
COPIA --from=builder extraído/spring-boot-loader/ ./
COPIA --from=builder extraído/instantánea-dependencias/ ./
COPIA --from=builder extraído/aplicación/ ./
EXPONER 8080

PUNTO DE ENTRADA ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Para gestionar la extracción del archivo JAR pesado del Dockerfile, utilizamos una compilación multitapa. Esto significa que hay un primer paso, llamado builder, que gestiona la extracción. La segunda etapa compila la imagen de Docker que se usará en tiempo de ejecución, seleccionando los archivos necesarios de la primera etapa. Con esta técnica, podemos gestionar toda la lógica de empaquetado del Dockerfile y, al mismo tiempo, minimizar el tamaño de la imagen de Docker final.

- La primera etapa comienza con esta línea:

```
DESDE eclipse-temurin:24_36-jre-noble AS constructor
```

En esta línea, podemos ver que se utiliza una imagen de Docker del proyecto Temurin, que contiene Java SE JRE para las versiones 24 y 36. También podemos ver que la etapa se llama builder.

- La etapa de creación establece el directorio de trabajo para extraer y agrega el archivo JAR fat desde la biblioteca de compilación de Gradle, build/libs, a esa carpeta.
- Luego, la etapa del constructor ejecuta el extracto java -Djarmode=layertools -jar app.jar comando, que realizará la extracción del archivo JAR fat en su directorio de trabajo, la carpeta extraída .

- La siguiente y última etapa comienza con esta línea:

```
DESDE eclipse-temurin:24_36-jre-noble
```

- Utiliza la misma imagen base de Docker que en la primera etapa y la carpeta de la aplicación como directorio de trabajo. Copia los archivos expandidos de la etapa de compilación , carpeta por carpeta, a la carpeta de la aplicación . Esto crea una capa por carpeta, como se acaba de describir. El parámetro --from=builder se utiliza para indicar a Docker que seleccione los archivos del sistema de archivos en la etapa de compilación .
- Después de exponer los puertos adecuados (8080 en este caso), el Dockerfile finaliza diciendo
Docker qué clase Java ejecutar para iniciar el microservicio en formato expandido, es decir, org.springframework.boot.loader.launch.JarLauncher.

Después de conocer los cambios necesarios en el código fuente, estamos listos para construir nuestra primera imagen de Docker.

Construyendo una imagen de Docker

Para crear la imagen de Docker, primero debemos crear nuestro artefacto de implementación (es decir, el archivo JAR pesado) para el producto-servicio:

```
cd $BOOK_HOME/Capítulo04  
./gradlew :microservicios:servicio-de-producto:compilación
```



Dado que solo queremos compilar el producto-servicio y los proyectos de los que depende (los proyectos API y Util), no usamos el comando de compilación habitual, que compila todos los microservicios. En su lugar, usamos una variante que indica a Gradle que compile solo el proyecto del producto-servicio : :microservices:product-service:build.

Podemos encontrar el archivo JAR fat en la biblioteca de compilación de Gradle, build/libs. El comando ls -l microservices/product-service/build/libs informará algo como lo siguiente:

```
bash
$ ls -l microservices/product-service/build/libs
total 46472
-rw-r--r-- 1 magnus  staff  23791030 Mar 21 09:41 product-service-1.0.0-SNAPSHOT.jar
$
```

Figura 4.7: Visualización de los detalles del archivo JAR fat



Como puedes ver, el archivo JAR tiene un tamaño cercano a los 20 MB, ¡no es de extrañar que se les llame archivos JAR pesados!

Si tiene curiosidad sobre su contenido real, puede verlo usando `unzip -l microservices/product-service/build/libs/product-service-1.0.0-SNAPSHOT.jar`.

A continuación, crearemos la imagen de Docker y la llamaremos `producto-servicio`, de la siguiente manera:

```
cd microservicios/producto-servicio
docker build -t servicio-producto.
```

Docker usará el `Dockerfile` del directorio actual para crear la imagen de Docker. Esta imagen se etiquetará con el nombre "product-service" y se almacenará localmente en Docker Engine.

Verifique que obtuvimos una imagen de Docker, como se esperaba, usando el siguiente comando:

```
imágenes de Docker | grep producto-servicio
```

El resultado esperado es el siguiente:

```
bash
$ docker images | grep product-service
product-service  latest  1896ce04468b  6 seconds ago  289MB
$
```

Figura 4.8: Verificando que construimos nuestra imagen de Docker

Entonces, ahora que hemos creado la imagen, veamos cómo podemos iniciar el servicio.

Puesta en marcha del servicio

Inicie el microservicio del producto como un contenedor usando el siguiente comando:

```
docker run --rm -p8080:8080 -e "SPRING_PROFILES_ACTIVE=docker" producto-servicio
```

Esto es lo que podemos inferir del comando:

1. `docker run`: El comando `docker run` iniciará el contenedor y mostrará el registro en la terminal. La terminal permanecerá bloqueada mientras el contenedor se esté ejecutando.
2. Ya hemos visto la opción `--rm`; le indicará a Docker que limpie el contenedor una vez que detengamos la ejecución desde la Terminal usando `Ctrl + C`.

- El resultado esperado es el siguiente:

Figura 4.9: Salida después de iniciar el microservicio del producto

El perfil que utiliza Spring es Docker. Busque " Los siguientes perfiles están activos: Docker" en la salida para verificarlo.

- El puerto asignado por el contenedor es 8080. Busque Netty iniciado en el puerto 8080 en la salida para verificar esto.
- El microservicio está listo para aceptar solicitudes una vez que se inicia el mensaje de registro. ¡ProductServiceApplication ha sido escrita!

Podemos usar el puerto 8080 del host local para comunicarnos con el microservicio, como se explicó anteriormente. Pruebe el siguiente comando en otra ventana de terminal:

```
curl localhost:8080/producto/3
```

El siguiente es el resultado esperado:



```
bash
$ curl localhost:8080/producto/3
{"productId":3,"name":"name-3","weight":123,"serviceAddress":"f308fea277a4/172.17.0.2:8080"}$
```

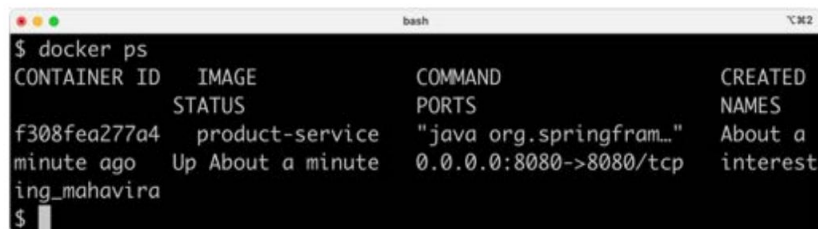
Figura 4.10: Solicitud de información sobre el producto 3

Esto es similar al resultado que recibimos del capítulo anterior, pero con una diferencia importante: ahora tenemos el contenido de "service Address": "9dc086e4a88b/172.17.0.2:8080", el puerto es 8080, como se esperaba, y la dirección IP, 172.17.0.2, es la dirección IP que se ha asignado al contenedor desde una red interna en Docker, pero ¿de dónde vino el nombre de host, 9dc086e4a88b? ¿venir de?

Solicita a Docker todos los contenedores en ejecución:

```
Docker PS
```

Veremos algo como lo siguiente:



```
bash
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS      PORTS
f308fea277a4   product-service "java org.springfram..." About a minute Up About a minute 0.0.0.0:8080->8080/tcp
ing_mahavira
$
```

Figura 4.11: Todos los contenedores en ejecución

Como podemos ver en el resultado anterior, el nombre de host es equivalente al ID del contenedor, lo cual es bueno saber si desea comprender qué contenedor respondió realmente a su solicitud.

Termine esto deteniendo el contenedor en la Terminal con el comando Ctrl + C. Una vez hecho esto, ahora podemos pasar a ejecutar el contenedor separado de la terminal.

Ejecutar el contenedor en modo separado

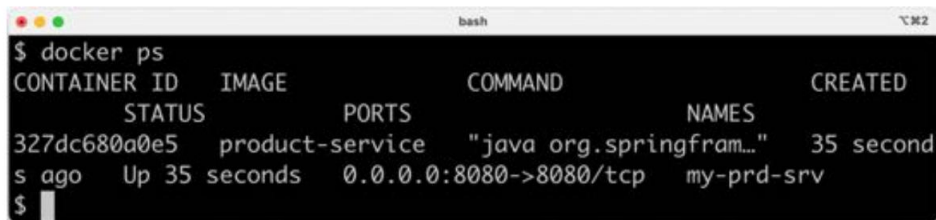
Genial, pero ¿qué pasa si no queremos bloquear la terminal desde donde iniciamos el contenedor? En la mayoría de los casos, es incómodo tener una sesión de terminal bloqueada para cada contenedor en ejecución. Es hora de aprender a iniciar el contenedor de forma independiente : ¡ejecutarlo sin bloquear la terminal!

Podemos hacer esto agregando la opción `-d` y, al mismo tiempo, dándole un nombre usando `--name`

Opción. Asignarle un nombre es opcional, y Docker generará uno si no lo hacemos, pero facilita el envío de comandos al contenedor desconectado con un nombre que hayamos decidido. La opción `--rm` ya no es necesaria, ya que detendremos y eliminaremos el contenedor explícitamente al terminar con él:

```
docker run -d -p8080:8080 -e "SPRING_PROFILES_ACTIVE=docker" --name my-prd-srv servicio-producto
```

Si ejecutamos nuevamente el comando `docker ps`, veremos nuestro nuevo contenedor, llamado `my-prd-srv`:



CONTAINER ID	IMAGE	PORTS	COMMAND	NAMES	CREATED
327dc680a0e5	product-service	0.0.0.0:8080->8080/tcp	"java org.springframework..."	my-prd-srv	35 seconds ago

Figura 4.12: Inicio del contenedor como separado

Pero ¿cómo obtenemos la salida del registro de nuestro contenedor?

Conozca el comando `docker logs` :

```
registros de Docker my-prd-srv -f
```

La opción `-f` indica al comando que siga la salida del registro; es decir, que no finalice el comando cuando se haya escrito toda la salida del registro actual en la Terminal, sino que espere más. Si espera muchos mensajes de registro antiguos que no desea ver, también puede agregar la opción `--tail 0` para ver solo los mensajes de registro nuevos. Como alternativa, puede usar la opción `--since` y especificar una marca de tiempo absoluta o relativa, por ejemplo, `--since 5m`, para ver los mensajes de registro con una antigüedad máxima de cinco minutos.

Pruebe esto con una nueva solicitud `curl`. Debería ver que se ha escrito un nuevo mensaje de registro en la salida del registro de la terminal.

Termine esto deteniendo y retirando el contenedor:

```
docker rm -f mi-srv-prd
```

La opción `-f` obliga a Docker a eliminar el contenedor, incluso si está en ejecución. Docker lo detendrá automáticamente antes de eliminarlo.

Ahora que sabemos cómo usar Docker con un microservicio, podemos ver cómo administrar un panorama de microservicios con la ayuda de Docker Compose.

Administrar un paisaje de microservicios mediante Docker Compose

Ya hemos visto cómo podemos ejecutar un único microservicio como un contenedor Docker, pero ¿qué pasa con la gestión de todo un panorama de sistemas de microservicios?

Como mencionamos anteriormente, este es el propósito de Docker Compose. Mediante comandos individuales, podemos compilar, iniciar, registrar y detener un grupo de microservicios cooperativos que se ejecutan como contenedores Docker.

Cambios en el código fuente

Para poder usar Docker Compose, necesitamos crear un archivo de configuración, `docker-compose.yml`, que describe los microservicios que Docker Compose gestionará. También debemos configurar Dockerfiles para los microservicios restantes y agregar un perfil de Spring específico de Docker a cada uno.

Los cuatro microservicios tienen su propio Dockerfile, pero todos tienen el mismo aspecto que el anterior.

En lo que respecta a los perfiles de Spring, los tres servicios principales (producto, recomendación y revisión) tienen el mismo perfil de Docker , que solo especifica que el puerto predeterminado es 8080. Debe utilizarse cuando se ejecuta como contenedor.

Para el servicio compuesto de producto, la situación es un poco más complicada, ya que necesita saber dónde encontrar los servicios principales. Al ejecutar todos los servicios en localhost, se configuró para usar localhost y los números de puerto individuales (7002-7004) para cada servicio principal. Al ejecutarse en Docker, cada servicio tendrá su propio nombre de host, pero será accesible en el mismo puerto (8080).

Aquí, el perfil de Docker para el servicio compuesto del producto se ve de la siguiente manera:

```
spring.config.activate.on-profile: docker
```

```
puerto del servidor: 8080
```



```
aplicación:

producto-servicio:
  anfitrión: producto
  puerto: 8080
servicio de recomendación:
  anfitrión: recomendación
  puerto: 8080
servicio de revisión:
  anfitrión: revisión
  puerto: 8080
```

Esta configuración se almacena en el archivo de propiedades, `application.yml`.

¿De dónde provienen los nombres de host del producto, la recomendación y la revisión ?

Estos se especifican en el archivo `docker-compose.yml` , que se encuentra en `$BOOK_HOME/Chapter04` Carpeta. Se ve así:

```
servicios:

producto:
  construir: microservicios/producto-servicio
  límite de memoria: 512 m
  ambiente:
    - PERFILES_DE_PRIMAVERA_ACTIVOS=docker

recomendación:
  compilación: microservicios/servicio de recomendación
  límite de memoria: 512 m
  ambiente:
    - PERFILES_DE_PRIMAVERA_ACTIVOS=docker

revisar:
  compilación: microservicios/servicio de revisión
  límite de memoria: 512 m
  ambiente:
    - PERFILES_DE_PRIMAVERA_ACTIVOS=docker

producto-compuesto:
  compilación: microservicios/servicio-compuesto-de-producto
```

```
límite de memoria: 512 m
puertos:
  - "8080:8080"
ambiente:
  - PERFILES_DE_PRIMAVERA_ACTIVOS=docker
```

Para cada microservicio, especificamos lo siguiente:

- El nombre del microservicio. Este también será el nombre de host del contenedor en el red interna de Docker.
- Una directiva de compilación que especifica dónde encontrar el Dockerfile que se utilizó para compilar el Imagen de Docker.
- Un límite de memoria de 512 MB. 512 MB deberían ser suficientes para todos nuestros microservicios en el ámbito de este libro. Para este capítulo, podría establecerse en un valor menor, pero a medida que agreguemos más capacidades a los microservicios en los próximos capítulos, sus requisitos de memoria... aumentará.
- Las variables de entorno que se configurarán para el contenedor. En nuestro caso, las usamos para especificar el perfil de Spring que se usará.

Para el servicio compuesto por productos , también especificaremos asignaciones de puertos: expondremos su puerto Para que se pueda acceder desde fuera de Docker. Los demás microservicios no serán accesibles desde el Afuera. A continuación, veremos cómo iniciar un entorno de microservicios.



En el Capítulo 10, Uso de Spring Cloud Gateway para ocultar microservicios detrás de un servidor perimetral, y el Capítulo 11, Cómo proteger el acceso a las API, aprenderemos más sobre cómo bloquear y proteger el acceso externo a un panorama de sistema de microservicios.

Puesta en marcha del panorama de microservicios

Con todos los cambios de código necesarios implementados, podemos crear nuestras imágenes de Docker, iniciar el entorno de microservicios y ejecutar algunas pruebas para verificar su correcto funcionamiento. Para ello, necesitamos hacer lo siguiente:

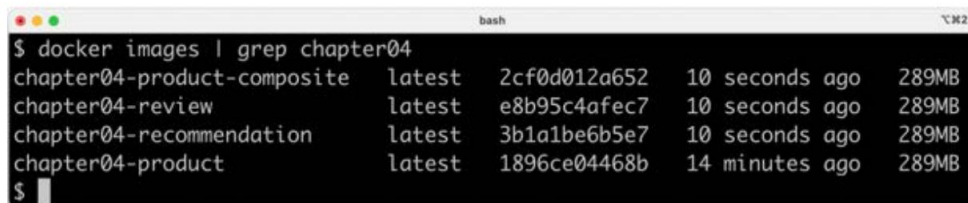
1. Primero, construimos nuestros artefactos de implementación con Gradle y luego las imágenes de Docker con Docker Compose:

```
cd $BOOK_HOME/Capítulo04
./gradlew compilación
compilación de docker compose
```

2. Luego, debemos verificar que podamos ver nuestras imágenes de Docker, de la siguiente manera:

```
Imágenes de Docker | grep capítulo 04
```

3. Deberíamos ver el siguiente resultado:



```
bash
$ docker images | grep chapter04
chapter04-product-composite   latest      2cf0d012a652   10 seconds ago   289MB
chapter04-review              latest      e8b95c4afec7   10 seconds ago   289MB
chapter04-recommendation      latest      3b1a1be6b5e7   10 seconds ago   289MB
chapter04-product             latest      1896ce04468b   14 minutes ago   289MB
$
```

Figura 4.13: Verificación de nuestras imágenes Docker

4. Inicie el entorno de microservicios con el siguiente comando:

```
docker composer -d
```

5. La opción `-d` hará que Docker Compose ejecute los contenedores en modo separado, lo mismo

En cuanto a Docker.

6. Podemos seguir el inicio monitoreando la salida que se escribe en el registro de cada contenedor. con el siguiente comando:

```
registros de composición de Docker -f
```

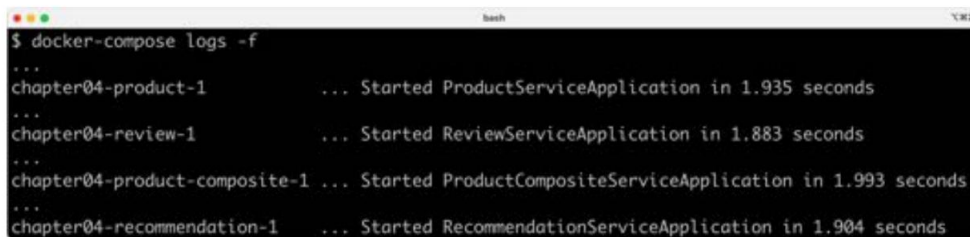
El comando `docker compose logs` admite los mismos `-f` y `--tail`

opciones como registros de Docker, como se describió anteriormente.



El comando `dockter-composelogs` también permite restringir la salida del registro a un grupo de contenedores. Simplemente agregue los nombres de los contenedores cuyos registros desea ver después del comando `logs`. Por ejemplo, para ver solo la salida del registro de los servicios de producto y revisión, use `docker compose logs -f product review`.

7. Cuando los cuatro microservicios hayan informado de su inicio, estaremos listos para probar el entorno de microservicios. Busque lo siguiente:



```
$ docker-compose logs -f
...
chapter04-product-1      ... Started ProductServiceApplication in 1.935 seconds
...
chapter04-review-1      ... Started ReviewServiceApplication in 1.883 seconds
...
chapter04-product-composite-1 ... Started ProductCompositeServiceApplication in 1.993 seconds
...
chapter04-recommendation-1 ... Started RecommendationServiceApplication in 1.904 seconds
```

Figura 4.14: Inicio de los cuatro microservicios

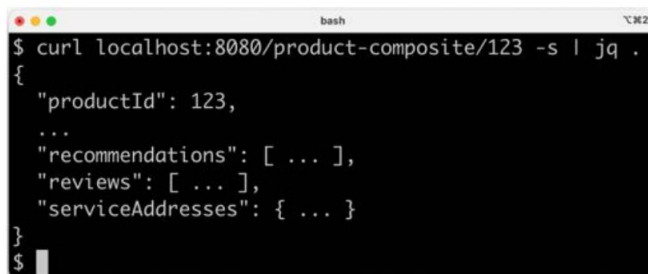


¡Tenga en cuenta que cada mensaje de registro tiene como prefijo el nombre del contenedor que produjo la salida!

Ahora, estamos listos para ejecutar algunas pruebas para verificar que esto funciona correctamente. El número de puerto es el único cambio que debemos realizar al llamar al servicio compuesto en Docker, a diferencia de cuando lo ejecutamos directamente en el host local, como hicimos en el capítulo anterior. Ahora usamos el puerto 8080:

```
curl localhost:8080/producto-compuesto/123 -s | jq .
```

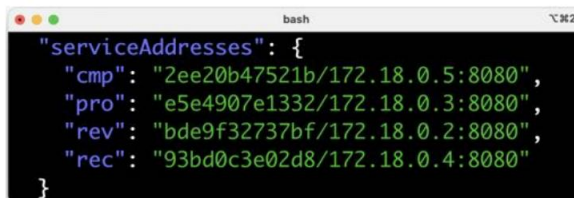
Obtendremos el mismo tipo de respuesta:



```
$ curl localhost:8080/product-composite/123 -s | jq .
{
  "productId": 123,
  ...
  "recommendations": [ ... ],
  "reviews": [ ... ],
  "serviceAddresses": { ... }
}
```

Figura 4.15: Llamada al servicio compuesto

Sin embargo, hay una gran diferencia: los nombres de host y los puertos informados por serviceAddresses en la respuesta:



```
"serviceAddresses": {
  "cmp": "2ee20b47521b/172.18.0.5:8080",
  "pro": "e5e4907e1332/172.18.0.3:8080",
  "rev": "bde9f32737bf/172.18.0.2:8080",
  "rec": "93bd0c3e02d8/172.18.0.4:8080"
}
```

Figura 4.16: Visualización de direcciones de servicio

Aquí podemos ver los nombres de host y las direcciones IP que se han asignado a cada uno de los Docker contenedores.

Hemos terminado; ahora sólo queda un paso:

Docker Compose Down

El comando anterior cerrará el entorno de microservicios. Hasta ahora, hemos visto cómo podemos probar manualmente los microservicios que cooperan ejecutando comandos Bash . En la siguiente sección, veremos cómo podemos mejorar nuestro script de prueba para automatizar estos pasos manuales.

Automatizar pruebas de microservicios cooperativos

Docker Compose es realmente útil cuando se trata de administrar manualmente un grupo de microservicios.

En esta sección, profundizaremos en este tema e integraremos Docker Compose en nuestro script de prueba, test-em-all.bash. Este script iniciará automáticamente el entorno de microservicios, ejecutará todas las pruebas necesarias para verificar su correcto funcionamiento y, finalmente, lo desmantelará sin dejar rastro.

El script de prueba se puede encontrar en \$BOOK_HOME/Chapter04/test-em-all.bash.

Antes de ejecutar el conjunto de pruebas, el script de prueba comprobará la presencia de un argumento de inicio en la invocación del script. Si lo encuentra, reiniciará los contenedores con el siguiente código:

```
si [[ $@ == *"inicio"* ]]
entonces
    echo "Reiniciando el entorno de prueba..."
    echo "$ docker compose down --remove-orphans"
    docker compose down --remove-orphans
    echo "$ docker compose up -d"
    docker composer -d
fi
```

Después de eso, el script de prueba esperará que el servicio compuesto del producto responda con OK:

```
esperarServicio http://$HOST:${PORT}/producto-compuesto/1
```

La función Bash waitForService se implementa de la siguiente manera:

```
función testUrl() {
    url=$@
    si curl $url -ks -f -o /dev/null
entonces
```

```

        devolver 0

    demás
        retorno 1

fi;

} función waitForService() {
    url=$@
    echo -n "Esperar: $url... "
    n=0
    hasta testUrl $url
    hacer
        n=$((n + 1)) si
            [[ $n == 100 ]]
            entonces
                eco "Ríndete"
                salida 1
            demás
                dormir 3
                echo -n ", reintentar #$n "
        fi
    hecho
    echo "HECHO, continúa..."
}

```

La función `waitForService` envía solicitudes HTTP a la URL proporcionada mediante `curl`. Las solicitudes se envían repetidamente hasta que `curl` indica que la solicitud ha sido respondida correctamente. La función espera 3 segundos entre cada intento y se da por vencida después de 100 intentos, deteniendo el script con una falla.

A continuación, todas las pruebas se ejecutan como antes. Después, el script dismantelará el entorno si encuentra el argumento de parada en los parámetros de invocación:

```

si [[ $@ == "stop"* ]]
    entonces
        echo "Hemos terminado, deteniendo el entorno de prueba..." echo "$ docker
        compose down"
        Docker Compose Down
    fi

```



Tenga en cuenta que el script de prueba no destruirá el paisaje si algunas pruebas fallan; simplemente se detendrá, dejando el paisaje listo para el análisis de errores.

El script de prueba también cambió el puerto predeterminado de 7001, que usamos cuando ejecutamos los microservicios sin Docker, a 8080, que usan nuestros contenedores Docker.

¡A probarlo! Para iniciar el paisaje, ejecutar las pruebas y desmontarlo después, ejecuta este comando:

```
./test-em-all.bash inicio parada
```

A continuación se muestra un ejemplo de resultado de una ejecución de prueba centrada en las fases de inicio y apagado. Se han eliminado los resultados de las pruebas reales (son los mismos que en el capítulo anterior):

```
@52ed908dac59 /
$ ./test-em-all.bash start stop
Start Tests: Wed MMM DD 17:57:28 CEST YYYY
HOST=localhost
PORT=8080
Restarting the test environment...
$ docker-compose down --remove-orphans
$ docker-compose up -d
[+] Running 5/5
  :: Network chapter04_default          C...    0.0s
  :: Container chapter04-recommendation-1 Started 0.4s
  :: Container chapter04-product-composite-1 Started 0.4s
  :: Container chapter04-review-1       Started 0.6s
  :: Container chapter04-product-1      Started 0.6s
Wait for: curl http://localhost:8080/product-composite/1... ,
  retry #1 DONE, continues...
...Test OK...
We are done, stopping the test environment...
$ docker-compose down
[+] Running 5/5
  :: Container chapter04-review-1       Removed 2.4s
  :: Container chapter04-product-1      Removed 2.2s
  :: Container chapter04-product-composite-1 Removed 2.4s
  :: Container chapter04-recommendation-1 Removed 2.4s
  :: Network chapter04_default          R...    0.1s
End, all tests OK: Wed MMM DD 17:57:37 CEST YYYY
$
```

Figura 4.17: Salida de muestra de una ejecución de prueba

Después de ejecutar estas pruebas, podemos pasar a ver cómo solucionar problemas en las pruebas que fallan.

Solución de problemas de una ejecución de prueba

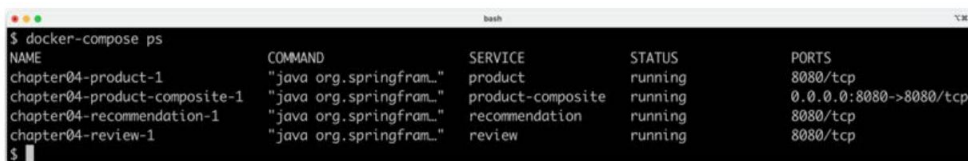
Si las pruebas que se estaban ejecutando `./test-em-all.bash start stop` fallan, seguir estos pasos puede ayudarle a identificar el problema y reanudar las pruebas una vez que se haya solucionado el problema:

1. Primero, verifique el estado de los microservicios en ejecución con el siguiente comando:

```
docker compose ps
```

2. Si todos los microservicios están en funcionamiento y en buen estado, recibirá lo siguiente

producción:



NAME	COMMAND	SERVICE	STATUS	PORTS
chapter04-product-1	"java org.springframework..."	product	running	8080/tcp
chapter04-product-composite-1	"java org.springframework..."	product-composite	running	0.0.0.0:8080->8080/tcp
chapter04-recommendation-1	"java org.springframework..."	recommendation	running	8080/tcp
chapter04-review-1	"java org.springframework..."	review	running	8080/tcp

Figura 4.18: Comprobación del estado de los microservicios en ejecución

3. Si alguno de los microservicios no tiene el estado "Activo", revise su registro para detectar errores con el comando `docker compose logs`. Por ejemplo, si desea revisar el registro del servicio del producto, utilice el siguiente comando:

```
producto de registros de Docker Compose
```

4. En esta etapa, no es fácil generar un registro de errores, dado que los microservicios son muy simples. En su lugar, a continuación se muestra un ejemplo de registro de errores del microservicio de producto del Capítulo 6, "Añadir persistencia". Supongamos que se encuentra lo siguiente en su salida de registro:



```
$ docker-compose logs product
...
chapter06-product-1 | ... INFO 1 --- [{}-mongodb:27017] org.mongodb.driver.cluster
                    : Exception in monitor thread while connecting to server mongodb:27017
chapter06-product-1 | com.mongodb.MongoSocketException: mongodb: Name or service not known
...
$
```

Figura 4.19: Información de error de muestra en la salida del registro

5. Al leer la salida del registro anterior, es evidente que el microservicio del producto no puede acceder a su base de datos MongoDB. Dado que la base de datos también se ejecuta como un contenedor Docker administrado por el mismo archivo Docker Compose, se puede usar el comando `docker compose logs` para identificar el problema.

6. Si es necesario, puede reiniciar un contenedor fallido con el comando `docker compose restart`. Por ejemplo, usaría el siguiente comando si quisiera reiniciar el contenedor.

microservicio de producto :

```
reiniciar el producto de Docker Compose
```

7. Si falta un contenedor, por ejemplo, debido a un fallo, se inicia con el comando `docker compose up -d --scale`. Por ejemplo, se usaría el siguiente comando para el microservicio de producto :

```
docker composer up -d --scale producto=1
```

8. Si los errores en la salida del registro indican que Docker se está quedando sin espacio en disco, partes de él pueden se puede recuperar con el siguiente comando:

```
sistema docker prune -f --volumes
```

9. Una vez que todos los microservicios estén en funcionamiento y en buen estado, ejecute el script de prueba nuevamente, pero sin iniciar los microservicios:

```
./pruébalos-a-todos.bash
```

10. ¡Las pruebas ahora deberían ejecutarse bien!

Cuando haya terminado con las pruebas, recuerde desmontar el panorama del sistema:

```
Docker Compose Down
```



Por último, un consejo sobre un comando combinado que crea artefactos de tiempo de ejecución y Toma imágenes de Docker de la fuente y luego ejecuta todas las pruebas en Docker:

```
./gradlew compilación limpia y compilación de docker compose y ./  
test-em-all.bash inicio parada
```

¡Esto es perfecto si quieres comprobar que todo funciona antes de enviar código nuevo a tu repositorio Git o como parte de una canalización de compilación en tu servidor de compilación!

Resumen

En este capítulo, hemos visto cómo se puede utilizar Docker para simplificar las pruebas de un entorno de microservicios que cooperan.

Aprendimos cómo Java SE, desde la versión 10, respeta las restricciones que imponemos a los contenedores respecto a la cantidad de CPU y memoria que pueden usar. También vimos lo poco que se necesita para ejecutar un microservicio basado en Java como contenedor Docker. Gracias a los perfiles de Spring, podemos ejecutar el microservicio en Docker sin necesidad de modificar el código.

Finalmente, hemos visto cómo Docker Compose puede ayudarnos a gestionar un paisaje de microservicios que cooperan con comandos únicos, ya sea manualmente o, mejor aún, automáticamente, cuando se integra con un script de prueba como `test-em-all.bash`.

En el próximo capítulo, estudiaremos cómo podemos agregar documentación de la API usando OpenAPI/Descripciones de Swagger.

Preguntas

1. ¿Cuáles son las principales diferencias entre una máquina virtual y un contenedor Docker?
2. ¿Cuál es el propósito de los espacios de nombres y cgroups en Docker?
3. ¿Qué sucede con una aplicación Java que no respeta la configuración de memoria máxima?
¿en un contenedor y asigna más memoria de la que le está permitida?
4. ¿Cómo podemos hacer que una aplicación basada en Spring se ejecute como un contenedor Docker sin...
¿Requiere modificaciones de su código fuente?
5. ¿Por qué no funcionará el siguiente fragmento de código de Docker Compose?

revisar:

compilación: microservicios/servicio de revisión

puertos:

- "8080:8080"

ambiente:

- PERFILES_DE_PRIMAVERA_ACTIVOS=docker

producto-compuesto:

compilación: microservicios/servicio-compuesto-de-producto

puertos:

- "8080:8080"

ambiente:

- PERFILES_DE_PRIMAVERA_ACTIVOS=docker

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



5

Agregar una descripción de API Uso de OpenAPI

El valor de una API, como un servicio RESTful, depende en gran medida de su facilidad de uso . Una documentación de calidad y fácil acceso es fundamental para la utilidad de una API.

En este capítulo, aprenderemos cómo podemos usar la especificación OpenAPI para documentar las API que podemos hacer accesibles externamente desde un entorno de microservicios.

Como mencionamos en el Capítulo 2, Introducción a Spring Boot, la Especificación OpenAPI, anteriormente conocida como la especificación Swagger, es una de las más utilizadas para documentar servicios RESTful. Muchas de las principales API Gateways son compatibles de forma nativa con la Especificación OpenAPI. Aprenderemos a usar el proyecto de código abierto springdoc-openapi.

Para generar dicha documentación. También aprenderemos a integrar un visor de documentación de API, Swagger UI Viewer, que puede usarse tanto para inspeccionar la documentación de API como para realizar solicitudes a la API.

Al finalizar este capítulo, contaremos con documentación de API basada en OpenAPI para la API externa expuesta por el microservicio producto-servicio-compuesto . El microservicio también expondrá un visor de interfaz de usuario Swagger que podremos usar para visualizar y probar la API.

En este capítulo se tratarán los siguientes temas:

- Introducción al uso de springdoc-openapi
- Agregar springdoc-openapi al código fuente
- Construcción e inicio del panorama de microservicios
- Probar la documentación de OpenAPI

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Este libro, consulte lo siguiente:

1. Capítulo 21 para macOS
2. Capítulo 22 para Windows

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter05`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver el proceso de creación de la documentación de la API basada en OpenAPI con `springdoc-openapi`, puede compararla con el código fuente del Capítulo 4, "Implementación de nuestros microservicios con Docker". Puede usar su herramienta de comparación preferida y comparar las dos carpetas: `$BOOK_HOME/Chapter04` y `$BOOK_HOME/Chapter05`.

Introducción al uso de `springdoc-openapi`

Usar `springdoc-openapi` permite mantener la documentación de la API junto con el código fuente que la implementa.

Con `springdoc-openapi`, se puede crear la documentación de la API sobre la marcha en tiempo de ejecución, inspeccionando las anotaciones de Java en el código. En mi opinión, esta es una característica importante. Si la documentación de la API se mantiene en un ciclo de vida independiente del código fuente de Java, ambos se separarán con el tiempo. En muchos casos, esto ocurrirá antes de lo esperado (según mi propia experiencia).

Como siempre, es importante separar la interfaz de un componente de su implementación. Al documentar una API RESTful, debemos agregar la documentación de la API a la interfaz Java que la describe, y no a la clase Java que la implementa. Para simplificar la actualización de las partes textuales de la documentación de la API (por ejemplo, descripciones más largas), podemos colocarlas en archivos de propiedades en lugar de directamente en el código Java.

Además de crear la especificación de la API sobre la marcha, `springdoc-openapi` también incluye un visor de API integrado llamado Swagger UI. Configuraremos el servicio `product-composite-service` para que exponga Swagger UI a su API.



Aunque Swagger UI es muy útil durante las fases de desarrollo y prueba, por razones de seguridad, no suele publicarse para las API en entornos de producción. En muchos casos, las API se publican mediante una puerta de enlace de API. Actualmente, la mayoría de los productos de puerta de enlace de API permiten publicar la documentación de la API basándose en un documento OpenAPI. Por lo tanto, en lugar de publicar Swagger UI, la documentación OpenAPI de la API (generada por springdoc-openapi) se exporta a una puerta de enlace de API que puede publicarla de forma segura.

Si se espera que las API sean utilizadas por desarrolladores externos, se puede configurar un portal para desarrolladores que contenga documentación y herramientas, utilizado para el autorregistro, por ejemplo. Swagger UI se puede utilizar en un portal para desarrolladores para permitirles aprender sobre la API leyendo la documentación y también probando las API usando una instancia de prueba.

En el Capítulo 11, "Asegurando el acceso a las API", aprenderemos a bloquear el acceso a las API mediante OAuth 2.1. También aprenderemos a configurar el componente Swagger UI para obtener tokens de acceso de OAuth 2.1 y usarlos cuando el usuario pruebe las API a través de Swagger UI.

La siguiente captura de pantalla es un ejemplo de cómo se ve Swagger UI:

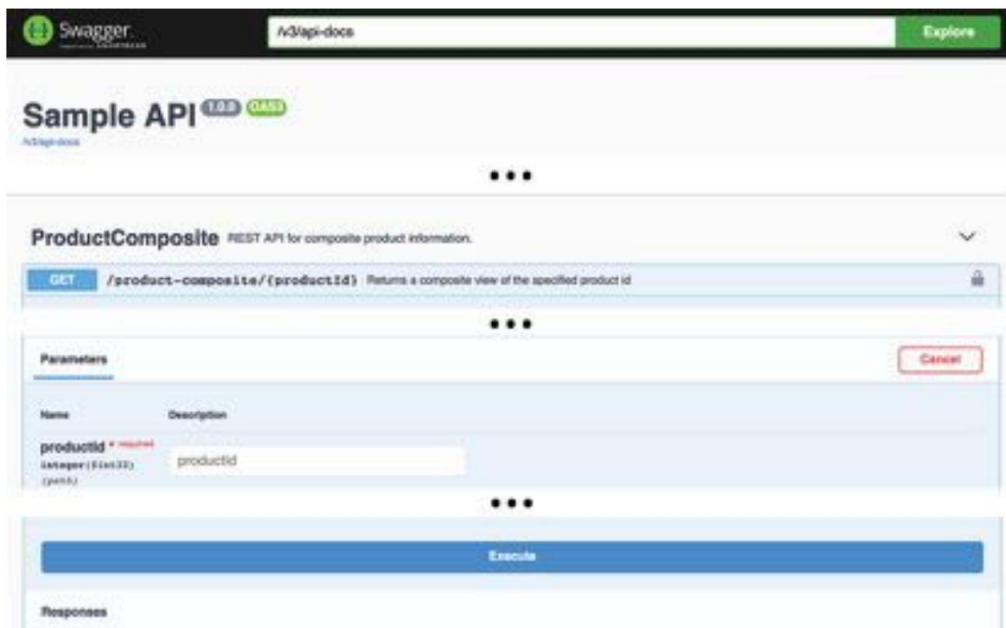


Figura 5.1: Ejemplo de interfaz de usuario de Swagger



Algunas partes de la captura de pantalla, por ahora sin importancia, se han reemplazado por "..." en la figura anterior. Volveremos a estos detalles más adelante en este capítulo.

Para que springdoc-openapi pueda crear la documentación de la API, necesitamos agregar dependencias a nuestros archivos de compilación y anotaciones a las interfaces Java que definen los servicios RESTful. Como se mencionó anteriormente, también colocaremos las partes descriptivas de la documentación de la API en un archivo de propiedades.



Si se han incluido partes de la documentación en archivos de propiedades para simplificar la actualización de la documentación de la API, es importante que estos archivos se gestionen en el mismo ciclo de vida y bajo el mismo control de versiones que el código fuente. De lo contrario, existe el riesgo de que se desvíen de la implementación, es decir, que queden obsoletos.

Con springdoc-openapi presentado, veamos cómo podemos comenzar a usarlo realizando los cambios necesarios en el código fuente.

Añadiendo springdoc-openapi al código fuente

Para agregar documentación basada en OpenAPI con respecto a la API externa expuesta por el microservicio producto -servicio compuesto , necesitamos cambiar el código fuente en dos proyectos:

- **product-composite-service:** Aquí, configuraremos una configuración de springdoc-openapi en la clase de aplicación Java, `ProductCompositeServiceApplication`, y agregue información general relacionada con la API.
- **API:** Aquí, añadiremos anotaciones a la interfaz Java, `ProductCompositeService`, que describen cada servicio RESTful y sus operaciones. En esta etapa, solo contamos con un servicio RESTful con una sola operación: acepta solicitudes HTTP GET a `/product-composite/{productId}`, que se utiliza para solicitar información compuesta sobre un producto específico.

Los textos reales que se utilizan para describir la operación de la API se colocarán en el archivo de propiedades predeterminado, `application.yml`, en el proyecto `product-composite-service` .

Antes de poder usar springdoc-openapi, debemos agregarlo como dependencia en los archivos de compilación de Gradle. ¡Comencemos!

Agregar dependencias a los archivos de compilación de Gradle

El proyecto springdoc-openapi se divide en varios módulos. Para el proyecto API , solo necesitamos el módulo que contiene las anotaciones que usaremos para documentar la API. Podemos agregarlo.

al archivo de compilación del proyecto API , build.gradle, de la siguiente manera:

```
implementación 'org.springdoc:springdoc-openapi-starter-common:2.8.6'
```

El proyecto producto-servicio-compuesto requiere un módulo más completo que incluya el visor de interfaz de usuario Swagger y compatibilidad con Spring WebFlux. Podemos agregar la dependencia al archivo de compilación, build.gradle, de la siguiente manera:

```
Implementación 'org.springdoc:springdoc-openapi-starter-webflux-ui:2.8.6'
```

Esas son todas las dependencias que necesitan agregarse; ahora es el momento de la configuración.

Agregar configuración de OpenAPI y documentación general de API a ProductCompositeService

Para habilitar springdoc-openapi en el microservicio product-composite-service , debemos agregar cierta configuración. Para mantener el código fuente compacto, lo agregaremos directamente a la clase de aplicación, ProductCompositeServiceApplication.java.



Si lo prefiere, puede colocar la configuración de springdoc-openapi en una clase de configuración de Spring separada.

Primero, necesitamos definir un bean Spring que devuelva un bean OpenAPI . El código fuente se ve así:

```
@Frijol
público OpenAPI getOpenApiDocumentation() {
    devolver nuevo OpenAPI()
        .info(new Info().title(apiTitle)
            .description(apiDescription)
            .version(apiVersion)
            .contact(nuevo Contacto()
                .name(nombreContactoAPI)
                .url(apiContactUrl)
                .email(apiContactEmail))
            .termsOfService(apiTérminosDeServicio)
```



```
.license(nueva Licencia()  
    .name(apiLicense)  
    .url(apiLicenseUrl)))  
.externalDocs(nueva DocumentaciónExterna()  
    .description(apiExternalDocDesc)  
    .url(apiExternalDocUrl));  
}
```

💡 Consejo rápido: Mejora tu experiencia de programación con las funciones AI Code Explainer y Quick Copy . Abre este libro en el Packt Reader de última generación. Haz clic en el botón Copiar.

(1) para copiar rápidamente el código en su entorno de codificación, o haga clic en el botón Explicar

(2) para que el asistente de IA le explique un bloque de código.



```
function calculate(a, b) {  
    return {sum: a + b};  
};
```

Copy

1

Explain

2

🔒 El lector Packt de próxima generación se incluye de forma gratuita con la compra de este libro.

Escanee el código QR O vaya a packtpub.com/unlock, Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.



Del código anterior, podemos ver que la configuración contiene información descriptiva general sobre la API, como la siguiente:

- El nombre, la descripción, la versión y la información de contacto de la API
- Condiciones de uso e información de licencia
- Enlaces a información externa sobre la API, si los hubiera

Las variables api* que se utilizan para configurar el bean OpenAPI se inicializan desde el archivo de propiedades mediante las anotaciones @Value de Spring . Son las siguientes:

```
@Value("${api.common.version}")          Cadena apiVersion;
@Value("${api.common.title}")            Cadena apiTitle;
@Value("${api.common.description}")      Cadena apiDescription;
@Value("${api.common.termsOfService}")  Cadena apiTermsOfService;
@Value("${api.common.license}")          Cadena apiLicense;
@Value("${api.common.licenseUrl}")       Cadena apiLicenseUrl;
@Value("${api.common.externalDocDesc}")  Cadena apiExternalDocDesc;
@Value("${api.common.externalDocUrl}")   Cadena apiExternalDocUrl;
@Value("${api.common.contact.name}")     Cadena apiContactName;
@Value("${api.common.contact.url}")      Cadena apiContactUrl;
@Value("${api.common.contact.email}")    Cadena apiContactEmail;
```

Los valores reales se establecen en el archivo de propiedades, application.yml, de la siguiente manera:

API:

```
común:
  versión: 1.0.0
  título: API de muestra
  Descripción: Descripción de la API...
  Términos de servicio: Mis términos de servicio
  licencia: MI LICENCIA
  licenseUrl: URL DE MI LICENCIA
  externalDocDesc: MI PÁGINA WIKI
  externalDocUrl: URL DE MI WIKI
  contacto:
    nombre: NOMBRE DEL CONTACTO
    url: URL DE CONTACTO
    correo electrónico: contact@mail.com
```

El archivo de propiedades también contiene parámetros de configuración para springdoc-openapi:

```
springdoc:
  swagger-ui.path: /openapi/swagger-ui.html
  api-docs.ruta: /openapi/v3/api-docs
  paquetesParaEscanear: se.magnus.microservices.composite.product
  rutasParaCoincidir: /**
```

Los parámetros de configuración tienen los siguientes propósitos:

- `springdoc.swagger-ui.path` y `springdoc.api-docs.path` se utilizan para especificar que las URL utilizadas por el visor de interfaz de usuario Swagger integrado están disponibles en la ruta `/openapi`.

Más adelante en este libro, al añadir diferentes tipos de servidores perimetrales y abordar los desafíos de seguridad, se simplificará la configuración de los servidores perimetrales utilizados. Consulte los siguientes capítulos para obtener más información:

- Capítulo 10, Uso de Spring Cloud Gateway para ocultar microservicios detrás de un servidor perimetral
 - Capítulo 11, Protección del acceso a las API
 - Capítulo 17, Implementación de características de Kubernetes para simplificar el panorama del sistema: el Reemplazo de la sección Spring Cloud Gateway
 - Capítulo 18, Uso de una malla de servicios para mejorar la observabilidad y la gestión: el Reemplazo del controlador de ingreso de Kubernetes con la sección de puerta de enlace de ingreso de Istio
- `springdoc.packagesToScan` y `springdoc.pathsToMatch` controlan en qué parte del código `springdoc-openapi` buscará anotaciones. Cuanto más limitado sea el alcance de `springdoc-openapi`, más rápido se realizará el escaneo.

Para obtener más información, consulte la clase de aplicación `ProductCompositeServiceApplication.java` y el archivo de propiedades `application.yml` en el proyecto `product-composite-service`. Ahora podemos ver cómo agregar documentación específica de la API a la interfaz de Java, `ProductCompositeService`.

`java`, en el proyecto `api`.

Agregar documentación específica de la API a la Interfaz de servicio compuesto de producto

Para documentar la API y sus operaciones RESTful, añadiremos una anotación `@Tag` a la declaración de la interfaz Java en `ProductCompositeService.java` del proyecto de la API. Para cada operación RESTful en la API, añadiremos una anotación `@Operation`, junto con anotaciones `@ApiResponse` en el método Java correspondiente, para describir la operación y sus respuestas esperadas.

Describiremos tanto las respuestas exitosas como las erróneas.

Además de leer estas anotaciones en tiempo de ejecución, `springdoc-openapi` también inspeccionará las anotaciones de Spring, como la anotación `@GetMapping`, para comprender qué argumentos de entrada toma la operación y cómo se verá la respuesta si se produce una respuesta exitosa. Para comprender la estructura de posibles respuestas de error, `springdoc-openapi` buscará `@RestControllerAdvice` y las anotaciones `@ExceptionHandler`. En el Capítulo 3, "Creación de un conjunto de microservicios cooperativos", añadimos una clase de utilidad, `GlobalControllerExceptionHandler.java`, al proyecto `util`.

Esta clase está anotada con `@RestControllerAdvice`. Consulte la sección "El controlador de excepciones del controlador REST global" para obtener más información. El controlador de excepciones gestiona los errores 404 (NOT_FOUND) y 422. Errores (ENTIDAD_INPROCESABLE) . Para permitir que springdoc-openapi también documente correctamente el error 400. (BAD_REQUEST) errores que Spring WebFlux genera cuando descubre argumentos de entrada incorrectos en una solicitud, hemos agregado un `@ExceptionHandler` para 400 errores (BAD_REQUEST) en `GlobalControllerExceptionHandler.java`.

La documentación de la API a nivel de recurso, correspondiente a la declaración de la interfaz Java, se ve de la siguiente manera:

```
@Tag(nombre = "ProductComposite", descripción =
    "API REST para información de productos compuestos.")
interfaz pública ProductCompositeService {
```

Para la operación de la API, hemos extraído el texto real utilizado en `@Operation` y `@ApiResponse` anotaciones en el archivo de propiedades. Estas anotaciones contienen marcadores de propiedades, como `{name- of-the-property}`, que springdoc-openapi usará para consultar el texto real del archivo de propiedades en tiempo de ejecución. El funcionamiento de la API se documenta a continuación:

```
@Operación(
    resumen =
        "${api.product-composite.get-composite-product.description}",
    descripción =
        "${api.product-composite.get-composite-product.notes}")
@ApiResponses(valor = {
    @ApiResponse(responseCode = "200", descripción =
        "${api.responseCodes.ok.description}"),
    @ApiResponse(responseCode = "400", descripción =
        "${api.responseCodes.badRequest.description}"),
    @ApiResponse(responseCode = "404", descripción = "${
        {api.responseCodes.notFound.description}"),
    @ApiResponse(códigoDeRespuesta = "422", descripción =
        "${api.responseCodes.unprocessableEntity.description}")
})
@GetMapping(
    valor = "/producto-compuesto/{productId}",
    produce = "aplicación/json")
ProductAggregate getProduct(@PathVariable int productId);
```

Del código fuente anterior, springdoc-openapi podrá extraer la siguiente información sobre la operación:

- La operación acepta solicitudes HTTP GET a la URL `/product-composite/{productid}`, donde la última parte de la URL, `{productid}`, se utiliza como parámetro de entrada para la solicitud.
- Una respuesta exitosa producirá una estructura JSON correspondiente a la clase Java, `Producto Agregado`
- En caso de error, se devolverá un código de error HTTP 400, 404 o 422 junto con la información de error en el cuerpo, como se describe en `@ExceptionHandler` en la clase Java `GlobalControllerExceptionHandler.java` en el proyecto `util`, como se describió anteriormente

Para los valores especificados en las anotaciones `@Operation` y `@ApiResponse`, podemos usar marcadores de propiedad directamente, sin usar las anotaciones `@Value` de Spring. Los valores reales se establecen en el archivo de propiedades, `application.yml`, de la siguiente manera:

API:

Códigos de respuesta:

`ok.description`: OK

`badRequest.description`: Solicitud incorrecta, formato no válido de la solicitud.

Consulte el mensaje de respuesta para obtener más información.

`notFound.description`: No encontrado, el id especificado no existe

`unprocessableEntity.description`: Entidad no procesable, entrada

Los parámetros provocaron un fallo en el procesamiento. Consulte el mensaje de respuesta para obtener más información.

producto-compuesto:

obtener-producto-compuesto:

`Descripción`: Devuelve una vista compuesta del ID del producto especificado.

`notas`: |

Respuesta normal

Si se encuentra el ID del producto solicitado, el método devolverá

Información sobre:

1. Información básica del producto

1. Reseñas

1. Recomendaciones

1. Direcciones de servicio\n(información técnica sobre el direcciones de los microservicios que crearon la respuesta)

```
# Respuestas parciales y de error esperadas

En los siguientes casos, solo se creará una respuesta parcial (utilizada para simplificar
la prueba de condiciones de error)

## Id. de producto 113
200 - Ok, pero no se devolverán recomendaciones

## Id. de producto 213
200 - Ok, pero no se devolverán reseñas

## Identificación de producto no numérica
400 - Se devolverá un error de **Solicitud incorrecta**

## Id. del producto 13
404 - Se devolverá un error **No encontrado**

## Identificadores de producto negativos
422 - Se devolverá un error de **Entidad no procesable**
```

De la configuración anterior podemos aprender lo siguiente:

- Se traducirá un marcador de propiedad como `${api.responseCodes.ok.description}`

Aceptar . Observe la estructura jerárquica del archivo de propiedades basado en YAML:

```
API:
  Códigos de
    respuesta: ok. Descripción: OK
```

- Un valor de varias líneas comienza con `|`, como el de `api.get-composite-product`.

Propiedad `description.notes` . Tenga en cuenta también que `springdoc-openapi` permite proporcionar una descripción de varias líneas mediante la sintaxis Markdown .

Para obtener más detalles, consulte la clase de interfaz de servicio, `ProductCompositeService.java`, en el proyecto `api` y el archivo de propiedades `application.yml` en el proyecto `product-composite-service` .



Si desea obtener más información sobre cómo se construye un archivo YAML, consulte la especificación en <https://yaml.org/spec/1.2/spec.html>.

Construyendo e iniciando el panorama de microservicios

¡Antes de que podamos probar la documentación de OpenAPI, necesitamos construir e iniciar el panorama de microservicios!

Esto se puede hacer con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo05
./gradlew compilación y docker compose compilación y docker compose up -d
```

Podría aparecer un mensaje de error indicando que el puerto 8080 ya está asignado. Se vería así:

```
ERROR: para product-composite No se puede iniciar el servicio
product-composite: el controlador no pudo programar la conectividad
externa en el punto final chapter05_product-composite_1
(0138d46f2a3055ed1b90b3b3daca92330919a1e7fec20351728633222db5e737): Falló el
enlace para 0.0.0.0:8080: el puerto ya está asignado
```

Si es así, es posible que hayas olvidado recuperar el entorno de microservicios del capítulo anterior. Para averiguar los nombres de los contenedores en ejecución, ejecuta el siguiente comando:

```
docker ps --format {{.Nombres}}
```

Una respuesta de muestra cuando un panorama de microservicios del capítulo anterior aún está en ejecución es como sigue:

```
capítulo 05_revisión_1
capítulo05_producto_1
capítulo05_recomendación_1
capítulo 04_revisión_1
capítulo 04_producto-compuesto_1
capítulo04_producto_1
capítulo04_recomendación_1
```

Si encuentra contenedores de otros capítulos en la salida del comando, por ejemplo, de Capítulo 4, Implementación de nuestros microservicios mediante Docker, como en el ejemplo anterior, debe saltar a la carpeta del código fuente de ese capítulo y descargar sus contenedores:

```
cd ../Capítulo04
Docker Compose Down
```

Ahora, puedes traer el contenedor que falta para este capítulo:

```
cd ../Capítulo05
docker composer -d
```

Tenga en cuenta que el comando solo inicia el contenedor faltante, product-composite, ya que los demás ya se iniciaron correctamente:

```
Iniciando el capítulo 05_producto-compuesto_1... listo
```

Para esperar a que se inicie el entorno de microservicios y verificar que funciona, puede ejecutar el siguiente comando:

```
./pruébalos-a-todos.bash
```

Tenga en cuenta que el script de prueba, test-em-all.bash, se ha ampliado con un conjunto de pruebas que verifican que los puntos finales de Swagger UI funcionan como se espera:

```
# Verificar el acceso a las URL de Swagger y OpenAPI
echo "Pruebas Swagger/OpenAPI"
assertCurl 302 "curl -s http://$HOST:$PORT/openapi/swagger-ui.html"
assertCurl 200 "curl -sL http://$HOST:$PORT/openapi/swagger-ui.html"
assertCurl 200 "curl -s http://$HOST:$PORT/openapi/swagger-ui/index.html"
assertCurl 200 "curl -s http://$HOST:$PORT/openapi/swagger-ui/oauth2-
redirigir.html"

assertCurl 200 "curl -s http://$HOST:$PORT/openapi/v3/api-docs"
assertEqual "3.1.0" "$(echo $RESPONSE | jq -r .openapi)"
assertEqual "http://$HOST:$PORT" "$(echo $RESPONSE | jq -r '.servers[0].
URL')"
assertCurl 200 "curl -s http://$HOST:$PORT/openapi/v3/api-docs.yaml"
```

Con el inicio exitoso de los microservicios, podemos avanzar y probar la documentación de OpenAPI expuesta por el microservicio compuesto por producto utilizando su interfaz de usuario Swagger incorporada. espectador.

Probando la documentación de OpenAPI

Para explorar la documentación de OpenAPI, usaremos el visor integrado de Swagger UI. Si abrimos `http://localhost:8080/openapi/swagger-ui.html` URL en un navegador web, veremos una página web que se parece a la siguiente captura de pantalla:

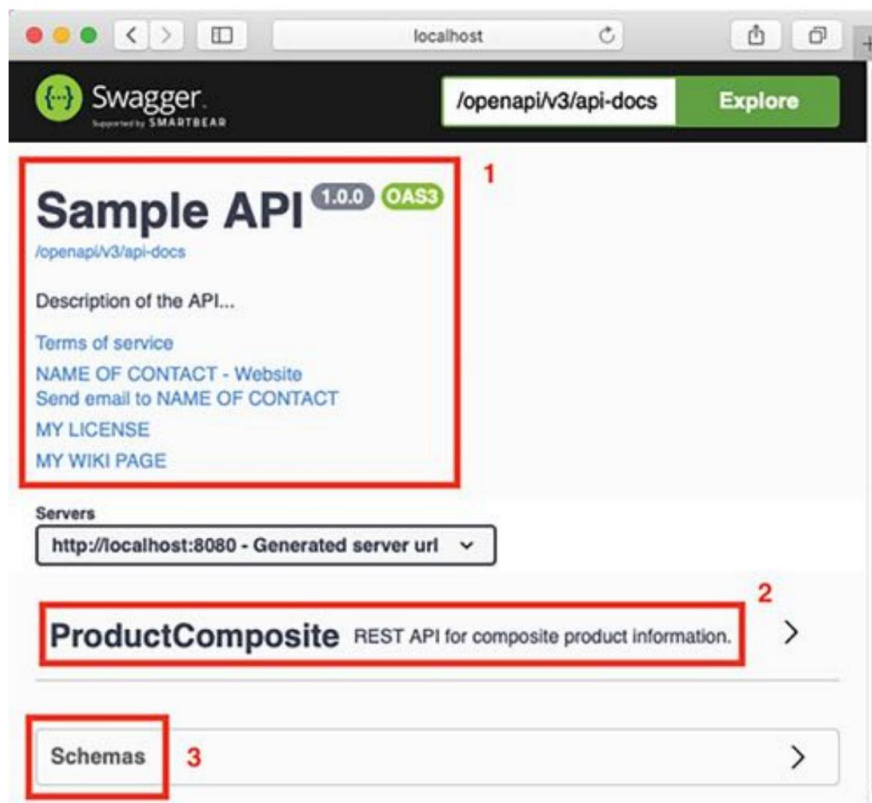


Figura 5.2: Documentación de OpenAPI con el visor de interfaz de usuario Swagger

Aquí podemos constatar lo siguiente:

1. Contiene información general que especificamos en el bean `OpenAPI springdoc-openapi` y un enlace al documento OpenAPI real, `/openapi/v3/api-docs`, que apunta a `http://localhost:8080/openapi/v3/api-docs`.



Tenga en cuenta que este es el enlace al documento OpenAPI que se puede exportar a una puerta de enlace API, como se analiza en Introducción al uso de `springdoc-openapi` sección anteriormente.

2. Hay una lista de recursos de API: en nuestro caso, la API ProductComposite .
3. En la parte inferior de la página, hay una sección donde podemos inspeccionar los esquemas utilizados en la API.

Proceda con el examen de la documentación de la API de la siguiente manera:

Haga clic en el recurso API de ProductComposite para expandirlo. Verá una lista de las operaciones disponibles en el recurso. Solo verá una operación: / producto-compuesto/{productId}.

Haga clic en él para expandirlo. Verá la documentación de la operación que especificamos en la interfaz Java de ProductCompositeService :

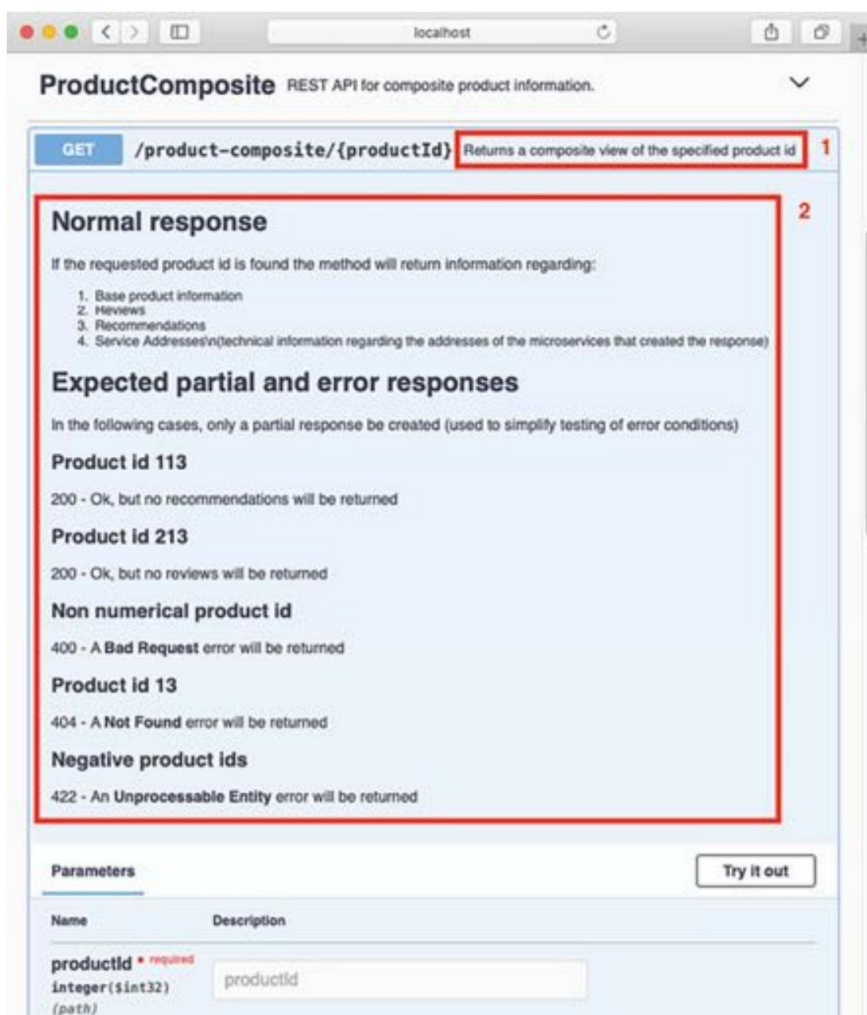


Figura 5.3: Documentación de la API de ProductComposite

Aquí podemos ver lo siguiente:

- La descripción de una línea de la operación.
- Una sección con detalles sobre el funcionamiento, incluidos los parámetros de entrada que admite.

¡Observe cómo se ha representado correctamente la sintaxis Markdown del campo de notas en la anotación `@ApiOperation` !

Si se desplaza hacia abajo en la página web, también encontrará documentación sobre las respuestas esperadas y su estructura. Esto muestra una respuesta normal de 200 (OK) :

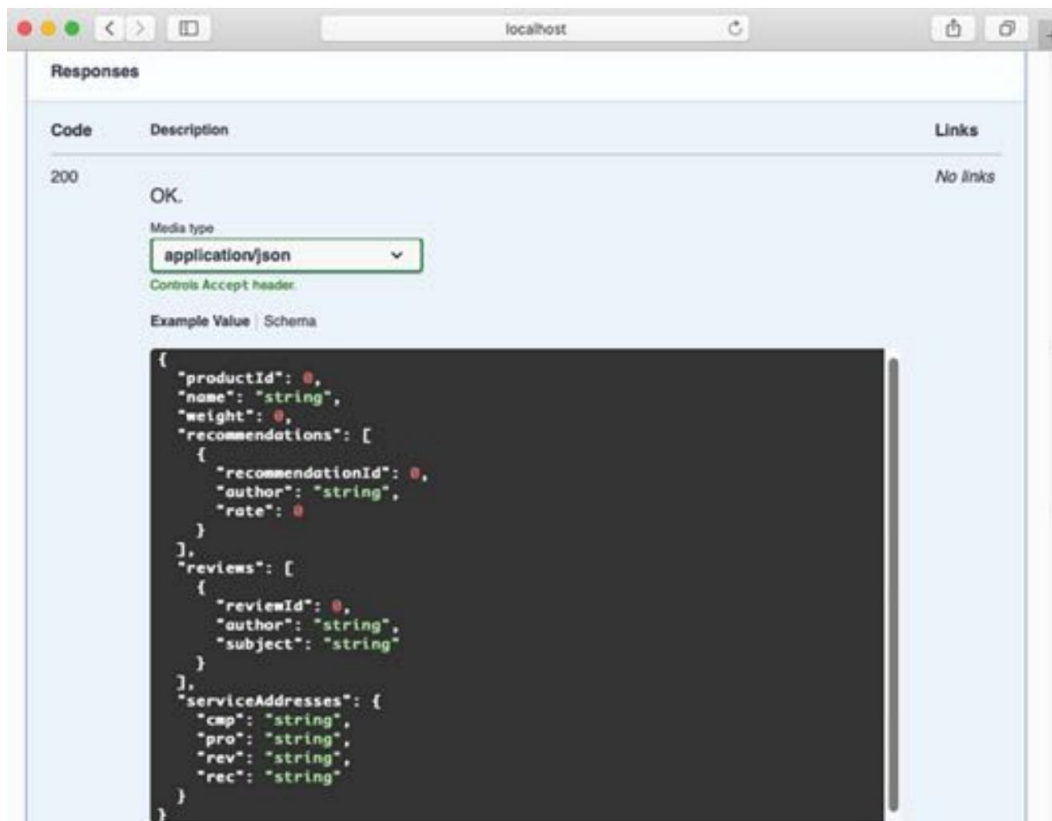


Figura 5.4: Documentación para una respuesta 200

Estas son las distintas respuestas de error 4xx que definimos anteriormente:

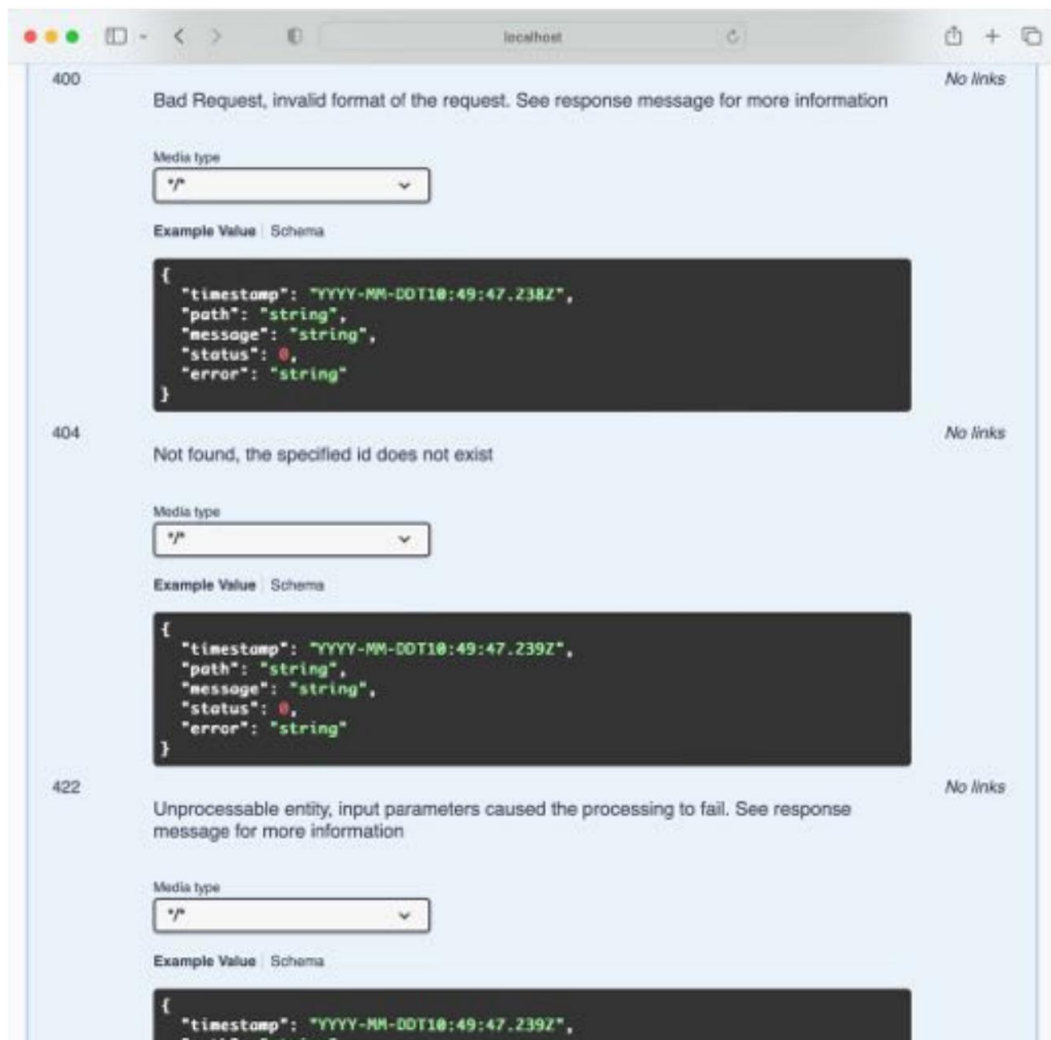


Figura 5.5: Documentación de las respuestas 4xx

Para cada respuesta de error potencial documentada, podemos aprender sobre su significado y la estructura del cuerpo de la respuesta.

Si nos desplazamos hacia arriba hasta la descripción del parámetro, encontraremos el botón "Pruébalo". Al hacer clic en él, podemos introducir los valores reales del parámetro y enviar una solicitud a la API haciendo clic en el botón "Ejecutar". Por ejemplo, si introducimos 123 en el campo "productId", obtendremos lo siguiente.

respuesta:

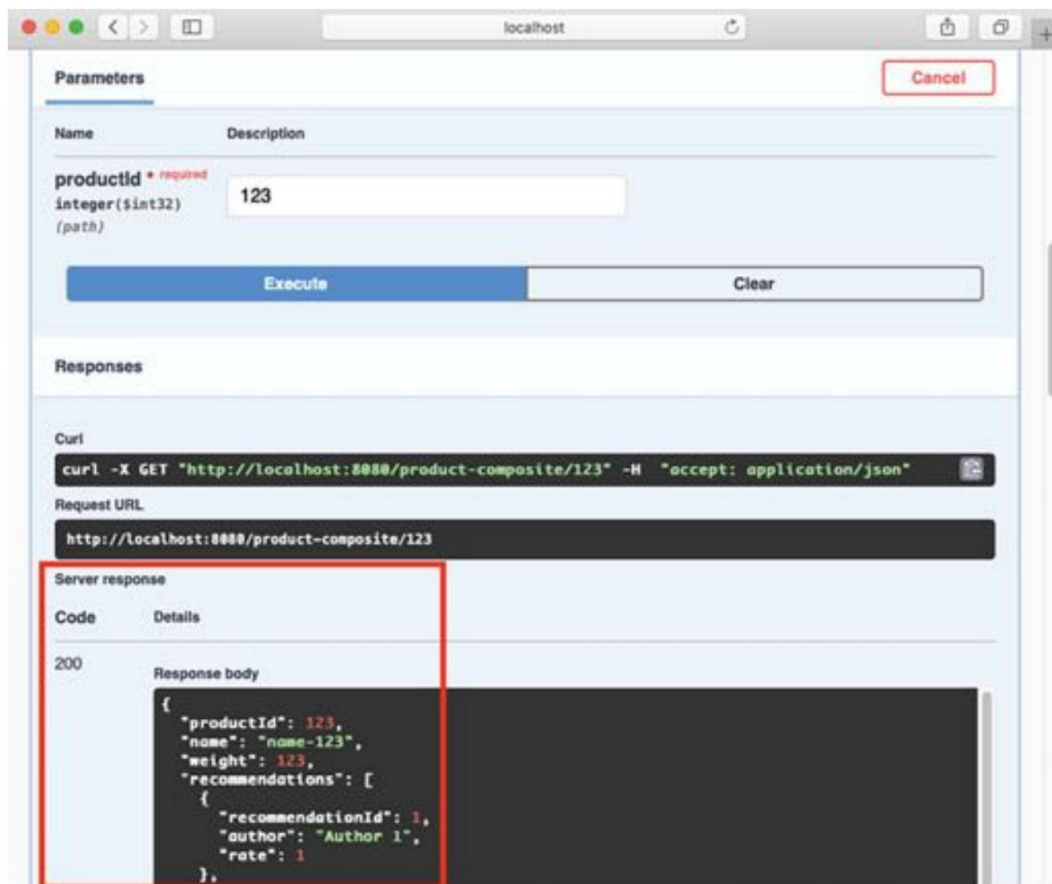


Figura 5.6: Respuesta después de enviar una solicitud de un producto existente

Consejo rápido: ¿Necesitas ver una versión en alta resolución de esta imagen? Abre este libro en el Packt Reader de próxima generación o verlo en copia PDF/ePub.

El lector Packt de última generación y una copia gratuita en PDF/ePub de este libro se incluyen con la compra. Escanea el código QR o visita packtpub.com/unlock. Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.



Obtendremos un 200 (OK) esperado como código de respuesta y una estructura JSON en el cuerpo de la respuesta con la que ya estamos familiarizados.

Si ingresamos una entrada incorrecta, como -1, obtendremos un código de error adecuado como código de respuesta, 422, y una descripción de error basada en JSON correspondiente en el cuerpo de la respuesta:

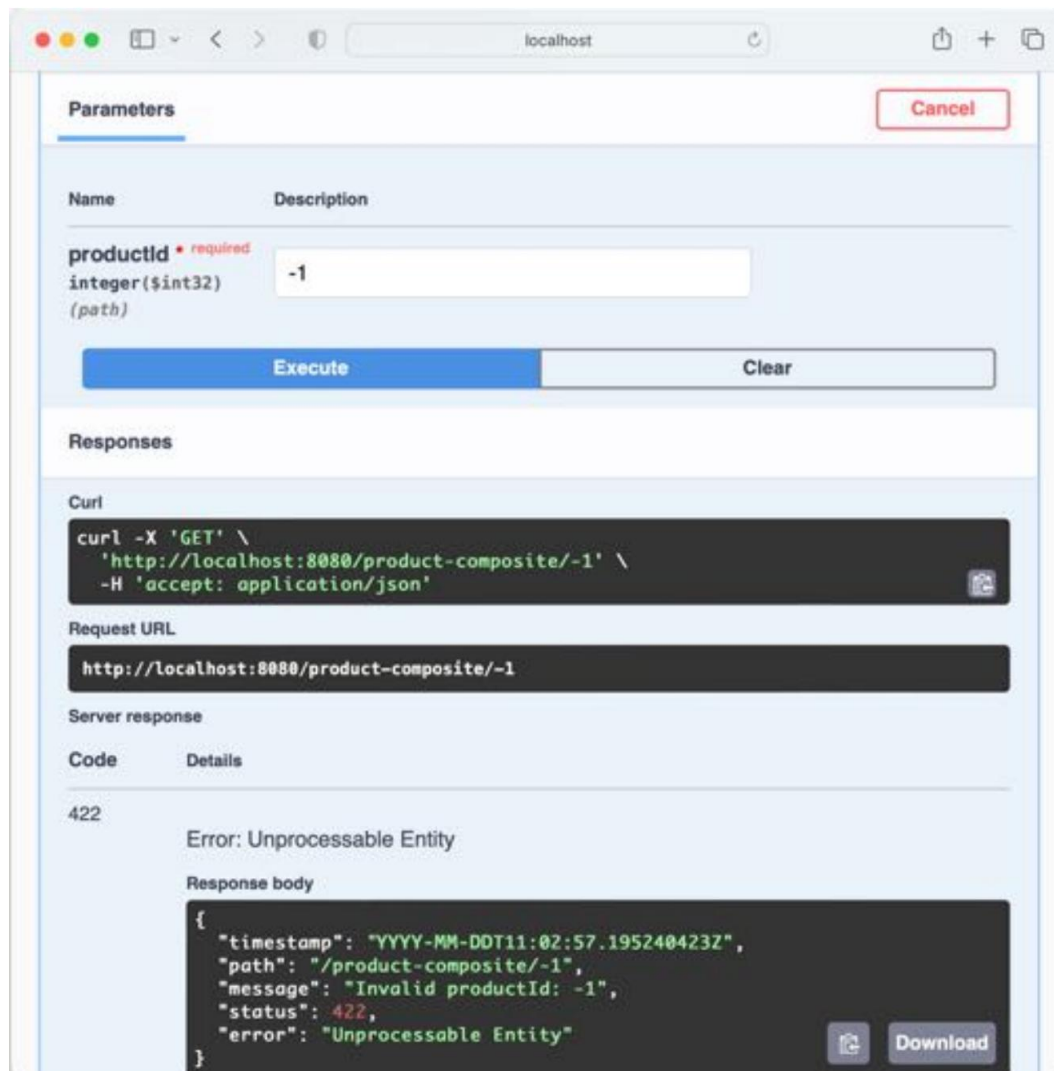


Figura 5.7: Respuesta después de enviar una solicitud con una entrada no válida

Tenga en cuenta que el campo de mensaje en el cuerpo de la respuesta señala claramente el problema: "Problema no válido".
ductId: -1".

Si desea intentar llamar a la API sin usar el visor de interfaz de usuario Swagger, puede copiar el comando curl correspondiente de la sección Respuestas y ejecutarlo en una ventana de Terminal, como se muestra en la captura de pantalla anterior:

```
curl -X GET "http://localhost:8080/producto-compuesto/123" -H "accept: aplicación/json"
```

Genial, ¿no?

Resumen

Una buena documentación de una API es esencial para su aceptación, y OpenAPI es una de las especificaciones más utilizadas cuando se trata de documentar servicios RESTful. springdoc-openapi

Es un proyecto de código abierto que permite crear documentación de API basada en OpenAPI sobre la marcha en tiempo de ejecución mediante la inspección de las anotaciones de Spring WebFlux y Swagger. Las descripciones textuales de una API se pueden extraer de las anotaciones del código fuente de Java y colocarse en un archivo de propiedades para facilitar su edición. springdoc-openapi se puede configurar para integrar un visor de interfaz de usuario Swagger en un microservicio, lo que facilita la lectura de las API expuestas por el microservicio y su uso desde el visor.

Ahora bien, ¿qué tal si mejoramos nuestros microservicios añadiendo persistencia, es decir, la capacidad de guardar sus datos en una base de datos? Para ello, necesitaremos añadir más API para poder crear y eliminar la información que gestionan los microservicios.

¡Vaya al siguiente capítulo para obtener más información!

Preguntas

1. ¿Cómo nos ayuda springdoc-openapi a crear documentación de API para servicios RESTful?
2. ¿Qué especificación para documentar API admite springdoc-openapi ?
3. ¿Cuál es el propósito del bean OpenAPI springdoc-openapi ?
4. Nombra algunas anotaciones que springdoc-openapi lee en tiempo de ejecución para crear la documentación de API.
Documentación sobre la marcha.
5. ¿Qué significa el código ": |" en un archivo YAML?
6. ¿Cómo se puede repetir una llamada a una API que se realizó utilizando el visor de interfaz de usuario Swagger integrado sin utilizar el visor nuevamente?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



6

Añadiendo persistencia

En este capítulo, aprenderemos a persistir los datos que utiliza un microservicio. Como ya se mencionó en el Capítulo 2, Introducción a Spring Boot, usaremos el proyecto Spring Data para persistir datos en bases de datos MongoDB y MySQL.

Los microservicios de producto y recomendación usarán Spring Data para MongoDB, y el microservicio de revisión usará Spring Data para la API de Persistencia de Java (JPA) para acceder a una base de datos MySQL. Añadiremos operaciones a las API RESTful para poder crear y eliminar datos en las bases de datos. Las API existentes para leer datos se actualizarán para acceder a las bases de datos. Ejecutaremos las bases de datos como contenedores Docker, gestionados por Docker Compose, es decir, de la misma manera.

mientras ejecutamos nuestros microservicios.

En este capítulo se tratarán los siguientes temas:

- Agregar una capa de persistencia a los microservicios centrales
- Escribir pruebas automatizadas que se centren en la persistencia
- Uso de la capa de persistencia en la capa de servicio
- Ampliación de la API de servicio compuesto
- Agregar bases de datos al entorno de Docker Compose
- Pruebas manuales de las nuevas API y la capa de persistencia
- Actualización de las pruebas automatizadas del panorama de microservicios

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Para acceder manualmente a las bases de datos, utilizaremos las herramientas CLI incluidas en las imágenes de Docker utilizadas para ejecutarlas. También expondremos los puertos estándar de cada base de datos en Docker Compose: 3306 para MySQL y 27017 para MongoDB. Esto nos permitirá usar nuestras herramientas de bases de datos favoritas para acceder a ellas como si se ejecutaran localmente en nuestros equipos.

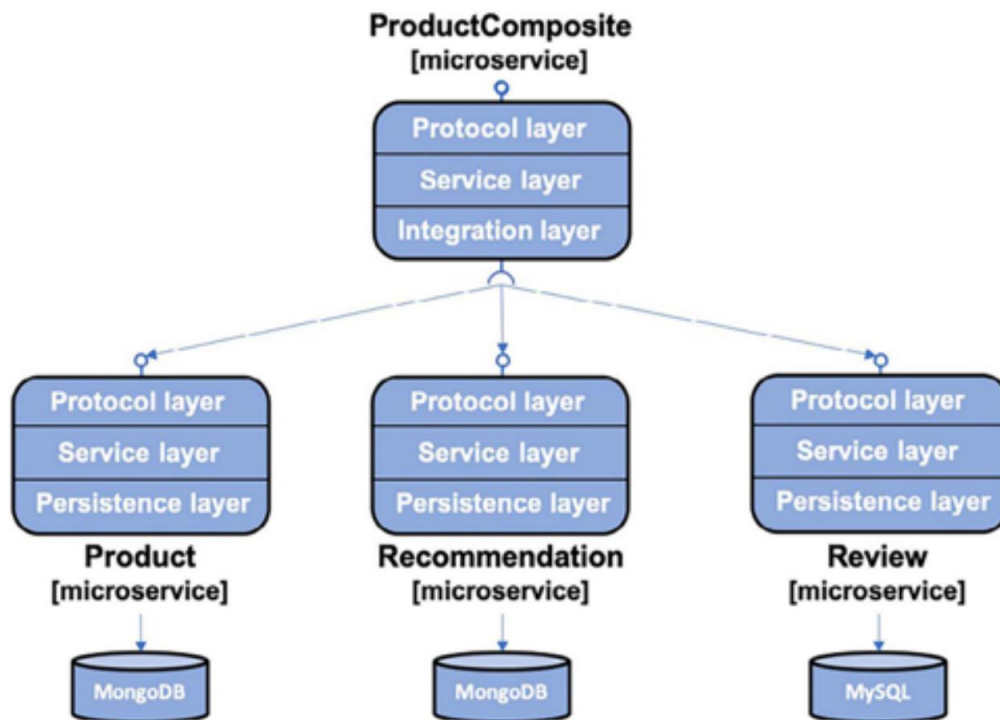
Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter06`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver el proceso para añadir persistencia a los microservicios con Spring Data, puede compararlo con el código fuente del Capítulo 5, "Añadir una descripción de API con OpenAPI". Puede usar su herramienta de comparación preferida y comparar las dos carpetas: `$BOOK_HOME/Chapter05` y `$BOOK_HOME/Chapter06`.

Antes de entrar en detalles, veamos hacia dónde nos dirigimos.

Objetivos del capítulo

Al final de este capítulo, tendremos capas dentro de nuestros microservicios que se verán como las siguientes:



The microservice landscape

[System boundary]

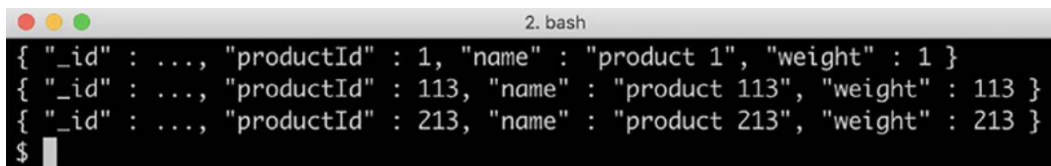
Figura 6.1: El panorama de microservicios que buscamos

La capa de protocolo gestiona la lógica específica del protocolo. Es muy limitada y solo consta de las anotaciones `RestController` en el proyecto de la API y la excepción `GlobalControllerException` común. Controlador en el proyecto util. La funcionalidad principal de cada microservicio reside en cada capa de servicio. El servicio compuesto por producto contiene una capa de integración que se utiliza para gestionar la... Comunicación con los tres microservicios principales. Todos los microservicios principales tendrán una Persistencia. capa utilizada para comunicarse con sus bases de datos.

Podremos acceder a los datos almacenados en MongoDB con un comando como el siguiente:

```
docker compose exec mongodb mongosh product-db --quiet --eval "db.
productos.find()"
```

El resultado del comando debería verse así:



```
2. bash
{ "_id" : ..., "productId" : 1, "name" : "product 1", "weight" : 1 }
{ "_id" : ..., "productId" : 113, "name" : "product 113", "weight" : 113 }
{ "_id" : ..., "productId" : 213, "name" : "product 213", "weight" : 213 }
$
```

Figura 6.2: Acceso a los datos almacenados en MongoDB

Respecto a los datos almacenados en MySQL, podremos acceder a ellos con un comando como este:

```
docker compose exec mysql mysql -uuser -p review-db -e "seleccionar * de
reseñas"
```

El resultado del comando debería verse así:



```
2. bash
+-----+-----+-----+-----+-----+
| id | author | content | product_id | review_id |
+-----+-----+-----+-----+-----+
| 1 | author 1 | content 1 | 1 | 1 |
| 2 | author 2 | content 2 | 1 | 2 |
+-----+-----+-----+-----+-----+
$
```

Figura 6.3: Acceso a los datos almacenados en MySQL



La salida de los comandos mongo y mysql se ha acortado para mejorar la legibilidad.

Veamos cómo implementar esto. Comenzaremos agregando la funcionalidad de persistencia a nuestro núcleo.
¡Microservicios!

Agregar una capa de persistencia a los microservicios centrales

Comencemos añadiendo una capa de persistencia a los microservicios principales. Además de usar Spring Data, también usaremos MapStruct, una herramienta de mapeo de beans Java, que facilita la transformación entre los objetos de entidad de Spring Data y las clases del modelo de la API. Para más detalles, consulte <http://mapstruct.org/>.

Primero, necesitamos agregar dependencias a MapStruct, Spring Data y los controladores JDBC para las bases de datos que vamos a usar. Después, podemos definir nuestras clases de entidad y repositorios de Spring Data. Estas clases de entidad y repositorios se guardarán en su propio paquete de Java, Persistence. Por ejemplo, para el microservicio de producto, se guardarán en `se.magnus`.

Paquete Java `microservices.core.product.persistence`.

Añadiendo dependencias

Usaremos MapStruct v1.6.3, por lo que comenzaremos definiendo una variable que contenga la información de la versión en el archivo de compilación para cada microservicio principal, `build.gradle`:

```
extensión {  
    mapstructVersion = "1.6.3.Final"  
}
```

A continuación, declaramos una dependencia en MapStruct:

```
implementación "org.mapstruct:mapstruct:${mapstructVersion}"
```

Dado que MapStruct genera la implementación de las asignaciones de beans en tiempo de compilación mediante el procesamiento de anotaciones de MapStruct, necesitamos agregar un `annotationProcessor` y un `testAnnotationProcessor` dependencia:

```
Procesador de anotaciones "org.mapstruct:mapstruct-  
processor:${mapstructVersion}"  
testAnnotationProcessor "org.mapstruct:mapstruct-processor:$  
{mapstructVersion}"
```

Para que la generación en tiempo de compilación funcione en IDE populares como IntelliJ IDEA, también necesitamos agregar la siguiente dependencia:

```
Solo compilación "org.mapstruct:mapstruct-processor:${mapstructVersion}"
```



Si utiliza IntelliJ IDEA, asegúrese de que el procesamiento de anotaciones esté habilitado. Abra Preferencias y vaya a Compilación, Ejecución, Implementación | Compilador | Procesadores de anotaciones. Compruebe que la casilla "Habilitar procesamiento de anotaciones" esté seleccionada.

Para los microservicios de productos y recomendaciones, declaramos las siguientes dependencias a Datos de Spring para MongoDB:

```
implementación 'org.springframework.boot:spring-boot-starter-data-mongodb'
```

Para el microservicio de revisión , declaramos una dependencia de Spring Data para JPA y un controlador JDBC para MySQL de esta manera:

```
implementación 'org.springframework.boot:spring-boot-starter-data-jpa'  
implementación 'mysql:mysql-connector-java'
```

Para habilitar el uso de MongoDB y MySQL al ejecutar pruebas de integración automatizadas, utilizaremos Testcontainers y su compatibilidad con JUnit 5, MongoDB y MySQL. Para los microservicios de producto y recomendación , declaramos las siguientes dependencias de prueba:

```
Implementación de prueba 'org.springframework.boot:spring-boot-testcontainers'  
Implementación de prueba 'org.testcontainers:junit-jupiter'  
Implementación de prueba 'org.testcontainers:mongodb'
```

Para los microservicios de revisión , declaramos las siguientes dependencias de prueba:

```
Implementación de prueba 'org.springframework.boot:spring-boot-testcontainers'  
Implementación de prueba 'org.testcontainers:junit-jupiter'  
Implementación de prueba 'org.testcontainers:mysql'
```

Para obtener más información sobre cómo se utiliza Testcontainers en las pruebas de integración, consulte la sección Escritura de pruebas automatizadas que se centran en la persistencia más adelante en este capítulo.

Almacenamiento de datos con clases de entidad

Las clases de entidad son similares a las clases del modelo API correspondientes en cuanto a los campos que contienen; consulte el paquete Java `se.magnus.api.core` en el proyecto API . Agregaremos dos campos: `id` y `versión`, en las clases de entidad en comparación con las clases del modelo API.

El campo `id` se utiliza para almacenar la identidad de la base de datos de cada entidad almacenada, correspondiente a la clave principal cuando se utiliza una base de datos relacional. Delegaremos la responsabilidad de generar valores únicos para el campo `id` a Spring Data. Dependiendo de la base de datos utilizada, Spring Data puede delegar esta responsabilidad al motor de base de datos o gestionarla por sí mismo. En cualquier caso, el código de la aplicación no necesita considerar cómo se establece un valor único de `id` de base de datos. El campo `id` no se expone en la API, como práctica recomendada desde el punto de vista de la seguridad. A los campos de las clases del modelo que identifican una entidad se les asignará un índice único en la clase de entidad correspondiente para garantizar la coherencia en la base de datos desde una perspectiva empresarial.

El campo de `versión` se utiliza para implementar el bloqueo optimista, lo que permite a Spring Data verificar que las actualizaciones de una entidad en la base de datos no sobrescriban una actualización simultánea. Si el valor del campo de `versión` almacenado en la base de datos es mayor que el valor del campo de `versión` en una solicitud de actualización, esto indica que la actualización se realiza en datos obsoletos; la información a actualizar ha sido actualizada por otra persona desde que se leyó de la base de datos.

Spring Data impedirá los intentos de realizar actualizaciones basadas en datos obsoletos. En la sección sobre la escritura de pruebas de persistencia, veremos pruebas que verifican que el mecanismo de bloqueo optimista de Spring Data impide las actualizaciones realizadas en datos obsoletos. Dado que solo implementamos APIs para operaciones de creación, lectura y eliminación, no expondremos el campo de versión en la API.

Las partes más interesantes de la clase de entidad de producto, utilizada para almacenar entidades en MongoDB, se ven como esto:

```
@Document(colección="productos")
clase pública ProductoEntidad {

    @Id
    cadena privada id;

    @Versión
    versión entera privada ;

    @Indexed(unique = verdadero)
    privado int productId;

    cadena privada nombre;
    int privado peso;
```

A continuación se muestran algunas observaciones del código anterior:

- La anotación `@Document(collection = "products")` se utiliza para marcar la clase como una clase de entidad utilizada para MongoDB, es decir, asignada a una colección en MongoDB con la productos de nombre
- Las anotaciones `@Id` y `@Version` se utilizan para marcar los campos de identificación y versión que se utilizarán por Spring Data, como se explicó anteriormente
- La anotación `@Indexed(unique = true)` se utiliza para obtener un índice único creado para la clave comercial, `productId`

Las partes más interesantes de la clase de entidad Recomendación, también se utilizan para almacenar entidades en MongoDB, se ve así:

```
@Document(collection="recomendaciones")
@CompoundIndex(nombre = "id-recomendación-producto", único = verdadero, def = "{ 'ID-producto': 1, 'ID-recomendación': 1 }")
clase pública EntidadRecomendación {
```



```

@Entidad
cadena privada id;

@Version
versión privada entera ;

privado int productId;
int privado recomendaciónId; cadena
privada autor;
calificación int privada;
contenido de cadena privada ;

```

Podemos ver cómo se crea un índice compuesto único utilizando la anotación `@CompoundIndex` para la clave comercial compuesta basada en los campos `productId` y `RecommendationId`.

Finalmente, las partes más interesantes de la clase de entidad `Review`, utilizada para almacenar entidades en una base de datos SQL como MySQL, se ven así:

```

@Entidad
@Table(nombre = "reseñas", índices = { @Index(nombre = "reseñas_idx_único",
único = verdadero, listaDeColumnas = "IdProducto,IdRevisión") })
clase pública ReviewEntity {

    @Id @ValorGenerado
    privado int id;

    @Version
    int privado versión;

    privado int productId;
    int privado reviewId; cadena
    privada autor;
    sujeto de cadena privada ;
    contenido de cadena privada;
}

```

Las siguientes son notas sobre el código anterior:

- Las anotaciones `@Entity` y `@Table` se utilizan para marcar la clase como una clase de entidad utilizada para JPA, asignada a una tabla en una base de datos SQL con el nombre `reviews`.

- La anotación `@Table` también se utiliza para especificar que se creará un índice compuesto único para la clave comercial compuesta en función de los campos `productId` y `reviewId`.

Las anotaciones `@Id` y `@Version` se utilizan para marcar los campos de id y versión que Spring Data utilizará, como se explicó anteriormente. Para dirigir Spring Data a JPA y generar automáticamente valores de id únicos para el campo `id`, utilizamos la anotación `@GeneratedValue`.

Para obtener el código fuente completo de las clases de entidad, consulte el paquete de persistencia en cada uno de los proyectos de microservicios principales.

Definición de repositorios en Spring Data

Spring Data incluye un conjunto de interfaces para definir repositorios. Usaremos `CrudRepository` y las interfaces `PagingAndSortingRepository`:

- La interfaz `CrudRepository` proporciona métodos estándar para realizar creaciones básicas, operaciones de lectura, actualización y eliminación de los datos almacenados en las bases de datos
- La interfaz `PagingAndSortingRepository` agrega soporte para paginación y clasificación a la Interfaz de `CrudRepository`

Utilizaremos la interfaz `CrudRepository` como base para los repositorios de Recomendación y Revisión y también la interfaz `PagingAndSortingRepository` como base para el repositorio de Productos.

También agregaremos algunos métodos de consulta adicionales a nuestros repositorios para buscar entidades utilizando la clave comercial, `productId`.

Spring Data permite definir métodos de consulta adicionales según las convenciones de nomenclatura para la firma del método. Por ejemplo, la firma del método `findByProductId(int productId)` puede usarse para indicar a Spring Data que cree automáticamente una consulta que devuelva entidades de la colección o tabla subyacente. En este caso, devolverá entidades cuyo campo `productId` esté establecido con el valor especificado en el parámetro `productId`. Para más detalles sobre cómo declarar consultas adicionales, consulte <https://docs.spring.io/spring-data/commons/reference/repositories/query-métodos-detalles.html>.

La clase de repositorio de productos se ve así:

```
La interfaz pública ProductRepository se extiende
PagingAndSortingRepository <EntidadProducto, Cadena>,
CrudRepository<ProductEntity, String> {

    Optional<ProductEntity> findByProductId(int productId);

}
```

Dado que el método `findByProductId` puede devolver cero o una entidad de producto, el valor de retorno se marca como opcional al envolverlo en un objeto `Optional`.

La clase de repositorio de recomendaciones se ve así:

```
La interfaz pública RecommendationRepository extiende CrudRepository
<EntidadRecomendada, Cadena> {
    Lista<EntidadDeRecomendación> findByProductId(int productId);
}
```

En este caso, el método `findByProductId` devolverá cero o más entidades de recomendación, por lo que el valor de retorno se define como una lista.

Finalmente, la clase de repositorio de revisión se ve así:

```
Interfaz pública ReviewRepository extiende CrudRepository<ReviewEntity,
Entero> {
    @Transactional(sólo lectura = verdadero)
    Lista<EntidadDeRevisión> findByProductId(int IdProducto);
}
```

Dado que las bases de datos SQL son transaccionales, debemos especificar el tipo de transacción predeterminado (solo lectura en nuestro caso) para el método de consulta, `findByProductId()`.

Eso es todo: esto es todo lo que se necesita para establecer una capa de persistencia para nuestros microservicios principales.

Para obtener el código fuente completo de las clases del repositorio, consulte el paquete de persistencia en cada uno de los proyectos de microservicios principales.

Comencemos a utilizar las clases de persistencia escribiendo algunas pruebas para verificar que funcionan como se espera.

Escribir pruebas automatizadas que se centren en la persistencia

Al escribir pruebas de persistencia, queremos iniciar una base de datos al inicio de las pruebas y desmantelarla al finalizar. Sin embargo, no queremos que las pruebas esperen a que se inicien otros recursos, por ejemplo, un servidor web como Netty (que se requiere en tiempo de ejecución).

Spring Boot viene con dos anotaciones a nivel de clase adaptadas a este requisito específico:

- `@DataMongoTest`: esta anotación inicia una base de datos MongoDB cuando comienza la prueba.

- `@DataJpaTest`: Esta anotación inicia una base de datos SQL cuando comienza la prueba.

De forma predeterminada, Spring Boot configura las pruebas para revertir las actualizaciones de la base de datos SQL y así minimizar el riesgo de efectos secundarios negativos en otras pruebas. En nuestro caso, este comportamiento provocará que algunas pruebas fallen. Por lo tanto, la reversión automática se desactiva con la anotación `@Transactional(propagation = NOT_SUPPORTED)` a nivel de clase.

Para gestionar el inicio y el cierre de las bases de datos durante la ejecución de las pruebas de integración, utilizaremos contenedores de prueba. Antes de analizar cómo escribir pruebas de persistencia, aprendamos a...

Utilice contenedores de prueba.

Uso de contenedores de prueba

Contenedores de prueba (<https://www.testcontainers.org>) Es una biblioteca que simplifica la ejecución de pruebas de integración automatizadas mediante la ejecución de administradores de recursos, como una base de datos o un agente de mensajes, como un contenedor Docker. Los contenedores de prueba se pueden configurar para iniciar automáticamente los contenedores Docker al iniciar las pruebas JUnit y desactivarlos al finalizar las pruebas.

Para habilitar Testcontainers en una clase de prueba existente para una aplicación Spring Boot, como los microservicios de este libro, podemos agregar la anotación `@Testcontainers` a la clase de prueba. Usando `@`

Anotación de contenedor: podemos, por ejemplo, declarar que las pruebas de integración del microservicio Review usarán un contenedor Docker con MySQL. El código se ve así:

```
@SpringBootTest
@Contenedores de prueba
clase SampleTests {
    @Recipiente
    base de datos privada estática MySQLContainer = nuevo MySQLContainer(
        "mysql:9.2.0");
```



La versión especificada para MySQL, 9.2.0, se copia de los archivos de Docker Compose para garantizar que se utilice la misma versión.

Una desventaja de este enfoque es que cada clase de prueba usará su propio contenedor Docker. Abrir MySQL en un contenedor Docker tarda unos segundos, normalmente 10 segundos en mi Mac. Ejecutar varias clases de prueba que usan el mismo tipo de contenedor añadirá esta latencia a cada una. Para evitar esta latencia adicional, podemos usar el patrón de contenedor único (ver https://www.testcontainers.org/test_framework_integration/manual_lifecycle_control/#singleton-contenedores). Siguiendo este patrón, se utiliza una clase base para iniciar un único contenedor Docker para MySQL. La clase base, `MySQLTestBase`, utilizada en el microservicio Review, tiene el siguiente aspecto:

```

class abstracta pública MySQLTestBase {

    @ConexiónDeServicio
    base de datos estática final JdbcDatabaseContainer =
        nuevo MySQLContainer("mysql:9.2.0").withStartupTimeoutSeconds(300);

    estática {
        base de datos.start();
    }
}

```

A continuación se presenta una explicación del código fuente anterior:

- El contenedor de la base de datos se declara de la misma manera que en el ejemplo anterior, con el
Adición de un período de espera extendido de cinco minutos para que el contenedor se ponga en marcha.
- Se utiliza un bloque estático para iniciar el contenedor de base de datos antes de invocar cualquier código JUnit.

Algunas propiedades se definirán para el contenedor de base de datos al iniciarse, como el puerto a usar. La anotación `@ServiceConnection` configurará automáticamente las propiedades de conexión de Spring correspondientes. Por ejemplo, establecerá las propiedades `spring.datasource.url`, `spring.datasource.username` y `spring.datasource.password` según la información del objeto de base de datos mediante los métodos `getJdbcUrl()`, `y getPassword().`

Las clases de prueba utilizan la clase base de la siguiente manera:

```

class PersistenceTests extiende MySQLTestBase {
class ReviewServiceApplicationTests extiende MySQLTestBase {

```

Para los microservicios de producto y revisión que utilizan MongoDB, se ha agregado una clase base correspondiente, `MongoDbTestBase`.

De forma predeterminada, la salida de registro de Testcontainers es bastante extensa. Un archivo de configuración de Logback

Se puede colocar en la carpeta `src/test/resource` para limitar la cantidad de registros generados. Logback es un marco de registro (<http://logback.qos.ch>). Se incluye en microservicios mediante la dependencia `spring-boot-starter-webflux`.

Para más detalles, consulte <https://www.testcontainers.org/>

entorno_docker_compatible/logging_config/. El archivo de configuración utilizado en este capítulo se llama `src/test/resources/logback-test.xml` y tiene este aspecto:

```
<?xml versión="1.0" codificación="UTF-8" ?>
<configuración>
  <include resource="org/springframework/boot/logging/logback/defaults.
xml"/>
  <include resource="org/springframework/boot/logging/logback/console-
appender.xml"/>

  <nivel raíz="INFO">
    <appender-ref ref="CONSOLA" />
  </root>
</configuración>
```

A continuación se presentan algunas notas sobre el archivo XML anterior:

- El archivo de configuración incluye dos archivos de configuración proporcionados por Spring Boot para definir los valores predeterminados, y se configura un anexo de registro que puede escribir eventos de registro en la consola.
- El archivo de configuración limita la salida del registro al nivel de registro INFO, descartando el registro DEBUG y TRACE registros emitidos por la biblioteca Testcontainers

Para obtener detalles sobre el soporte de Spring Boot para el registro y el uso de Logback, consulte <https://docs.spring.io/spring-boot/how-to/logging.html#howto.logging.logback>.

Con la introducción de Testcontainers, estamos listos para ver cómo se pueden escribir pruebas de persistencia.

Escritura de pruebas de persistencia

Las pruebas de persistencia para los tres microservicios principales son similares entre sí, por lo que solo repasaremos las pruebas de persistencia para el microservicio del producto.

La clase de prueba, PersistenceTests, declara un método, setupDb(), anotado con @BeforeEach, que se ejecuta antes de cada método de prueba. El método de configuración elimina cualquier entidad de pruebas anteriores en la base de datos e inserta una entidad que los métodos de prueba pueden usar como base para sus pruebas:

```
@DataMongoTest
clase PruebasDePersistencia {

    @Autowired
    repositorio privado ProductRepository ;
    EntidadProductoprivada EntidadGuardada ;

    @BeforeEach
    void setupDb() {
        repositorio.deleteAll();
        ProductEntity entidad = new ProductEntity(1, "n", 1);
        entidadGuardada = repositorio.guardar(entidad);
        afirmarIgualProducto(entidad, entidadGuardada);
    }
}
```

A continuación, se presentan los distintos métodos de prueba. El primero es una prueba de creación :

```
@Prueba
void crear() {
    ProductoEntidad nuevaEntidad = nueva ProductoEntidad(2, "n", 2);
    repositorio.save(newEntity);

    ProductoEntidad foundEntity = repositorio.findById(
        nuevaEntidad.getId()).get();
    assertEqualsProduct(nuevaEntidad, entidadEncontrada);

    assertEquals(2, repositorio.count());
}
```

Esta prueba crea una nueva entidad, verifica que se pueda encontrar usando el método findById y finaliza afirmando que hay dos entidades almacenadas en la base de datos, la creada por el método de configuración y la creada por la prueba misma.

La prueba de actualización se ve así:

```
@Test
void update()
{ entidadGuardada.setName("n2");
  repositorio.save(entidadGuardada);

  ProductoEntidad foundEntity =
    repositorio.findById(savedEntity.getId()).get(); assertEquals(1,
    (long)foundEntity.getVersion()); assertEquals("n2",
    foundEntity.getName());
}
```

Esta prueba actualiza la entidad creada por el método de configuración , la vuelve a leer de la base de datos mediante el método estándar findById() y confirma que contiene los valores esperados para algunos de sus campos. Tenga en cuenta que, al crear una entidad, Spring Data establece su campo de versión en 0 , por lo que esperamos que sea 1 después de la actualización.

La prueba de eliminación se ve así:

```
@Test
void eliminar() {
  repositorio.delete(savedEntity);
  assertFalse(repositorio.existsById(savedEntity.getId()));
}
```

Esta prueba elimina la entidad creada por el método de configuración y verifica que ya no exista en la base de datos.

La prueba de lectura se ve así:

```
@Test
void getByProductId()
{ Opcional<ProductoEntidad> entidad =

  repositorio.findById( savedEntity.getId()); assertTrue(entity.isPresent()); assertEqualsProduct(savedEntity)
}
```


Esta prueba utiliza el método `findByProductId()` para obtener la entidad creada por el método de configuración, verifica que se haya encontrado y luego utiliza el método auxiliar local, `assertEqualsProduct()`, para verificar que la entidad devuelta por `findByProductId()` se vea igual que la entidad almacenada por el método de configuración.

A continuación, se presentan dos métodos de prueba que verifican flujos alternativos para gestionar condiciones de error. El primero es una prueba que verifica que los duplicados se gestionen correctamente:

```
@Prueba
void duplicateError() {
    assertThrows(DuplicateKeyException.class, () -> {
        EntidadProducto = nuevaEntidadProducto (EntidadGuardada.getProductId(),
            "n", 1);
        repositorio.save(entidad);
    });
}
```

La prueba intenta almacenar una entidad con la misma clave de negocio que la entidad creada por el método de configuración. La prueba fallará si la operación de guardado se realiza correctamente o si falla con una excepción distinta a la esperada `DuplicateKeyException`.

La otra prueba negativa es, en mi opinión, la más interesante de la clase. Verifica la correcta gestión de errores al actualizar datos obsoletos; verifica el funcionamiento del mecanismo de bloqueo optimista. Su aspecto es el siguiente:

```
@Prueba
void optimismLockError() {

    // Almacena la entidad guardada en dos objetos de entidad separados
    EntidadProductoentidad1 = repositorio.findByld (
        entidadGuardada.getId()).get();
    EntidadProductoentidad2 = repositorio.findByld(savedEntity.getId()).get() ;

    // Actualizar la entidad utilizando el primer objeto de entidad
    entidad1.setName("n1");
    repositorio.save(entidad1);

    // Actualice la entidad utilizando el segundo objeto de entidad.
    // Esto debería fallar ya que la segunda entidad ahora contiene una versión antigua
```

```
// número, es decir, un error de bloqueo optimista
assertThrows(OptimisticLockingFailureException.class, () -> {
    entidad2.setName("n2");
    repositorio.save(entidad2);
});

// Obtener la entidad actualizada de la base de datos y verificar su nuevo estado
ProductoEntidadactualizadaEntidad = repositorio.findById(
    entidadGuardada.getId()).get();
assertEquals(1, (int)updatedEntity.getVersion());
assertEquals("n1", updatedEntity.getName());
}
```

Del código se observa lo siguiente:

- Primero, la prueba lee la misma entidad dos veces y la almacena en dos variables diferentes, entidad1 y entidad2.
- A continuación, utiliza una de las variables, entidad1, para actualizar la entidad. La actualización de la entidad en la base de datos hará que Spring Data incremente automáticamente su campo de versión . La otra variable, entidad2, ahora contiene datos obsoletos, como lo demuestra su campo de versión , que tiene un valor inferior al correspondiente en la base de datos.
- Cuando la prueba intenta actualizar la entidad utilizando la variable entity2 , que contiene datos obsoletos, se espera que falle arrojando una excepción OptimisticLockingFailureException .
- La prueba finaliza afirmando que la entidad en la base de datos refleja la primera actualización, es decir, contiene el nombre "n1", y que el campo de versión tiene el valor 1; solo se ha realizado una actualización en la entidad en la base de datos.

Finalmente, el servicio del producto contiene una prueba que demuestra el uso del soporte integrado para la clasificación y paginación en Spring Data:

```
@Prueba
paginación vacía () {
    repositorio.deleteAll();
    Lista<ProductEntity> nuevosProductos = rangoCerrado(1001, 1010)
        .mapToObj(i -> new ProductEntity(i, "nombre" + yo, yo))
        .collect(Collectors.toList());
    repositorio.saveAll(nuevosProductos);
}
```

```

Paginable nextPage = PageRequest.of(0, 4, ASC, "productId");
nextPage = testNextPage(nextPage, "[1001, 1002, 1003, 1004]", verdadero);
nextPage = testNextPage(nextPage, "[1005, 1006, 1007, 1008]", verdadero);
nextPage = testNextPage(nextPage, "[1009, 1010]", falso);
}

```

La siguiente es una explicación del código anterior:

- La prueba comienza eliminando todos los datos existentes y luego inserta 10 entidades con el productId campo que va desde 1001 a 1010.
- A continuación, crea PageRequest, solicitando un recuento de páginas de 4 entidades por página y una clasificación Orden basado en ProductId en orden ascendente.
- Por último, utiliza un método auxiliar, testNextPage, para leer las tres páginas esperadas, verificar los ID de producto esperados en cada página y verificar que Spring Data informe correctamente si existen más páginas o no.

El método auxiliar testNextPage se ve así:

```

prueba privada Pageable nextPage (Pageable nextPage, String
expectedProductIds, booleano esperaPáginaSiguiente) {
    Página<ProductEntity> productPage = repositorio.findAll(nextPage);
    assertEquals(expectedProductIds, productPage.getContent()
        .stream().map(p -> p.getProductId()).collect(
        Coleccionistas.toList()).toString());
    assertEquals(esperaPróximaPágina, productPage.hasNext());
    devolver productPage.nextPageable();
}

```

El método auxiliar utiliza el objeto de solicitud de página, nextPage, para obtener la siguiente página del método del repositorio, findAll(). Con base en el resultado, extrae los ID de producto de las entidades devueltas en una cadena y la compara con la lista esperada de ID de producto. Finalmente, devuelve la siguiente página.

Para obtener el código fuente completo de las pruebas de persistencia, consulte la clase de prueba PersistenceTests en cada uno de los proyectos de microservicios principales.

Las pruebas de persistencia en el microservicio del producto se pueden ejecutar usando Gradle con un comando como esto:

```

cd $BOOK_HOME/Capítulo06
./gradlew microservicios:producto-servicio:prueba --pruebas Pruebas de persistencia

```

Después de ejecutar las pruebas, debería responder con lo siguiente:

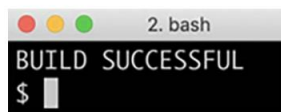


Figura 6.4: RESPUESTA DE CONSTRUCCIÓN EXITOSA

Con una capa de persistencia en su lugar, podemos actualizar la capa de servicio en nuestros microservicios principales para usar la capa de persistencia.

Uso de la capa de persistencia en la capa de servicio

En esta sección, aprenderemos a usar la capa de persistencia en la capa de servicio para almacenar y recuperar datos de una base de datos. Seguiremos los siguientes pasos:

1. Registro de la URL de conexión a la base de datos
2. Agregar nuevas API
3. Llamar a la capa de persistencia desde la capa de servicio
4. Declaración de un asignador de beans Java
5. Actualización de las pruebas de servicio

Registrar la URL de conexión a la base de datos

Al ampliar el número de microservicios, cada uno con su propia base de datos, puede resultar difícil controlar qué base de datos utiliza realmente cada uno. Para evitar esta confusión, se recomienda agregar una declaración LOG justo después del inicio del microservicio que registre la información de conexión utilizada para conectarse a la base de datos.

Por ejemplo, el código de inicio para el servicio del producto se ve así:

```
clase pública ProductoServicioAplicación {  
    Registrador final estático privado LOG =  
        LoggerFactory.getLogger(ProductServiceApplication.class);  
  
    público estático void main(String[] args) {  
        Contexto de aplicación configurable ctx =  
            SpringApplication.run(ProductServiceApplication.class, args);  
        Cadena mongodbHost = ctx.getEnvironment().getProperty(  
            "spring.data.mongodb.host");  
        Cadena mongodbPort = ctx.getEnvironment().getProperty(  

```

```

        "spring.data.mongodb.port");
    LOG.info("Conectado a MongoDB: " + mongodDbHost + ":" + mongodDbPort);
}
}

```

La llamada al método LOG.info escribirá algo como lo siguiente en el registro:

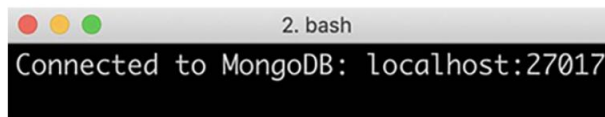


Figura 6.5: Salida de registro esperada

Para obtener el código fuente completo, consulte la clase de aplicación principal en cada uno de los proyectos de microservicios principales, por ejemplo, ProductServiceApplication en el proyecto de producto-servicio .

Agregar nuevas API

Antes de poder usar la capa de persistencia para crear y eliminar información en la base de datos, necesitamos crear las operaciones de API correspondientes en nuestras API de servicio principales.

Las operaciones de API para crear y eliminar una entidad de producto se ven así:

```

@PostMapping(
    valor      = "/producto",
    consume = "aplicación/json",
    produce = "aplicación/json")
Producto createProduct(@RequestBody Cuerpo del producto );

@DeleteMapping(valor = "/producto/{productId}")
void eliminarProduct(@PathVariable int productId);

```



La implementación de la operación de eliminación será idempotente, es decir, volverá a Obtener el mismo resultado si se llama varias veces. Esta es una característica valiosa en caso de fallo.

Escenarios. Por ejemplo, si un cliente experimenta un tiempo de espera de red durante una llamada a una operación de eliminación, puede simplemente volver a llamar a la operación de eliminación sin preocuparse por las respuestas variables (por ejemplo, OK (200) en la primera llamada y No encontrado (404) en llamadas posteriores), ni por cualquier efecto secundario inesperado. Esto implica que la operación debería devolver el código de estado OK (200) aunque la entidad ya no exista en la base de datos.

Las operaciones de API para las entidades de recomendación y revisión son similares; sin embargo, tenga en cuenta que cuando se trata de la operación de eliminación para las entidades de recomendación y revisión, se eliminarán todas las recomendaciones y revisiones para el productId especificado.

Para obtener el código fuente completo, consulte las declaraciones de interfaz (ProductService, RecommendationService y ReviewService) de los microservicios principales en el proyecto de API.

Llamar a la capa de persistencia desde la capa de servicio

El código fuente de la capa de servicio para usar la capa de persistencia está estructurado de la misma manera para todos los microservicios principales. Por lo tanto, solo revisaremos el código fuente del producto microservicio.

Primero, necesitamos inyectar la clase de repositorio desde la capa de persistencia y un mapeador de beans Java. clase en el constructor:

```
servicioUtil privado final serviceUtil;
repositorio final privado ProductRepository ;
mapeador final privado del producto ;

público ProductServiceImpl(ProductRepository repositorio, ProductMapper mapeador,
ServiceUtil serviceUtil) {
    este.repositorio = repositorio;
    este.mapper = mapeador;
    este.serviceUtil = serviceUtil;
}
```

En la siguiente sección veremos cómo se define la clase mapeadora de Java.

A continuación, el método createProduct se implementa de la siguiente manera:

```
Producto público createProduct( Cuerpo del producto) {
    intentar {
        EntidadProductoEntidad = mapper.apiToEntity(cuerpo);
        ProductoEntidad nuevaEntidad = repositorio.save(entidad);
        devolver mapper.entityToApi(newEntidad);
    } captura (DuplicateKeyException dke) {
        lanzar nueva InvalidInputException(" Clave duplicada, Id. del producto: "
        cuerpo.getProductid());
    }
}
```

El método `createProduct` utilizó el método `save` en el repositorio para almacenar una nueva entidad. Cabe destacar que la clase `mapper` se utiliza para convertir beans Java entre una clase de modelo de API y una clase de entidad mediante los dos métodos `mapper: apiToEntity()` y `entityToApi()`. El único error que gestionamos para el método `create` es la excepción `DuplicateKeyException`, que se convierte en una excepción `InvalidInputException`.

El método `getProduct` se ve así:

```
público Producto getProduct(int productId) {  
    si (productId < 1) arrojar nueva InvalidInputException(  
        "ProductId no válido: " + productId);  
    EntidadProducto = repositorio.findByProductId(productId)  
        .orElseThrow(() -> nueva NotFoundException(  
            "No se encontró ningún producto para productId: " + productId));  
    Respuesta del producto = mapper.entityToApi(entidad);  
    respuesta.setServiceAddress(serviceUtil.getServiceAddress());  
    respuesta de retorno ;  
}
```

Tras una validación de entrada básica (es decir, garantizar que `productId` no sea negativo), se utiliza el método `findByProductId()` del repositorio para encontrar la entidad del producto. Dado que el método del repositorio devuelve un producto opcional, podemos utilizar el método `orElseThrow()` en el producto opcional.

Clase para lanzar una excepción `NotFoundException` si no se encuentra ninguna entidad de producto. Antes de devolver la información del producto, se utiliza el objeto `serviceUtil` para completar la dirección actual del microservicio.

Por último, veamos el método `deleteProduct`:

```
public void deleteProduct(int IdProducto) {  
    repositorio.findByProductId(productId).ifPresent(e ->  
        repositorio.delete(e));  
}
```

El método `delete` también utiliza el método `findByProductId()` del repositorio y el método `ifPresent()` de la clase `Optional` para eliminar la entidad solo si existe. Tenga en cuenta que la implementación es idempotente; no informará ningún error si no se encuentra la entidad.

Para obtener el código fuente completo, consulte la clase de implementación del servicio en cada uno de los proyectos de microservicios principales, por ejemplo, `ProductServiceImpl` en el proyecto `producto-servicio`.

Declaración de un asignador de beans Java

Entonces, ¿qué pasa con el mapeador mágico de beans Java?

Como ya se mencionó, MapStruct se utiliza para declarar nuestras clases mapeadoras. El uso de MapStruct es similar en los tres microservicios principales, por lo que solo revisaremos el código fuente del objeto mapeador en el microservicio de producto .

La clase mapeadora para el servicio de producto se ve así:

```
@Mapper(componentModel = "resorte")
interfaz pública ProductMapper {

    @Mappings({
        @Mapping(target = "direcciónDeServicio", ignore = verdadero)
    })
    Producto entityToApi( Entidad ProductoEntidad);

    @Mappings({
        @Mapping(objetivo = "id", ignorar = verdadero),
        @Mapping(target = "versión", ignore = verdadero)
    })
    ProductoEntity apiToEntity(Product api);
}
```

Del código se puede observar lo siguiente:

El método entityToApi() asigna objetos de entidad al objeto del modelo API. Dado que la clase de entidad no tiene un campo para serviceAddress, el método entityToApi() está anotado para ignorar el campo serviceAddress en el objeto del modelo API.

El método apiToEntity() asigna objetos del modelo API a objetos de entidad. De igual manera, el método apiToEntity() está anotado para ignorar los campos id y version que faltan en la clase del modelo API.

MapStruct no solo permite mapear campos por nombre, sino que también permite asignar campos con nombres diferentes. En la clase de mapeo del servicio de recomendaciones , el campo de entidad de calificación se asigna al campo del modelo de API, "rate", mediante las siguientes anotaciones:

```
@Mapping(objetivo = "tasa", fuente="entidad.calificación"),
Entidad de recomendaciónToApi ( EntidadRecommendationEntity);
```



```
@Mapping(objetivo = "calificación", fuente="api.rate"),
EntidadRecomendación apiToEntity(Recomendación api);
```

Después de una compilación exitosa de Gradle, la implementación del mapeo generado se puede encontrar en la carpeta build/classes para cada proyecto, por ejemplo, ProductMapperImpl.java en el proyecto product-service .

Para obtener el código fuente completo, consulte la clase mapeadora en cada uno de los proyectos de microservicios principales, por ejemplo, ProductMapper en el proyecto producto-servicio .

Actualización de las pruebas de servicio

Las pruebas de las API expuestas por los microservicios principales se han actualizado desde el capítulo anterior con pruebas que cubren las operaciones de creación y eliminación de API.

Las pruebas agregadas son similares en los tres microservicios principales, por lo que solo revisaremos el código fuente de las pruebas de servicio en el microservicio del producto .

Para garantizar un estado conocido para cada prueba, se declara un método de configuración, setupDb(), y se anota con @BeforeEach, de modo que se ejecuta antes de cada prueba. El método de configuración elimina cualquier entidad creada previamente:

```
@Autowired
repositorio privado ProductRepository ;

@BeforeEach
vacío setupDb() {
    repositorio.deleteAll();
}
```

El método de prueba para la API de creación verifica que una entidad de producto se pueda recuperar después de haber sido creada y que la creación de otra entidad de producto con el mismo productId genere un error esperado, UNPROCESSABLE_ENTITY, en la respuesta a la solicitud de API:

```
@Prueba
void duplicateError() {
    int productold = 1;
    postAndVerifyProduct(productId, OK);
    assertTrue(repositorio.findByProductId(productId).isPresent());

    postAndVerifyProduct(productId, ENTIDAD NO PROCESABLE)
```

```

        .jsonPath("$.path").isEqualTo("/
        product") .jsonPath("$.message").isEqualTo(" Clave duplicada, Id. del producto: " +
            ID del producto);
    }

```

El método de prueba para la API de eliminación verifica que se pueda eliminar una entidad de producto y que una segunda solicitud de eliminación sea idempotente; también devuelve el código de estado OK , aunque la entidad ya no esté existe en la base de datos:

```

@Prueba
void deleteProduct() { int
    productId = 1;
    postAndVerifyProduct(productId, OK);
    assertTrue(repository.findByProductId(productId).isPresent());

    eliminarYVerificarProducto(productId, OK);
    assertFalse(repository.findByProductId(productId).isPresent());

    eliminarYVerificarProducto(productId, OK);
}

```

Para simplificar el envío de solicitudes de creación, lectura y eliminación a la API y verificar el estado de la respuesta, se han creado tres métodos auxiliares:

- publicarYVerificarProducto()
- obtenerYVerificarProducto()
- eliminarYVerificarProducto()

El método postAndVerifyProduct() se ve así:

```

privado WebTestClient.BodyContentSpec postAndVerifyProduct(int productId, HttpStatus
expectedStatus) {
    Producto producto = nuevo Producto(productId, "Nombre " + ID del producto,
    productId, "SA");
    return client.post().uri("/")

        .body(just(product),
        Product.class) .accept(APPLICATION_JSON)

        .exchange().expectStatus().isEqualTo(estadosesperado)

```

```

        .expectHeader().contentType(APLICACIÓN_JSON)
        .expectBody();
    }

```

El método auxiliar ejecuta la solicitud HTTP y verifica el código de respuesta y el tipo de contenido del cuerpo de la respuesta. Además, también devuelve el cuerpo de la respuesta para que el emisor pueda realizar más investigaciones, si es necesario. Los otros dos métodos auxiliares para solicitudes de lectura y eliminación son similares.

El código fuente de las tres clases de prueba de servicio se puede encontrar en cada uno de los microservicios principales. proyectos, por ejemplo, ProductServiceApplicationTests en el proyecto producto-servicio .

Ahora, pasemos a ver cómo ampliamos una API de servicio compuesto.

Ampliación de la API de servicio compuesto

En esta sección, veremos cómo podemos ampliar la API compuesta con operaciones para crear y eliminar entidades compuestas. Seguiremos los siguientes pasos:

1. Agregar nuevas operaciones a la API de servicio compuesto
2. Agregar métodos a la capa de integración
3. Implementación de las nuevas operaciones de API compuestas
4. Actualización de las pruebas de servicio compuestas

Agregar nuevas operaciones a la API de servicio compuesto

Las versiones compuestas de creación y eliminación de entidades, así como la gestión de entidades agregadas, son similares a las operaciones de creación y eliminación de las API de servicio principales. La principal diferencia radica en que incorporan anotaciones para la documentación basada en OpenAPI. Para obtener una explicación del uso de las anotaciones `@Operation` y `@ApiResponse` de OpenAPI , consulte el Capítulo 5, "Añadir una descripción de API mediante OpenAPI", específicamente la sección "Añadir documentación específica de la API a la interfaz ProductCompositeService" .

La operación de API para crear una entidad de producto compuesta se declara de la siguiente manera:

```

@Operación(
    resumen = "${
        api.producto-compuesto.crear-producto-compuesto.descripcion}",
    descripción = "${api.product-composite.create-composite-product.notes}")
@ApiResponses(valor = {
    @ApiResponse(códigoDeRespuesta = "400", descripción = "${

```

```

        api.responseCodes.badRequest.description}"),
        @ApiResponse(responseCode = "422", descripción = "${
            api.responseCodes.unprocessableEntity.description}") })

@PostMapping(
    valor      = "/producto-compuesto",
    consume    = "aplicación/json")
void createProduct(@RequestBody Cuerpo del agregador del producto );

```

La operación de API para eliminar una entidad de producto compuesta se declara de la siguiente manera:

```

@Operation( resumen = "${api.product-composite.delete-composite-product.
    descripción}",
    descripción = "${api.product-composite.delete-composite-product.notes}")
@ApiResponses(valor = {
    @ApiResponse(responseCode = "400", descripción = "${
        {api.responseCodes.badRequest.description}"), @ApiResponse(responseCode = "422", descripción = "${api.responseCodes
        Entidad no procesable.descripcion}") })

>DeleteMapping(valor = "/compuesto-del-producto/{Id-del-producto}") void
deleteProduct(@PathVariable int Id-del-producto);

```

Para obtener el código fuente completo, consulte la interfaz Java ProductCompositeService en el proyecto API .

También necesitamos, como antes, agregar el texto descriptivo de la documentación de la API al archivo de propiedades, application.yml, en el proyecto compuesto del producto :

crear-producto-compuesto:

Descripción: Crea un producto compuesto **notas:** |

Respuesta normal

La información del producto compuesto publicada en la API será

Dividir y almacenar como información de producto separada, recomendación
y entidades de revisión.

Respuestas de error esperadas

1. Si ya existe un producto con el mismo ID de producto que el especificado
en la información publicada, se generará un ****422 - No procesable**

Se generará un error de entidad** con un mensaje de error de "clave duplicada".
Devuelto

eliminar-producto-compuesto:

Descripción: Elimina un producto compuesto de notas:

|

Respuesta normal

Entidades para información de productos, recomendaciones y reseñas

Se eliminarán los relacionados con el productId especificado.

La implementación del método delete es idempotente, es decir,

Se puede llamar varias veces con la misma respuesta.

Esto significa que una solicitud de eliminación de un producto inexistente será...
devolver **200 Ok**.

Usando el visor Swagger UI, la documentación actualizada de OpenAPI se verá así:

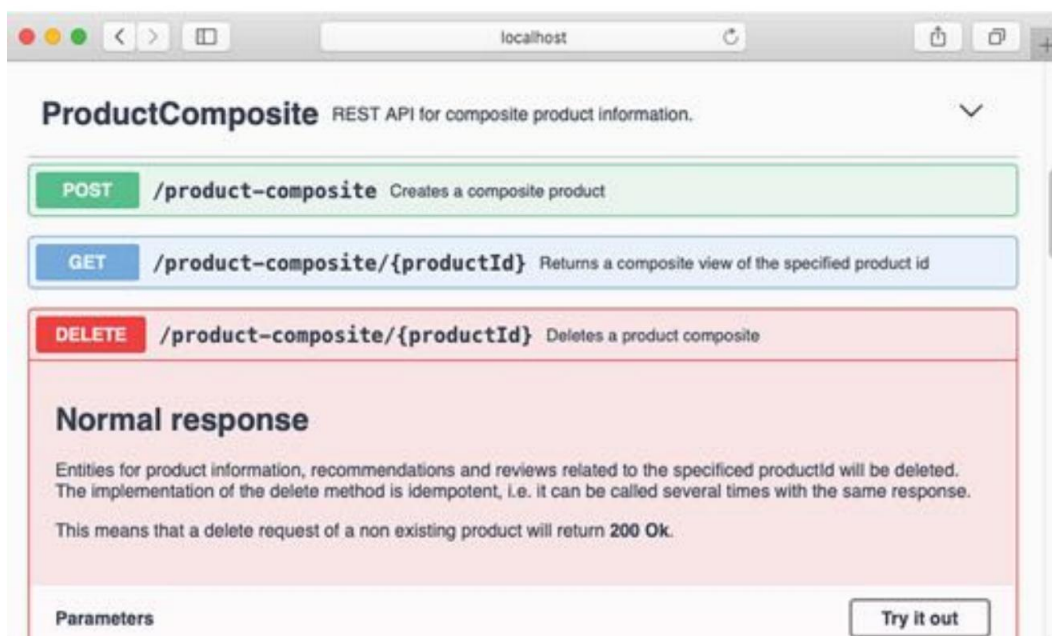


Figura 6.6: Documentación de OpenAPI actualizada

Más adelante en este capítulo, utilizaremos el visor de interfaz de usuario Swagger para probar las nuevas operaciones de API compuestas.

Agregar métodos a la capa de integración

Antes de que podamos implementar las nuevas API de creación y eliminación en los servicios compuestos, necesitamos ampliar la capa de integración para que pueda llamar a las operaciones de creación y eliminación subyacentes en las API de los microservicios principales.

Los métodos en la capa de integración para llamar a las operaciones de creación y eliminación en los tres microservicios principales son sencillos y similares entre sí, por lo que solo revisaremos el código fuente de los métodos que llaman al microservicio del producto .

El método `createProduct()` se ve así:

```
@Anular
Producto público createProduct( Cuerpo del producto) {
    intentar {
        devolver restTemplate.postForObject(
            productServiceUrl, cuerpo, Producto.class);
    } captura (HttpClientErrorException ex) {
        lanzar handleHttpClientException(ex);
    }
}
```

Simplemente delega la responsabilidad de enviar la solicitud HTTP al objeto `RestTemplate` y delega el manejo de errores al método auxiliar, `handleHttpClientException`.

El método `deleteProduct()` se ve así:

```
@Anular
public void deleteProduct(int IdProducto) {
    intentar {
        restTemplate.delete(productServiceUrl + "/" + productId);
    } captura (HttpClientErrorException ex) {
        lanzar handleHttpClientException(ex);
    }
}
```

Se implementa de la misma manera que el método de creación, pero realiza una solicitud de eliminación HTTP. en cambio.

El código fuente completo de la capa de integración se puede encontrar en `ProductCompositeIntegration` clase en el proyecto compuesto de producto .

Implementación de las nuevas operaciones de API compuestas

¡Ahora podemos implementar los métodos compuestos de creación y eliminación!

El método de creación compuesto dividirá el objeto de producto agregado en objetos discretos para producto, recomendación y revisión , y llamará a los métodos de creación correspondientes en la capa de integración:

```
@Override
public void createProduct(ProductAggregate cuerpo) {
    prueba
    { Producto producto = new Producto(body.getProductId(),
    body.getName(), body.getWeight(), null);
    integración.createProduct(producto); si
    (body.getRecommendations() != null) {
        body.getRecommendations().forEach(r ->
        { Recomendación recomendación = nueva Recomendación(
        cuerpo.getProductId(),
        r.getRecommendationId(), r.getAuthor(), r.getRate(),
        r.getContent(), null);
        integración.createRecommendation(recomendación); });
    }

    si (cuerpo.getReviews() != null) {
        cuerpo.getReviews().forEach(r -> {
            Revisar revisión = nueva Revisión(body.getProductId(),
            r.getReviewId(), r.getAuthor(), r.getSubject(), r.getContent(),
            null);
            integración.createReview(revisión); });
    }
} catch (RuntimeException re) {
    LOG.warn("la creación de ProductoCompuesto falló", re);
    throw re;
}
}
```

El método de eliminación compuesto simplemente llama a los tres métodos de eliminación en la capa de integración para eliminar las entidades correspondientes en las bases de datos subyacentes:

```
@Anular
public void deleteProduct(int IdProducto) {
    integración.deleteProduct(productId);
    integración.deleteRecommendations(productId);
    integración.deleteReviews(productId);
}
```

El código fuente completo para la implementación del servicio se puede encontrar en `ProductCompositeServiceImpl` clase en el proyecto compuesto de producto .

Para escenarios de días felices, esta implementación funcionará bien, pero si consideramos varios escenarios de error, veremos que esta implementación causará problemas.

¿Qué sucede, por ejemplo, si uno de los microservicios principales subyacentes no está disponible temporalmente, por ejemplo, debido a problemas internos, de red o de base de datos?

Esto podría resultar en la creación o eliminación parcial de productos compuestos. En el caso de la operación de eliminación, esto se puede solucionar simplemente llamando al método de eliminación del compuesto hasta que se complete correctamente. Sin embargo, si el problema persiste, es probable que el solicitante abandone la operación, lo que resultará en un estado inconsistente del producto compuesto, lo cual es inaceptable en la mayoría de los casos.

En el próximo capítulo, Capítulo 7, Desarrollo de Microservicios Reactivos, veremos cómo podemos abordar este tipo de deficiencias con API sincrónicas como una API RESTful.

Por ahora, sigamos adelante con este frágil diseño en mente.

Actualización de las pruebas de servicio compuestas

Las pruebas de servicios compuestos, como ya se mencionó en el Capítulo 3, " Creación de un conjunto de microservicios cooperativos" (consulte la sección "Añadir pruebas automatizadas de microservicios de forma aislada "), se limitan al uso de componentes simulados simples en lugar de los servicios principales. Esto nos impide probar escenarios más complejos, como la gestión de errores al intentar crear duplicados en las bases de datos subyacentes. Por lo tanto, las pruebas para las operaciones de creación y eliminación de la API compuesta son relativamente sencillas:

```
@Prueba
void crearProductoCompuesto1() {
    ProductoAgregado compuestoProducto = nuevo ProductoAgregado(1, "nombre",
        1, nulo, nulo, nulo);
}
```



```

        postAndVerifyProduct(productocompuesto, OK);
    }

    @Prueba
    void crearProductoCompuesto2() {
        ProductoAgregado compuestoProducto = nuevo ProductoAgregado(1, "nombre",
            1, singletonList(nuevo ResumenDeRecomendaciones(1, "a", 1, "c")), singletonList(nuevo
                ResumenDeRevisiones(1, "a", "s", "c")), null);
        postAndVerifyProduct(productocompuesto, OK);
    }

    @Prueba
    void eliminarProductoCompuesto() {
        ProductoAgregado compuestoProducto = nuevo ProductoAgregado(1, "nombre",
            1, singletonList(nuevoResumenRecomendaciones (1, "a", 1, "c")),
            singletonList(nuevo ReviewSummary(1, "a", "s", "c")), null);
        postAndVerifyProduct(productocompuesto, OK);
        deleteAndVerifyProduct(productocompuesto.getProductId(), OK);
        deleteAndVerifyProduct(productocompuesto.getProductId(), OK);
    }

```

El código fuente completo para la prueba de servicio se puede encontrar en la clase `ProductCompositeServiceApplicationTests` en el proyecto `compuesto de producto`.

Estos son todos los cambios necesarios en el código fuente. Antes de poder probar los microservicios juntos, debemos aprender a agregar bases de datos al entorno del sistema administrado por Docker Compose.

Agregar bases de datos al entorno de Docker Compose

Ahora tenemos todo el código fuente listo. Antes de iniciar el entorno de microservicios y probar las nuevas API junto con la nueva capa de persistencia, debemos iniciar algunas bases de datos.

Incorporaremos MongoDB y MySQL al panorama del sistema controlado por Docker Compose y agregaremos configuración a nuestros microservicios para que puedan encontrar sus bases de datos cuando se ejecuten.

La configuración de Docker Compose

MongoDB y MySQL se declaran de la siguiente manera en el archivo de configuración de Docker Compose, `docker-composer.yml`:

```

mongodb:
  imagen: mongo:8.0.5
  mem_limit: 512m puertos:
    - "27017:27017"
  Comando: mongod
  healthcheck:
    prueba: echo 'db.runCommand("ping").ok' | mongosh --quiet
    intervalo: 5 s
    tiempo de espera: 2 s
    reintentos: 60

MySQL:
  imagen: mysql:9.2.0 límite de
  memoria: 512m
  puertos:
    - "3306:3306"
  ambiente:
    - CONTRASEÑA_RAÍZ_MYSQL=contraseña_raíz
    - MYSQL_DATABASE=revisión-db
    - MYSQL_USER=usuario
    - MYSQL_PASSWORD=contraseña
  control de salud:
    prueba: "/usr/bin/mysql --user=usuario --contraseña=pwd --ejecutar \"MOSTRAR
BASES DE DATOS;\n\"
  intervalo: 5 s
  tiempo de espera: 2 s
  reintentos: 60

```

Las siguientes son notas sobre el código anterior:

- Usaremos la imagen oficial de Docker para MongoDB v8.0.5 y MySQL v9.2.0 y reenviaremos sus puertos predeterminados, 27017 y 3306, al host de Docker, que también están disponibles en localhost cuando se usa Docker Desktop para Mac.
- Para MySQL, también declaramos algunas variables de entorno, definiendo lo siguiente:
 - La contraseña de root
 - El nombre de la base de datos que se creará al iniciar el contenedor
 - Un nombre de usuario y una contraseña para un usuario configurado para la base de datos en el contenedor

puesta en marcha

- También declaramos una comprobación de estado que Docker ejecutará para determinar el estado del Mon-bases de datos goDB y MySQL

Para evitar que los microservicios intenten conectarse a las bases de datos antes de que estén en funcionamiento, el producto y los servicios de recomendación se declaran como dependientes de la base de datos MongoDB, de la siguiente manera:

```
depende_de:
  MongoDB:
    condición: servicio_saludable
```

Por la misma razón, el servicio de revisión se declara como dependiente de la base de datos MySQL:

```
depende_de:
  MySQL:
    condición: servicio_saludable
```

Esto significa que Docker Compose no iniciará los contenedores de microservicios hasta que se inicien los contenedores de base de datos y sus controles de estado los informen como en buen estado.

Configuración de la conexión a la base de datos

Con la base de datos instalada, necesitamos configurar los microservicios principales para que sepan cómo conectarse a sus bases de datos. Esto se configura en el archivo de configuración de cada microservicio principal (application.yml) en los proyectos de servicio de producto, servicio de recomendación y servicio de reseña .

La configuración de los servicios de producto y recomendación es similar, por lo que solo analizaremos la configuración del servicio de producto . La siguiente parte de la configuración es de interés:

```
spring.data.mongodb:
  anfitrión: localhost
  puerto: 27017
  base de datos: product-db

explotación florestal:
  nivel:
    org.springframework.data.mongodb.core.MongoTemplate: DEBUG

spring.config.activate.on-profile: docker
```

```
spring.data.mongodb.host: mongodb
```

Las siguientes son partes importantes del código anterior:

- Cuando se ejecuta sin Docker utilizando el perfil Spring predeterminado, se espera la base de datos
Para ser accesible en localhost:27017
- Establecer el nivel de registro para MongoTemplate en DEBUG nos permitirá ver qué MongoDB
Las declaraciones se ejecutan en el registro
- Cuando se ejecuta dentro de Docker utilizando el perfil Spring, se espera que la base de datos de Docker...
Para ser accesible en mongodb:27017

La configuración del servicio de revisión , que afecta la forma en que se conecta a su base de datos SQL, se parece a la siguiente:

```
spring.jpa.hibernate.ddl-auto: actualización
```

primavera.fuente de datos:

```
URL: jdbc:mysql://localhost/review-db
```

```
nombre de usuario: usuario
```

```
contraseña: pwd
```

```
spring.datasource.hikari.initializationFailTimeout: 60000
```

```
explotación florestal:
```

```
nivel:
```

```
org.hibernate.SQL: DEPURACIÓN
```

```
org.hibernate.type.descriptor.sql.BasicBinder: TRACE
```

```
...
```

```
spring.config.activate.on-profile: docker
```

primavera.fuente de datos:

```
URL: jdbc:mysql://mysql/review-db
```

La siguiente es una explicación del código anterior:

- De forma predeterminada, Spring Data JPA utilizará Hibernate como EntityManager de JPA.

- La propiedad `spring.jpa.hibernate.ddl-auto` se utiliza para indicarle a Spring Data JPA que cree crear tablas SQL nuevas o actualizar las existentes durante el inicio.



Nota: Se recomienda encarecidamente configurar `spring.jpa.hibernate.ddl-auto`. Establezca la propiedad como "ninguna" o valide en un entorno de producción. Esto evita que Spring Data JPA manipule la estructura de las tablas SQL. Para más información, consulte <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#como-inicializar-una-base-de-datos>.

- Cuando se ejecuta sin Docker, utilizando el perfil Spring predeterminado, se espera la base de datos Ser accesible en localhost usando el puerto predeterminado 3306.

De forma predeterminada, Spring Data JPA utiliza HikariCP como pool de conexiones JDBC. Para minimizar los problemas de arranque en equipos con recursos de hardware limitados, el parámetro `initializationFailTimeout` se establece en 60 segundos. Esto significa que la aplicación Spring Boot esperará hasta 60 segundos durante el arranque para establecer una base de datos. conexión.

La configuración de registro de Hibernate hará que muestre las sentencias SQL utilizadas y los valores reales. Tenga en cuenta que, al usar Hibernate en un entorno de producción, se debe evitar escribir los valores reales en el registro por motivos de privacidad.

- Cuando se ejecuta dentro de Docker utilizando el perfil Spring, se espera que la base de datos de Docker... Ser accesible en el nombre de host MySQL usando el puerto predeterminado 3306.

Con esta configuración, estamos listos para iniciar el entorno del sistema. Pero antes, aprendamos cómo ejecutar las herramientas CLI de la base de datos.

Las herramientas CLI de MongoDB y MySQL

Una vez que hayamos comenzado a ejecutar algunas pruebas con los microservicios, será interesante ver qué datos se almacenan realmente en sus bases de datos. Cada contenedor Docker de base de datos incluye herramientas basadas en CLI que permiten consultar las tablas y colecciones de la base de datos. Para ejecutar las herramientas CLI de la base de datos, se puede usar el comando "exec" de Docker Compose.

Los comandos descritos en esta sección se usarán al realizar las pruebas manuales en la siguiente sección. No intente ejecutarlos ahora; fallarán, ya que aún no tenemos bases de datos en funcionamiento.

Para iniciar la herramienta CLI de MongoDB, mongo, dentro del contenedor mongod , ejecute el siguiente comando:

```
docker compose exec mongod mongosh —quiet
```

```
>
```

Ingrese exit para salir de la CLI de mongo .

Para iniciar la herramienta CLI de MySQL, mysql, dentro del contenedor mysql e iniciar sesión en review-db con el usuario creado al inicio, ejecute el siguiente comando:

```
docker compose exec mysql mysql -uuser -p review-db
MySQL>
```



La herramienta CLI de MySQL le solicitará una contraseña; puede encontrarla en el archivo docker-compose.yml . Busque el valor de la variable de entorno MYSQL_PASSWORD .

Ingrese exit para salir de la CLI de MySQL .

Veremos el uso de estas herramientas en la siguiente sección.

Pruebas manuales de las nuevas API y la capa de persistencia

Ahora tenemos todo listo para probar los microservicios juntos. Construiremos nuevas imágenes de Docker e iniciaremos el entorno del sistema usando Docker Compose, basándonos en ellas.

A continuación, usaremos el visor de interfaz de usuario de Swagger para ejecutar algunas pruebas manuales. Finalmente, usaremos los datos... Herramientas CLI base para ver qué datos se insertaron en las bases de datos.

Construya e inicie el panorama del sistema con el siguiente comando:

```
cd $BOOK_HOME/Capítulo06
./gradlew compilación y docker compose compilación y docker compose up
```

Abra Swagger UI en un navegador web, <http://localhost:8080/openapi/swagger-ui.html>, y realice los siguientes pasos en la página web:

1. Haga clic en el servicio ProductComposite y en el método POST para expandirlos.
2. Haga clic en el botón Probarlo y baje hasta el campo del cuerpo.
3. Reemplace el valor predeterminado, 0, del campo productId con 123456.
4. Desplácese hacia abajo hasta el botón Ejecutar y haga clic en él.
5. Verifique que el código de respuesta devuelto sea 200.

La siguiente es una captura de pantalla de muestra después de presionar el botón Ejecutar :

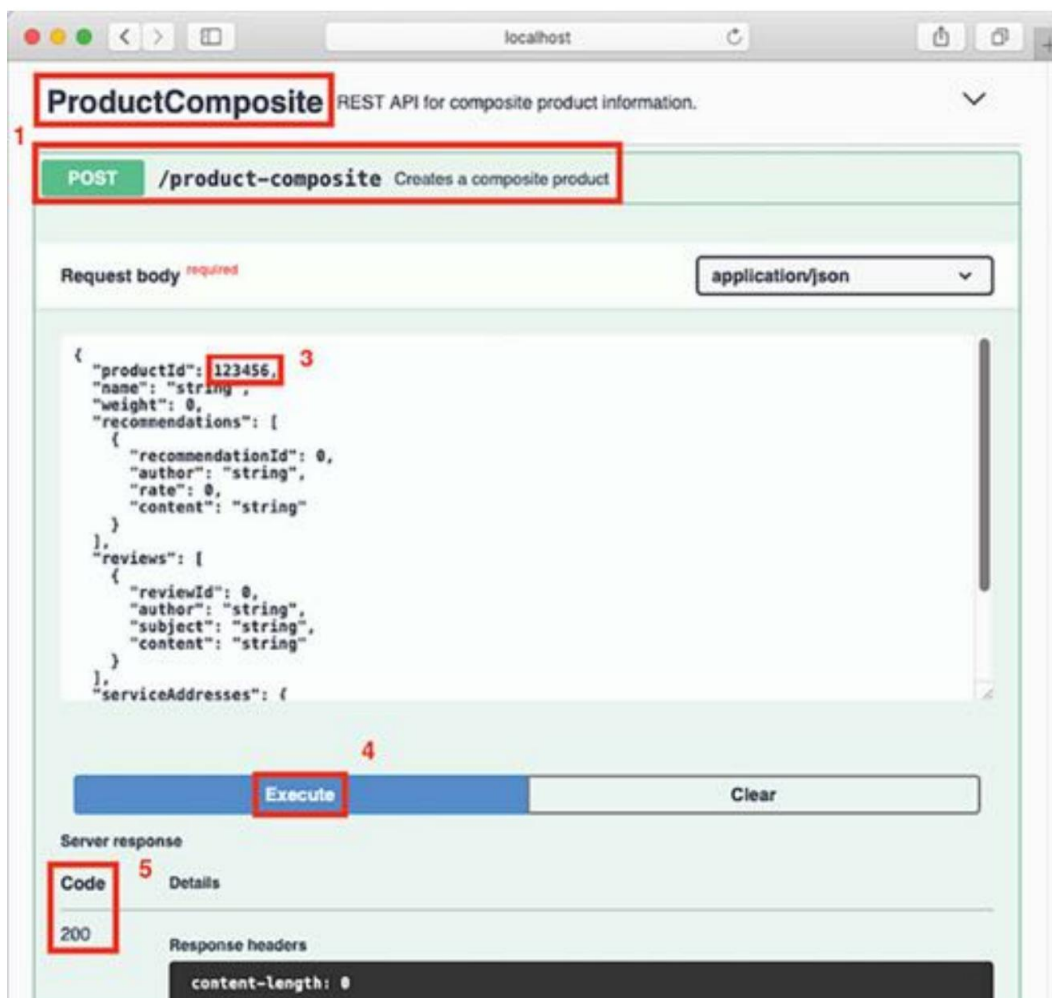


Figura 6.7: Prueba de la respuesta del servidor

En la salida del registro del comando docker compose up , deberíamos poder ver una salida como la siguiente (abreviada para mayor legibilidad):

```
product-composite_1 | ... createCompositeProduct: creates a new composite entity for productId: 123456
product_1 | ... createProduct: entity created for productId: 123456
recommendation_1 | ... createRecommendation: created a recommendation entity: 123456/0
review_1 | ... createReview: created a review entity: 123456/0
```

Figura 6.8: Salida del registro de Docker Compose Up

También podemos utilizar las herramientas CLI de la base de datos para ver el contenido real en las diferentes bases de datos.

Busque el contenido en el servicio de producto , es decir, la colección de productos en MongoDB, con el siguiente comando:

```
docker compose exec mongodb mongosh product-db --quiet --eval "db.products.find()"
```

Espere una respuesta como esta:



```
$ docker-compose exec mongodb mongosh product-db --quiet --eval "db.products.find()"
[
  {
    _id: ObjectId("6419b18e62925d253399fe46"),
    version: 0,
    productId: 123456,
    name: 'string',
    weight: 0,
    _class: 'se.magnus.microservices.core.product.persistence.ProductEntity'
  }
]
```

Figura 6.9: Búsqueda de productos

Busque el contenido en el servicio de recomendaciones , es decir, la colección de recomendaciones en MongoDB, con el siguiente comando:

```
docker compose exec mongodb mongosh recomendación-db --quiet --eval "db.recomendaciones.find()"
```

Espere una respuesta como esta:



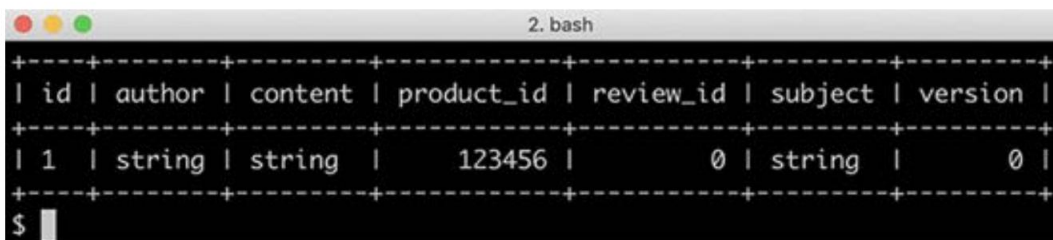
```
$ docker-compose exec mongodb mongosh recommendation-db --quiet --eval "db.recommendations.find()"
[
  {
    _id: ObjectId("6419b18ef5533a465e39565a"),
    version: 0,
    productId: 123456,
    recommendationId: 0,
    author: 'string',
    rating: 0,
    content: 'string',
    _class: 'se.magnus.microservices.core.recommendation.persistence.RecommendationEntity'
  }
]
```

Figura 6.10: Búsqueda de recomendaciones

Busque el contenido en el servicio de revisión , es decir, la tabla de revisiones en MySQL, con lo siguiente dominio:

```
docker compose exec mysql mysql -uuser -p review-db -e "seleccionar * de las reseñas"
```

La herramienta CLI de MySQL le solicitará una contraseña; puede encontrarla en docker-compose.yml Archivo. Busque el valor de la variable de entorno MYSQL_PASSWORD . Espere una respuesta como la siguiente:



```

+-----+-----+-----+-----+-----+-----+-----+
| id | author | content | product_id | review_id | subject | version |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | string | string | 123456 | 0 | string | 0 |
+-----+-----+-----+-----+-----+-----+-----+
$ 

```

Figura 6.11: Búsqueda de reseñas

Desactive el entorno del sistema interrumpiendo el comando docker compose up con Ctrl + C, seguido del comando docker compose down . Después, veamos cómo actualizar las pruebas automatizadas en un entorno de microservicios.

Actualización de las pruebas automatizadas del entorno de microservicios

Las pruebas automatizadas del entorno de microservicios, test-em-all.bash, deben actualizarse para garantizar que la base de datos de cada microservicio tenga un estado conocido antes de ejecutar las pruebas.

El script se amplía con una función de configuración, setupTestdata(), que utiliza las API compuestas de creación y eliminación para configurar los datos de prueba utilizados por las pruebas.

La función setupTestdata se ve así:

```
función setupTestdata() {

    cuerpo=\
        '{"productId":1,"name":"producto 1","weight":1, "recommendations":[
            {"recommendationId":1,"author":"autor
              1", "tasa":1, "contenido":"contenido 1"},
            {"recommendationId":2,"author":"autor
```

```

        2", "calificación":2, "contenido":"contenido 2"},
        {"recomendaciónId":3, "autor":"autor
        3", "tasa":3, "contenido":"contenido 3"}
    ], "reseñas":[
        {"reviewId":1, "author":"autor 1", "subject":"asunto
        1", "contenido":"contenido 1"},
        {"reviewId":2, "author":"autor 2", "subject":"asunto
        2", "contenido":"contenido 2"},
        {"reviewId":3, "author":"autor 3", "subject":"asunto
        3", "contenido":"contenido 3"}
    ]}'
    recreateComposite 1 "$cuerpo"

    cuerpo=\
        '{"productId":113,"name":"producto 113","weight":113, "reviews":[
            {"reviewId":1,"author":"autor 1","subject":"asunto
            1", "contenido":"contenido 1"},
            {"reviewId":2,"author":"autor 2","subject":"asunto
            2", "contenido":"contenido 2"},
            {"reviewId":3,"author":"autor 3","subject":"asunto
            3", "contenido":"contenido 3"} ]}'

    recreateComposite 113 "$cuerpo"

    cuerpo=\
        '{"productId":213,"name":"producto 213","peso":213,
        "recomendaciones":[
            {"recommendationId":1,"author":"autor
            1", "calificación":1, "contenido":"contenido 1"},
            {"recomendaciónId":2, "autor":"autor
            2", "calificación":2, "contenido":"contenido 2"},
            {"recomendaciónId":3, "autor":"autor
            3", "tasa":3, "contenido":"contenido 3"}
        ]}'
    recreateComposite 213 "$cuerpo"
}

```

Utiliza una función auxiliar, `recreateComposite()`, para realizar las solicitudes reales de eliminación y crear API:

```
función recreateComposite() {
  Id. del producto local=$1
  compuesto local=$2

  assertCurl 200 "curl -X ELIMINAR http://$HOST:$PORT/producto-
    compuesto/${productId} -s"
  curl -X POST http://$HOST:$PORT/compuesto-de-producto -H "Tipo-de-contenido:
    aplicación/json" --data "$compuesto"
}
```

La función `setupTestdata` se llama directamente después de la función `waitForService` :

```
waitForService curl -X ELIMINAR http://$HOST:$PORT/producto-compuesto/13
configuraciónTestdata
```

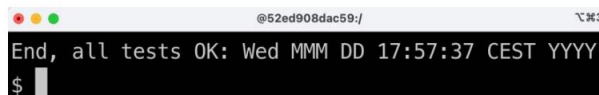
El objetivo principal de la función `waitForService` es verificar que todos los microservicios estén en funcionamiento. En el capítulo anterior, se utilizó la API `get` del servicio de producto compuesto. En este capítulo, se utiliza la API `delete`. Al usar la API `get`, solo se llama al microservicio principal del producto si no se encuentra la entidad; no se llamará a los servicios de recomendación y revisión para verificar su funcionamiento. La llamada a la API `delete` también garantizará que se detecte el error "No encontrado".

La prueba con el ID de producto 13 se realizará correctamente. En el siguiente capítulo, veremos cómo definir API específicas para verificar el estado de un entorno de microservicios.

Ejecute el script de prueba actualizado con el siguiente comando:

```
cd $BOOK_HOME/Capítulo06
./test-em-all.bash inicio parada
```

La ejecución debe finalizar escribiendo un mensaje de registro como este:



```
@52ed908dac59:/
End, all tests OK: Wed MMM DD 17:57:37 CEST YYYY
$
```

Figura 6.12: Mensaje de registro al final de la ejecución de la prueba

Con esto concluyen las actualizaciones de las pruebas automatizadas del panorama de microservicios.

Resumen

En este capítulo, vimos cómo podemos usar Spring Data para agregar una capa de persistencia a los microservicios principales . Usamos los conceptos básicos de Spring Data, repositorios y entidades para almacenar datos tanto en MongoDB como en MySQL. El modelo de programación es similar para una base de datos NoSQL como MongoDB y una base de datos SQL como MySQL, aunque no es completamente portable. También vimos cómo las anotaciones de Spring Boot, `@DataMongoTest` y `@DataJpaTest`, se pueden usar para configurar convenientemente pruebas dirigidas a la persistencia; aquí es donde una base de datos se inicia automáticamente antes de que se ejecute la prueba, pero no se inicia ninguna otra infraestructura que el microservicio necesitará en tiempo de ejecución, por ejemplo, un servidor web como Netty. Para manejar el inicio y el desmontaje de las bases de datos, hemos usado Testcontainers, que ejecuta las bases de datos en contenedores Docker. Esto da como resultado pruebas de persistencia que son fáciles de configurar y que comienzan con una sobrecarga mínima.

También hemos visto cómo la capa de persistencia puede ser utilizada por la capa de servicio y cómo podemos agregar APIs para crear y eliminar entidades, tanto centrales como compuestas.

Finalmente, aprendimos lo conveniente que es iniciar bases de datos como MongoDB y MySQL en tiempo de ejecución usando Docker Compose y cómo usar las nuevas API de creación y eliminación para configurar datos de prueba antes de ejecutar pruebas automatizadas del panorama del sistema basado en microservicios.

Sin embargo, en este capítulo se identificó una preocupación importante. Actualizar (crear o eliminar) una entidad compuesta (una entidad cuyas partes se almacenan en varios microservicios) mediante API síncronas puede generar inconsistencias si no se actualizan correctamente todos los microservicios involucrados.

En general, esto no es aceptable. Esto nos lleva al siguiente capítulo, donde analizaremos por qué y cómo construir microservicios reactivos, es decir, microservicios escalables y robustos.

Preguntas

1. Spring Data, un modelo de programación común basado en entidades y repositorios, puede utilizarse para diferentes tipos de motores de bases de datos. A partir de los ejemplos de código fuente de este capítulo, ¿cuáles son las diferencias más importantes en el código de persistencia para MySQL y MongoDB?
2. ¿Qué se requiere para implementar el bloqueo optimista usando Spring Data?
3. ¿Para qué se utiliza MapStruct?
4. ¿Qué significa si una operación es idempotente y por qué es útil?
5. ¿Cómo podemos acceder a los datos almacenados en las bases de datos MySQL y MongoDB sin
 ¿Utilizando la API?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



7

Desarrollo reactivo Microservicios

En este capítulo, aprenderemos a desarrollar microservicios reactivos, es decir, a desarrollar API REST síncronas no bloqueantes y servicios asíncronos basados en eventos. También aprenderemos a elegir entre estas dos alternativas. Finalmente, veremos cómo crear y ejecutar pruebas manuales y automatizadas de un entorno de microservicios reactivos.

Como ya se describió en el Capítulo 1, Introducción a los Microservicios, la base de los sistemas reactivos es que se basan en mensajes y utilizan comunicación asíncrona. Esto les permite ser elásticos (es decir, escalables y resilientes), lo que significa que toleran fallos. La elasticidad y la resiliencia juntas permiten que un sistema reactivo responda.

En este capítulo se tratarán los siguientes temas:

- Elegir entre API síncronas sin bloqueo y servicios asíncronos controlados por eventos. vicios
- Desarrollo de API REST síncronas sin bloqueo
- Desarrollo de servicios asíncronos basados en eventos
- Ejecución de pruebas manuales del panorama de microservicios reactivos
- Ejecución de pruebas automatizadas del panorama de microservicios reactivos

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter07`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver qué se necesita para que los microservicios sean reactivos, puede compararlo con el código fuente del Capítulo 6, "Añadir persistencia". Puede usar su herramienta de comparación preferida y comparar las dos carpetas, es decir, `$BOOK_HOME/` Capítulo 06 y `$BOOK_HOME/` Capítulo 07.

Elegir entre API sincrónicas sin bloqueo y servicios asíncronos controlados por eventos

Al desarrollar microservicios reactivos, no siempre es evidente cuándo usar API síncronas sin bloqueo y cuándo usar servicios asíncronos basados en eventos. En general, para que un microservicio sea robusto y escalable, es importante que sea lo más autónomo posible, por ejemplo, minimizando sus dependencias en tiempo de ejecución. Esto también se conoce como acoplamiento flexible. Por lo tanto, el paso asíncrono de mensajes de eventos es preferible al uso de API síncronas. Esto se debe a que el microservicio solo dependerá del acceso al sistema de mensajería en tiempo de ejecución, en lugar de depender del acceso síncrono a otros microservicios.

Sin embargo, hay una serie de casos en los que las API sincrónicas podrían ser favorables, como los siguientes:

- Para operaciones de lectura donde un usuario final está esperando una respuesta
- Donde las plataformas de cliente son más adecuadas para consumir API sincrónicas, por ejemplo, aplicaciones móviles o aplicaciones web SPA
- Donde los clientes se conectarán al servicio desde otras organizaciones, donde podría ser difícil acordar un sistema de mensajería común para usar en todas las organizaciones

Para el panorama del sistema en este libro, utilizaremos lo siguiente:

Los servicios de creación, lectura y eliminación expuestos por el microservicio compuesto del producto se basarán en API síncronas sin bloqueo. Se asume que el microservicio compuesto tiene clientes en plataformas web y móviles, así como clientes que provienen de otras organizaciones, además de las que operan el entorno del sistema. Por lo tanto, la sincronización...
Las API crónicas parecen una combinación natural.

- Los servicios de lectura proporcionados por los microservicios principales también se desarrollarán como API síncronas sin bloqueo, ya que hay un usuario final esperando sus respuestas.
- Los servicios de creación y eliminación proporcionados por los microservicios principales se desarrollarán como servicios asíncronos controlados por eventos, lo que significa que escucharán las operaciones de creación y eliminación. Eventos sobre temas dedicados a cada microservicio.

Las API síncronas proporcionadas por los microservicios compuestos para crear y eliminar información agregada de productos publicarán eventos de creación y eliminación en estos temas. Si la operación de publicación se realiza correctamente, devolverá una respuesta 202 (Aceptado) ; de lo contrario, se devolverá una respuesta de error. La respuesta 202 difiere de una respuesta 200 (OK) normal : indica que la solicitud se ha aceptado, pero no se ha procesado completamente. En cambio, el procesamiento se completará de forma asíncrona e independiente de la respuesta 202 .

Esto queda ilustrado en el siguiente diagrama:

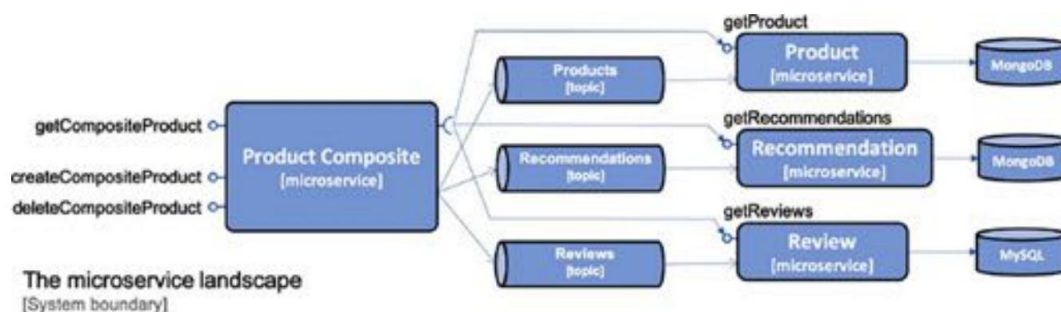


Figura 7.1: El panorama de microservicios

Primero, aprendamos cómo podemos desarrollar API REST síncronas sin bloqueo y, luego, veremos cómo desarrollar servicios asíncronos controlados por eventos.

Desarrollo de API REST síncronas sin bloqueo

En esta sección, aprenderemos a desarrollar versiones no bloqueantes de las API de lectura. El servicio compuesto realizará llamadas reactivas (es decir, no bloqueantes) en paralelo a los tres servicios principales.

Cuando el servicio compuesto haya recibido respuestas de todos los servicios principales, creará una respuesta compuesta y la enviará al emisor. Esto se ilustra en el siguiente diagrama:

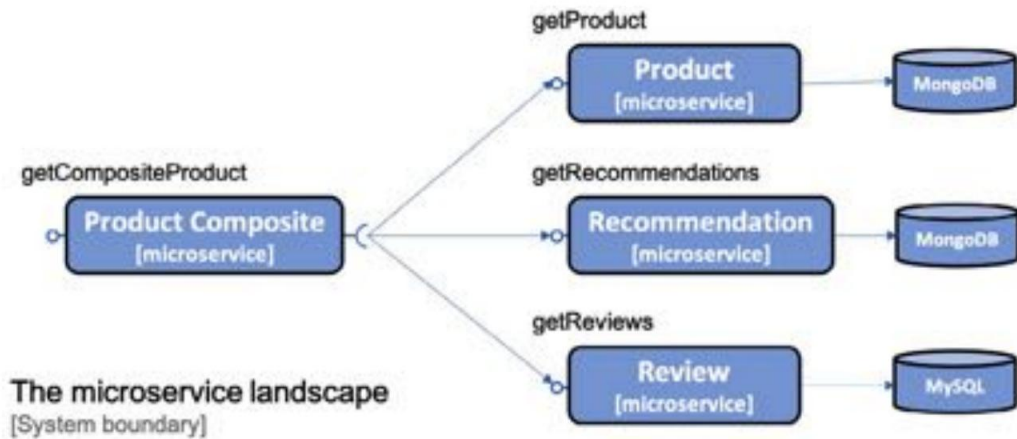


Figura 7.2: La parte getCompositeProduct del paisaje

En esta sección cubriremos lo siguiente:

- Una introducción al Proyecto Reactor
- Persistencia sin bloqueo utilizando Spring Data para MongoDB
- API REST sin bloqueo en los servicios principales, incluido cómo manejar el código de bloqueo para la capa de persistencia basada en JPA
- API REST sin bloqueo en el servicio compuesto

Una introducción al Proyecto Reactor

Como mencionamos en la sección Spring WebFlux del Capítulo 2, Introducción a Spring Boot, el soporte reactivo en Spring 5 se basa en Project Reactor (<https://projectreactor.io>). El proyecto Reactor se basa en la especificación Reactive Streams (<http://www.reactive-streams.org>), Un estándar para la creación de aplicaciones reactivas. Project Reactor es fundamental: es en lo que se basan Spring WebFlux, Spring WebClient y Spring Data para ofrecer sus funciones reactivas y sin bloqueo.

El modelo de programación se basa en el procesamiento de flujos de datos, y los tipos de datos principales en Project Reactor son Flux y Mono. Un objeto Flux se utiliza para procesar un flujo de 0...n elementos, y un objeto Mono se utiliza para procesar un flujo vacío o que devuelve como máximo un elemento. Veremos numerosos ejemplos de su uso en este capítulo. A modo de breve introducción, veamos la siguiente prueba:

```
@Prueba
vacío testFlux() {
```

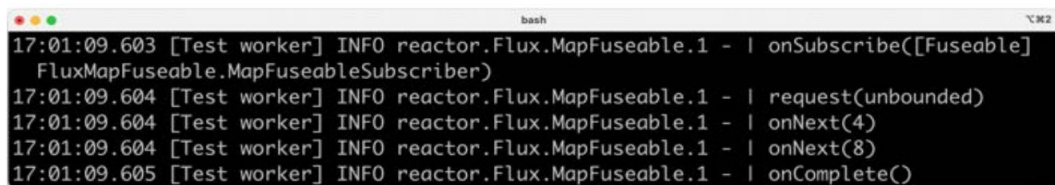
```
Lista<Entero> lista = Flux.just(1, 2, 3, 4)
    .filter(n -> n % 2 == 0)
    .map(n -> n * 2)
    .registro()
    .collectList().block();

assertThat(lista).contieneExactamente(4, 8);
}
```

A continuación se muestra una explicación del código fuente anterior:

1. Iniciamos la secuencia con los números enteros 1, 2, 3 y 4 usando el ayudante estático `Flux.just()` método.
2. A continuación, filtramos los números impares: solo permitimos que los números pares pasen. El arroyo. En esta prueba, estos son el 2 y el 4.
3. A continuación, transformamos (o mapeamos) los valores en el flujo multiplicándolos por 2, de modo que se convierten en 4 y 8.
4. Luego, registramos los datos que fluyen a través del flujo después de la operación del mapa .
5. Usamos el método `collectList` para recopilar todos los elementos del flujo en una lista, emitida una vez que se complete la transmisión.
6. Hasta ahora, solo hemos declarado el procesamiento de un flujo. Para que se procese , necesitamos que alguien se suscriba. La última llamada al método de bloque registrará un suscriptor que espera a que se complete el procesamiento.
7. La lista resultante se guarda en una variable miembro llamada `lista`.
8. Ahora podemos finalizar la prueba usando el método `assertThat` para afirmar que la lista después El procesamiento del flujo contiene el resultado esperado: los números enteros 4 y 8.

La salida del registro se verá así:



```
17:01:09.603 [Test worker] INFO reactor.Flux.MapFuseable.1 - | onSubscribe([Fuseable]
  FluxMapFuseable.MapFuseableSubscriber)
17:01:09.604 [Test worker] INFO reactor.Flux.MapFuseable.1 - | request(unbounded)
17:01:09.604 [Test worker] INFO reactor.Flux.MapFuseable.1 - | onNext(4)
17:01:09.604 [Test worker] INFO reactor.Flux.MapFuseable.1 - | onNext(8)
17:01:09.605 [Test worker] INFO reactor.Flux.MapFuseable.1 - | onComplete()
```

Figura 7.3: Salida del registro para el código anterior

De la salida del registro anterior, podemos ver lo siguiente:

1. El procesamiento de la transmisión lo inicia un suscriptor que se suscribe a la transmisión y solicita su contenido.
2. A continuación, los números enteros 4 y 8 pasan por la operación logarítmica .
3. El procesamiento concluye con una llamada al método `onComplete` en el suscriptor, notificando- confirmando que el flujo ha llegado a su fin.

Para obtener el código fuente completo, consulte la clase de prueba `ReactorTests` en el proyecto `util` .



Normalmente, no iniciamos el procesamiento del flujo. En su lugar, solo definimos cómo se procesará, y un componente de infraestructura será responsable de iniciarlo. Por ejemplo, Spring WebFlux lo hará como respuesta a una solicitud HTTP entrante. Una excepción a esta regla general es cuando el código de bloqueo necesita una respuesta de un flujo reactivo. En estos casos, el código de bloqueo puede llamar al método `block()` del objeto `Flux` o `Mono` para obtener la respuesta de forma bloqueante.

Persistencia sin bloqueo con Spring Data para MongoDB

Hacer que los repositorios basados en MongoDB para los servicios de productos y recomendaciones sean reactivos es muy sencillo:

- Cambiar la clase base de los repositorios a `ReactiveCrudRepository`
- Cambie los métodos de búsqueda personalizados para que devuelvan un objeto `Mono` o `Flux`

`ProductRepository` y `RecommendationRepository` se ven así después del cambio:

```
La interfaz pública ProductRepository extiende ReactiveCrudRepository
<ProductEntity, String> {
    Mono<ProductEntity> findByProductId(int productId);
}
```

```
La interfaz pública RecommendationRepository se extiende
RepositorioReactivoCrud<EntidadDeRecomendación, Cadena> {
    Flux<EntidadDeRecomendación> findByProductId(int IdProducto);
}
```

No se aplican cambios al código de persistencia del servicio de revisión ; este permanecerá bloqueado mediante el repositorio JPA. Consulte la siguiente sección, " Gestión del código de bloqueo", para saber cómo gestionar el código de bloqueo en la capa de persistencia del servicio de revisión .

Para ver el código fuente completo, consulte las siguientes clases:

- `ProductRepository` en el proyecto del producto
- `RecommendationRepository` en el proyecto de recomendación

Cambios en el código de prueba

Al probar la capa de persistencia, debemos realizar algunos cambios. Dado que nuestros métodos de persistencia ahora devuelven un objeto `Mono` o `Flux`, los métodos de prueba deben esperar a que la respuesta esté disponible en los objetos reactivos devueltos. Los métodos de prueba pueden usar una llamada explícita al método `block()` en el objeto `Mono/Flux` para esperar hasta que haya una respuesta disponible, o pueden usar la clase auxiliar `StepVerifier` de `Project Reactor` para declarar una secuencia verificable de asyn- eventos cronológicos.

Veamos cómo podemos cambiar el siguiente código de prueba para que funcione con la versión reactiva del repositorio:

```
ProductoEntidad foundEntity = repositorio.findById(newEntity.getId()).get();
assertEqualsProduct(nuevaEntidad, entidadEncontrada);
```

Podemos usar el método `block()` en el objeto `Mono` devuelto por el repositorio `findById()` método y mantener el estilo de programación imperativo, como se muestra aquí:

```
ProductoEntidad foundEntity = repositorio.findById(
    nuevaEntidad.getId()).block();
assertEqualsProduct(nuevaEntidad, entidadEncontrada);
```

Como alternativa, podemos usar la clase `StepVerifier` para configurar una secuencia de pasos de procesamiento que ejecute la operación de búsqueda del repositorio y verifique el resultado. La secuencia se inicializa con la última llamada al método `verifyComplete()`, como se muestra a continuación:

```
StepVerifier.create(repositorio.findById(newEntity.getId()))
    .expectNextMatches(entidadEncontrada -> areProductEqual(entidadNueva,
        Entidad encontrada))
    .verifyComplete();
```

Para obtener ejemplos de pruebas que utilizan la clase `StepVerifier`, consulte la clase de prueba `PersistenceTests` en el proyecto del producto.

Para ver ejemplos correspondientes de pruebas que utilizan el método `block()`, consulte `PersistenceTests` clase de prueba en el proyecto de recomendación.

API REST sin bloqueo en los servicios principales

Con una capa de persistencia no bloqueante implementada, es hora de que las API de los servicios principales también sean no bloqueantes. Necesitamos realizar los siguientes cambios:

- Cambiar las API para que solo devuelvan tipos de datos reactivos
- Cambiar las implementaciones del servicio para que no contengan ningún código de bloqueo
- Cambiar nuestras pruebas para que puedan probar los servicios reactivos
- Tratar el código bloqueador: aislar el código que aún necesita ser bloqueado del código no bloqueador

Cambios en las API

Para que las API de los servicios principales sean reactivas, necesitamos actualizar sus métodos para que devuelvan un objeto Mono o Flux .

Por ejemplo, `getProduct()` en el servicio de producto ahora devuelve `Mono<Product>` en lugar de un

Objeto del producto :

```
Mono<Producto> getProduct(@PathVariable int productId);
```

Para obtener el código fuente completo, eche un vistazo a las siguientes interfaces principales en el proyecto API :

- `ProductoServicio`
- `Servicio de recomendación`
- `Servicio de revisión`

Cambios en las implementaciones del servicio

Para las implementaciones de servicios en los proyectos de producto y recomendación , que utilizan una capa de persistencia reactiva, podemos usar la API fluida de Project Reactor. Por ejemplo, la implementación del método `getProduct()` se muestra en el siguiente código:

```
público Mono<Producto> obtenerProducto(int IdProducto) {  
  
    si (productId < 1) {  
        lanzar nueva IllegalArgumentException(" ProductId no válido: " + productId);  
    }  
  
    devolver repositorio.findById(productId)  
        .switchIfEmpty(Mono.error(new NotFoundException("No se encontró ningún producto  
        para productId: " + ID del producto)))  
}
```

```

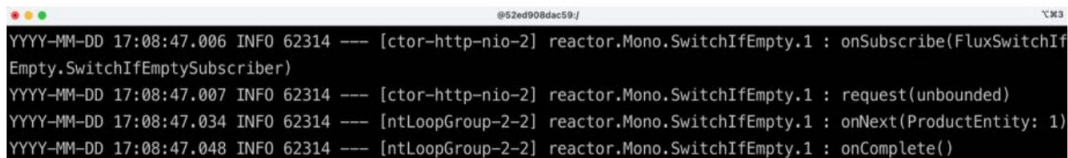
        .log(LOG.getName(), FINE)
        .map(e -> mapper.entityToApi(e))
        .map(e -> setServiceAddress(e));
    }

```

Examinemos lo que hace el código:

1. El método devolverá un objeto Mono ; el procesamiento solo se declara aquí. El procesamiento lo activa el framework web Spring WebFlux, que se suscribe al objeto Mono al recibir una solicitud a este servicio.
2. Se recuperará un producto utilizando su productId de la base de datos subyacente mediante el Método `findByProductId()` en el repositorio de persistencia.
3. Si no se encuentra ningún producto para el productId indicado, se generará un error `NotFoundException`.
4. El método de registro producirá una salida de registro.
5. Se llamará al método `mapper.entityToApi()` para transformar la entidad devuelta de la capa de persistencia en un objeto de modelo API.
6. El método de mapa final utilizará un método auxiliar, `setServiceAddress()`, para establecer el nombre DNS y la dirección IP de los microservicios que procesaron la solicitud en `serviceAddress` campo del objeto modelo.

A continuación se muestra un ejemplo de salida de registro para un procesamiento exitoso:



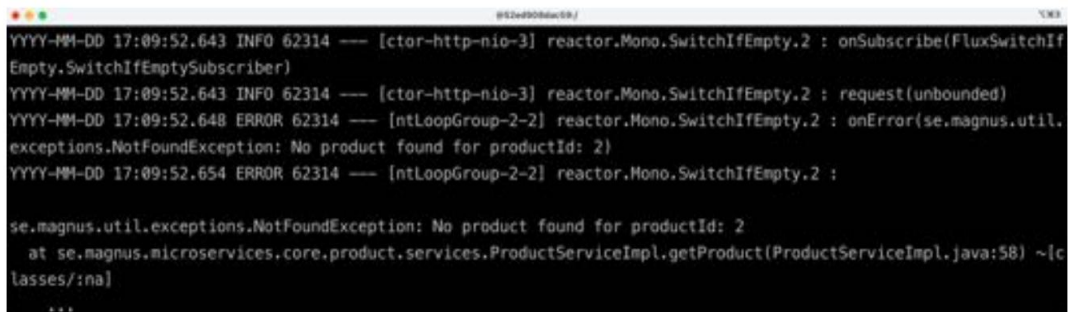
```

@52ed908dec59/
YYYY-MM-DD 17:08:47.006 INFO 62314 --- [ctor-http-nio-2] reactor.Mono.SwitchIfEmpty.1 : onSubscribe(FluxSwitchIfEmpty.SwitchIfEmptySubscriber)
YYYY-MM-DD 17:08:47.007 INFO 62314 --- [ctor-http-nio-2] reactor.Mono.SwitchIfEmpty.1 : request(unbounded)
YYYY-MM-DD 17:08:47.034 INFO 62314 --- [ntLoopGroup-2-2] reactor.Mono.SwitchIfEmpty.1 : onNext(ProductEntity: 1)
YYYY-MM-DD 17:08:47.048 INFO 62314 --- [ntLoopGroup-2-2] reactor.Mono.SwitchIfEmpty.1 : onComplete()

```

Figura 7.4: Salida del registro cuando el procesamiento es exitoso

El siguiente es un ejemplo de salida de registro de un procesamiento fallido (que genera una excepción `NotFoundException`):



```

@52ed908dec59/
YYYY-MM-DD 17:09:52.643 INFO 62314 --- [ctor-http-nio-3] reactor.Mono.SwitchIfEmpty.2 : onSubscribe(FluxSwitchIfEmpty.SwitchIfEmptySubscriber)
YYYY-MM-DD 17:09:52.643 INFO 62314 --- [ctor-http-nio-3] reactor.Mono.SwitchIfEmpty.2 : request(unbounded)
YYYY-MM-DD 17:09:52.648 ERROR 62314 --- [ntLoopGroup-2-2] reactor.Mono.SwitchIfEmpty.2 : onError(se.magnus.util.exceptions.NotFoundException: No product found for productId: 2)
YYYY-MM-DD 17:09:52.654 ERROR 62314 --- [ntLoopGroup-2-2] reactor.Mono.SwitchIfEmpty.2 : 
se.magnus.util.exceptions.NotFoundException: No product found for productId: 2
    at se.magnus.microservices.core.product.services.ProductServiceImpl.getProduct(ProductServiceImpl.java:58) ~[classes/:na]
    ...

```

Figura 7.5: Salida del registro cuando falla el procesamiento

Para ver el código fuente completo, consulte las siguientes clases:

- `ProductServiceImpl` en el proyecto del producto
- `RecommendationServiceImpl` en el proyecto de recomendación

Cambios en el código de prueba

El código de prueba para las implementaciones de servicios se ha modificado de la misma manera que las pruebas de la capa de persistencia descritas anteriormente. Para gestionar el comportamiento asíncrono de los tipos de retorno reactivos, Mono y Flux, las pruebas combinan la llamada al método `block()` con el uso de la clase auxiliar `StepVerifier`.

Para obtener el código fuente completo, consulte las siguientes clases de prueba:

- `ProductServiceApplicationTests` en el proyecto de producto
- `RecommendationServiceApplicationTests` en el proyecto de recomendación

Cómo lidiar con el código de bloqueo

En el caso del servicio de revisión, que utiliza JPA para acceder a sus datos en una base de datos relacional, no se admite un modelo de programación no bloqueante. En su lugar, podemos ejecutar el código bloqueante mediante un `Scheduler`, capaz de ejecutarlo en un hilo desde un grupo de hilos dedicado con un número limitado de hilos. El uso de un grupo de hilos para el código bloqueante evita agotar los hilos disponibles en el microservicio y evitar que se afecte el procesamiento no bloqueante concurrente en el microservicio, si lo hay.

Veamos cómo se puede configurar esto en los siguientes pasos:

1. Primero, configuramos un bean programador y su grupo de subprocessos en la clase principal,

`ReviewServiceApplication`, de la siguiente manera:

```

Aplicación de servicio de revisión pública (
    @Value("${app.threadPoolSize:10}") Entero tamaño del grupo de hilos,
    @Value("${app.taskQueueSize:100}") Entero taskQueueSize
){
    este.threadPoolSize = threadPoolSize;
    esto.taskQueueSize = taskQueueSize;
}

@Frijol
Programador público jdbcScheduler() {
    devuelve Schedulers.newBoundedElastic(threadPoolSize,

```

```
tamañoDeColaDeTarea, "jdbc-pool");
}
```

Del código anterior, podemos ver que el bean del programador se llama jdbcScheduler y que podemos configurar su grupo de subprocesos usando las siguientes propiedades:

- app.threadPoolSize, que especifica el número máximo de subprocesos en el grupo.

El valor predeterminado es 10

- app.taskQueueSize, que especifica la cantidad máxima de tareas que se pueden colocar en una cola esperando subprocesos disponibles (el valor predeterminado es 100)

2. A continuación, inyectamos el programador llamado jdbcScheduler en la implementación del servicio de revisión .

clase de información, como se muestra aquí:

```
@RestController
class pública ReviewServiceImpl implementa ReviewService {

    Planificador final privado jdbcScheduler;

    públicoReviewServiceImpl (
        @Qualifier("jdbcScheduler")
        Programador jdbcScheduler, ...)
    { this.jdbcScheduler = jdbcScheduler;
    }
}
```

3. Finalmente, utilizamos el grupo de subprocesos del programador en la implementación reactiva del

método getReviews() , de la siguiente manera:

```
@Override
público Flux<Revisión> obtenerReseñas(int productId) {

    si (productId < 1) {
        lanzar nueva InputException(" Producto no válido: " +
            ID del producto);
    }

    LOG.info("Recibiré reseñas del producto con id={}", productId);

    devuelve Mono.fromCallable() -> internalGetReviews(productId)
        .flatMapMany(Flux::fromIterable)
```



```

        .log(LOG.getName(), FINE)
        .subscribeOn(jdbcScheduler);
    }

    Lista privada <Revisión> internalGetReviews(int productId) {

        Lista<ReviewEntity> entityList = repositorio.
            findByProductId(productId);
        Lista<Revisión> lista = mapper.entityListToApiList(entityList);
        lista.forEach(e -> e.setServiceAddress(serviceUtil.
            obtenerDirecciónDeServicio()));

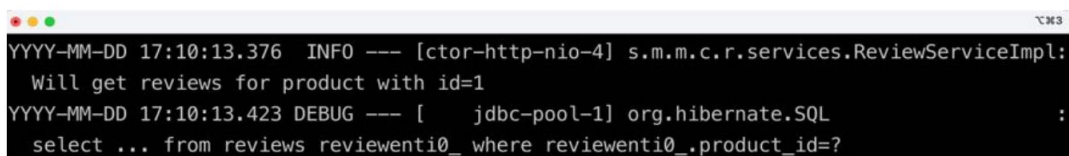
        LOG.debug(" Tamaño de la respuesta: {}", list.size());

        lista de retorno ;
    }

```

Aquí, el código de bloqueo se coloca en el método `internalGetReviews()` y se encapsula en un objeto `Mono` mediante el método `Mono.fromCallable()`. El método `getReviews()` utiliza el método `subscribeOn()` para ejecutar el código de bloqueo en un hilo del grupo de hilos de `jdbcScheduler`.

Al ejecutar pruebas más adelante en este capítulo, podremos consultar la salida del registro del servicio de revisión y comprobar que las sentencias SQL se ejecutan en subprocesos del grupo dedicado del programador. Veremos una salida del registro como esta:



```

YYYY-MM-DD 17:10:13.376 INFO --- [ctor-http-nio-4] s.m.m.c.r.services.ReviewServiceImpl:
  Will get reviews for product with id=1
YYYY-MM-DD 17:10:13.423 DEBUG --- [ jdbc-pool-1] org.hibernate.SQL :
  select ... from reviews reviewenti0_ where reviewenti0_.product_id=?

```

Figura 7.6: Salida del registro del servicio de revisión

De la salida del registro anterior, podemos ver lo siguiente:

- La primera salida del registro proviene de la llamada `LOG.info()` en el método `getReviews()` y se ejecuta en un hilo HTTP llamado `ctor-http-nio-4`, que es un hilo utilizado por WebFlux.
- En la segunda salida del registro, podemos ver la sentencia SQL generada por Spring Data JPA, utilizando Hibernate en segundo plano. La sentencia SQL corresponde al repositorio. Llamada al método `findByProductId()`. Se ejecuta en un hilo llamado `jdbc-pool-1`, lo que significa que se ejecuta en un hilo del grupo de hilos dedicado al código de bloqueo, como era de esperar.

Para obtener el código fuente completo, consulte las clases `ReviewServiceApplication` y `ReviewServiceImpl` en el proyecto de revisión .

Con la lógica para gestionar el código bloqueante establecida, hemos terminado de implementar las API REST no bloqueantes en los servicios principales. Veamos cómo hacer que las API REST en los servicios compuestos también sean no bloqueantes.

API REST sin bloqueo en los servicios compuestos

Para que nuestra API REST en el servicio compuesto no sea bloqueante, debemos hacer lo siguiente:

- Cambiar la API para que sus operaciones solo devuelvan tipos de datos reactivos
- Cambiar la implementación del servicio para que llame a las API de los servicios creados en paralelo y de manera no bloqueante.
- Cambiar la capa de integración para que utilice un cliente HTTP sin bloqueo
- Cambiar nuestras pruebas para que puedan probar el servicio reactivo

Cambios en la API

Para que la API del servicio compuesto sea reactiva, debemos aplicar el mismo tipo de cambio que aplicamos a las API de los servicios principales descritos anteriormente. Esto significa que el tipo de retorno del método `getProduct()` , `ProductAggregate`, debe reemplazarse por `Mono<ProductAggregate>`. Los métodos `createProduct()` y `deleteProduct()` deben actualizarse para que devuelvan `Mono<Void>` en lugar de `void`; de lo contrario, no podremos propagar las respuestas de error a quienes llaman a la API.

Para obtener el código fuente completo, consulte la interfaz `ProductCompositeService` en el proyecto API .

Cambios en la implementación del servicio

Para poder llamar a las tres API en paralelo, la implementación del servicio utiliza el `zip()` estático Método de la clase `Mono` . El método `zip` puede gestionar varias solicitudes reactivas paralelas y comprimirlas una vez completadas. El código se ve así:

```
@Anular
público Mono<ProductAggregate> obtenerProducto(int productId) {
    devolver Mono.zip(

        valores -> createProductAggregate(
            Valores (del producto) [0],
            (Lista<Recomendación>) valores[1],
            (Lista<Revisión>) valores[2],
```

```
        serviceUtil.getServiceAddress()),

        integración.getProduct(productId),
        integración.getRecommendations(productId).collectList(),
        integración.getReviews(productId).collectList())

        .doOnError(ex ->
            LOG.warn("Error al obtener el producto compuesto: {}",
                ej.toString()))
        .log(LOG.getName(), FINE);
    }
```

Veamos más de cerca:

El primer parámetro del método `zip` es una función lambda que recibirá las respuestas en una matriz denominada `values`. La matriz contendrá un producto, una lista de recomendaciones y una lista de reseñas. La agregación de las respuestas de las tres llamadas a la API se gestiona mediante el mismo método auxiliar que antes, `createProductAggregate()`, sin modificaciones.

Los parámetros después de la función lambda son una lista de las solicitudes que el método `zip` llamará en paralelo, un objeto `Mono` por solicitud. En nuestro caso, enviamos tres objetos `Mono` creados por los métodos de la clase de integración, uno por cada solicitud enviada a cada microservicio principal.

Para obtener el código fuente completo, consulte la clase `ProductCompositeServiceImpl` en el producto-compuesto proyecto.



Para obtener información sobre cómo se implementan las operaciones de API `createProduct` y `deleteProduct` en el servicio compuesto de producto, consulte la sección [Publicación de eventos](#) en el servicio compuesto más adelante.

Cambios en la capa de integración

En la clase de integración `ProductCompositeIntegration`, hemos reemplazado el cliente HTTP bloqueante, `RestTemplate`, por un cliente HTTP no bloqueante, `WebClient`, que viene con Spring 5.

Para crear una instancia de `WebClient`, se utiliza un patrón de constructor. Si se requiere personalización, por ejemplo, para configurar encabezados o filtros comunes, se puede realizar mediante el constructor. Para conocer las opciones de configuración disponibles, consulte <https://docs.spring.io/spring/docs/current/spring-framework-reference/web-reactive.html#webflux-client-builder>.

El WebClient se utiliza de la siguiente manera:

1. En el constructor, el WebClient se inyecta automáticamente. Construimos la instancia del WebClient con:
fuera cualquier configuración:

```
La clase pública ProductCompositeIntegration implementa ProductService,
Servicio de recomendación, Servicio de revisión {

    cliente web final privado cliente web;

    público ProductoCompuestoIntegración(
        WebClient.Builder cliente web, ...
    ) {
        este.webClient = webClient.build();
    }
}
```

2. A continuación, utilizamos la instancia webClient para realizar nuestras solicitudes no bloqueantes para llamar a la
Servicio de producto :

```
@Anular
público Mono<Producto> obtenerProducto(int IdProducto) {
    Cadena url = productServiceUrl + "/producto/" + productId;

    devolver webClient.get().uri(url).retrieve()
        .bodyToMono(Producto.clase)
        .log(LOG.getName(), FINE)
        .onErrorMap(WebClientResponseException.clase,
            ex -> manejarExcepción(ex)
        );
}
```

Si la llamada a la API al servicio del producto falla con una respuesta de error HTTP, toda la solicitud a la API fallará. El método `onErrorMap()` de WebClient llamará a nuestro método `handleException(ex)`, que asigna las excepciones HTTP generadas por la capa HTTP a nuestras propias excepciones, por ejemplo, `NotFoundException` o `InvalidInputException`.

Sin embargo, si las llamadas al servicio de producto son exitosas, pero la llamada a la API de recomendación o de reseñas falla, no queremos que la solicitud falle por completo. En su lugar, queremos devolver al emisor la mayor cantidad de información disponible. Por lo tanto, en lugar de propagar una excepción en

En estos casos, devolveremos una lista vacía de recomendaciones o reseñas. Para suprimir el error, realizaremos la llamada `onErrorResume(error -> empty())`. El código es similar al siguiente:

```
@Anular
público Flux<Recomendación> obtenerRecomendaciones(int productId) {

    Cadena url = RecommendationServiceUrl + "/recomendación?
productId=" + productId;

    // Devuelve un resultado vacío si algo sale mal para hacerlo
    // Es posible que el servicio compuesto devuelva respuestas parciales
    devolver webClient.get().uri(url).retrieve()
        .bodyToFlux(Recomendación.class)
        .log(LOG.getName(), FINE)
        .onErrorResume(error -> vacío());
}
```

La clase `GlobalControllerExceptionHandler`, del proyecto `util`, capturará, como antes, las excepciones y las transformará en respuestas de error HTTP adecuadas que se envían al llamador de la API compuesta. De esta forma, podemos decidir si una respuesta de error HTTP específica de las llamadas a la API subyacente generará una respuesta de error HTTP o simplemente una respuesta parcialmente vacía.

Para obtener el código fuente completo, consulte la clase `ProductCompositeIntegration` en el `producto-compuesto` proyecto.

Cambios en el código de prueba

El único cambio necesario en las clases de prueba es actualizar la configuración de Mockito y su mock de la clase de integración. El mock debe devolver objetos `Mono` y `Flux`. El método `setup()` utiliza los métodos auxiliares `Mono.just()` y `Flux.fromIterable()`, como se muestra en el siguiente código:

```
class ProductCompositeServiceApplicationTests {
    @AntesDeCada
    void setup() {
        cuando(compositeIntegration.getProduct(PRODUCT_ID_OK)).
            luegoReturn(Mono.just(nuevo Producto(PRODUCT_ID_OK, "nombre", 1,
                "dirección simulada")));
        cuando(compositeIntegration.getRecommendations(PRODUCT_ID_OK)).
            luegoReturn(Flux.fromIterable(singletonList(nueva Recomendación(
```

```

PRODUCT_ID_OK, 1, "autor", 1, "contenido",
    "dirección simulada"))));
cuando(compositeIntegration.getReviews(PRODUCT_ID_OK)).
    luegoReturn(Flux.fromIterable(singletonList(nueva Revisión(
        PRODUCT_ID_OK, 1, "autor", "tema", "contenido",
            "dirección simulada"))));

```

Para obtener el código fuente completo, consulte la clase de prueba `ProductCompositeServiceApplicationTests` en el proyecto `product-composite`.

Con esto finalizamos la implementación de nuestras API REST síncronas sin bloqueo. Ahora es el momento de desarrollar nuestros servicios asíncronos basados en eventos.

Desarrollo de servicios asíncronos basados en eventos

En esta sección, aprenderemos cómo desarrollar versiones asíncronas y controladas por eventos de la función `create` y eliminar servicios. El servicio compuesto publicará eventos de creación y eliminación en cada tema de servicio principal y luego devolverá una respuesta correcta al emisor sin esperar a que se procese en los servicios principales. Esto se ilustra en el siguiente diagrama:

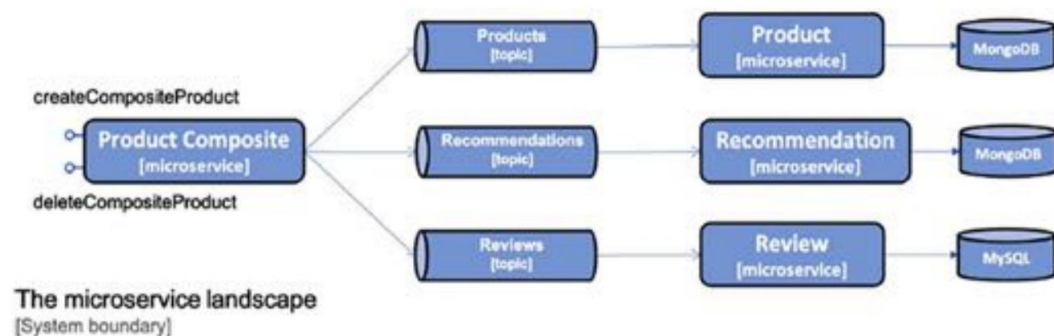


Figura 7.7: Las partes `createCompositeProduct` y `deleteCompositeProduct` del paisaje

Cubriremos los siguientes temas:

- Manejo de desafíos con la mensajería
- Definición de temas y eventos
- Cambios en los archivos de compilación de Gradle
- Consumir eventos en los servicios principales
- Publicación de eventos en el servicio compuesto

Cómo afrontar los desafíos de la mensajería

Para implementar los servicios de creación y eliminación basados en eventos, utilizaremos Spring Cloud Stream. En el Capítulo 2, Introducción a Spring Boot, vimos lo fácil que es publicar y consumir mensajes en un tema utilizando Spring Cloud Stream. El modelo de programación se basa en un paradigma funcional, donde las funciones que implementan una de las interfaces funcionales (Proveedor, Función o Consumidor) del paquete `java.util.function` pueden encadenarse para realizar un procesamiento basado en eventos desacoplado. Para activar dicho procesamiento funcional externamente, desde código no funcional, se puede utilizar la clase auxiliar `StreamBridge`.

Por ejemplo, para publicar el cuerpo de una solicitud HTTP a un tema, solo tenemos que escribir lo siguiente:

```
@Autowired
StreamBridge privado streamBridge;

@PostMapping
void sampleCreateAPI(@RequestBody String cuerpo) {
    streamBridge.send("tema", cuerpo);
}
```

La clase auxiliar `StreamBridge` se utiliza para activar el procesamiento. Publicará un mensaje en un tema. Se puede definir una función que consuma eventos de un tema (sin crear nuevos eventos) implementando la interfaz funcional `java.util.function.Consumer` de la siguiente manera:

```
@Frijol
público Consumidor<Cadena> miSuscriptor() {
    devolver s -> System.out.println("ML RECIBIDO: " + s);
}
```

Para vincular las distintas funciones, utilizamos la configuración. Veremos ejemplos de dicha configuración más adelante en [Agregar configuración para publicar eventos](#) y [Agregar configuración para consumir Secciones de eventos](#).

¡Este modelo de programación se puede utilizar independientemente del sistema de mensajería utilizado, por ejemplo, RabbitMQ o Apache Kafka!

Si bien se prefiere el envío de mensajes asíncronos a las llamadas API síncronas, esto conlleva sus propios desafíos. Veremos cómo podemos usar Spring Cloud Stream para gestionar algunos de ellos. Se abordarán las siguientes características de Spring Cloud Stream:

- Grupos de consumidores
- Reintentos y colas de mensajes muertos
- Pedidos y particiones garantizados

Estudiaremos cada uno de ellos en las siguientes secciones.

Grupos de consumidores

El problema aquí es que si ampliamos la cantidad de instancias de un consumidor de mensajes, por ejemplo, si iniciamos dos instancias del microservicio de producto, ambas instancias del microservicio de producto consumirán los mismos mensajes, como lo ilustra el siguiente diagrama:

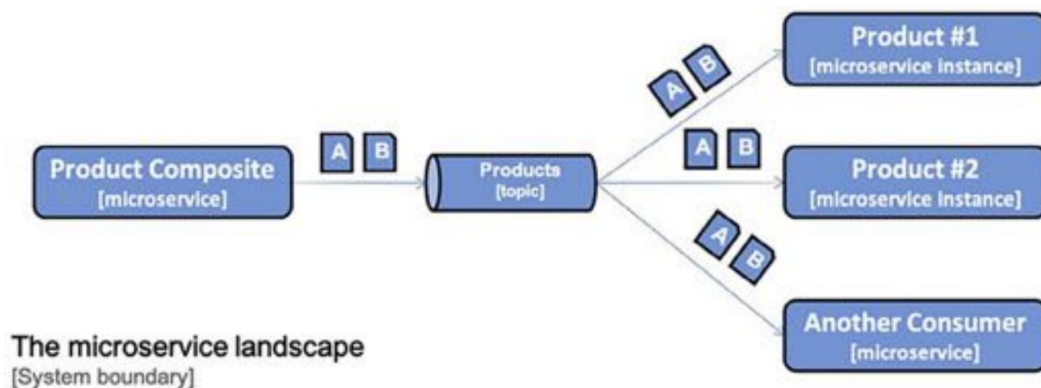


Figura 7.8: Productos n.º 1 y n.º 2 que consumen los mismos mensajes

Esto podría provocar que un mensaje se procese dos veces, lo que podría generar duplicados u otras inconsistencias no deseadas en la base de datos. Por lo tanto, solo queremos que una instancia por consumidor procese cada mensaje. Esto se puede solucionar introduciendo un grupo de consumidores, como se ilustra en el siguiente diagrama:

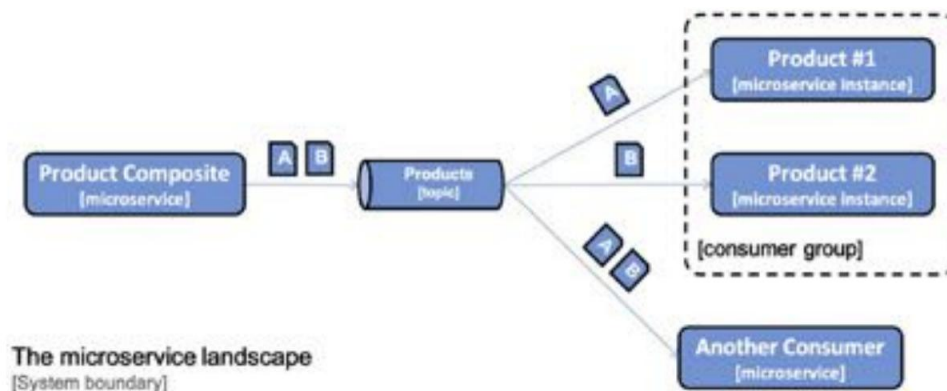


Figura 7.9: Grupo de consumidores

En Spring Cloud Stream, se puede configurar un grupo de consumidores en el lado del consumidor. Por ejemplo, para el microservicio de producto, se vería así:

```
primavera.nube.stream:
  enlaces.messageProcessor-en-0:
    destino: productos
    grupo: productosGroup
```

De esta configuración podemos aprender lo siguiente:

- Spring Cloud Stream aplica, por defecto, una convención de nomenclatura para vincular una configuración a una función. Para los mensajes enviados a una función, el nombre de la vinculación es `<functionName>-in-<índice>`:
 - `functionName` es el nombre de la función, que es `messageProcessor` en el ejemplo anterior.
 - El índice se establece en 0, a menos que la función requiera múltiples argumentos de entrada o salida. No utilizaremos funciones con múltiples argumentos, por lo que el índice siempre se establecerá en 0 en nuestros ejemplos.
 - Para los mensajes salientes, la convención de nombre de enlace es `<functionName>-out-<índice>`.
- La propiedad de destino especifica el nombre del tema desde el cual se consumirán los mensajes, que en este caso son `productos`.

La propiedad "group" especifica a qué grupo de consumidores se agregarán las instancias del microservicio de producto; en este ejemplo, "productsGroup". Esto significa que los mensajes enviados al tema "products" solo serán entregados por Spring Cloud Stream a una de las instancias del microservicio de producto.

Reintentos y colas de mensajes muertos

Si un consumidor no procesa un mensaje, este puede volver a ponerse en cola para el consumidor fallido hasta que se procese correctamente. Si el contenido del mensaje no es válido, también conocido como mensaje envenenado, el mensaje impedirá que el consumidor procese otros mensajes hasta que se elimine manualmente. Si el fallo se debe a un problema temporal, por ejemplo, si no se puede acceder a la base de datos debido a un error temporal de red, el procesamiento probablemente se realizará correctamente tras varios reintentos.

Debe ser posible especificar el número de reintentos hasta que un mensaje se transfiera a otro almacenamiento para su análisis y corrección. Un mensaje fallido suele transferirse a una cola dedicada denominada cola de mensajes fallidos. Para evitar la sobrecarga de la infraestructura durante un fallo temporal (por ejemplo, un error de red), debe ser posible configurar la frecuencia de los reintentos, preferiblemente con un intervalo de tiempo creciente entre cada uno.

En Spring Cloud Stream, esto se puede configurar en el lado del consumidor, por ejemplo, para el producto microservicio, como se muestra aquí:

```
spring.cloud.stream.bindings.messageProcessor-in-0.consumer:
```

```
Intentos máximos: 3
```

```
IntervaloInicialdeRetroceso: 500
```

```
Intervalo máximo de retroceso: 1000
```

```
Multiplicador de retroceso: 2.0
```

```
spring.cloud.stream.rabbit.bindings.messageProcessor-in-0.consumer:
```

```
autoBindDlq: verdadero
```

```
republicarToDlq: verdadero
```

```
spring.cloud.stream.kafka.bindings.messageProcessor-in-0.consumer:
```

```
enableDlq: verdadero
```

En el ejemplo anterior, especificamos que Spring Cloud Stream debe realizar tres reintentos antes de colocar un mensaje en la cola de mensajes fallidos. El primer reintento se realizará después de 500 ms y los dos restantes, después de 1000 ms.

Habilitar el uso de colas de mensajes inactivos es específico del enlace; por lo tanto, tenemos una configuración para RabbitMQ y otra para Kafka.

Orden y particiones garantizadas

Si la lógica de negocio requiere que los mensajes se consuman y procesen en el mismo orden en que se enviaron, no podemos usar varias instancias por consumidor para aumentar el rendimiento del procesamiento; por ejemplo, no podemos usar grupos de consumidores. Esto podría, en algunos casos, generar una latencia inaceptable en el procesamiento de los mensajes entrantes.

Podemos usar particiones para garantizar que los mensajes se entreguen en el mismo orden en que fueron enviados pero sin perder rendimiento ni escalabilidad.

En la mayoría de los casos, solo se requiere un orden estricto en el procesamiento de mensajes que afectan a las mismas entidades comerciales. Por ejemplo, los mensajes que afectan al producto con ID de producto 1 pueden, en muchos casos, procesarse independientemente de los mensajes que afectan al producto con ID de producto 2.

Esto significa que el orden solo debe garantizarse para los mensajes que tengan el mismo ID de producto.

La solución es especificar una clave para cada mensaje, que el sistema de mensajería puede usar para garantizar que se mantenga el orden entre mensajes con la misma clave. Esto se puede solucionar introduciendo subtemas, también conocidos como particiones, en un tema. El sistema de mensajería coloca los mensajes en una partición específica según su clave.

Los mensajes con la misma clave siempre se almacenan en la misma partición. El sistema de mensajería solo necesita garantizar el orden de entrega de los mensajes en la misma partición. Para garantizar el orden de los mensajes, configuramos una instancia de consumidor por partición dentro de un grupo de consumidores. Al aumentar el número de particiones, podemos permitir que un consumidor aumente su número de instancias. Esto mejora el rendimiento del procesamiento de mensajes sin perder el orden de entrega. Esto se ilustra en el siguiente diagrama:

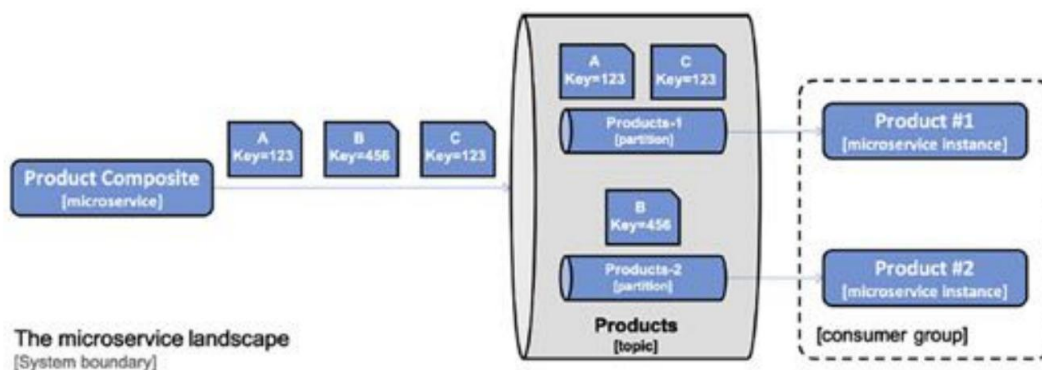


Figura 7.10: Especificación de claves para mensajes

Como se ve en el diagrama anterior, todos los mensajes con la clave establecida en 123 siempre van a Productos-1 partición, mientras que los mensajes con la clave establecida en 456 van a la partición Products-2 .

En Spring Cloud Stream, esto debe configurarse tanto en el lado del editor como en el del consumidor. Del lado del editor, se deben especificar la clave y el número de particiones. Por ejemplo, para el servicio compuesto por producto , tenemos lo siguiente:

```
spring.cloud.stream.bindings.products-out-0.producer:  
  expresión de clave de partición: headers['partitionKey']  
  número de particiones: 2
```

Esta configuración significa que la clave se tomará del encabezado del mensaje con el nombre partitionKey y que se utilizarán dos particiones.

Cada consumidor puede especificar la partición desde la que desea consumir mensajes. Por ejemplo, para el microservicio de producto , tenemos lo siguiente:

```
spring.cloud.stream.bindings.messageProcessor-en-0:  
  destino: productos  
  grupo: grupodeproductos  
  consumidor:  
    particionado: verdadero  
    índice de instancia: 0
```

Esta configuración le dice a Spring Cloud Stream que este consumidor solo consumirá mensajes de la partición número 0, es decir, la primera partición.

Definición de temas y eventos

Como ya mencionamos en la sección Spring Cloud Stream en el Capítulo 2, Introducción a Spring Boot, Spring Cloud Stream se basa en el patrón de publicación y suscripción, donde un editor publica mensajes en temas y los suscriptores se suscriben a los temas de los que están interesados en recibir mensajes.

Utilizaremos un tema por tipo de entidad: productos, recomendaciones y reseñas.

Los sistemas de mensajería gestionan mensajes que suelen constar de encabezados y cuerpo. Un evento es un mensaje que describe un evento. En el caso de los eventos, el cuerpo del mensaje puede utilizarse para describir el tipo de evento, sus datos y una marca de tiempo de cuándo ocurrió.

Un evento, para el alcance de este libro, se define de la siguiente manera:

- El tipo de evento, por ejemplo, un evento de creación o eliminación
- Una clave que identifica los datos, por ejemplo, un ID de producto

- Un elemento de datos , es decir, los datos reales del evento.
- Una marca de tiempo, que describe cuándo ocurrió el evento

La clase de evento que utilizaremos tendrá el siguiente aspecto:

```
clase pública Evento<K, T> {

    enumeración pública Tipo {CREAR, ELIMINAR}

    privado Event.Type eventType; clave K
    privada;
    datos T privados;
    evento ZonedDateTime privadoCreatedAt;

    Evento público () {
        este.eventType = null; este.key
        = null;
        este.data = null;
        este.eventCreatedAt = null;
    }

    Evento público (Tipo eventType, K clave, T datos) {
        este.eventType = tipoDeEvento;
        esta.clave = clave;
        esta.datos = datos;
        esta.eventoCreadoAt = ahora();
    }

    tipo público getEventType() {
        devolver tipoDeEvento;
    }

    público K obtenerClave() {
        tecla de retorno ;
    }

    público T obtenerDatos() {
        devolver datos;
    }
}
```

```

    }

    público ZonedDateTime getEventCreatedAt() {
        devolver eventoCreadoEn;
    }
}

```

Explicuemos el código fuente anterior en detalle:

- La clase Evento es una clase genérica parametrizada sobre los tipos de sus campos clave y de datos , K y T
- El tipo de evento se declara como un enumerador con los valores permitidos, es decir, CREAR y ELIMINAR
- La clase define dos constructores, uno vacío y otro que se puede utilizar para inicializar la miembros de tipo, clave y valor
- Finalmente, la clase define métodos getter para sus variables miembro.

Para obtener el código fuente completo, consulte la clase Event en el proyecto API .

Cambios en los archivos de compilación de Gradle

Para integrar Spring Cloud Stream y sus enlazadores para RabbitMQ y Kafka, necesitamos agregar las dos dependencias de inicio spring-cloud-starter-stream-rabbit y spring-cloud-starter-stream-kafka en los cuatro proyectos de microservicios. También necesitamos una dependencia de prueba en el proyecto compuesto por producto , spring-cloud-stream::test-binder, para incorporar la compatibilidad con pruebas. El siguiente código lo muestra:

```

dependencias {
    implementación 'org.springframework.cloud:spring-cloud-starter-stream-
conejo'

    implementación 'org.springframework.cloud:spring-cloud-starter-stream-
Kafka

    Implementación de prueba 'org.springframework.cloud:spring-cloud-stream::test-
aglutinante'
}

```

Para especificar qué versión de Spring Cloud queremos usar, primero declaramos una variable para la versión:

```

extensión {
    springCloudVersion = "2025.0.0"
}

```

A continuación, usamos la variable para configurar la gestión de dependencias para la versión de Spring Cloud especificada, como se ve aquí:

```
gestión de dependencias {  
    importaciones  
        { mavenBom "org.springframework.cloud:spring-cloud-dependencies:  
            ${springCloudVersion}"  
        }  
    }  
}
```

Para obtener el código fuente completo, consulte el archivo build.gradle en cada uno de los proyectos de microservicios.

Con las dependencias requeridas agregadas a los archivos de compilación de Gradle, podemos comenzar a aprender cómo consumir eventos en los servicios principales.

Consumir eventos en los servicios principales

Para poder consumir eventos en los servicios principales, necesitamos hacer lo siguiente:

- Declarar procesadores de mensajes que consumen eventos publicados en el tema del servicio principal
- Cambiar nuestras implementaciones de servicio para utilizar la capa de persistencia reactiva
- Agregar configuración requerida para consumir eventos
- Cambiar nuestras pruebas para que puedan probar el procesamiento asíncrono de los eventos

El código fuente para consumir eventos está estructurado de la misma manera en los tres servicios principales, por lo que solo revisaremos el código fuente del servicio del producto .

Declaración de procesadores de mensajes

Las API REST para crear y eliminar entidades se han reemplazado por un procesador de mensajes en cada microservicio principal que consume eventos de creación y eliminación en el tema de cada entidad. Para poder consumir los mensajes publicados en un tema, necesitamos declarar un bean Spring que implemente la interfaz funcional `java.util.function.Consumer` .

El procesador de mensajes para el servicio de producto se declara de la siguiente manera:

```
@Configuración  
clase pública MessageProcessorConfig {  
  
    Servicio de producto final privado productService;  
  
    público MessageProcessorConfig(ProductService productService)
```

```

{
    este.productService = productoServicio;
}

@Frijol
público Consumidor<Evento<Entero,Producto>> messageProcessor() {
    ...

```

Del código anterior podemos ver lo siguiente:

- La clase está anotada con `@Configuration`, lo que le indica a Spring que busque beans Spring en la clase.
- Inyectamos una implementación de la interfaz `ProductService` en el constructor. El bean `productService` contiene la lógica de negocio para crear y eliminar las entidades de producto.
- Declaramos el procesador de mensajes como un bean Spring que implementa la interfaz funcional `Consumer`, aceptando un evento como parámetro de entrada de `Event<Integer,Product>` tipo.

La implementación de la función `Consumidor` se ve así:

```

evento de retorno -> {

    cambiar (evento.getEventType()) {

        caso CREAR:
            Producto producto = evento.getData();
            productService.createProduct(producto).block();
            romper;

        caso ELIMINAR:
            int productId = evento.getKey();
            productService.deleteProduct(productId).block();
            romper;

        por defecto:
            Cadena errorMessage = "Tipo de evento incorrecto: " +
                evento.getEventType() +
                ", se esperaba un evento CREATE o DELETE";

```



```

        lanzar nueva EventProcessingException(errorMessage);
    }

};

```

La implementación anterior hace lo siguiente:

- Toma un evento del tipo `Event<Integer,Product>` como parámetro de entrada
- Al utilizar una declaración `switch`, según el tipo de evento, se creará o eliminará un producto.
entidad
- Utiliza el bean `productService` inyectado para realizar las operaciones reales de creación y eliminación.
eración
- Si el tipo de evento no es crear ni eliminar, se lanzará una excepción

Para garantizar que podamos propagar las excepciones lanzadas por el bean `productService` al sistema de mensajería, llamamos al método `block()` en las respuestas que recibimos del bean `productService`. Esto garantiza que el procesador de mensajes espere a que el bean `productService` complete su creación o eliminación en la base de datos subyacente. Sin llamar al método `block()`, no podríamos propagar excepciones y el sistema de mensajería no podría volver a poner en cola un intento fallido ni mover el mensaje a una cola de mensajes fallidos; en su lugar, el mensaje se descartaría silenciosamente.



En general, llamar a un método `block()` se considera una mala práctica desde el punto de vista del rendimiento y la escalabilidad. Sin embargo, en este caso, solo procesaremos unos pocos mensajes entrantes en paralelo, uno por partición, como se describió anteriormente. Esto significa que solo bloquearemos algunos subprocesos simultáneamente, lo que no afectará negativamente el rendimiento ni la escalabilidad.

Para obtener el código fuente completo, consulte las clases `MessageProcessorConfig` en los proyectos de producto, recomendación y revisión.

Cambios en las implementaciones del servicio

Las implementaciones de los métodos de creación y eliminación para el servicio de productos y recomendaciones se han reescrito para utilizar la capa de persistencia reactiva no bloqueante de MongoDB. Por ejemplo, la creación de entidades de producto se realiza de la siguiente manera:

```

@Anular
público Mono<Product> createProduct( Cuerpo del producto) {

```

```

    si (cuerpo.getProductId() < 1) {
        lanzar nueva InvalidInputException(" ProductId no válido: " +
            cuerpo.getProductId());
    }

    EntidadProductoEntidad = mapper.apiToEntity(cuerpo);
    Mono<Producto> newEntity = repositorio.save(entidad)
        .log(LOG.getName(), FINE)
        .onErrorMap(
            DuplicateKeyException.clase,
            ex -> nueva InvalidInputException
                ("Clave duplicada, Id. de producto: " + body.getProductId()))
        .map(e -> mapper.entityToApi(e));
    devolver nuevaEntidad;
}

```

Tenga en cuenta que en el código anterior se utiliza el método `onErrorMap()` para asignar la excepción de persistencia `DuplicateKeyException` a nuestra propia excepción `InvalidInputException`.



Para el servicio de revisión, que utiliza la capa de persistencia de bloqueo para JPA, los métodos de creación y eliminación se han actualizado de la misma manera que se describe en la sección *Cómo tratar el código de bloqueo*.

Para ver el código fuente completo, consulte las siguientes clases:

- `ProductServiceImpl` en el proyecto del producto
- `RecommendationServiceImpl` en el proyecto de recomendación
- `ReviewServiceImpl` en el proyecto de revisión

Agregar configuración para consumir eventos

También necesitamos configurar el sistema de mensajería para que pueda consumir eventos. Para ello, debemos completar los siguientes pasos:

1. Declaramos que RabbitMQ es el sistema de mensajería predeterminado y que el contenido predeterminado

El tipo es JSON:

```

primavera.nube.stream:
    defaultBinder: conejo
    predeterminado.contentType: aplicación/json

```

2. A continuación, vinculamos la entrada a los procesadores de mensajes a nombres de temas específicos, de la siguiente manera:

```
primavera.nube.stream:  
  enlaces.messageProcessor-en-0:  
    destino: productos
```

3. Finalmente, declaramos la información de conectividad tanto para Kafka como para RabbitMQ:

```
spring.cloud.stream.kafka.binder:  
  corredores: 127.0.0.1  
  puerto de intermediario predeterminado: 9092  
  
primavera.rabbitmq:  
  host: 127.0.0.1  
  puerto: 5672  
  nombre de usuario: invitado  
  contraseña: invitado  
  
---  
  
spring.config.activate.on-profile: docker  
  
spring.rabbitmq.host: conejomq  
spring.cloud.stream.kafka.binder.brokers: kafka
```

En el perfil predeterminado de Spring, especificamos los nombres de host que se usarán al ejecutar nuestro entorno de sistema sin Docker en el host local con la dirección IP 127.0.0.1. En el perfil de Spring de Docker , especificamos los nombres de host que usaremos al ejecutar en Docker y usar Docker Compose, es decir, RabbitMQ y Kafka.

Además de esta configuración, la configuración del consumidor también especifica los grupos de consumidores, el manejo de reintentos, las colas de mensajes no entregados y las particiones tal como se describieron anteriormente en la sección Manejo de desafíos con la mensajería .

Para obtener el código fuente completo, consulte los archivos de configuración application.yml en los proyectos de producto, recomendación y revisión .

Cambios en el código de prueba

Dado que los servicios principales ahora reciben eventos para crear y eliminar sus entidades, las pruebas deben actualizarse para que envíen eventos en lugar de llamar a las API REST , como lo hacían en el modelo anterior.

capítulos. Para poder llamar al procesador de mensajes desde la clase de prueba, inyectamos el mensaje pro-
convertir el bean cesador en una variable miembro:

```
@SpringBootTest
class ProductServiceApplicationTests {

    @Autowired
    @Qualifier("procesador de mensajes")
    privado Consumidor<Evento<Entero, Producto>> messageProcessor;
```

Del código anterior, podemos ver que no solo inyectamos cualquier función de consumidor , sino que también usamos la anotación `@Qualifier` para especificar que queremos inyectar la función de consumidor que tiene la nombre `messageProcessor`.

Para enviar eventos de creación y eliminación al procesador de mensajes, agregamos dos métodos auxiliares, `sendCreateProductEvent` y `sendDeleteProductEvent`, en la clase de prueba:

```
void privado sendCreateProductEvent(int productId) {
    Producto producto = nuevo Producto(productId, "Nombre " + ID del producto,
                                         ID del producto, "SA");
    Evento<Entero, Producto> evento = nuevo Evento<>(CREATE, productId,
                                                    producto);
    messageProcessor.accept(evento);
}

void privado sendDeleteProductEvent(int productId) {
    Evento<Entero, Producto> evento = nuevo Evento<>(ELIMINAR, productId, nulo);
    messageProcessor.accept(evento);
}
```

Tenga en cuenta que usamos el método `accept()` en la declaración de la interfaz de la función `Consumer` para invocar al procesador de mensajes. Esto significa que omitimos el sistema de mensajería en las pruebas y llamamos directamente al procesador de mensajes.

Las pruebas para crear y eliminar entidades se actualizan para utilizar estos métodos auxiliares.

Para obtener el código fuente completo, consulte las siguientes clases de prueba:

- `ProductServiceApplicationTests` en el proyecto de producto
- `RecommendationServiceApplicationTests` en el proyecto de recomendación
- `ReviewServiceApplicationTests` en el proyecto de revisión

Hemos visto lo necesario para consumir eventos en los microservicios principales. Ahora, veamos cómo podemos publicar eventos en el microservicio compuesto.

Publicación de eventos en el servicio compuesto. Cuando el servicio compuesto recibe solicitudes HTTP para la creación y eliminación de productos compuestos, publica los eventos correspondientes en los servicios principales sobre sus temas. Para publicar eventos en el servicio compuesto, debemos realizar los siguientes pasos:

1. Publicar eventos en la capa de integración.
2. Agregar configuración para publicar eventos.
3. Cambiar las pruebas para que puedan probar la publicación de eventos.



Tenga en cuenta que no se requieren cambios en la clase de implementación del servicio compuesto: ¡ la capa de integración se encarga de ello!

Publicación de eventos en la capa de integración

Para publicar un evento en la capa de integración, necesitamos hacer lo siguiente:

1. Cree un objeto de evento basado en el cuerpo de la solicitud HTTP.
2. Cree un objeto Mensaje donde el objeto Evento se utiliza como carga útil y el campo clave en el objeto de evento se utiliza como clave de partición en el encabezado.
3. Utilice la clase auxiliar StreamBridge para publicar el evento en el tema deseado.

El código para enviar eventos de creación de productos se ve así:

```
@Override
public Mono<Product> createProduct( Cuerpo del producto) {
    devuelve Mono.fromCallable(() -> {

        sendMessage("productos-salidos-0",
            nuevo Evento<>(CREATE, cuerpo.getProductId(), cuerpo));
        devolver cuerpo;
    }).subscribeOn(publishEventScheduler);
}

void privado sendMessage(String bindingName, Event evento) { Mensaje mensaje
    = MessageBuilder.withPayload(evento)
```

```

        .setHeader("partitionKey", evento.getKey())
        .construir();
    streamBridge.send(nombreEnlace, mensaje);
}

```

En el código anterior podemos ver lo siguiente:

- La capa de integración implementa el método `createProduct()` en `ProductService` Interfaz mediante un método auxiliar, `sendMessage()`. Este método auxiliar toma el nombre de un enlace de salida y un objeto de evento. El nombre del enlace, `products-out-0`, se vinculará al tema del servicio de producto en la siguiente configuración.

Dado que `sendMessage()` usa código de bloqueo, al llamar a `streamBridge`, se ejecuta en un hilo proporcionado por un programador dedicado, `publishEventScheduler`. Este es el mismo enfoque que para gestionar el código JPA de bloqueo en el microservicio de revisión. Consulte la sección "Gestión del código de bloqueo" para obtener más información.

El método auxiliar `sendMessage()` crea un objeto `Message` y establece la carga útil y el encabezado de la clave de partición, como se describió anteriormente. Finalmente, utiliza el objeto `streamBridge` para enviar el evento al sistema de mensajería, que lo publicará en el tema definido en la configuración.

Para obtener el código fuente completo, consulte la clase `ProductCompositeIntegration` en el producto-compuesto proyecto.

Agregar configuración para publicar eventos

También necesitamos configurar el sistema de mensajería para poder publicar eventos; esto es similar a lo que hicimos para los consumidores. Declarar RabbitMQ como sistema de mensajería predeterminado, JSON como tipo de contenido predeterminado y Kafka y RabbitMQ para la información de conectividad es lo mismo que para los consumidores.

Para declarar qué temas se deben utilizar para los nombres de enlace de salida, tenemos la siguiente configuración:

```

primavera.nube.stream:
  fijaciones:
    productos-fuera-0:
      destino: productos
    recomendaciones-de-0:
      destino: recomendaciones
    reseñas-de-0:
      destino: reseñas

```

Al usar particiones, también necesitamos especificar la clave de partición y el número de particiones. que se utilizará:

```
spring.cloud.stream.bindings.products-out-0.producer:
    expresión de clave de partición: headers['partitionKey']
    número de particiones: 2
```

En la configuración anterior podemos ver lo siguiente:

- La configuración se aplica al nombre de enlace products-out-0
- La clave de partición utilizada se tomará del encabezado del mensaje de partición
- Se utilizarán dos particiones

Para obtener el código fuente completo, consulte el archivo de configuración application.yml en el componente del producto. proyecto.

Cambios en el código de prueba

Probar microservicios asíncronos basados en eventos es, por naturaleza, difícil. Las pruebas suelen necesitar sincronizarse con el procesamiento asíncrono en segundo plano para poder verificar el resultado. Spring Cloud Stream incluye un enlazador de pruebas, que permite verificar los mensajes enviados sin usar ningún sistema de mensajería durante las pruebas.



Consulte la sección Cambios en los archivos de compilación de Gradle anterior para saber cómo se incluye el soporte de pruebas en el proyecto compuesto del producto .

El soporte para pruebas incluye una clase auxiliar `OutputDestination` , que permite obtener los mensajes enviados durante una prueba. Se ha añadido una nueva clase de prueba, `MessagingTests`, para ejecutar pruebas que verifican el envío de los mensajes esperados. Repasemos las partes más importantes de...

clase de prueba:

Para poder inyectar un bean `OutputDestination` en la clase de prueba, también necesitamos importar su configuración desde la clase `TestChannelBinderConfiguration` . Esto se hace con el siguiente código:

```
@SpringBootTest
@Import({TestChannelBinderConfiguration.class})
clase PruebasDeMensajería {
```

```
@Autowired
```

```
destino de salida privado ;
```

2. A continuación, declaramos un par de métodos auxiliares para leer mensajes y también para poder purgar un tema. El código se ve así:

```
void privado purgeMessages(String bindingName) {
    obtenerMensajes(nombreEnlace);
}

Lista privada <Cadena> obtenerMensajes(Cadena bindingName){
    Lista<String> mensajes = nueva ArrayList<>();
    booleano anyMoreMessages = verdadero;

    mientras (cualquierMásMensajes) {
        Mensaje<byte[]> mensaje =
            obtenerMensaje(nombreEnlace);

        si (mensaje == nulo) {
            cualquierMásMensaje = falso;

        } else
            { mensajes.add(new String(mensaje.getPayload()));
            }

    } devolver mensajes;
}

Mensaje privado <byte[]> obtenerMensaje(String bindingName){
    intentar
        { devolver objetivo.recibir(0, bindingName);
        } catch (NullPointerException npe) {
        LOG.error("getMessage() recibió un NPE con enlace = {}", bindingName);

        devuelve nulo;
    }
}
```


Del código anterior podemos ver lo siguiente:

- El método `getMessage()` devuelve un mensaje de un tema específico utilizando el Bean `OutputDestination`, destino nombrado
- El método `getMessages()` utiliza el método `getMessage()` para devolver todos los mensajes en un tema
- El método `purgeMessages()` utiliza el método `getMessages()` para purgar un tema de todos los mensajes actuales

3. Cada prueba comienza con la purga de todos los temas involucrados en las pruebas utilizando un método `setup()` anotado con `@BeforeEach`:

```
@BeforeEach
void setUp() {
    purgeMessages("productos");
    purgeMessages("recomendaciones");
    purgeMessages("reseñas");
}
```

4. Una prueba real puede verificar los mensajes de un tema mediante el método `getMessages()`. Por ejemplo, consulte la siguiente prueba para la creación de un producto compuesto:

```
@Test
void crearProductoCompuesto1() {

    ProductoAgregado compuesto = nuevo ProductoAgregado(
        1, "nombre", 1, nulo, nulo, nulo);
    postAndVerifyProduct(compuesto, ACEPTADO);

    Lista final <String> productMessages = getMessages("productos");
    Lista final <String> mensajesDeRecomendación = obtenerMensajes(
        "recomendaciones");
    Lista final <String> reviewMessages = getMessages("reseñas");

    // Afirmar un nuevo evento de producto esperado en cola
    assertEquals(1, productMessages.size());

    Evento<Entero, Producto> eventoEsperado =
        nuevo Evento<>(CREAR, composite.getProductId(),
```

```

        nuevo Producto(compuesto.getProductId(), compuesto.getName(),
            compuesto.getWeight(), null));
    assertThat(productMessages.get(0),
        es(mismoEventoExceptoCreadoEn(evento esperado)));

    // Afirmar que no hay recomendaciones y revisar eventos
    assertEquals(0, mensajesDeRecomendación.tamaño());
    assertEquals(0, reviewMessages.size());
}

```

En el código anterior, podemos ver un ejemplo donde una prueba hace lo siguiente:

- En primer lugar, realiza una solicitud HTTP POST , solicitando la creación de un producto compuesto.

producto.
- A continuación, obtiene todos los mensajes de los tres temas, uno para cada servicio principal subyacente.
- Para estas pruebas, la marca de tiempo específica de cuándo se creó un evento es irrelevante.

Para comparar un evento real con uno esperado, ignorando las diferencias en el campo `eventCreatedAt` , se puede usar una clase auxiliar llamada `IsSameEvent` . El método `sameEventExceptCreatedAt()` es un método estático de la clase `IsSameEvent` que compara objetos `Event` y los trata como iguales si todos los campos son iguales, excepto el campo `eventCreatedAt` .
- Por último, verifica que se puedan encontrar los eventos esperados y no otros.

Para obtener el código fuente completo, consulte las clases de prueba `MessagingTests` e `IsSameEvent` en el proyecto `compuesto del producto` .

Ejecución de pruebas manuales del entorno de microservicios reactivos

Ahora contamos con microservicios totalmente reactivos, tanto en términos de API REST síncronas sin bloqueo como de servicios asíncronos basados en eventos. ¡Probémoslos!

Aprenderemos a ejecutar pruebas utilizando RabbitMQ y Kafka como intermediario de mensajes. Dado que RabbitMQ puede usarse con y sin particiones, probaremos ambos casos. Se utilizarán tres configuraciones diferentes, cada una definida en un archivo Docker Compose independiente:

- Uso de RabbitMQ sin el uso de particiones
- Uso de RabbitMQ con dos particiones por tema
- Uso de Kafka con dos particiones por tema

Sin embargo, antes de probar estas tres configuraciones, necesitamos agregar dos características para poder probar el procesamiento asincrónico:

- Guardar eventos para inspección posterior al usar RabbitMQ
- Una API de salud que se puede utilizar para monitorear el estado del panorama de microservicios

Guardado de eventos

Tras realizar algunas pruebas en servicios asíncronos basados en eventos, podría ser interesante ver qué eventos se enviaron realmente. Al usar Spring Cloud Stream con Kafka, los eventos se conservan en los temas, incluso después de que los consumidores los hayan procesado. Sin embargo, al usar Spring Cloud Stream con RabbitMQ, los eventos se eliminan tras su procesamiento correcto.

Para ver qué eventos se han publicado en cada tema, Spring Cloud Stream está configurado para guardar los eventos publicados en un grupo de consumidores independiente, `auditGroup`, por tema. Para los productos

Tema, la configuración se ve así:

```
primavera.nube.stream:
  fijaciones:
    productos-fuera-0:
      destino: productos
      productor:
        grupos obligatorios: auditGroup
```

Al utilizar RabbitMQ, esto dará como resultado que se creen colas adicionales donde se almacenan los eventos para su posterior inspección.

Para obtener el código fuente completo, consulte el archivo de configuración `application.yml` en el componente del producto. proyecto.

Agregar una API de salud

Probar un entorno de microservicios que combina API síncronas y mensajería asíncrona es un desafío. Por ejemplo, ¿cómo sabemos cuándo un entorno de microservicios recién creado, junto con sus bases de datos y sistema de mensajería, está listo para procesar solicitudes y mensajes?

Para facilitar la identificación de la disponibilidad de todos los microservicios, hemos añadido API de estado a los microservicios. Estas API se basan en la compatibilidad con los endpoints de estado que incluye el módulo Spring Boot Actuator. De forma predeterminada, un endpoint de estado basado en Actuator responde con "UP".

(y da 200 como estado de retorno HTTP) si el microservicio en sí y todas las dependencias Spring

Boot sabe si están disponibles. Las dependencias que Spring Boot conoce incluyen bases de datos y sistemas de mensajería. Si el microservicio o alguna de sus dependencias no están disponibles, el endpoint de estado responde con DOWN (y devuelve 500 como estado HTTP).

También podemos extender los endpoints de estado para cubrir dependencias que Spring Boot desconoce. Usaremos esta función para extender el endpoint de estado del compuesto de producto, de modo que también incluya el estado de los tres servicios principales. Esto significa que el endpoint de estado del compuesto de producto solo responderá con UP si él mismo y los tres microservicios principales están en buen estado. Esto se puede usar de forma manual o automática con el script test-em-all.bash para determinar cuándo todos los microservicios y sus dependencias están en funcionamiento.

En la clase `ProductCompositeIntegration`, hemos agregado métodos auxiliares para verificar el estado de los tres microservicios principales, de la siguiente manera:

```
público Mono<Salud> getProductHealth() {  
    devolver getHealth(productServiceUrl);  
}  
  
público Mono<Salud> obtenerRecomendaciónSalud() {  
    devolver getHealth(UriServicioRecomendación);  
}  
  
público Mono<Salud> obtenerRevisiónSalud() {  
    devolver getHealth(reviewServiceUrl);  
}  
  
privado Mono<Salud> obtenerSalud(String url) {  
    url += "/actuador/salud";  
    LOG.debug(" Llamará a la API de salud en la URL: {}", url);  
    devolver webClient.get().uri(url).retrieve().bodyToMono(String.class)  
        .map(s -> new Salud.Builder().up().build())  
        .onErrorResume(ex -> Mono.just(nuevo  
            Salud.Builder().down(ex).build()))  
        .log(LOG.getName(), FINE);  
}
```

Este código es similar al que usamos anteriormente para llamar a los servicios principales y leer las API. Tenga en cuenta que el punto final de salud está configurado, por defecto, en `/actuador/health`.

Para obtener el código fuente completo, consulte la clase `ProductCompositeIntegration` en el proyecto `product-composite`.

En la clase de configuración, `HealthCheckConfiguration`, utilizamos estos métodos auxiliares para registrar un control de estado compuesto mediante la clase Spring Actuator llamada `CompositeReactiveHealthContributor`:

```
@Configuration
class pública HealthCheckConfiguration {

    @Autowired
    Integración de ProductCompositeIntegration ;

    @Frijol
    Servicios básicos de ReactiveHealthContributor() {

        Mapa final <String, ReactiveHealthIndicator> registro =
            nuevo LinkedHashMap<>();

        registro.put("producto", () -> integración.getProductHealth());
        registro.put("recomendación", () ->
            integración.getRecommendationHealth());
        registro.put("revisión", () -> integración.getReviewHealth());

        devuelve CompositeReactiveHealthContributor.fromMap(registro);
    }
}
```

Para obtener el código fuente completo, consulte la clase `HealthCheckConfiguration` en el proyecto compuesto del producto.

Finalmente, en el archivo de configuración `application.yml` de los cuatro microservicios, configuramos el Spring Boot Actuator para que haga lo siguiente:

- Muestra detalles sobre el estado de salud, que no solo incluye ARRIBA o ABAJO sino también información sobre sus dependencias
- Expone todos sus puntos finales a través de HTTP

La configuración para estos dos ajustes es la siguiente:

```
management.endpoint.health.show-details: "SIEMPRE"
administración.puntos finales.web.exposición.incluir: "**"
```

Para obtener un ejemplo del código fuente completo, consulte el archivo de configuración `application.yml` en el proyecto compuesto del producto .



Advertencia: Estas opciones de configuración son útiles durante el desarrollo, pero revelar demasiada información en los puntos finales de los actuadores en sistemas de producción puede representar un problema de seguridad. Por lo tanto, planifique minimizar la información expuesta por los puntos finales de los actuadores en producción.

Esto se puede hacer reemplazando `***` con, por ejemplo, `health,info` en la configuración anterior de la propiedad `management.endpoints.web.exposure.include` .

Para obtener detalles sobre los puntos finales expuestos por Spring Boot Actuator, consulte <https://docs.spring.io/spring-boot/docs/current/reference/html/production-ready-endpoints.html>.

El punto final de salud se puede usar manualmente con el siguiente comando (no lo intente todavía, espere hasta que hayamos iniciado el entorno de microservicios):

```
curl localhost:8080/actuador/salud -s | jq .
```

Esto dará como resultado una respuesta que contendrá lo siguiente:

```
bash
{
  "status": "UP",
  "components": {
    "coreServices": {
      "status": "UP",
      "components": {
        "product": {
          "status": "UP"
        }
      },
      "recommendation": {
        "status": "UP"
      },
      "review": {
        "status": "UP"
      }
    }
  },
  ...
}
```

Figura 7.11: Respuesta del punto final de salud

En la salida anterior, podemos ver que el servicio compuesto informa que está en buen estado, es decir, su estado es ACTIVO. Al final de la respuesta, podemos ver que los tres microservicios principales también se reportan en buen estado.

Con una API de salud implementada, estamos listos para probar nuestros microservicios reactivos.

Usar RabbitMQ sin usar particiones

En esta sección, probaremos los microservicios reactivos junto con RabbitMQ pero sin usar particiones.

Para esta configuración se utiliza el archivo Docker Compose predeterminado docker-compose.yml . Se han añadido los siguientes cambios al archivo:

- Se ha agregado RabbitMQ, como se muestra aquí:

```
conejomq:
  imagen: rabbitmq:4.0.7-management
  límite de memoria: 512 m
  puertos:
    - 5672:5672
    - 15672:15672
  control de salud:
    prueba: ["CMD", "rabbitmqctl", "estado"]
    intervalo: 5 s
    tiempo de espera: 2 s
    reintentos: 60
```

De la declaración anterior de RabbitMQ, podemos ver lo siguiente:

- Utilizamos una imagen de Docker para RabbitMQ v4.0.7 incluido el complemento de administración y la interfaz web de administración
- Exponemos los puertos estándar para conectarse a RabbitMQ y la Web de administración. UI, 5672 y 15672
- Agregamos una verificación de estado para que Docker pueda saber cuándo RabbitMQ está listo para aceptar conexiones

Los microservicios ahora tienen una dependencia declarada en el servicio RabbitMQ. Esto significa que Docker no iniciará los contenedores de microservicios hasta que se informe que el servicio RabbitMQ está en buen estado.

```
depende_de:
  rabbitmq:
    condición: servicio_saludable
```

Para ejecutar pruebas manuales, realice los siguientes pasos:

1. Construya e inicie el panorama del sistema con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo07 ./
gradlew compilación y compilación de docker compose y compilación de docker compose up -d
```

2. Ahora, debemos esperar a que el entorno de microservicios esté listo y funcionando. Intente ejecutar el siguiente comando varias veces:

```
curl -s localhost:8080/actuador/salud | jq -r .estado
```

¡Cuando vuelva a estar activo, estaremos listos para ejecutar nuestras pruebas!

3. Primero, cree un producto compuesto con los siguientes comandos:

```
cuerpo={"productId":1,"name":"nombre del producto C","peso":300,
  "recomendaciones":[
    {"recommendationId":1,"author":"autor 1",
      "rate":1,"content":"contenido 1"},
    {"recommendationId":2,"author":"autor 2",
      "tasa":2,"contenido":"contenido 2"},
    {"recommendationId":3,"author":"autor 3",
      "tasa":3,"contenido":"contenido 3"}
  ], "reseñas":[
    {"reviewId":1,"author":"autor 1","subject":"sujeto 1",
      "contenido":"contenido 1"},
    {"reviewId":2,"author":"autor 2","subject":"sujeto 2",
      "contenido":"contenido 2"},
    {"reviewId":3,"author":"autor 3","subject":"sujeto 3",
      "contenido":"contenido 3"} ]}'
```



```
curl -X POST localhost:8080/producto-compuesto -H "Tipo-de-contenido: aplicación/json" --data "$body"
```

Al usar Spring Cloud Stream con RabbitMQ, se creará un intercambio de RabbitMQ por tema y un conjunto de colas, según nuestra configuración. ¡Veamos qué colas ha creado Spring Cloud Stream!

- 4. Abra la siguiente URL en un navegador web: <http://localhost:15672/#/queues>. Inicie sesión con el nombre de usuario y la contraseña predeterminados: invitado. Debería ver las siguientes colas:

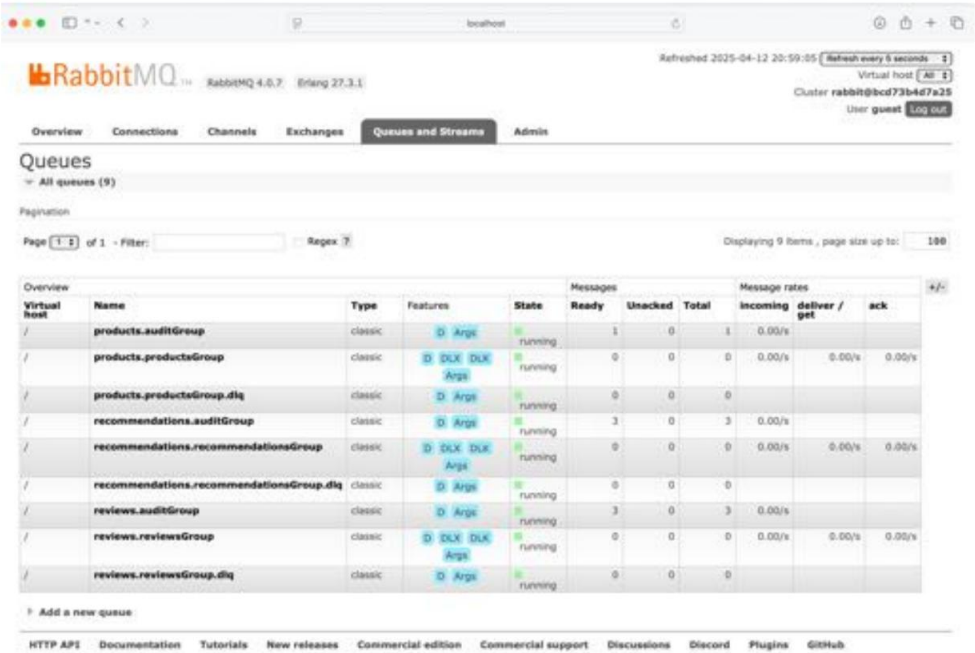


Figura 7.12: Lista de colas

Para cada tema, podemos ver una cola para auditGroup, una cola para el grupo de consumidores utilizado por el microservicio principal correspondiente y una cola de mensajes fallidos. También podemos ver que las colas de auditGroup contienen mensajes, como era de esperar.

- 5. Haga clic en la cola products.auditGroup y desplácese hacia abajo hasta la sección Obtener mensajes , expándala y haga clic en el botón Obtener mensaje(s) para ver el mensaje en la cola:

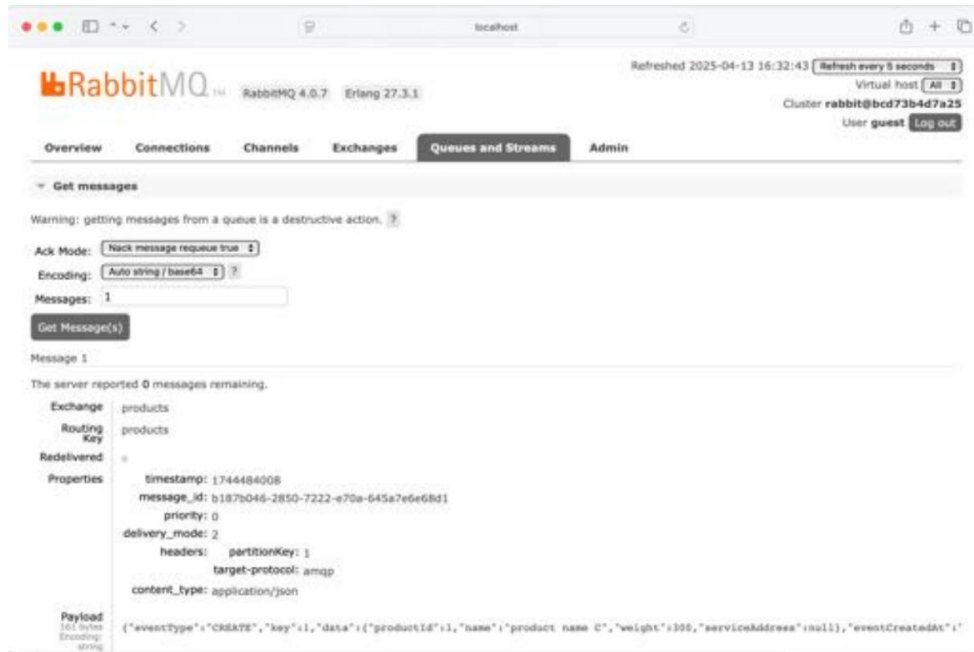


Figura 7.13: Visualización del mensaje en la cola

En la captura de pantalla anterior, observe la sección Carga útil , pero también el encabezado, partición-Key, que usaremos en la siguiente sección donde probaremos RabbitMQ con particiones.

6. A continuación, intente obtener el compuesto del producto utilizando el siguiente código:

```
curl -s localhost:8080/producto-compuesto/1 | jq
```

7. Finalmente, elimínelo con el siguiente comando:

```
curl -X DELETE localhost:8080/producto-compuesto/1
```

8. Intente obtener el producto eliminado de nuevo. Debería aparecer un error 404: "No encontrado" .
9. Si vuelve a mirar las colas de auditoría de RabbitMQ, debería poder encontrar nuevos mensajes que contengan eventos de eliminación .
10. Finalice la prueba desglosando el panorama de microservicios con el siguiente comando:

Mand:

```
Docker Compose Down
```

Con esto finalizamos las pruebas donde usamos RabbitMQ sin particiones. Ahora, sigamos probando RabbitMQ con particiones.

Uso de RabbitMQ con particiones

¡Ahora, probemos el soporte de partición en Spring Cloud Stream!

Disponemos de un archivo Docker Compose independiente para usar RabbitMQ con dos particiones por tema: `docker-compose-partitions.yml`. Este archivo también iniciará dos instancias por microservicio principal, una para cada partición. Por ejemplo, una segunda instancia de producto se configura de la siguiente manera:

```
producto-p1:
  construir: microservicios/producto-servicio
  límite de memoria: 512 m
  ambiente:
    - SPRING_PROFILES_ACTIVE=docker,transmisión_particionada,
      instancia_de_transmisión_1
  depende_de:
    MongoDB:
      condición: servicio_saludable
    conejomq:
      condición: servicio_saludable
```

A continuación se muestra una explicación de la configuración anterior:

- Usamos el mismo código fuente y Dockerfile que usamos para la primera instancia del producto .
pero configúrelos de manera diferente.
- Para que todas las instancias de microservicios sepan que usarán particiones, hemos agregado el **Perfil de Spring, particionado en streaming**, a su entorno `SPRING_PROFILES_ACTIVE` variable.
- Asignamos las dos instancias de producto a particiones diferentes utilizando perfiles Spring distintos. El perfil `Spring streaming_instance_0` lo utiliza la primera instancia de producto y el `streaming_instance_1` lo utiliza la segunda instancia (`product-p1`).

La segunda instancia del producto solo procesará eventos asíncronos; no responderá a las llamadas de la API. Dado que tiene un nombre diferente, `product-p1` (también usado como su nombre DNS), no responderá a las llamadas a una URL que comience por `http://product:8080`.

Inicie el panorama de microservicios con el siguiente comando:

```
exportar COMPOSE_FILE=docker-compose-partitions.yml
docker compose compilación && docker compose up -d
```

Cree un producto compuesto de la misma manera que para las pruebas de la sección anterior, pero también con el ID de producto establecido en 2. Si observa las colas configuradas por Spring Cloud Stream, verá una cola por partición y que las colas de auditoría de producto ahora contienen un mensaje cada una; el evento del ID de producto 1 se colocó en una partición y el del ID de producto 2 en la otra. Si regresa a `http://localhost:15672/#/`

colas En su navegador web debería ver algo como lo siguiente:

Overview				Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	Incoming	deliver / get	ack
/	products.auditGroup-0	classic	D Arqst	running	1	0	1	0.00/s		
/	products.auditGroup-1	classic	D Arqst	running	1	0	1	0.00/s		
/	products.productsGroup-0	classic	D DLX DLX Arqst	running	0	0	0	0.00/s	0.00/s	0.00/s
/	products.productsGroup-1	classic	D DLX DLX Arqst	running	0	0	0	0.00/s	0.00/s	0.00/s
/	products.productsGroup.dlq	classic	D Arqst	running	0	0	0			
/	recommendations.auditGroup-0	classic	D Arqst	running	3	0	3	0.00/s		
/	recommendations.auditGroup-1	classic	D Arqst	running	3	0	3	0.00/s		
/	recommendations.recommendationsGroup-0	classic	D DLX DLX Arqst	running	0	0	0	0.00/s	0.00/s	0.00/s
/	recommendations.recommendationsGroup-1	classic	D DLX DLX Arqst	running	0	0	0	0.00/s	0.00/s	0.00/s
/	recommendations.recommendationsGroup.dlq	classic	D Arqst	running	0	0	0			
/	reviews.auditGroup-0	classic	D Arqst	running	3	0	3	0.00/s		
/	reviews.auditGroup-1	classic	D Arqst	running	3	0	3	0.00/s		
/	reviews.reviewsGroup-0	classic	D DLX DLX Arqst	running	0	0	0	0.00/s	0.00/s	0.00/s
/	reviews.reviewsGroup-1	classic	D DLX DLX Arqst	running	0	0	0	0.00/s	0.00/s	0.00/s
/	reviews.reviewsGroup.dlq	classic	D Arqst	running	0	0	0			

Figura 7.14: Lista de colas

Para finalizar la prueba con RabbitMQ usando particiones, derribe el panorama de microservicios con el siguiente comando:

```
Docker Compose Down
desconfigurar COMPOSE_FILE
```

Hemos terminado las pruebas con RabbitMQ, con y sin particiones. La configuración final que probaremos consiste en probar los microservicios junto con Kafka.

Usando Kafka con dos particiones por tema

Ahora, probaremos una característica muy interesante de Spring Cloud Stream: ¡cambiar el sistema de mensajería de RabbitMQ a Apache Kafka!

Esto se puede hacer simplemente cambiando el valor de la propiedad `spring.cloud.stream.defaultBinder` de Rabbit a Kafka. Esto se gestiona mediante el archivo Docker Compose `docker-compose-kafka.yml`, que también ha reemplazado RabbitMQ con Kafka y ZooKeeper. La configuración de Kafka y ZooKeeper es la siguiente:

```
Kafka:
  imagen: confluentinc/cp-kafka:7.9.0
  reiniciar: siempre
  límite de memoria: 1024m
  puertos:
    - "9092:9092"
  ambiente:
    - KAFKA_ZOOKEEPER_CONNECT=guardián del zoológico:2181
    - OYENTES_ANUNCIADOS_KAFKA=TEXTO_PLAINTEXT://kafka:9092
    - KAFKA_BROKER_ID=1
    - FACTOR DE REPLICACIÓN DEL TEMA DE DESPLAZAMIENTOS DE KAFKA=1
  depende_de:
    - cuidador del zoológico

cuidador del zoológico:
  imagen: confluentinc/cp-zookeeper:7.9.0
  reiniciar: siempre
  límite de memoria: 512 m
  puertos:
    - "2181:2181"
  ambiente:
    - PUERTO_CLIENTE_ZOOKEEPER=2181
```

Kafka también está configurado para usar dos particiones por tema y, como antes, iniciamos dos instancias por microservicio principal, una para cada partición. Para más detalles, consulta el archivo de Docker Compose, `docker-compose-kafka.yml`.

Inicie el panorama de microservicios con el siguiente comando:

```
exportar COMPOSE_FILE=docker-compose-kafka.yml
docker compose compilación && docker compose up -d
```

Repita las pruebas de la sección anterior: cree dos productos, uno con el ID del producto establecido en 1 y uno con el ID del producto establecido en 2.



Desafortunadamente, Kafka no incluye herramientas gráficas que permitan inspeccionar temas, particiones y los mensajes que contienen. En cambio, Podemos ejecutar comandos CLI en el contenedor Docker de Kafka.

Para ver una lista de temas, ejecute el siguiente comando:

```
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 --list
```

Espere un resultado como el que se muestra aquí:

```
bash
error.products.productsGroup
error.recommendations.recommendationsGroup
error.reviews.reviewsGroup
products
recommendations
reviews
$
```

Figura 7.15: Visualización de una lista de temas

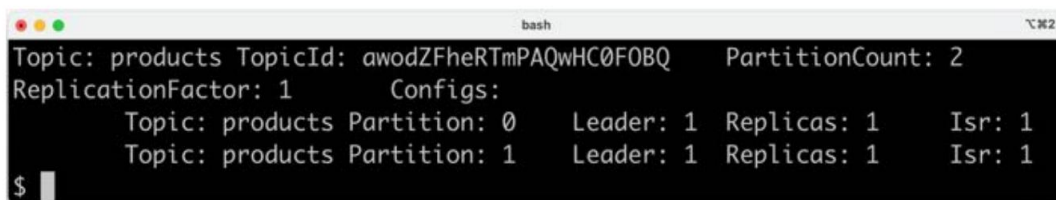
Esto es lo que vemos en la salida anterior:

- Los temas que tienen el prefijo error son los temas correspondientes a las colas de mensajes muertos.
- No encontrará grupos auditGroup como en RabbitMQ. Dado que Kafka conserva los eventos en los temas, incluso después de que los consumidores los hayan procesado, no se necesita un grupo auditGroup adicional .

Para ver las particiones en un tema específico (por ejemplo, el tema de productos), ejecute el siguiente comando: Mand:

```
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 --describe --topic productos
```

Espera un resultado como el que se muestra aquí:



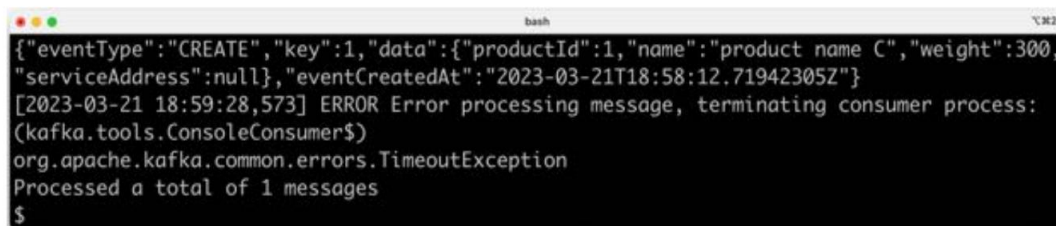
```
bash
Topic: products TopicId: awodZFheRTmPAQwHC0FOBQ PartitionCount: 2
ReplicationFactor: 1 Configs:
  Topic: products Partition: 0 Leader: 1 Replicas: 1 Isr: 1
  Topic: products Partition: 1 Leader: 1 Replicas: 1 Isr: 1
$
```

Figura 7.16: Visualización de particiones en el tema de productos

Para ver todos los mensajes en una partición específica (por ejemplo, la partición 1 en el tema de productos), ejecute el siguiente comando:

```
docker compose exec kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic productos --desde-el-
principio --timeout-ms 1000 --partición 1
```

Espera un resultado como el que se muestra aquí:



```
bash
{"eventType":"CREATE","key":1,"data":{"productId":1,"name":"product name C","weight":300,
"serviceAddress":null},"eventCreatedAt":"2023-03-21T18:58:12.71942305Z"}
[2023-03-21 18:59:28,573] ERROR Error processing message, terminating consumer process:
(kafka.tools.ConsoleConsumer$)
org.apache.kafka.common.errors.TimeoutException
Processed a total of 1 messages
$
```

Figura 7.17: Visualización de todos los mensajes en la partición 1 en el tema de productos

La salida finalizará con una excepción de tiempo de espera ya que detenemos el comando al especificar un tiempo de espera para el comando de 1000 ms.

Derriba el panorama de microservicios con el siguiente comando:

```
Docker Compose Down
desconfigurar COMPOSE_FILE
```

Ahora, hemos aprendido cómo usar Spring Cloud Stream para migrar un agente de mensajes de RabbitMQ a Kafka sin necesidad de modificar el código fuente. Solo requiere algunos cambios en el archivo Docker Compose.

Pasemos a la última sección de este capítulo: ¡aprender cómo ejecutar estas pruebas automáticamente!

Ejecución de pruebas automatizadas del panorama de microservicios reactivos

Para poder ejecutar pruebas del entorno de microservicios reactivos de forma automática en lugar de manual, se ha mejorado el script de prueba automatizado `test-em-all.bash`. Los cambios más importantes son los siguientes:

- El script utiliza el nuevo punto final de salud para saber cuándo el panorama del microservicio está operacional, como se muestra aquí:

```
waitForService curl http://$HOST:$PORT/actuador/salud
```

El script incluye una nueva función `waitForMessageProcessing()`, que se llama después de configurar los datos de prueba. Su propósito es simplemente esperar a que los servicios de creación asíncrona completen la creación de los datos de prueba.

Para utilizar el script de prueba para ejecutar automáticamente las pruebas con RabbitMQ y Kafka, realice los siguientes pasos:

1. Ejecute las pruebas utilizando el archivo Docker Compose predeterminado (es decir, con RabbitMQ sin particiones) con los siguientes comandos:

```
desconfigurar COMPOSE_FILE  
./test-em-all.bash inicio parada
```

2. Ejecute las pruebas para RabbitMQ con dos particiones por tema usando el archivo `docker-compose-partitions.yml` de Docker Compose con los siguientes comandos:

```
exportar COMPOSE_FILE=docker-compose-partitions.yml  
./test-em-all.bash inicio parada  
desconfigurar COMPOSE_FILE
```

3. Finalmente, ejecute las pruebas con Kafka y dos particiones por tema usando Docker Compose Archivo `docker-compose-kafka.yml` con los siguientes comandos:

```
exportar COMPOSE_FILE=docker-compose-kafka.yml  
./test-em-all.bash inicio parada  
desconfigurar COMPOSE_FILE
```

En esta sección, aprendimos a usar el script de prueba `test-em-all.bash` para ejecutar automáticamente pruebas del entorno de microservicios reactivos, que se ha configurado para usar RabbitMQ o Kafka como su agente de mensajes.

Resumen

¡En este capítulo hemos visto cómo podemos desarrollar microservicios reactivos!

Usando Spring WebFlux y Spring WebClient, podemos desarrollar API síncronas sin bloqueo que pueden manejar solicitudes HTTP entrantes y enviar solicitudes HTTP salientes sin bloquear ningún subproceso. Usando la compatibilidad reactiva de Spring Data con MongoDB, también podemos acceder a bases de datos MongoDB sin bloqueo, es decir, sin bloquear ningún subproceso mientras esperamos respuestas de la base de datos. Spring WebFlux, Spring WebClient y Spring Data se basan en Project Reactor para proporcionar sus funciones reactivas y sin bloqueo. Cuando necesitamos usar código de bloqueo (por ejemplo, al usar Spring Data para JPA), podemos encapsular el procesamiento del código de bloqueo programándolo en un grupo de subprocesos dedicado.

También hemos visto cómo Spring Data Stream puede utilizarse para desarrollar servicios asíncronos basados en eventos que funcionan como sistemas de mensajería tanto en RabbitMQ como en Kafka sin necesidad de modificar el código. Mediante la configuración, podemos usar funciones de Spring Cloud Stream, como grupos de consumidores, reintentos, colas de mensajes fallidos y particiones, para gestionar los diversos desafíos de la mensajería asíncrona.

También hemos aprendido a probar de forma manual y automática un panorama de sistemas que consta de microservicios reactivos.

Este fue el capítulo final sobre cómo utilizar las características fundamentales en Spring Boot y Spring Frame. trabajar.

A continuación, se presenta una introducción a Spring Cloud y cómo se puede utilizar para hacer que nuestros servicios estén listos para producción, sean escalables, robustos, configurables, seguros y resilientes.

Preguntas

1. ¿Por qué es importante saber cómo desarrollar microservicios reactivos?
2. ¿Cómo elegir entre API síncronas sin bloqueo y servicios asíncronos controlados por eventos o mensajes?
3. ¿Qué hace que un evento sea diferente de un mensaje?
4. Mencione algunos desafíos de los servicios asíncronos basados en mensajes. ¿Cómo los abordamos?
¿a ellos?

5. ¿Por qué no falla la siguiente prueba?

```
@Prueba
void testStepVerifier() {
    Verificador de pasos.crear(Flux.just(1, 2, 3, 4)
        .filter(n -> n % 2 == 0)
        .map(n -> n * 2)
        .registro())
        .esperarSiguiente(4, 8, 12);
}
```

Primero, asegúrese de que la prueba falle. Luego, corrijala para que tenga éxito.

6. ¿Cuáles son los desafíos de escribir pruebas con código reactivo usando JUnit y cómo podemos...
¿Manejarlos?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



Parte 2

Aprovechar Spring Cloud para gestionar microservicios

En esta parte del libro, comprenderá cómo se puede utilizar Spring Cloud para

Gestionar los desafíos que se enfrentan al desarrollar microservicios (es decir, construir un sistema distribuido).

Esta parte del libro incluye los siguientes capítulos:

- Capítulo 8, Introducción a Spring Cloud
- Capítulo 9, Agregar descubrimiento de servicios mediante Netflix Eureka
- Capítulo 10, Uso de Spring Cloud Gateway para ocultar microservicios detrás de un servidor perimetral
- Capítulo 11, Protección del acceso a las API
- Capítulo 12, Configuración centralizada
- Capítulo 13, Mejorar la resiliencia mediante Resilience4j
- Capítulo 14, Comprensión del rastreo distribuido

8

Introducción a Spring Cloud

Hasta ahora, hemos visto cómo podemos usar Spring Boot para construir microservicios con API bien documentadas, junto con Spring WebFlux y springdoc-openapi; persistir datos en bases de datos MongoDB y SQL usando Spring Data para MongoDB y JPA; microservicios reactivos ya sea como API no bloqueantes usando Project Reactor o como servicios asíncronos controlados por eventos usando Spring Cloud Stream con RabbitMQ o Kafka, junto con Docker; y administrar y probar un panorama de sistema que consiste en microservicios, bases de datos y sistemas de mensajería.

Ahora es el momento de ver cómo podemos utilizar Spring Cloud para hacer que nuestros servicios estén listos para producción, es decir, escalables, robustos, configurables, seguros y resilientes.

En este capítulo, le presentaremos cómo se puede usar Spring Cloud para implementar los siguientes patrones de diseño del Capítulo 1, Introducción a los microservicios, en Patrones de diseño para microservicios. sección:

- Descubrimiento de servicios
- Servidor de borde
- Configuración centralizada
- Disyuntor
- Rastreo distribuido

Requisitos técnicos

Este capítulo no contiene ningún código fuente, por lo que no es necesario instalar ninguna herramienta.

La evolución de Spring Cloud

En su lanzamiento inicial 1.0, en marzo de 2015, Spring Cloud era principalmente un envoltorio de herramientas de Netflix OSS, que son las siguientes:

- Netflix Eureka, un servidor de descubrimiento
- Netflix Ribbon, un balanceador de carga del lado del cliente
- Netflix Zuul, un servidor de borde
- Netflix Hystrix, un disyuntor

La versión inicial de Spring Cloud también incluía un servidor de configuración e integración con Spring Security, que proporcionaba APIs protegidas por OAuth 2.0. En mayo de 2016, se lanzó la versión Brixton (v1.1) de Spring Cloud. Con esta versión, Spring Cloud obtuvo compatibilidad con el rastreo distribuido basado en Spring Cloud Sleuth y Zipkin, desarrollados en Twitter.

Estos componentes iniciales de Spring Cloud podrían usarse para implementar los patrones de diseño anteriores.

Para obtener más detalles, consulte <https://spring.io/blog/2015/03/04/spring-cloud-1-0-0-available-ahora> y <https://spring.io/blog/2016/05/11/spring-cloud-brixton-release-is-available>.

Desde sus inicios, Spring Cloud ha crecido considerablemente a lo largo de los años y ha agregado soporte para lo siguiente, entre otros:

- Descubrimiento de servicios y configuración centralizada basada en HashiCorp Consul y Apache

Guardián del zoológico

- Microservicios basados en eventos que utilizan Spring Cloud Stream
- Proveedores de nube como Microsoft Azure, Amazon Web Services y Google Cloud Platform.

forma



Ver <https://spring.io/projects/spring-cloud> para obtener una lista completa de herramientas.

Desde el lanzamiento de Spring Cloud Greenwich (v2.1) en enero de 2019, algunas de las herramientas de Netflix mencionadas anteriormente se han colocado en modo de mantenimiento en Spring Cloud.

Esto se debe a que Netflix ya no añade nuevas funciones a algunas herramientas y a que Spring Cloud ha incorporado mejores alternativas. El proyecto Spring Cloud recomienda las siguientes alternativas:

Componente actual	Reemplazado por
Netflix Hystrix	Resiliencia4j
Panel de control de Netflix Hystrix/Turbina de Netflix	Micrómetro y sistema de monitoreo
Cinta de Netflix	Balanceador de carga de Spring Cloud
Netflix Zuul	Puerta de enlace de Spring Cloud

Tabla 8.1: Reemplazos de herramientas de Spring Cloud

Para más detalles, consulte lo siguiente:

- <https://spring.io/blog/2019/01/23/spring-cloud-greenwich-release-is-now-disponible>
- <https://github.com/Netflix/Hystrix#hystrix-status>
- <https://github.com/Netflix/ribbon#project-status-on-maintenance>

Con el lanzamiento de Spring Cloud Ilford (v2020.0.0) en diciembre de 2020, el único componente de Netflix restante en Spring Cloud es Netflix Eureka.

La última versión de Spring Cloud, Northfields (v2025.0.0), se lanzó en mayo de 2025 junto con Spring Boot 3.5.0. Como ya se mencionó en el Capítulo 2, Introducción a Spring Boot, en la sección "Novedades de Spring Boot 3.0 a 3.5", Spring Cloud Sleuth ha sido reemplazado por Micrometer Tracing para soportar el rastreo distribuido.

En este libro, utilizaremos los componentes de software de la siguiente tabla para implementar los patrones de diseño mencionados anteriormente:

Patrón de diseño	Componente de software
Descubrimiento de servicios	Balanceador de carga de Netflix Eureka y Spring Cloud
Servidor de borde	Spring Cloud Gateway y Spring Security OAuth
Configuración centralizada	Servidor de configuración de Spring Cloud
Cortacircuitos	Resiliencia4j
Rastreo distribuido	Trazado micrométrico y Zipkin

Tabla 8.2: Componentes de software por patrón de diseño

¡Ahora, repasemos los patrones de diseño y presentemos los componentes de software que se utilizarán para implementarlos!

Uso de Netflix Eureka para el descubrimiento de servicios

El descubrimiento de servicios es probablemente la función de soporte más importante para que un entorno de microservicios cooperativos esté listo para producción. Como ya describimos en el Capítulo 1, Introducción a los Microservicios, en la sección Descubrimiento de servicios , un servicio de descubrimiento de servicios (o servicio de descubrimiento , como abreviatura) puede utilizarse para realizar un seguimiento de los microservicios existentes y sus instancias.

El primer servicio de descubrimiento que Spring Cloud admitió fue Netflix Eureka.

Lo usaremos en el Capítulo 9, Agregar descubrimiento de servicio usando Netflix Eureka, junto con un equilibrador de carga basado en Spring Cloud LoadBalancer.

Veremos lo fácil que es registrar microservicios con Netflix Eureka al usar Spring Cloud.

También aprenderemos cómo un cliente puede enviar solicitudes HTTP, como una llamada a una API RESTful, a una de las instancias registradas en Netflix Eureka. Además, el capítulo explicará cómo escalar el número de instancias de un microservicio y cómo se distribuirá la carga de las solicitudes a un microservicio entre sus instancias disponibles (basándose, por defecto, en la programación round-robin).

La siguiente captura de pantalla muestra la interfaz web de Eureka, donde podemos ver qué microservicios hemos registrado:

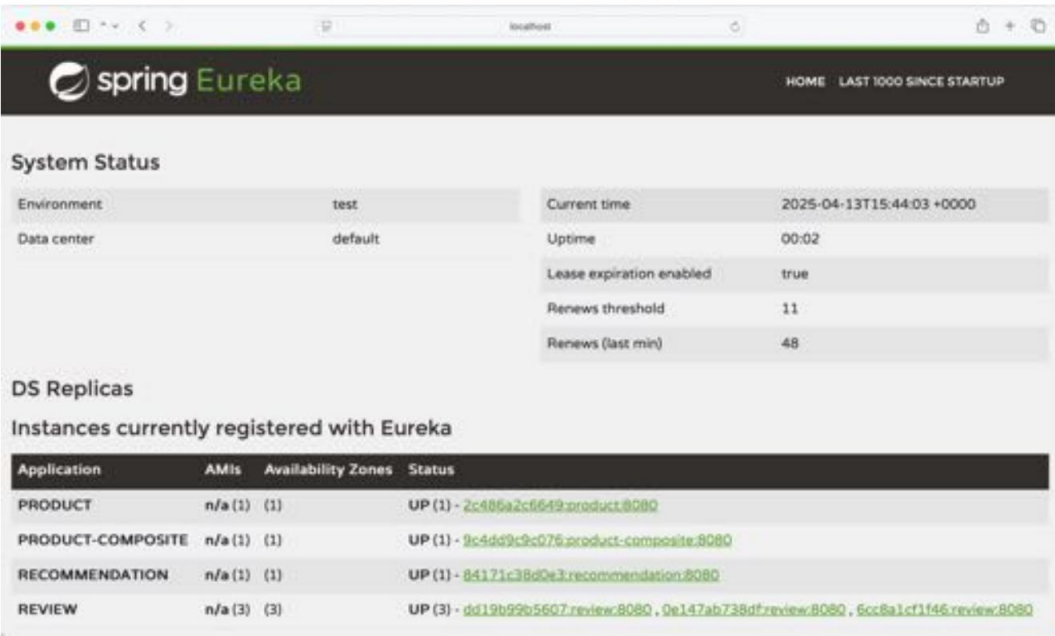


Figura 8.1: Visualización de microservicios actualmente registrados en Eureka

En la captura de pantalla anterior, podemos ver que el servicio de revisión tiene tres instancias disponibles, mientras que los otros tres servicios solo tienen una instancia cada uno.

Con la presentación de Netflix Eureka, presentemos cómo Spring Cloud puede ayudar a proteger un panorama de sistemas de microservicios mediante un servidor de borde.

Uso de Spring Cloud Gateway como servidor perimetral

Otra función de soporte muy importante es un servidor perimetral. Como ya describimos en el Capítulo 1, Introducción a los Microservicios, en la sección Servidor Perimetral, este puede utilizarse para proteger un entorno de microservicios, lo que implica ocultar los servicios privados del uso externo y proteger los servicios públicos cuando son utilizados por clientes externos.

Inicialmente, Spring Cloud utilizaba Netflix Zuul v1 como servidor perimetral. Desde el lanzamiento de Spring Cloud Greenwich, se recomienda usar Spring Cloud Gateway. Spring Cloud Gateway ofrece compatibilidad similar con funciones críticas, como el enrutamiento basado en rutas URL y la protección de endpoints mediante OAuth 2.0 y OpenID Connect (OIDC).

Una diferencia importante entre Netflix Zuul v1 y Spring Cloud Gateway es que Spring Cloud Gateway se basa en API sin bloqueo que utilizan Spring 6, Project Reactor y Spring Boot 3, mientras que Netflix Zuul v1 se basa en API con bloqueo. Esto significa que Spring Cloud Gateway debería ser capaz de gestionar un mayor número de solicitudes simultáneas que Netflix Zuul v1, lo cual es importante para un servidor perimetral por el que pasa todo el tráfico externo.

El siguiente diagrama muestra cómo todas las solicitudes de clientes externos pasan por Spring Cloud Gateway como servidor perimetral. Según las rutas URL, enruta las solicitudes al microservicio deseado:

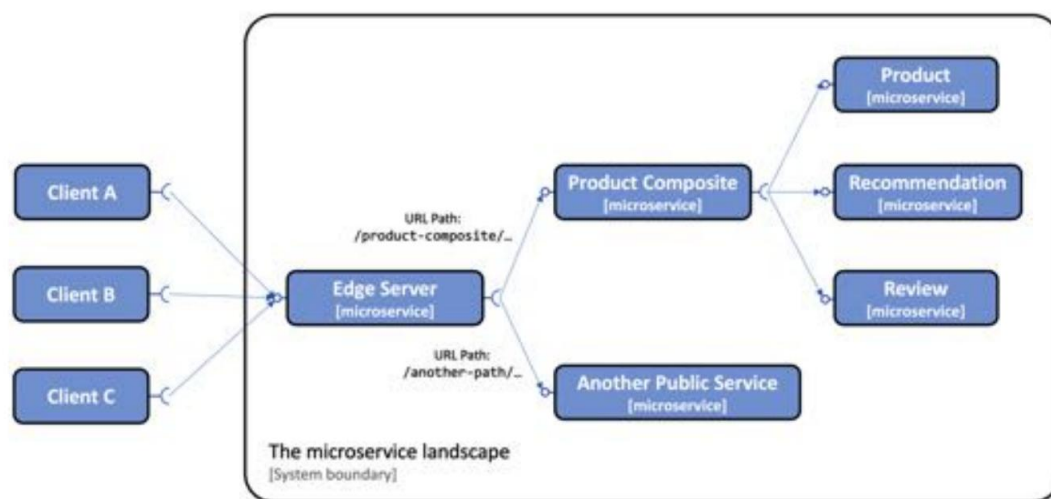


Figura 8.2: Solicitudes que se enrutan a través de un servidor perimetral

En el diagrama anterior, podemos ver cómo el servidor perimetral envía solicitudes externas cuya URL comienza con `/product-composite/` al microservicio "Product Composite". Los servicios principales de Producto, Recomendación y Revisión no son accesibles desde clientes externos.

En el Capítulo 10, Uso de Spring Cloud Gateway para ocultar microservicios detrás de un servidor perimetral, veremos cómo configurar Spring Cloud Gateway con nuestros microservicios.

En el Capítulo 11, "Asegurando el acceso a las API", veremos cómo podemos usar Spring Cloud Gateway junto con Spring Security OAuth2 para proteger el acceso al servidor edge mediante OAuth 2.0 y OIDC. También veremos cómo Spring Cloud Gateway puede propagar la información de identidad del usuario que realiza la llamada a nuestros microservicios, por ejemplo, su nombre de usuario o dirección de correo electrónico.

Con la introducción de Spring Cloud Gateway, veamos cómo Spring Cloud puede ayudar a administrar la configuración de un panorama de sistemas de microservicios.

Uso de Spring Cloud Config para una configuración centralizada

Para administrar la configuración de un paisaje de sistema de microservicios, Spring Cloud contiene Spring Cloud Config, que proporciona la gestión centralizada de los archivos de configuración según los requisitos descritos en el Capítulo 1, Introducción a los microservicios, en la configuración central. sección.

Spring Cloud Config admite el almacenamiento de archivos de configuración en varios backends diferentes, como los siguientes:

- Un repositorio Git, por ejemplo, en GitHub o Bitbucket
- Un sistema de archivos local
- Bóveda de HashiCorp
- Una base de datos JDBC

Spring Cloud Config nos permite manejar la configuración en una estructura jerárquica; por ejemplo, podemos colocar partes comunes de la configuración en un archivo común y configuraciones específicas del microservicio en archivos de configuración separados.

Spring Cloud Config también permite detectar cambios en la configuración y enviar notificaciones a los microservicios afectados. Utiliza Spring Cloud Bus para transportar las notificaciones. Spring Cloud Bus es una abstracción sobre Spring Cloud Stream, con la que ya estamos familiarizados; es decir, admite el uso de RabbitMQ o Kafka como sistema de mensajería para transportar las notificaciones de forma predeterminada.

El siguiente diagrama ilustra la cooperación entre Spring Cloud Config, sus clientes, un repositorio Git y Spring Cloud Bus:

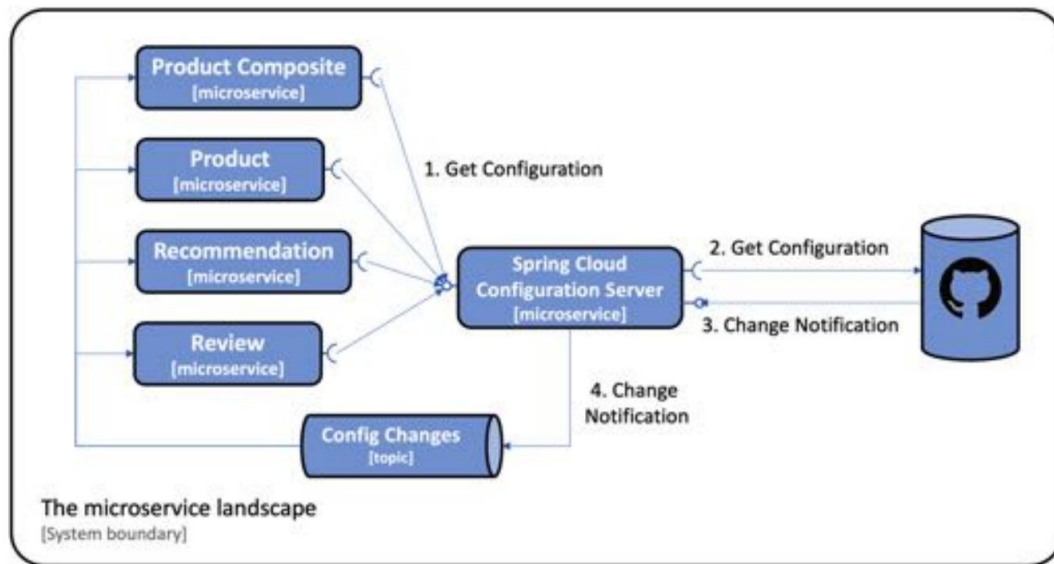


Figura 8.3: Cómo Spring Cloud Config encaja en el panorama de microservicios

El diagrama muestra lo siguiente:

1. Cuando los microservicios se inician, solicitan su configuración al servidor de configuración.
2. El servidor de configuración obtiene la configuración, en este caso, de un repositorio Git.
3. Opcionalmente, el repositorio Git se puede configurar para enviar notificaciones a la configuración.
servidor cuando las confirmaciones de Git se envían al repositorio de Git.
4. El servidor de configuración publicará los eventos de cambio mediante Spring Cloud Bus. Los microservicios afectados por el cambio reaccionarán y recuperarán la configuración actualizada del servidor de configuración.

Por último, Spring Cloud Config también admite el cifrado de información confidencial en la configuración, como las credenciales.

Aprenderemos sobre Spring Cloud Config en el Capítulo 12, Configuración centralizada.

Con la introducción de Spring Cloud Config, veamos cómo Spring Cloud puede ayudar a que los microservicios sean más resistentes a las fallas que ocurren de vez en cuando en un panorama de sistemas.

Usando Resilience4j para mejorar la resiliencia

En un entorno de sistemas a gran escala de microservicios cooperativos, debemos asumir que algo falla constantemente. Los fallos deben considerarse normales, y el entorno del sistema debe estar diseñado para gestionarlos.

Inicialmente, Spring Cloud incluía Netflix Hystrix, un interruptor de seguridad de eficacia probada. Sin embargo, como ya se mencionó, desde el lanzamiento de Spring Cloud Greenwich, se recomienda reemplazar Netflix Hystrix con Resilience4j. Resilience4j es una biblioteca de tolerancia a fallos de código abierto. Incorpora una gama más amplia de mecanismos de tolerancia a fallos en comparación con Netflix Hystrix:

- Se utiliza un disyuntor para evitar una reacción en cadena de fallos si se detiene un servicio remoto.
respondiendo.
- Un limitador de velocidad se utiliza para limitar la cantidad de solicitudes a un servicio durante un tiempo específico.
período.
- Se utiliza un mamparo para limitar el número de solicitudes simultáneas a un servicio.
- Los reintentos se utilizan para manejar errores aleatorios que puedan ocurrir de vez en cuando.
- Se utiliza un limitador de tiempo para evitar tener que esperar demasiado tiempo para obtener una respuesta de un servidor lento o que no responde.
servicio ivo.

Puede descubrir más sobre Resilience4j en <https://github.com/resilience4j/resilience4j>.

En el Capítulo 13, «Mejora de la resiliencia con Resilience4j», nos centraremos en el interruptor automático de Resilience4j. Este sigue el diseño clásico de un interruptor automático, como se ilustra en el siguiente diagrama de estados:

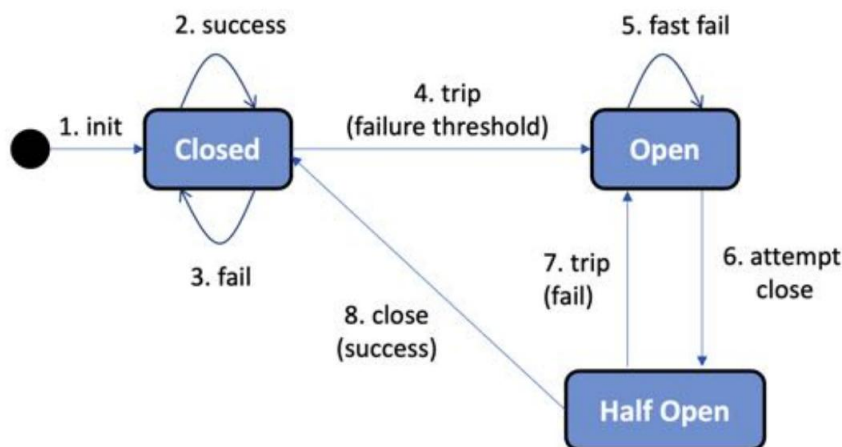


Figura 8.4: Diagrama de estados del disyuntor

Echemos un vistazo al diagrama de estados con más detalle:

1. Un disyuntor comienza como Cerrado, lo que permite procesar las solicitudes.
2. Mientras las solicitudes se procesen con éxito, permanecerá en el estado Cerrado .
3. Si empiezan a ocurrir fallos, un contador empieza a contar hacia adelante.
4. Si se alcanza un umbral de fallos dentro de un período de tiempo específico, el disyuntor se disparará, es decir, pasará al estado Abierto , impidiendo el procesamiento de más solicitudes. Tanto el umbral de fallos como el período de tiempo son configurables.
5. En cambio, una solicitud fallará rápidamente, lo que significa que regresará inmediatamente con una excepción.
6. Después de un período de tiempo configurable, el disyuntor entrará en un estado medio abierto y permitirá que pase una solicitud, a modo de prueba, para ver si se ha resuelto la falla.
7. Si la solicitud de sonda falla, el disyuntor vuelve al estado Abierto .
8. Si la solicitud de sonda tiene éxito, el disyuntor pasa al estado inicial Cerrado , lo que permite Nuevas solicitudes a procesar.

Ejemplo de uso del disyuntor en Resilience4j

Supongamos que tenemos un servicio REST, llamado myService, que está protegido por un disyuntor que utiliza Resilience4j.

Si el servicio comienza a producir errores internos (por ejemplo, porque no puede acceder a un servicio del que depende), podríamos recibir una respuesta del servicio como 500 Error interno del servidor.

Tras varios intentos configurables, el circuito se abrirá y se producirá un fallo rápido que devuelve un mensaje de error como « El disyuntor 'myService' está abierto ». Cuando se resuelva el error y se realice un nuevo intento (tras el tiempo de espera configurable), el disyuntor permitirá un nuevo intento como prueba. Si la llamada tiene éxito, el disyuntor se cerrará de nuevo; es decir, funcionará con normalidad.

Al usar Resilience4j junto con Spring Boot, podremos monitorear el estado de los interruptores automáticos en un microservicio mediante su endpoint de salud del actuador de Spring Boot . Si funciona con normalidad, es decir, si el circuito está cerrado, responderá con algo como lo siguiente: